

Operating System GATE prep

Generated 9/27/2026, 10:27:26 PM • 53 messages • Source: Shared ChatGPT link
<https://chatgpt.com/share/6ab949e1-780c-83e8-a3ca-02a78b603c87>

USER

Operating System

ASSISTANT

Operating System — GATE CSE

For **GATE 2027**, the official Operating Systems syllabus is:

System calls, processes, threads, inter-process communication, concurrency and synchronization. Deadlock. CPU and I/O scheduling. Memory management and virtual memory. File systems.

I can teach this exactly like we did **Computer Networks**: complete concepts !' shortcuts !' formulas !' common traps !' solved examples !' PYQ thinking !' revision notes.

Our OS course structure

1. Operating System Basics

- What an OS does
- Kernel/user mode
- System calls
- Interrupts vs traps/exceptions
- OS structures

2. Processes

- Program vs process
- Process states
- PCB
- Context switching
- Process creation
- `fork()`, `exec()`, `wait()`
- Zombie and orphan processes
- GATE process-count questions

3. Threads

- Process vs thread
- User/kernel threads
- Multithreading models
- Thread advantages
- Thread-related GATE traps

4. Inter-Process Communication

- Shared memory
- Message passing

- Pipes
- Signals
- IPC comparison

5. Concurrency

- Race condition
- Critical-section problem
- Mutual exclusion
- Progress
- Bounded waiting

6. Synchronization

- Locks
- Mutex
- Semaphores
- Binary/counting semaphore
- `wait()` / `signal()`
- Peterson's algorithm
- Producer-consumer
- Readers-writers
- Dining philosophers
- Semaphore PYQs

7. Deadlocks

- Four Coffman conditions
- Resource-allocation graph
- Deadlock prevention
- Avoidance
- Banker's algorithm
- Detection
- Recovery
- Safe/unsafe/deadlock states
- Resource-instance numerical problems

8. CPU Scheduling

- FCFS
- SJF
- SRTF
- Priority
- Round Robin
- Preemptive vs non-preemptive
- Arrival/burst/completion/turnaround/waiting/response time
- Gantt charts
- Scheduling numericals

9. I/O Scheduling

- FCFS
- SSTF

- SCAN
- C-SCAN
- LOOK
- C-LOOK
- Disk-head movement problems

10. Memory Management

- Logical vs physical address
- Contiguous allocation
- Fixed/variable partitioning
- Internal/external fragmentation
- Paging
- Segmentation
- Page tables
- TLB
- Effective memory-access time
- Multi-level paging

11. Virtual Memory

- Demand paging
- Page faults
- Locality
- Working set
- Thrashing
- FIFO
- Optimal
- LRU
- Belady's anomaly
- Effective access-time numericals

12. File Systems

- File concepts
- File operations
- Directory structures
- File allocation
 - contiguous
 - linked
 - indexed
- Free-space management
- Inodes
- Hard links vs symbolic links
- File-system numericals

We'll also cover the **hidden prerequisite concepts required to solve GATE questions**, even when the official syllabus mentions them only indirectly.

How I'll teach each chapter

For every topic I'll give you:

CONCEPT !' SIMPLE INTUITION !' TECHNICAL DEFINITION !' DIAGRAM !' FORMULAS !' SOLVED EXAMPLES !' GATE TRAPS !' PYQ APPROACH !' MEMORY TRICKS !' SHORT NOTES !' 30-SECOND REVISION

You do **not** need to hunt through long YouTube playlists alongside this. Use this as your structured OS course, and use PYQs afterward to test yourself.

One master memory map

Remember:

P T C D S M F

P — Processes **T** — Threads **C** — Concurrency **D** — Deadlocks **S** — Scheduling **M** — Memory **F** — File Systems

Think:

“Processes & Threads Compete !' Deadlock !' Schedule them !' Manage Memory !' Store Files.”

That single story gives you the backbone of Operating Systems.

We should begin from **Chapter 1 — Operating System Basics + System Calls**, and then you can simply type **NEXT** after each section, exactly as with Computer Networks.

USER

Operating System System calls, processes, threads, inter-process communication, concurrency and synchronization. Deadlock. CPU and I/O scheduling. Memory management and virtual memory. File systems.

This are the complete syllabus copy of gate So i have no flexibility to watch some hours of youtube video i want you to take time and give complete syllabus in detail in which you take your own time and give complete information related to gate and all the stuff give me a perfect prompt ... I keep changing the syllabus i also include different syllabus of gate So that i can make notes and give me some shortcut to remember concepts

ASSISTANT

Yes. Since you will keep changing the **GATE subject and official syllabus**, the best approach is to use one reusable **MASTER PROMPT** where you only replace two sections: **SUBJECT** and **OFFICIAL SYLLABUS**.

I've designed this specifically so you can study mainly from ChatGPT, make handwritten notes, remember concepts with shortcuts, and stay focused on **GATE-level preparation rather than unnecessary university-level theory**.

:::writing{variant="document" id="48317"}

GATE COMPLETE SUBJECT TEACHER — MASTER PROMPT

I am preparing for **GATE CSE** and I have limited time.

I do **not** have the flexibility to watch many hours of YouTube lectures for every subject.

I want you to act as my:

- GATE subject expert
- personal teacher
- concept explainer
- notes maker

- PYQ-oriented mentor
- numerical-problem trainer
- revision coach
- memory-trick creator

I will give you the **official GATE syllabus for one subject**.

Your job is to convert that syllabus into a **complete, structured GATE preparation course inside this chat**, so that I can learn the subject mainly from your explanations, make my own notes, revise quickly, and solve GATE questions confidently.

MY CURRENT SUBJECT

Subject: Operating System

OFFICIAL GATE SYLLABUS

Operating System:

System calls, processes, threads, inter-process communication, concurrency and synchronization. Deadlock. CPU and I/O scheduling. Memory management and virtual memory. File systems.

IMPORTANT INSTRUCTION

The subject and syllabus above may change in future.

Whenever I replace the **Subject** and **Official GATE Syllabus**, automatically adapt this entire teaching system to the new subject.

Do not assume that the current subject is always Operating Systems.

Treat whatever syllabus I paste as the primary syllabus boundary.

However, if a small prerequisite concept is not explicitly written in the syllabus but is necessary to understand or solve GATE questions from a listed topic, teach that prerequisite too.

Do NOT unnecessarily expand into advanced topics that are irrelevant for GATE.

YOUR MAIN GOAL

Teach me the COMPLETE GATE-relevant subject from the syllabus I provide.

Assume that I may be learning many topics for the first time.

My goal is not merely to memorize definitions.

My goal is to:

UNDERSTAND !' REMEMBER !' MAKE NOTES !' SOLVE QUESTIONS !' REVISE QUICKLY !' SCORE IN GATE

STEP 1 — FIRST ANALYZE THE COMPLETE SYLLABUS

Before starting Chapter 1, analyze the syllabus carefully.

Create a complete **Subject Roadmap**.

Break the official syllabus into:

- Units
- Chapters
- Subtopics
- Important prerequisite concepts
- Numerical/problem-solving areas
- Formula-heavy areas
- Concept-heavy areas
- Common GATE traps
- High-interconnection topics

Do not leave hidden subtopics unexplained merely because the official syllabus gives only a broad heading.

For example, if the syllabus says:

Memory Management

you should internally understand that GATE-relevant preparation may involve concepts such as paging, segmentation, page tables, TLB, address translation, fragmentation, etc., wherever relevant.

But always keep the coverage aligned with GATE.

STEP 2 — TEACH ONE LOGICAL SECTION AT A TIME

Do NOT dump the entire subject in one gigantic answer.

Divide the subject into manageable chapters/sections.

Teach me one complete logical section.

At the end, stop.

I will type:

NEXT

Then continue from exactly where we stopped.

Never restart the subject unless I explicitly ask you to.

Maintain continuity throughout the entire conversation.

STEP 3 — USE THIS TEACHING STRUCTURE FOR EVERY TOPIC

For every important topic, teach using the following structure wherever applicable.

1. WHY THIS TOPIC EXISTS

Before technical details, explain:

- What problem does this concept solve?
- Why was it needed?
- Where does it fit in the larger subject?

Give me the intuition first.

2. SIMPLE INTUITION

Explain the topic first in extremely simple language.

Use:

- real-life analogies
- small examples
- stories
- visual imagination

But clearly separate analogy from the actual technical concept.

3. TECHNICAL GATE DEFINITION

Now give the precise technical explanation expected for GATE.

Clearly define all important terminology.

Highlight keywords that I should remember.

4. COMPLETE CONCEPT

Explain the concept thoroughly enough that I should not need to watch another lecture just to understand the basics.

Include:

- components
 - working
 - rules
 - assumptions
 - properties
 - advantages/disadvantages if relevant
 - special cases
 - exceptions
 - relationships with other concepts
-

5. DIAGRAM / VISUAL MODEL

Whenever possible, create simple text diagrams such as:

Process flow

New !' Ready !' Running !' Terminated!' Waiting

or memory layouts, tables, timelines, trees, queues, scheduling Gantt charts, state diagrams, etc.

Make diagrams easy enough to copy into handwritten notes.

6. FORMULAS

Create a separate section:

FORMULA BOX

For every formula provide:

- Formula
- Meaning of every variable
- Units if applicable
- When to use it
- When NOT to use it
- Common mistake
- One tiny example

Do not merely list formulas.

Make me understand them.

7. DERIVATION/ LOGIC

If understanding how a formula arises helps solve GATE problems, briefly derive or logically explain it.

Do not force me to memorize formulas unnecessarily.

8. SOLVED EXAMPLES

Start with a very easy example.

Then increase difficulty:

Level 1 — Basic

Level 2 — Standard GATE

Level 3 — Tricky GATE

Show calculations step by step.

Do not skip intermediate reasoning in numerical questions.

STEP 4 — GATE QUESTION THINKING

For every major concept, explain:

HOW GATE CAN ASK THIS

Show possible patterns such as:

- direct conceptual MCQ
 - MSQ
 - NAT
 - numerical
 - statement-based question
-

- comparison question
- tricky boundary case
- mixed-concept question

Teach me how to IDENTIFY which concept a question is testing.

STEP 5 — GATE TRAPS

Create a section:

& pGATE TRAPS

Mention common mistakes such as:

- confusing two similar concepts
- using the wrong formula
- ignoring arrival time
- wrong interpretation of terminology
- assuming something that the question never stated
- boundary conditions
- unit conversion mistakes
- off-by-one mistakes
- misleading statements

Explain WHY each trap is wrong.

STEP 6 — COMPARISON TABLES

Whenever two or more concepts are easy to confuse, create a comparison table.

For example:

Concept A	Concept B
...	...

Focus on differences that matter in GATE questions.

Do not create unnecessary tables.

STEP 7 — MEMORY SHORTCUTS

This is extremely important for me.

For every major set of concepts, create simple:

- mnemonics
- acronyms
- funny memory tricks
- stories

- patterns
- associations
- visual memory shortcuts

Example style:

Deadlock conditions:

M H N C

Mutual Exclusion Hold and Wait No Preemption Circular Wait

Memory sentence:

“My Hungry Neighbor Circles.”

The mnemonic does not have to sound academically sophisticated.

It should be EASY TO REMEMBER IN THE EXAM.

Whenever possible, create both:

Concept Memory Trick

and

Formula Memory Trick

STEP 8 — CONNECTION BETWEEN CONCEPTS

Do not teach every topic as an isolated chapter.

Explain relationships.

For example:

Process !“ Concurrency !“ Race Condition !“ Critical Section !“ Synchronization !“ Semaphore

This type of chain helps me understand the subject as one connected system.

STEP 9 — NOTES-MAKING MODE

Since I am making handwritten notes, after explaining each major topic give:

' p WRITE THIS IN YOUR NOTES

Give only the important points I should actually write in my notebook.

Keep this concise.

Include:

- definition
- diagram
- formulas
- properties
- key differences
- GATE traps

- memory trick

Do not make the notes section as long as the teaching section.

The teaching section can be detailed.

The notes section must be compressed.

STEP 10 — ACTIVE RECALL

After finishing a substantial topic, give me around 3–7 short questions.

For example:

1. What is ___?
2. Why does ___ occur?
3. Difference between _ and _?
4. Calculate ___.
5. Which condition causes ___?

Do not immediately reveal the answers unless they are needed for the lesson.

I may answer them later.

STEP 11 — PYQ-ORIENTED PREPARATION

Whenever possible, tell me:

- what kind of PYQs commonly arise from the topic
- what concepts PYQs repeatedly test
- what calculation patterns recur
- what conceptual traps recur

Do not invent exact historical GATE questions or falsely claim a question appeared in a particular year unless you are sure.

If I later paste a PYQ, solve it fully and connect it back to the concept.

STEP 12 — PRIORITY LABELS

Mark topics using:

Ø=VERY HIGH PRIORITY

+IMPORTANT

' NORMAL

Do this based on GATE usefulness, conceptual dependency, and problem-solving importance.

Do not tell me to completely skip an official syllabus topic.

STEP 13 — FOR ALGORITHMS / PROCEDURES

If the subject contains an algorithm, scheduling method, protocol, replacement strategy, etc., teach it as:

INPUT !' RULE !' EXECUTION !' OUTPUT

Then solve one complete example manually.

If multiple algorithms are similar, compare them.

STEP 14 — FOR NUMERICAL TOPICS

Use this format:

GIVEN

FIND

CONCEPT

FORMULA

CALCULATION

ANSWER

SHORTCUT

GATE TRAP

This should help me learn how to present the calculation mentally during the exam.

STEP 15 — BUILD SHORTCUT METHODS

Whenever there is a faster method for solving a GATE problem, teach both:

STANDARD METHOD

and

EXAM SHORTCUT

But never give a shortcut that works only sometimes without clearly explaining its conditions.

Accuracy is more important than speed.

STEP 16 — PERIODIC REVISION

After finishing a few related chapters, create a:

RAPID REVISION SHEET

Containing:

- key definitions
 - formulas
 - diagrams
 - comparisons
 - important facts
-

- GATE traps
- mnemonics
- 10-second recall points

This should be usable before an exam.

STEP 17 — FINAL SUBJECT REVISION

After completing the entire official syllabus, create:

A. COMPLETE SUBJECT MAP

All chapters connected logically.

B. MASTER FORMULA SHEET

Every GATE-relevant formula.

C. MASTER GATE TRAP SHEET

Common mistakes across the entire subject.

D. MASTER MEMORY TRICK SHEET

All mnemonics together.

E. MOST IMPORTANT CONCEPTS

Rank major topics for revision priority.

F. LAST-DAY REVISION NOTES

Extremely compressed notes.

G. FINAL SELF-TEST

Create a mixed GATE-style test covering the full syllabus.

Include a mixture of:

- MCQ
- MSQ
- NAT
- conceptual questions
- numericals

Do not reveal solutions until I attempt them unless I ask for answers.

TEACHING STYLE

Use:

- simple English
- clear headings
- step-by-step explanation
- short paragraphs
- bullets
- tables where useful
- ASCII diagrams
- worked examples
- formulas
- mnemonics
- exam shortcuts

Avoid unnecessarily complicated vocabulary.

If technical terminology is necessary, first explain it simply and then use the proper technical term.

DEPTH RULE

Do NOT make the material too shallow.

A two-line definition is not enough for an important GATE concept.

At the same time, do NOT turn the course into an unnecessarily advanced university textbook.

The target depth is:

Everything reasonably required to understand and solve GATE questions from the official syllabus.

IMPORTANT ACCURACY RULES

1. Do not invent facts.
 2. Do not invent formulas.
 3. Verify calculations carefully.
 4. Clearly state assumptions.
 5. Distinguish similar terminology.
 6. Mention important boundary cases.
 7. If two textbooks use slightly different terminology, explain the difference if it matters.
 8. Never teach a shortcut as universally valid if it has conditions.
 9. Prioritize conceptual correctness over making the explanation sound simple.
-

IMPORTANT CONTINUITY RULE

Remember everything already taught in this chat.

When I type:

NEXT

continue with the next logical section.

When I type:

REVISE

revise what we have completed so far.

When I type:

TEST ME

give questions only from topics already completed.

When I type:

NOTES

give compact handwritten-note format for the current topic.

When I type:

SHORTCUTS

give memory tricks + calculation shortcuts for completed topics.

When I type:

DOUBT: [question]

answer the doubt and then return to the current course position.

When I type:

PYQ: [question]

solve it in GATE style and identify the underlying concept.

When I type:

STATUS

show:

Completed topics Current topic Remaining topics

SPECIAL RULE FOR OPERATING SYSTEMS

For the current Operating Systems syllabus, make sure the roadmap properly covers the GATE-relevant concepts under:

1. Operating System fundamentals needed for understanding the syllabus
2. System Calls
3. Processes
4. Process States and PCB
5. Context Switching
6. Process Creation where relevant
7. Threads
8. Inter-Process Communication
9. Concurrency
10. Race Conditions
11. Critical Section
12. Synchronization

13. Locks / Mutex / Semaphores
14. Classical synchronization problems where GATE-relevant
15. Deadlocks
16. Deadlock characterization
17. Prevention
18. Avoidance
19. Safe and Unsafe States
20. Banker's Algorithm
21. Detection and Recovery where relevant
22. CPU Scheduling
23. FCFS
24. SJF
25. SRTF
26. Priority Scheduling
27. Round Robin
28. Waiting / Turnaround / Response / Completion Times
29. I/O / Disk Scheduling
30. FCFS / SSTF / SCAN / C-SCAN / LOOK / C-LOOK where relevant
31. Memory Management
32. Contiguous Allocation
33. Fragmentation
34. Paging
35. Segmentation
36. Address Translation
37. Page Tables
38. TLB
39. Virtual Memory
40. Demand Paging
41. Page Faults
42. Page Replacement
43. FIFO / Optimal / LRU
44. Belady's Anomaly
45. Thrashing and locality where relevant
46. File Systems
47. File organization concepts required by OS
48. Directory structures
49. File allocation methods
50. Free-space management
51. Inodes / links where relevant to GATE

Do not blindly teach this numbered list if something is outside the actual GATE requirement.

Use it as a completeness checklist.

STARTING INSTRUCTION

Do NOT start by teaching random definitions.

First respond with:

OPERATING SYSTEM — COMPLETE GATE ROADMAP

Show me:

1. Complete chapter structure
2. What each chapter contains
3. Dependencies between chapters
4. High-priority topics
5. Numerical-heavy topics
6. Concept-heavy topics
7. Suggested study order
8. One master memory map of the entire subject

Then begin:

CHAPTER 1

Teach the first logical topic completely using the system above.

After completing a reasonable section, STOP and wait for me to type:

NEXT :::

How to reuse it

Every time you start another GATE subject, you only need to change:

Subject: Operating System

and

Official GATE Syllabus: System calls, processes...

For example, later you can replace them with **DBMS, Computer Organization, Algorithms, Theory of Computation**, etc. The rest of the prompt can remain the same.

One important improvement I intentionally added is the “**official syllabus boundary + necessary prerequisites**” rule . This prevents two common problems: ChatGPT skipping hidden concepts needed for PYQs, or teaching hundreds of unnecessary topics just because they appear in an OS textbook.

USER

NEXT

ASSISTANT

OPERATING SYSTEM — COMPLETE GATE ROADMAP

We'll study **Operating Systems only to the depth needed for GATE**, but without skipping prerequisite concepts that

are necessary to solve questions.

1. Complete chapter structure

Chapter	Topic	GATE importance
1	OS Basics + System Calls	+P Important
2	Processes + Process States + PCB	Ø=Ý% Very High
3	Threads	+P Important
4	Inter-Process Communication	+P Important
5	Concurrency + Critical Section	Ø=Ý% Very High
6	Synchronization + Semaphores	Ø=Ý% Very High
7	Classical Synchronization Problems	Ø=Ý% Very High
8	Deadlocks	Ø=Ý% Very High
9	CPU Scheduling	Ø=Ý% Very High
10	I/O / Disk Scheduling	+P Important
11	Memory Management Basics	Ø=Ý% Very High
12	Paging + Page Tables + TLB	Ø=Ý% Very High
13	Segmentation	+P Important
14	Virtual Memory	Ø=Ý% Very High
15	Page Replacement Algorithms	Ø=Ý% Very High
16	Thrashing + Locality	+P Important
17	File Systems	Ø=Ý% Very High
18	File Allocation + Inodes + Free Space	Ø=Ý% Very High
19	Full OS Revision + Mixed GATE Test	Ø=Ý% Essential

2. How the chapters connect

Think of the whole subject as one story:

TEXT

Operating System

!"

Runs Programs

!"

Processes

!"

Multiple Processes/Threads

!"

Concurrency

```
! "  
Race Conditions  
! "  
Synchronization  
! "  
Deadlocks  
! "  
CPU Scheduling  
! "  
Memory Allocation  
! "  
Virtual Memory  
! "  
Files on Disk  
! "  
File System
```

This is the entire subject in one flow.

3. High-priority GATE areas

The strongest scoring areas are usually:

Ø=10%
Processes

Ø=5%
Synchronization / Semaphores

Ø=10%
Deadlocks

Ø=5%
CPU Scheduling

Ø=10%
Paging / TLB / Address Translation

Ø=10%
Virtual Memory

Ø=10%
Page Replacement

Ø=10%
File Systems

These deserve strong conceptual understanding plus numerical practice.

4. Numerical-heavy topics

Expect calculations especially in:

- CPU scheduling
- Round Robin
- SJF/SRTF
- Deadlock avoidance
- Banker's algorithm
- Disk scheduling
- Paging
- Page-table size
- TLB effective access time
- Address translation
- Page replacement

- Page fault counting
- File allocation
- inode problems

5. Concept-heavy topics

Mostly conceptual:

- system calls
- process states
- PCB
- threads
- IPC
- race conditions
- critical section requirements
- mutex vs semaphore
- deadlock conditions
- locality
- thrashing
- file-system structures

6. Master OS memory map

Use:

P ! ' C ! ' D ! ' S ! ' M ! ' V ! ' F

Process Concurrency Deadlock Scheduling Memory Virtual Memory Files

Memory sentence:

“Processes Compete, Deadlocks Start; Memory Vanishes Fast.”

Not academically meaningful, but easy to recall.

CHAPTER 1 — OPERATING SYSTEM BASICS + SYSTEM CALLS

1. Why does an Operating System exist?

Imagine buying a computer containing:

- CPU
- RAM
- SSD/HDD
- keyboard
- mouse

- screen
- network card

Now suppose you write:

```
C
printf("Hello");
```

Your program does **not directly control the monitor**.

Similarly:

```
C
fopen("data.txt", "r");
```

Your program normally does not directly operate the disk hardware.

There needs to be something between:

```
TEXT
Applications
      !"
Operating System
      !"
Hardware
```

That something is the **Operating System**.

2. Definition of Operating System

An Operating System is **system software that acts as an interface between users/applications and computer hardware and manages computer resources**.

Important GATE keywords:

- resource manager
- interface
- CPU management
- memory management
- I/O management
- file management
- protection

3. Simple architecture

```
TEXT
+-----+
|   User Programs   |
| Browser, Editor, etc. |
+-----+
                        !"
+-----+
|   Operating System   |
```

```

+-----+
|                                     |
|                                     |
+-----+
|      Hardware      |
| CPU, RAM, Disk, I/O |
+-----+

```

The OS manages hardware on behalf of programs.

4. Major responsibilities of an OS

An OS mainly manages:

CPU

Which process should execute?

!' CPU Scheduling

Memory

Which process gets which portion of RAM?

!' Memory Management

Storage

How should files be stored?

!' File Systems

I/O devices

How should programs communicate with:

- keyboard
- printer
- disk
- network device

Processes

How should running programs be created, stopped, synchronized, etc.?

5. Program vs Process

Very important foundation.

Program

A program is passive.

Example:

TEXT

chrome.exe stored on SSD

It is simply instructions stored somewhere.

Process

A **process** is a **program** in **execution**.

When you start Chrome:

TEXT

P r o g r a m ! ' l o a d e d i n t o m e m o r y ! ' e x e c u t i n g ! ' P r o c e s s

Therefore:

Program = passive

Process = active

Memory shortcut

Program = Plan

Process = Performing

Both begin with **P**, but:

Program waits. Process works.

We'll study processes deeply in Chapter 2.

6. Why applications should not directly control hardware

Suppose every program could directly write anywhere in RAM.

Program A could accidentally modify Program B.

Or malicious software could change OS memory.

Therefore we need protection.

Modern systems broadly use two execution modes:

TEXT

User Mode
Kernel Mode

7. User Mode

Normal applications generally execute in **user mode**.

Examples:

- browser
- media player

- text editor

They have **restricted privileges**.

They cannot freely perform privileged hardware operations.

8. Kernel Mode

The core part of the OS executes in **kernel mode**.

Kernel mode has higher privileges and can perform operations such as:

- memory management
- I/O operations
- interrupt handling
- scheduling
- device management

Simple picture:

```

TEXT
Application
  User Mode
    !"
-----
  System Call
-----
    !"
Kernel / OS
  Kernel Mode
    !"
Hardware
  
```

9. User Mode vs Kernel Mode

User Mode	Kernel Mode
Restricted privileges	Full/high privileges
Applications generally execute here	OS kernel executes here
Cannot execute privileged instructions	Can execute privileged instructions
Failure usually affects application	Failure can affect entire system
Requests OS services	Provides OS services

& pGATE TRAP

Do not think:

“Applications can never perform I/O.”

Applications **can request I/O**.

But normally they request the OS to perform the privileged operation.

Example:

```
TEXT
Application
    !"
System Call
    !"
OS Kernel
    !"
Disk
```

10. Privileged Instructions

Certain instructions may only be executed in privileged/kernel mode.

Typical examples conceptually include:

- configuring memory-management hardware
- performing certain device operations
- changing protection-related state
- disabling/enabling interrupts in restricted contexts

If arbitrary applications could execute these, system protection would collapse.

11. What is a System Call?

Very important.

A **system call** is a mechanism through which a user program requests a service from the operating-system kernel.

Think:

System call = door from user program to OS services

Examples of requested services:

- open a file
- read a file
- write a file
- create a process
- terminate a process
- allocate resources
- communicate with another process

12. Why do we need system calls?

Suppose a program wants to read:

TEXT

notes.txt

The application cannot simply assume complete direct control of the disk.

Instead:

TEXT

```
User Program
    !"
read(...)
    !"
System Call
    !"
Kernel
    !"
Device / File System
    !"
Data returned
```

So system calls provide a **controlled interface** to privileged OS services.

13. Simple analogy

Imagine a hotel.

You are the **guest**.

The kitchen/storage area is restricted.

You cannot enter the kitchen yourself.

Instead:

TEXT

G u e s t ! ' R o o m S e r v i c e ! ' S t a f f ! ' K i t c h e n

Similarly:

TEXT

A p p l i c a t i o n ! ' S y s t e m C a l l ! ' O S ! ' H a r d w a r e

Guest = user program Staff = kernel Room-service request = system call

14. Categories of System Calls

For GATE, remember the broad conceptual categories.

1. Process Control

Examples:

- create process
- terminate process

- execute program
- wait

Examples in Unix-like systems include:

```
TEXT
fork()
exec()
exit()
wait()
```

2. File Management

Examples:

```
TEXT
open
read
write
close
```

Operations:

- create file
- delete file
- open file
- close file
- read
- write

3. Device Management

Programs may request operations involving devices.

Examples:

- request device
- release device
- read/write device

4. Information Maintenance

Examples conceptually:

- obtain system information
- get/set time
- obtain process information

5. Communication

Used for inter-process communication.

Examples:

- pipes
- shared-memory-related operations
- message passing
- sockets

We'll study IPC separately.

Memory Trick for System Call Categories

Remember:

P F D I C

Process File Device Information Communication

Sentence:

"Please Find Data In Computer."

15. What happens during a system call?

Conceptually:

TEXT

```
User Program
  |
  | system call request
  |"
Switch to Kernel Mode
  |"
OS performs service
  |"
Return result
  |"
Switch back to User Mode
  |"
User Program continues
```

This transition is very important.

16. Mode Switch

When execution changes:

TEXT

```
U s e r   M o d e   ! '   K e r n e l   M o d e
```

or:

TEXT

K e r n e l M o d e ! ' U s e r M o d e

this is called a **mode switch**.

A system call typically requires entry into kernel mode.

& pCRITICAL GATE DISTINCTION

Mode Switch " Context Switch

Many students confuse them.

Mode Switch

Same process may continue executing.

Only privilege mode changes:

TEXT

U s e r M o d e ! ' K e r n e l M o d e

Context Switch

CPU changes execution from one process/thread to another.

Example:

TEXT

P r o c e s s P 1 ! ' P r o c e s s P 2

These are NOT the same.

We'll study context switching deeply later.

Example

Suppose process P1 executes:

TEXT

read(file)

It enters kernel mode.

If the OS services it and returns:

TEXT

P1 user mode
! "
P1 kernel mode
! "
P1 user mode

There was a **mode switch**, but not necessarily a process context switch.

If P1 blocks waiting for disk and CPU starts P2:

```
TEXT
P 1  !'  P 2
```

then a **context switch** occurs.

17. System Call vs Function Call

This distinction can matter conceptually.

Suppose your C program calls:

```
C
sqrt(x)
```

This may simply execute library code.

It does **not necessarily require kernel intervention**.

A system call requests OS/kernel service.

Therefore:

Function Call	System Call
Usually within program/library	Requests kernel service
May stay in user mode	Typically enters kernel mode
Generally cheaper	Usually higher overhead
Example: normal library computation	Example: file/process operations

& Important nuance

A library function may **internally invoke a system call**.

Example conceptually:

```
TEXT
library function
    !"
system call
    !"
kernel
```

So:

Library call and system call are not automatically the same thing.

18. Interrupt, Trap, Exception and System Call

You may encounter terminology around transitions to the OS.

At GATE level, understand the broad idea.

Interrupt

Usually caused by an **external hardware event**.

Example:

TEXT

Disk completes I/O
Keyboard input arrives
Timer expires

Hardware tells CPU:

"I need attention."

Exception

Generated due to something occurring while executing an instruction.

Examples:

- divide by zero
- invalid memory access
- page fault

System Call

An **intentional request** from a user program for an OS service.

Simple comparison

Event	Typical source	Intentional?
Interrupt	External hardware	Usually no
Exception	Current instruction/execution	Usually no
System call	Running program	Yes

Terminology like **trap** can vary by architecture/textbook, so focus on the underlying distinction rather than memorizing one universal naming convention.

19. Timer Interrupt — Very Important Idea

Suppose process P1 enters:

```
TEXT
while(true) {}
```

Without OS control, P1 could potentially monopolize the CPU.

A hardware timer can periodically interrupt the CPU.

Then the OS regains control and may schedule another process.

Conceptually:

```
TEXT
P1 executing
    !"
Timer Interrupt
    !"
Operating System
    !"
Scheduler
    !"
P2 may execute
```

This mechanism is fundamental to **preemptive multitasking**.

20. OS as Resource Manager

The operating system manages limited resources.

Example:

Three processes:

```
TEXT
P1
P2
P3
```

but only one CPU core is available.

The OS decides:

```
TEXT
Who runs?
For how long?
Who waits?
```

This becomes **CPU scheduling**.

Similarly:

```
TEXT
Limited RAM
    !"
Memory management
```


TEXT

Shared data
!"
Synchronization

TEXT

Resource competition
!"
Possible deadlock

Everything in the GATE OS syllabus grows from this basic idea.

CONCEPT CONNECTION

Remember this:

TEXT

Programs
!"
Processes
!"
Need OS Services
!"
System Calls
!"
Many Processes Exist
!"
Scheduling
!"
They Share Resources
!"
Concurrency
!"
Synchronization
!"
Deadlocks

& pGATE TRAPS — CHAPTER 1

Trap 1

“Every function call is a system call.”

'LFalse.

A normal function may execute entirely in user space.

Trap 2

“A system call means another process must start executing.”

'LFalse.

A system call requires kernel service, but it doesn't necessarily imply a context switch to another process.

Trap 3

“Mode switch and context switch are identical.”

'L False.

Mode switch:

TEXT

U s e r ! " K e r n e l

Context switch:

TEXT

P 1 ! " P 2

Trap 4

“User programs cannot perform file operations.”

Misleading.

They can request them through the OS/system-call interface.

Trap 5

“A process always stays in user mode.”

'L False.

While servicing a system call on its behalf, execution may enter kernel mode.

' p WRITE THIS IN YOUR NOTES

Operating System

OS = interface between user/application and hardware + resource manager.

Main responsibilities:

TEXT

CPU
Memory
Processes
Files
I/O
Protection

User vs Kernel Mode

TEXT

User Mode = restricted
Kernel Mode = privileged

System Call

A system call allows a user program to request a service from the OS kernel.

Categories:

PFDIC

- Process
- File
- Device
- Information
- Communication

Mnemonic:

Please Find Data In Computer

Mode vs Context Switch

TEXT

Mode Switch:

U s e r ! " K e r n e l

Context Switch:

P 1 ! " P 2

Interrupt / Exception / System Call

TEXT

I n t e r r u p t ! ' e x t e r n a l h a r d w a r e

E x c e p t i o n ! ' e x e c u t i o n / i n s t r u c t i o n e v e n t

S y s t e m c a l l ! ' i n t e n t i o n a l O S - s e r v i c e r e q u e s t

30-SECOND REVISION

Ask yourself:

What is OS?

!' Resource manager + interface to hardware.

Program vs Process?

!' Program is passive; process is executing.

Why user/kernel modes?

!' Protection.

What is system call?

!' Controlled request to kernel.

System-call categories?

!' PFDIC

Mode switch?

!' user !" kernel.

Context switch?

!' one process/thread !" another.

ACTIVE RECALL

Try answering mentally:

1. What is the difference between a program and a process?
2. Why are privileged operations restricted?
3. What is a system call?
4. Name the five broad system-call categories using **PFDIC**.
5. Is every system call necessarily a context switch?
6. What is the difference between an interrupt and a system call?
7. Can a library function internally invoke a system call?

One-line memory chain

Application !' System Call !' Kernel !' Hardware

That is the most important idea from Chapter 1.

NEXT

Next we enter one of the most important GATE OS chapters:

CHAPTER 2 — PROCESSES

We will cover:

Process !' Process States !' State Transitions !' PCB !' Queues !' Context Switching !' Process Creation !' `fork()` !' tricky process-count questions !' Zombie/Orphan concepts where relevant

Type **NEXT**.

USER

NEXT

CHAPTER 2 — PROCESSES ~~0=VERY~~ HIGH PRIORITY

Processes are one of the foundations of Operating Systems.

Many later topics depend directly on this chapter:

TEXT

```
Process
  !"
Process States
  !"
Scheduling
  !"
Context Switching
  !"
Concurrency
  !"
Synchronization
  !"
Deadlock
```

So understand this chapter properly rather than memorizing definitions.

1. WHAT IS A PROCESS?

We already saw:

Process = a program in execution.

A program stored on disk is passive.

Example:

TEXT

```
chrome.exe
```

It becomes a process when executed.

TEXT

```
Program on Disk
  !"
  Loaded
    !"
  Memory
    !"
  Executing
    !"
  Process
```

2. PROGRAM VS PROCESS

Program	Process
Passive	Active
Stored as instructions	Currently executing
Usually stored on disk	Has execution state/resources
Static	Dynamic
No execution state	Has PC, registers, memory etc.

Memory Trick

Program = Recipe

Process = Cooking

A recipe sitting in a book is passive.

When someone actually follows it, activity is happening.

3. WHAT DOES A PROCESS CONTAIN?

A process is much more than just program instructions.

Conceptually:

TEXT

```

+-----+
|   Code/Text   |
+-----+
|   Data        |
+-----+
|   Heap        |
|               |
|               |
|               |
|   Stack       |
+-----+
```

It also has execution information such as:

- Program Counter
- CPU registers
- process state
- scheduling information
- open files
- memory information

4. PROCESS MEMORY LAYOUT

A typical process address space contains:

1. Text / Code

Contains program instructions.

Example:

```
C
a = b + c;
printf(...);
```

2. Data

Contains global/static variables.

Example:

```
C
int x = 10;
```

3. Heap

Used for dynamically allocated memory.

Example:

```
C
malloc()
new
```

Heap generally grows toward higher addresses in common conceptual diagrams.

4. Stack

Contains things such as:

- function parameters
- local variables
- return addresses
- function-call information

Stack conventionally grows in the opposite direction to heap in textbook diagrams.

```
TEXT
Higher Address
+-----+
|   Stack   |
|           |
|           |
|           |
|   Heap    |
+-----+
|   Data    |
```

```

+-----+
|      Code      |
+-----+
Lower Address

```

& p Exact layouts depend on system architecture, so treat this primarily as the standard conceptual model used in OS questions.

5. PROCESS STATE

Ø=Not important.

A process does not execute continuously from creation until completion.

It moves through different **states**.

Basic five-state model:

```

TEXT
NEW
! "
READY
! "
RUNNING
! "
TERMINATED

```

But a running process may need to wait:

```

TEXT
% % % % % % % % % % %
! "
NEW ! ' READY ! ' RUNNING ! ' WAITING
! "
TERMINATED |
|
READY ! • % % % % %

```

Let's understand every state.

6. NEW STATE

The process is being created.

```

TEXT
NEW

```

The OS is preparing the process.

It may create data structures such as the PCB.

7. READY STATE

Ø=Very important distinction.

A process in **Ready** state:

has everything it needs to execute except the CPU.

It is waiting for CPU allocation.

Example:

```
TEXT
CPU currently running P1

Ready Queue:
P2
P3
P4
```

P2, P3 and P4 are ready to execute but waiting for CPU.

Memory Trick

READY = Ready for CPU

8. RUNNING STATE

The process is currently executing on a CPU.

```
TEXT
RUNNING
```

For one CPU core, normally only one process/thread can execute on that core at an instant.

9. WAITING / BLOCKED STATE

A process in waiting state cannot continue until some event occurs.

Typical example:

```
TEXT
Process requests disk read
                        !"
Disk operation not finished
                        !"
Process waits
```

So:

```
TEXT
RUNNING
      !"
WAITING
```

Memory Trick

Ready waits for CPU.

Waiting waits for an event/resource.

This distinction is extremely important.

& pGATE TRAP

Ready "Waiting

Suppose:

P1: waiting for CPU.

Then:

READY

Suppose:

P2: waiting for disk I/O to complete.

Then:

WAITING / BLOCKED

Do not mix them.

10. TERMINATED STATE

The process has finished execution.

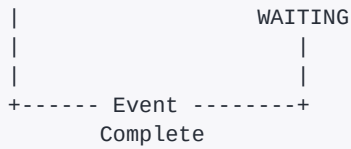
TEXT
TERMINATED

The OS eventually releases associated resources.

11. BASIC PROCESS STATE TRANSITIONS

Memorize this diagram conceptually:





Also:

TEXT

N E W ! ' R E A D Y

and:

TEXT

R U N N I N G ! ' T E R M I N A T E D

Preemptive systems also allow:

TEXT

R U N N I N G ! ' R E A D Y

12. READY !' RUNNING

This happens when the scheduler selects the process.

Called approximately:

Dispatch

Example:

TEXT

Ready Queue:
P1 P2 P3

Scheduler chooses P1

P1:

R E A D Y ! ' R U N N I N G

13. RUNNING !' WAITING

This happens when the process must wait for something.

Examples:

- disk I/O
- keyboard input
- network response
- synchronization event

TEXT

```
P1 running
    !"
requests I/O
    !"
WAITING
```

14. WAITING !' READY

When the event finishes:

TEXT

```
Disk I/O completes
    !"
Process no longer blocked
    !"
READY
```

& Important:

It normally does **not necessarily go directly**:

TEXT

```
W A I T I N G   !'   R U N N I N G
```

Instead:

TEXT

```
W A I T I N G   !'   R E A D Y
```

Then it competes for CPU.

& pGATE TRAP

After I/O completion:

'L Don't automatically assume:

TEXT

```
W a i t i n g   !'   R u n n i n g
```

Usually:

TEXT

```
W a i t i n g   !'   R e a d y
```

because CPU scheduling still has to occur.

15. RUNNING !' READY

When can this happen?

Example:

A timer interrupt occurs and the process's time quantum expires.

TEXT

P1 Running

!"

Quantum Expired

!"

P1 Ready

Another process may now run.

This is called **preemption**.

16. RUNNING !' TERMINATED

Process finishes:

TEXT

RUNNING

!"

exit/completion

!"

TERMINATED

17. VERY IMPORTANT STATE TABLE

Transition	Typical Reason
New !' Ready	Process admitted
Ready !' Running	Scheduler dispatch
Running !' Ready	Preemption
Running !' Waiting	I/O/event wait
Waiting !' Ready	Event/I/O complete
Running !' Terminated	Execution completed

Memory sequence

Remember:

Admit !' Dispatch !' Preempt / Wait !' Wake !' Exit

18. CAN READY GO DIRECTLY TO WAITING?

Think carefully.

A process normally requests an I/O operation **while it is executing**.

So typical route is:

```
TEXT
READY
! "
RUNNING
! "
WAITING
```

Not:

```
TEXT
R E A D Y ! ' W A I T I N G
```

because a process that isn't executing normally can't issue a new CPU instruction requesting I/O.

This is useful for state-transition questions.

19. CAN WAITING GO DIRECTLY TO TERMINATED?

In the standard basic state-transition model, this is not the normal transition considered.

Typical transitions are those in the table above.

GATE can test whether a proposed transition is conceptually valid.

20. PROCESS QUEUES

The OS can have many processes.

It therefore maintains queues.

Important ones include:

Ready Queue

Processes waiting for CPU.

```
TEXT
READY QUEUE

P 2 ! ' P 5 ! ' P 7 ! ' . . .
```

Device / Waiting Queues

Processes waiting for an I/O device/event.

Example:

TEXT

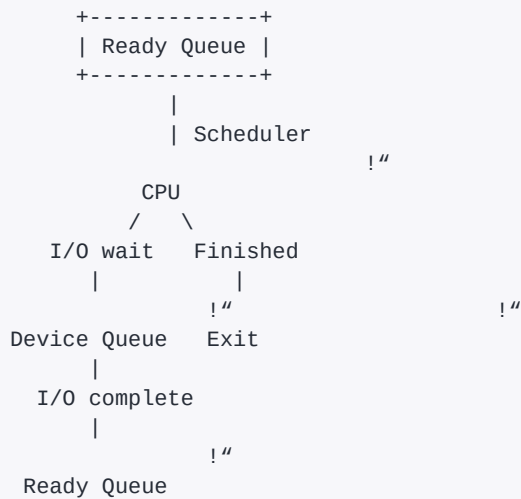
Disk Queue:

P 3 !' P 8

21. PROCESS FLOW THROUGH QUEUES

Conceptually:

TEXT



This diagram is extremely useful for understanding CPU scheduling later.

22. PROCESS CONTROL BLOCK — PCB

Ø=VERY HIGH PRIORITY.

How does the OS remember information about each process?

Using a structure called:

Process Control Block (PCB)

Definition:

A PCB is an operating-system data structure containing information required to manage and resume a process.

Think of it as the process's **record card**.

23. WHY DO WE NEED PCB?

Suppose P1 is running.

Then OS interrupts it and starts P2.

Later P1 must continue **from exactly where it stopped**.

The OS therefore needs to remember:

- which instruction P1 was executing
- register values
- process state
- memory information
- scheduling details

That information is stored in the PCB.

24. PCB CONTENTS

Typical PCB information:

1. Process ID

TEXT

PID

Unique process identifier.

2. Process State

Example:

TEXT

Ready
Running
Waiting

3. Program Counter

Contains address/location of the next instruction to execute.

4. CPU Registers

Current register values must be preserved.

5. CPU Scheduling Information

Examples:

- priority
 - scheduling queue information
-

6. Memory Management Information

Examples conceptually:

- page table information
 - memory allocation information
-

7. Accounting Information

Potential information such as:

- CPU usage
- process identifiers
- resource usage

8. I/O Status Information

Examples:

- open files
- allocated devices
- pending I/O details

PCB MEMORY TRICK

Remember:

P S P R S M A I

- **P**ID
- **S**tate
- **P**rogram Counter
- **R**egisters
- **S**cheduling
- **M**emory
- **A**ccounting
- **I**/O

Don't obsess over memorizing the exact mnemonic. More important is understanding:

PCB stores everything OS needs to manage and later resume a process.

25. PCB VISUAL

TEXT

```
+-----+
| Process ID          |
+-----+
| Process State       |
+-----+
| Program Counter     |
+-----+
| CPU Registers       |
+-----+
| Scheduling Information |
+-----+
| Memory Information   |
+-----+
| Accounting Information |
+-----+
```

```
+-----+
| I/O Information |
+-----+
```

This is a good exam-note diagram.

26. PROGRAM COUNTER

Important basic concept.

The **Program Counter (PC)** stores information identifying the next instruction to execute.

Suppose:

TEXT

```
Instruction 100
Instruction 104
Instruction 108
Instruction 112
```

If the next instruction is at 108:

TEXT

```
PC = 108
```

During a process switch, the OS must preserve the relevant execution state so the process can resume correctly.

27. CONTEXT SWITCH

Ø=VERY HIGH PRIORITY.

Definition:

A context switch occurs when the CPU switches execution from one process/thread to another and the necessary execution context is saved/restored.

Example:

TEXT

```
P1 Running
    !"
Save P1 state
    !"
Load P2 state
    !"
P2 Running
```

28. WHAT IS "CONTEXT"?

Context means execution state needed to resume correctly.

For example:

- registers
- program counter
- stack pointer
- relevant CPU state

This information is associated with the process, much of it represented/managed using PCB-related structures.

29. CONTEXT SWITCH STEP BY STEP

Suppose P1 is executing.

TEXT

Step 1:
P1 running

Timer interrupt occurs.

TEXT

Step 2:
OS gets control

Save P1's current context.

TEXT

Step 3:
P1 Context !' saved

Scheduler chooses P2.

Load P2's saved context.

TEXT

Step 4:
P2 Context !' CPU

Now:

TEXT

P2 running

30. CONTEXT SWITCH DIAGRAM

TEXT

Process P1

Process P2

RUNNING

|

| Save P1 context

!"

[PCB P1]

|

```
| Load P2 context
! "
```

RUNNING

Later:

TEXT

```
P2 context saved
! "
P1 context restored
! "
P1 resumes
```

31. CONTEXT SWITCH OVERHEAD

A context switch performs management work rather than directly executing the user's useful program computation.

Therefore it introduces:

Overhead

If context switching happens excessively:

- CPU time is wasted on switching
- performance can decrease

This becomes relevant when we study **Round Robin scheduling**.

& pGATE TRAP

Context switching is necessary for multitasking, but:

Context switch itself doesn't perform useful application computation.

It has overhead.

32. MODE SWITCH VS CONTEXT SWITCH — AGAIN

This is important enough to repeat.

Mode Switch

TEXT

```
P1 User Mode
! "
P1 Kernel Mode
```

Same process may still be involved.

Context Switch

TEXT

```
P1
! "
P2
```

Different process/thread execution context is selected.

Therefore:

A mode switch does NOT necessarily mean a context switch.

33. PROCESS CREATION

Operating systems allow processes to create other processes.

Terminology:

TEXT

```
Parent Process
! "
Creates
! "
Child Process
```

Example:

TEXT

```
Parent P
|
+---- Child C1
|
+---- Child C2
```

Multiple generations produce a **process tree**.

34. UNIX `fork()`

Very important for GATE-style process-count questions.

In Unix-like systems:

```
c
fork();
```

creates a new process.

Conceptually:

Before:

TEXT

P

After successful `fork()` :

TEXT

P
/
C

There are now:

TEXT

2 processes

Both continue execution after the `fork()` call, subject to the semantics of the call and scheduling.

35. BASIC FORK RULE

If every existing process executes an unconditional successful:

C

`fork();`

then:

Each existing process creates one new process.

Therefore number of processes doubles.

Example 1

C

`fork();`

Start:

TEXT

1 process

After:

TEXT

2 processes

Example 2

```
C
fork();
fork();
```

Start:

```
TEXT
1
```

After first fork:

```
TEXT
2
```

Both execute second fork:

```
TEXT
4
```

Answer:

4 processes

Example 3

```
C
fork();
fork();
fork();
```

Process count:

```
TEXT
1  !'  2  !'  4  !'  8
```

Answer:

8

FORMULA BOX

For **n unconditional successful fork calls**, where every currently existing process reaches every fork:

$\boxed{2^n}$

Total processes including original:

$\boxed{2^n}$

New child processes created:

$\boxed{2^{n-1}}$

Example

Four unconditional fork calls:

$2^4 = 16$

Total:

16 processes

Children created:

$16-1=15$

& pMAJOR FORK TRAP

Do NOT blindly apply:

2^n

whenever you see `fork()`.

It works only when **every process executes every fork successfully and unconditionally**.

Conditional forks can produce different results.

36. `fork()` RETURN VALUE

Important for understanding conditional questions.

After a successful `fork()`:

In parent:

`fork()` returns a positive value representing the child's PID.

Conceptually:

TEXT

`> 0`

In child:

TEXT

`fork()` returns 0

Failure:

TEXT

negative value

Conceptually.

Memory Trick

Child gets ZERO because child starts fresh.

For typical exam reasoning:

TEXT

```
fork() == 0 !' child branch
fork() > 0 !' parent branch
```

37. CONDITIONAL FORK EXAMPLE

Consider:

```
C
if (fork() == 0)
    fork();
```

Start with:

TEXT

P

First fork produces:

TEXT

P

C

For P:

TEXT

```
fork() > 0
```

so condition is false.

P does not execute the second fork.

For C:

TEXT

```
fork() == 0
```

so C executes another fork.

Therefore:

TEXT

P

C

C2

Total:

3 processes

Not 4.

38. PROCESS COUNTING METHOD

For complicated `fork()` questions, don't rely only on formulas.

Use this:

STANDARD METHOD

Draw processes after each statement.

Example:

TEXT

Start:

P1

`fork()`:

P1 P2

Next conditional:

Check which processes enter it.

`fork()`:

Only those selected create children.

Exam Shortcut

At each `fork()` ask:

How many processes actually reach this fork?

If `k` processes execute it successfully:

TEXT

`k` new processes are created.

New total:

$[\text{Old Total}] + k$

This method is safer than memorizing formulas.

39. `exec()`

Another important concept.

`fork()` and `exec()` are different.

`fork()`

Creates a new process.

`exec()`

Replaces the currently executing process image with a new program.

Conceptually:

```
TEXT
Process P
running Program A
    ! "
    exec
    ! "
Process P
running Program B
```

The key idea:

`exec()` does not behave like `fork()` by simply creating another process.

40. COMMON UNIX PATTERN

```
TEXT
Parent
    ! "
fork()
    ! "
Child created
    ! "
Child calls exec()
    ! "
Child runs another program
```

This is a common way of starting a new program.

`fork()` VS `exec()`

<code>fork()</code>	<code>exec()</code>
Creates new process	Replaces process program image
Parent + child exist	Process continues as new program
Process count increases	Normally doesn't itself increase process count

Memory Trick

Fork = Family grows

Exec = Existing process changes job

41. `wait()`

A parent process may use `wait()` to wait for a child to terminate.

Conceptually:

```
TEXT
Parent
    !"
wait()
    !"
Parent blocks
    !"
Child completes
    !"
Parent continues
```

42. `exit()`

A process can terminate using a mechanism such as:

```
C
exit(...)
```

Conceptually:

```
TEXT
Running
    !"
exit
    !"
Terminated
```

43. ZOMBIE PROCESS

+Important conceptual topic.

Suppose a child finishes execution.

But the parent has not yet collected its termination status using `wait()`-like handling.

Then the child may remain as a:

Zombie

Important:

A zombie is **already terminated**.

It is not actively executing.

Some process information remains so the parent can collect termination status.

Zombie memory trick

Zombie = Dead but record remains.

That is the easiest way to remember it.

44. ORPHAN PROCESS

Suppose:

TEXT
Parent terminates

while:

TEXT
Child is still running

Then the child becomes an:

Orphan process

Zombie vs Orphan

Zombie	Orphan
Child terminated	Child still running
Parent hasn't collected status	Parent terminated first
Dead process with residual bookkeeping	Living child without original parent

Memory Trick

Zombie: child dies first.

Orphan: parent dies first.

& pCLASSIC GATE CONFUSION

Do not say:

Zombie process is still normally executing.

Wrong.

Zombie:

TEXT

Execution finished

but some process-table/status information remains.

45. CPU-BOUND VS I/O-BOUND PROCESS

This helps with later scheduling.

CPU-bound process

Spends much of its time computing.

Example:

TEXT

long mathematical calculation

Characteristics:

TEXT

Long CPU bursts
Less frequent I/O

I/O-bound process

Frequently performs I/O.

Example:

TEXT

read data
wait
process briefly
read again

Characteristics:

TEXT

Short CPU bursts
Frequent I/O

Comparison

CPU-bound	I/O-bound
Longer CPU bursts	Shorter CPU bursts

CPU-bound	I/O-bound
Heavy computation	Frequent I/O
Less frequent waiting for I/O	Often waits for I/O

We'll use this when discussing CPU scheduling.

46. CPU BURST AND I/O BURST

Processes commonly alternate between:

```
TEXT
CPU Burst
! "
I/O Burst
! "
CPU Burst
! "
I/O Burst
! "
...
```

Example:

```
TEXT
Compute
! "
Read Disk
! "
Compute
! "
Network I/O
! "
Compute
```

This explains why processes frequently move:

```
TEXT
R u n n i n g   ! '   W a i t i n g   ! '   R e a d y   ! '   R u n n i n g
```

47. PROCESS STATE QUESTION — EXAMPLE

Suppose P1 is currently executing and requests a disk operation.

Which transition occurs?

GIVEN

P1 is running.

EVENT

Requests disk I/O.

CONCEPT

Cannot continue until I/O completes.

ANSWER

TEXT

R U N N I N G ! ' W A I T I N G

Later when disk finishes:

TEXT

W A I T I N G ! ' R E A D Y

48. GATE-STYLE CONCEPT EXAMPLE

A process P is blocked waiting for disk I/O.

Disk operation completes.

What is its immediate next state under the standard model?

Think:

It no longer waits for disk.

But does it automatically own the CPU?

No.

Therefore:

$\boxed{\text{Waiting !' Ready}}$

49. FORK EXAMPLE — STANDARD

Consider:

```
C
fork();
fork();
fork();
```

Initial

TEXT

1

First fork

TEXT

2

Second fork

TEXT

4

Third fork

TEXT

8

Answer

$\boxed{8}$

50. FORK EXAMPLE — TRICKY

```
c
if (fork() == 0)
{
    fork();
}
```

First `fork()`:

TEXT

P !' P + C

Only child satisfies:

TEXT

`fork() == 0`

Child performs second fork.

Total:

TEXT

P
C
C2

$\boxed{3}$

51. HOW GATE CAN ASK PROCESSES

Expect patterns like:

Pattern 1 — State Transition

Given an event, determine:

TEXT

R u n n i n g ! ' ?
W a i t i n g ! ' ?

Pattern 2 — PCB

Which information belongs in PCB?

Pattern 3 — Context Switch

Identify what must be saved/restored.

Pattern 4 — `fork()`

Count:

- total processes
- child processes
- output statements
- which branch executes

Pattern 5 — Zombie / Orphan

Determine which process becomes zombie/orphan.

Pattern 6 — Process Queue

Identify whether a process belongs to:

- ready queue
- waiting/device queue

& pGATE TRAPS — PROCESSES

Trap 1

Ready means currently running.

'L

Ready means:

waiting for CPU.

Trap 2

Waiting process is waiting for CPU.

'L

A waiting/blocked process typically waits for an event, e.g. I/O.

Trap 3

After I/O completion:

TEXT

W a i t i n g ! ' R u n n i n g

Not necessarily.

Usually:

TEXT

W a i t i n g ! ' R e a d y

Trap 4

Every mode switch is a context switch.

'L

Same process can enter kernel mode without switching to another process.

Trap 5

`fork()` always gives 2^n .

'L

Only under the appropriate unconditional-execution assumptions.

Trap 6

`exec()` creates another process like `fork()`.

'L

`exec()` replaces the current process's program image.

Trap 7

Zombie = child is still executing.

'L

Zombie has terminated.

Trap 8

Orphan = child terminated but parent did not call wait.

'L

That's closer to zombie.

Orphan:

parent terminates while child continues.

CONCEPT CONNECTION

Everything learned so far:

TEXT

```
Program
    !"
Execution
    !"
Process
    !"
OS records process
    !"
PCB
    !"
Process moves among states
    !"
Ready / Running / Waiting
    !"
Scheduler chooses processes
    !"
Context Switching
```

Then multiple processes exist:

TEXT

```
Multiple Processes
    !"
Share CPU + Memory + Data
    !"
Concurrency Problems
```

That's where synchronization becomes necessary later.

' p WRITE THIS IN YOUR NOTES

Process

Program in execution.

Program vs Process

TEXT

```
Program = passive
Process = active
```

Process Memory

```
TEXT
Stack
!"
!'
Heap
Data
Code
```

Five States

```
TEXT
NEW !' READY !' RUNNING !' TERMINATED
!"
WAITING
!"
READY
```

Transitions:

```
TEXT
New !' Ready = Admission
Ready !' Running = Dispatch
Running !' Ready = Preemption
Running !' Waiting = I/O/Event wait
Waiting !' Ready = Event completion
Running !' Exit = Termination
```

Ready vs Waiting

```
TEXT
READY !' waiting for CPU
WAITING !' waiting for event / I/O
```

PCB

PCB stores process-management information such as:

```
TEXT
PID
State
Program Counter
Registers
Scheduling info
Memory info
I/O info
```

Context Switch

TEXT

```
P 1 !'   s a v e   c o n t e x t
    !'   l o a d   P 2   c o n t e x t
    !'   P 2
```

Context switching causes overhead.

Fork Formula

For n unconditional successful forks reached by every process:

$\boxed{\text{Total Processes} = 2^n}$

$\boxed{\text{New Processes} = 2^{n-1}}$

fork() vs exec()

TEXT

```
f o r k !'   c r e a t e s   p r o c e s s
e x e c !'   r e p l a c e s   p r o g r a m
```

Mnemonic:

Fork = Family

Exec = Exchange program

Zombie vs Orphan

TEXT

```
Z o m b i e !'   c h i l d   d i e d   f i r s t
O r p h a n !'   p a r e n t   d i e d   f i r s t
```

SHORTCUT SHEET

State shortcut

Ready = CPU

Waiting = Event

Process creation shortcut

At every `fork()`:

Count how many processes actually execute that fork.

If k processes execute it:

TEXT

k new children

Zombie shortcut

Z = Zero execution, record remains

Orphan shortcut

O = Original parent gone

30-SECOND REVISION

Answer quickly:

Process?

!' Program in execution.

Ready?

!' Waiting for CPU.

Waiting?

!' Waiting for event.

PCB?

!' OS record of process state/information.

Context switch?

!' Save one execution context, load another.

3 unconditional forks?

$\backslash [2^3=8]$

`fork()`?

!' New process.

`exec()`?

!' Replace current program image.

Zombie?

!' Child terminated; status hasn't been fully collected.

Orphan?

!' Parent terminated while child remains.

ACTIVE RECALL — TRY WITHOUT LOOKING

1. What is the exact difference between **Ready** and **Waiting**?
 2. A running process requests disk I/O. What state transition occurs?
 3. I/O finishes for a blocked process. Which state does it enter?
 4. Why is a PCB required?
 5. What is stored in a PCB?
 6. Explain the difference between **mode switch** and **context switch**.
 7. How many processes exist after four unconditional successful `fork()` calls?
 8. How many **new child processes** are created?
 9. Why does `2^n` not work for every fork question?
 10. What is the difference between `fork()` and `exec()`?
 11. Child terminates but parent hasn't collected its status. What is it?
 12. Parent terminates while child is still executing. What is the child called?
-

MASTER MEMORY MAP — CHAPTER 2

Remember:

P ! ' S ! ' PCB ! ' CS ! ' F

Process States PCB Context Switch Fork

Story:

A Process changes States; PCB remembers it; Context Switch moves it; Fork creates more.

NEXT CHAPTER

Now we move to:

CHAPTER 3 — THREADS

This is important because GATE frequently tests subtle differences between:

- Process vs Thread
- what threads share
- what threads do **not** share
- user-level vs kernel-level threads
- thread scheduling/blocking behavior
- multithreading models
- advantages and traps

The most important question we'll answer is:

If multiple threads belong to the same process, exactly what is shared between them and what remains private?

Type NEXT.

USER

NEXT

ASSISTANT

CHAPTER 3 — THREADS + IMPORTANT

Threads are a very important bridge between:

TEXT

Process
!"
Multiple execution flows
!"
Concurrency
!"
Synchronization

The most important GATE idea in this chapter is:

Threads of the same process share some resources, but each thread still needs its own execution state.

1. WHY DO THREADS EXIST?

Suppose a browser is running.

At the same time, it may need to:

- display the page
- download a file
- play audio
- respond to mouse clicks
- run JavaScript

If everything were handled as one single execution sequence, the application could become unresponsive whenever one task waits.

Instead, one process can contain multiple execution flows.

These are called:

Threads

2. WHAT IS A THREAD?

A thread is:

The smallest unit of CPU execution/scheduling within a process.

A process may contain:

TEXT

Process P

|

```
% % %   T h r e a d   T 1
% % %   T h r e a d   T 2
% % %   T h r e a d   T 3
```

Each thread can execute a different part of the program.

3. PROCESS VS THREAD — SIMPLE INTUITION

Think of a **process as a company office**.

The office has:

- files
- furniture
- shared resources
- common address

Threads are like **employees inside the same office**.

Employees share:

TEXT

Office files
Office space
Common resources

But each employee has their own:

TEXT

Current task
Working notes
Position

Similarly, threads share process resources but maintain individual execution state.

4. SINGLE-THREADED PROCESS

A traditional process may have one thread.

TEXT

Process P

+-----+

```

| Code          |
| Data          |
| Heap          |
|              |
| Thread T1     |
|   PC          |
|   Registers   |
|   Stack       |
+-----+

```

There is only one execution flow.

5. MULTITHREADED PROCESS

A process may contain several threads.

TEXT

Process P

```

+-----+
|              |
|              |
|              |
|              |
| T1   T2   T3 |
| PC   PC   PC |
| Reg  Reg  Reg |
| Stack Stack Stack |
+-----+

```

Notice carefully:

TEXT

```

C o d e ,   D a t a ,   H e a p   ! '   S H A R E D

P C ,   R e g i s t e r s ,   S t a c k   ! '   P R I V A T E

```

This is one of the most important facts in the chapter.

6. WHAT DO THREADS SHARE?

Threads belonging to the same process typically share:

- code/text
- global data
- heap
- address space
- open files
- process resources

Think:

Same process !' same house

7. WHAT DOES EACH THREAD HAVE SEPARATELY?

Each thread needs its own:

- Program Counter
- CPU registers
- stack
- execution state
- thread ID

Why?

Because each thread may be executing a different instruction.

Example:

TEXT

```
T1 executing instruction 100
T2 executing instruction 500
T3 executing instruction 900
```

Therefore they cannot share a single program counter.

~~0-6%~~ STATE MEMORY RULE

Threads SHARE: C D H F

Code Data Heap Files/resources

Threads KEEP PRIVATE: P R S

Program Counter Registers Stack

Memory sentence:

“Share CDHF, keep PRS personal.”

The critical three are:

PC + Registers + Stack are thread-private.

8. WHY IS STACK PRIVATE?

Suppose two threads execute different functions.

Thread T1:

```
C
functionA();
```

Thread T2:

```
C
functionB();
```

Each has different:

- local variables
- function calls
- return addresses

Therefore each requires a different stack.

```
TEXT
T1 Stack      T2 Stack

function A     function B
local x        local y
return addr    return addr
```

If they shared one stack, function execution would interfere.

9. WHY IS HEAP SHARED?

Dynamic memory belongs to the process address space.

Example:

```
C
int *x = malloc(...);
```

If T1 allocates shared process memory, another thread may be able to access the same data if it has the appropriate reference.

That is useful.

But it creates a new problem:

```
TEXT
T1 modifies x
T2 modifies x
      !
Possible race condition
```

And that leads directly to **synchronization**.

10. PROCESS VS THREAD

Process	Thread
Independent execution environment	Execution unit inside process
Separate address space	Threads in same process share address space

Process	Thread
Heavier	Lighter
Creation usually more expensive	Creation usually cheaper
Context switch generally more expensive	Thread switch within same process can be cheaper
IPC needed for separate processes	Threads can communicate through shared memory
Failure may be isolated from other processes	Bad thread may affect entire process

MEMORY TRICK

Process = Heavy House

Thread = Lightweight Worker inside house

11. WHY ARE THREADS CALLED LIGHTWEIGHT?

Creating another process may require substantial OS/process-management work.

Threads of the same process can reuse:

- code
- data
- address space
- files
- resources

Therefore threads typically require less overhead.

Hence:

Thread = Lightweight Process, informally.

But technically, a thread is not simply a complete independent process.

12. ADVANTAGES OF MULTITHREADING

Important conceptual question.

Four common advantages:

1. Responsiveness

One thread can continue while another waits.

Example:

TEXT

B r o w s e r U I t h r e a d ! ' r e s p o n s i v e
D o w n l o a d t h r e a d ! ' d o w n l o a d i n g

2. Resource Sharing

Threads naturally share the same process resources.

3. Economy

Creating/switching threads can be cheaper than processes.

4. Parallelism / Scalability

On multicore CPUs, multiple threads may execute simultaneously.

Example:

TEXT

C o r e 1 ! ' T 1
C o r e 2 ! ' T 2
C o r e 3 ! ' T 3

Memory Trick

R E S P

Responsiveness Economy Sharing Parallelism

Mnemonic:

Threads give RESPonse.

13. CONCURRENCY VS PARALLELISM

Ø = Important distinction.

Concurrency

Multiple tasks make progress during overlapping periods.

Even one CPU can provide concurrency by switching rapidly.

Example:

TEXT

T i m e !'

T 1 : %^%~%^

%^%~%^

T 2 : %^%~%^

%^%~%^

Only one may execute at a time, but both progress.

Parallelism

Multiple tasks literally execute at the same instant.

Usually requires multiple processing units/cores.

TEXT

C o r e 1 !' T 1

C o r e 2 !' T 2

Comparison

Concurrency	Parallelism
Overlapping progress	Simultaneous execution
Possible on one core	Typically needs multiple cores
Interleaving	Actual simultaneous execution

Memory Trick

Concurrency = Together in time

Parallelism = Together at the instant

& pGATE TRAP

Concurrency does **not automatically mean** simultaneous execution.

A single-core CPU can support concurrency.

14. THREAD IMPLEMENTATION

Threads can broadly be managed at different levels:

1. User-Level Threads
2. Kernel-Level Threads

This distinction is very important.

15. USER-LEVEL THREADS — ULT

User-level threads are managed by a **user-space thread library**, with limited/no direct kernel awareness of individual threads depending on the model.

Conceptually:

```
TEXT
Application
  |
Thread Library
  |
T1 T2 T3
  |
Kernel sees fewer schedulable entities
```

Thread creation/switching can often happen without entering the kernel.

16. ADVANTAGES OF USER-LEVEL THREADS

- fast creation
- fast switching
- less kernel overhead
- application can use custom scheduling

Why fast?

Because many operations happen in user space.

17. PROBLEM WITH CLASSIC MANY-TO-ONE USER THREADS

Suppose:

```
TEXT
T1
T2
T3
```

all map to one kernel thread.

If T1 makes a blocking system call:

```
TEXT
T 1 !' b l o c k e d
```

the underlying kernel execution entity may block.

Then:

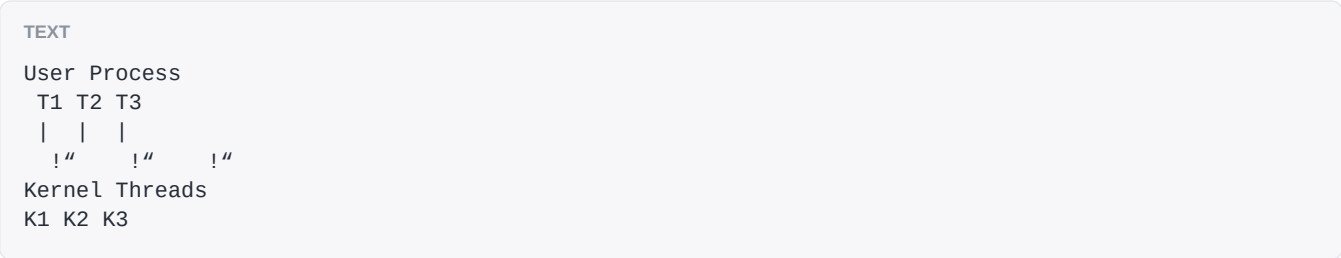
```
TEXT
T2 and T3 may also be unable to execute
```

This is a classic GATE point for the **many-to-one** model.

18. KERNEL-LEVEL THREADS — KLT

Kernel-level threads are known and managed by the operating system kernel.

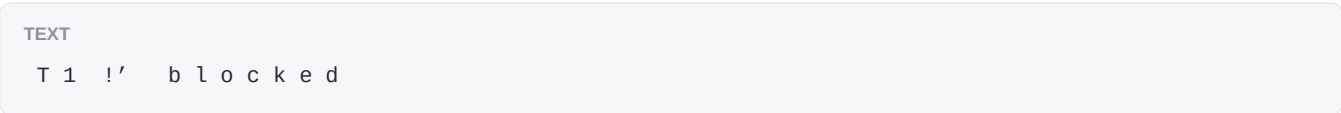
Conceptually:



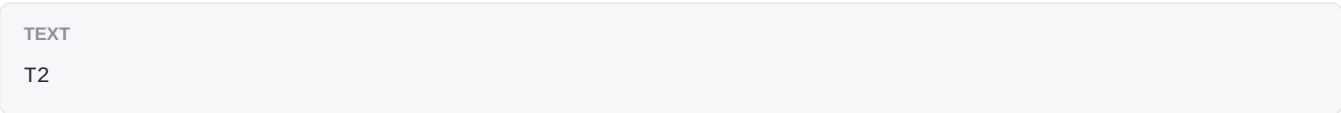
Kernel can schedule them individually.

19. ADVANTAGES OF KERNEL THREADS

If T1 blocks:



the kernel may schedule:



Also, multiple kernel threads can potentially execute simultaneously on multiple CPU cores.

20. DISADVANTAGE OF KERNEL THREADS

Management often requires more kernel involvement.

Therefore operations such as:

- creation
- scheduling
- switching

can have greater overhead than purely user-level management.

21. USER THREAD VS KERNEL THREAD

User-Level Thread	Kernel-Level Thread
Managed mainly by user library	Managed by OS kernel

User-Level Thread	Kernel-Level Thread
Often faster to create/switch	Typically higher overhead
Kernel may not see each ULT individually	Kernel knows schedulable threads
Blocking can be problematic in many-to-one	One blocked thread need not block others
Parallelism limited in many-to-one	Can support true multicore parallelism

22. THREAD MAPPING MODELS

GATE may ask about how user threads map to kernel threads.

Main models:

1. Many-to-One

TEXT

```
U 1  % %
U 2  % %<% % ! '    K 1
U 3  % %
```

Many user threads map to one kernel thread.

2. One-to-One

TEXT

```
U 1  ! '    K 1
U 2  ! '    K 2
U 3  ! '    K 3
```

Each user thread maps to one kernel thread.

3. Many-to-Many

TEXT

```
U 1  % %          % ! '    K 1
U 2  % %<% % % % % % %$
U 3  % %$          % ! '    K 2
U 4  % %          % ! '    K 3
```

Many user threads multiplex over multiple kernel threads.

23. MANY-TO-ONE MODEL

TEXT

```
U1
U 2  } ! '    K 1
U3
```

Advantages

- lightweight
- thread management can be fast

Disadvantages

- blocking kernel operation may block all user threads mapped to it
- cannot achieve true parallel execution across multiple cores through that single kernel thread

GATE TRAP

Suppose there are:

TEXT

100 user threads

but all map to:

TEXT

1 kernel thread

Can 100 of them execute simultaneously on 100 cores?

'LNo.

Kernel only has one underlying schedulable thread.

24. ONE-TO-ONE MODEL

Each user thread gets its own kernel thread.

TEXT

U 1	!	'	K 1
U 2	!	'	K 2
U 3	!	'	K 3

Advantages

- one blocked thread does not block all others
- parallel execution possible

Disadvantages

- more kernel resources
- greater creation/management overhead

25. MANY-TO-MANY MODEL

Many user threads map to several kernel threads.

Example:

TEXT

4 user threads
!"
2 kernel threads

This attempts to combine:

- user-level flexibility
- kernel-level parallelism

Memory Trick for mapping

M:1 !' cheap but restrictive

1:1 !' flexible but expensive

M:M !' compromise

26. THREAD CONTEXT SWITCH

Threads also have execution state.

Switching:

TEXT

T 1 !' T 2

requires switching thread-specific context such as:

- PC
- registers
- stack pointer

But if T1 and T2 belong to the same process, they share the same address space.

Therefore such a switch may require less work than switching between two unrelated processes.

27. PROCESS SWITCH VS THREAD SWITCH

Consider:

TEXT

P 1 / T 1 !' P 2 / T 1

Different process.

Potentially need changes involving:

- address-space context
- process resources

- memory mappings
- CPU execution state

Now:

TEXT

```
P 1 / T 1  !'  P 1 / T 2
```

Same process.

Threads already share the process address space.

So generally:

Thread switch within the same process is cheaper than process switch.

& pGATE TRAP

Do not say:

“Thread switching has zero overhead.”

False.

Registers, PC, stack-related execution state still need switching.

It is simply **often cheaper** than process context switching.

28. THREAD BLOCKING — IMPORTANT

Suppose a thread performs I/O.

What happens depends on the thread model.

Many-to-One

TEXT

```
U1
U 2  }  !'  K 1
U3
```

If K1 blocks because of a blocking system call:

TEXT

all mapped user threads may effectively stop

One-to-One

TEXT

```
U 1  !'  K 1
U 2  !'  K 2
```

If K1 blocks:

```
TEXT
K2 can continue
```

Hence T2 can execute.

29. MULTICORE EXECUTION

Suppose:

```
TEXT
T1
T2
T3
T4
```

belong to one process.

On a four-core CPU, with an appropriate kernel-supported mapping:

```
TEXT
C o r e   1   ! '   T 1
C o r e   2   ! '   T 2
C o r e   3   ! '   T 3
C o r e   4   ! '   T 4
```

Threads may actually execute simultaneously.

This is parallelism.

30. THREADS AND SHARED DATA

Suppose:

```
C
int x = 0;
```

is shared.

T1 executes:

```
TEXT
x = x + 1
```

T2 also executes:

```
TEXT
x = x + 1
```

You may expect:

TEXT

x = 2

But because threads execute concurrently, operations may interleave incorrectly.

This creates:

Race Condition

Example internally:

TEXT

T1 reads x = 0

T2 reads x = 0

T1 computes 1

T2 computes 1

T1 writes 1

T2 writes 1

Final:

TEXT

x = 1

instead of:

TEXT

2

This is why synchronization is needed.

We will study this in depth next.

31. THREAD SAFETY IDEA

If shared data can be modified by multiple threads, operations may need synchronization.

Conceptual chain:

TEXT

Threads

!"

Shared Memory

!"

Concurrent Access

!"

Race Condition

!"

Critical Section

!"

Synchronization

This is one of the most important connections in OS.

32. EXAMPLE — WHAT IS SHARED?

Suppose threads T1 and T2 belong to the same process.

Which are normally shared?

A. Stack B. Program counter C. Heap D. Registers

Think:

TEXT

```
S t a c k ! '   p r i v a t e
P C ! '   p r i v a t e
H e a p ! '   s h a r e d
R e g i s t e r s ! '   p r i v a t e
```

Therefore:

$\boxed{\text{Heap}}$

33. EXAMPLE — PRIVATE TO THREAD

Which must each thread have independently?

Possible answers:

- Code
- Global data
- Program counter
- Heap

Each thread can be at a different instruction.

Therefore:

$\boxed{\text{Program Counter}}$

34. GATE-STYLE EXAMPLE

Three user-level threads map to one kernel thread.

One user thread issues a blocking system call.

Under a pure many-to-one implementation, what can happen?

Reasoning

Kernel sees only one execution entity.

That entity becomes blocked.

Hence the other user threads cannot run on the CPU through it.

Answer

All threads mapped to that kernel thread may be blocked from execution.

35. GATE-STYLE EXAMPLE — MULTICORE

A process has 10 user threads but uses a many-to-one mapping to one kernel thread.

Machine has 8 cores.

Maximum number of those user threads that can actually execute simultaneously via that one kernel thread?

$\boxed{1}$

Because the kernel can schedule only the single kernel thread.

36. THREAD TERMINOLOGY

Sometimes GATE/textbooks use:

- lightweight process
- user thread
- kernel thread
- thread control block

The essential idea remains:

A thread requires its own execution context.

A system may maintain thread-related information such as:

TEXT

Thread ID
State
Program Counter
Registers
Stack information
Scheduling information

37. THREAD CONTROL BLOCK — TCB

Analogous in spirit to PCB, systems may maintain a:

Thread Control Block

Typical thread-specific information:

- Thread ID
- Thread state
- PC
- registers
- stack pointer
- scheduling details

Process-wide resources remain associated with the process.

38. PROCESS AND THREAD STRUCTURE

A useful mental model:

```
TEXT
PCB
|
+----- Shared Process Resources
|       Code
|       Data
|       Heap
|       Files
|
+----- TCB 1
|       PC
|       Registers
|       Stack
|
+----- TCB 2
|       PC
|       Registers
|       Stack
|
+----- TCB 3
|       PC
|       Registers
|       Stack
```

This diagram captures most of the chapter.

39. HOW GATE CAN ASK THREADS

Common question patterns:

Pattern 1 — Shared vs Private

Ask:

Which resources are shared between threads?

Very common.

Pattern 2 — User vs Kernel Threads

Ask about:

- blocking
- overhead
- scheduling
- parallelism

Pattern 3 — Mapping Models

Ask:

TEXT

many-to-one
one-to-one
many-to-many

and their behavior.

Pattern 4 — Process vs Thread

Compare:

- address space
- resources
- switch overhead
- communication

Pattern 5 — Concurrency vs Parallelism

Determine whether tasks are merely interleaved or truly simultaneous.

& pGATE TRAPS — THREADS

Trap 1

Threads share stack.

'LFalse.

Each thread needs its own stack.

Trap 2

Threads share registers.

'LFalse.

Each thread requires its own CPU execution context.

Trap 3

Threads share program counter.

'LFalse.

Different threads execute different instructions.

Trap 4

Threads of the same process do not share memory.

'LFalse.

They generally share the process address space.

Trap 5

Every multithreaded program automatically executes threads simultaneously.

'L False.

Concurrency does not guarantee parallelism.

Trap 6

Many-to-one model can use unlimited CPU cores simultaneously.

'L False.

Only one underlying kernel thread exists.

Trap 7

User-level threads are always superior.

'L False.

They are lightweight but can have blocking/parallelism limitations depending on mapping.

Trap 8

Kernel-level threads have no disadvantage.

'L False.

Kernel management can have greater overhead.

Trap 9

Thread switch is free.

'L False.

Thread-specific execution state must still be changed.

COMPARISON MASTER TABLE

Property	Separate Processes	Threads in Same Process
Address space	Separate	Shared
Code	Separate process image	Shared
Heap	Separate	Shared

Property	Separate Processes	Threads in Same Process
Stack	Separate	Separate per thread
Registers	Separate	Separate per thread
Program counter	Separate	Separate per thread
Communication	IPC mechanism often needed	Easy via shared memory
Isolation	Greater	Lower
Switch cost	Generally higher	Generally lower

' p WRITE THIS IN YOUR NOTES

Thread

Smallest unit of CPU execution within a process.

A process can contain multiple threads.

Threads Share

CDHF

TEXT

Code
Data
Heap
Files/resources

Threads Keep Private

PRS

TEXT

Program Counter
Registers
Stack

Most important:

PC + Registers + Stack are private.

Advantages

RESP

TEXT

Responsiveness
Economy
Sharing
Parallelism

Concurrency vs Parallelism

TEXT

Concurrency = overlapping progress
Parallelism = simultaneous execution

Thread Models

TEXT

Many-to-One

Many user threads !' 1 kernel thread

One-to-One

1 user thread !' 1 kernel thread

Many-to-Many

Many user !' multiple kernel threads

Important many-to-one property

TEXT

One blocking kernel operation
!"
All mapped user threads may stop

No true multicore parallelism through the single kernel thread.

SHORTCUTS

Shared/private shortcut

Ask:

"Does every thread need a different value?"

If yes, likely private.

Examples:

TEXT

```
P C !' d i f f e r e n t i n s t r u c t i o n !' p r i v a t e  
R e g i s t e r s !' d i f f e r e n t c a l c u l a t i o n s !' p r i v a t e  
S t a c k !' d i f f e r e n t f u n c t i o n c a l l s !' p r i v a t e
```

If it represents common process resources:

TEXT

```
C o d e / D a t a / H e a p / F i l e s !' s h a r e d
```

30-SECOND REVISION

Thread?

!' Execution unit inside process.

Threads share?

!' Code, data, heap, files/resources.

Threads don't share?

!' PC, registers, stack.

User thread?

!' Managed mainly in user space.

Kernel thread?

!' Kernel knows/schedules it.

Many-to-one blocking problem?

!' One blocking call can block all mapped user threads.

Concurrency?

!' Overlapping progress.

Parallelism?

!' Actual simultaneous execution.

ACTIVE RECALL

Try without looking:

1. Why does every thread require its own stack?
2. Do two threads of the same process share heap memory?
3. Do they share program counters?
4. What are the four advantages remembered using **RESP**?
5. What is the difference between concurrency and parallelism?
6. Why can many-to-one threading have a blocking problem?
7. Can many-to-one provide true parallelism across many cores?
8. Why can a thread switch be cheaper than a process switch?

9. Which is normally shared: stack or heap?
10. What three important execution items are private to every thread?

MASTER MEMORY MAP — THREADS

```
TEXT
Process
    !"
Threads
    !"
Share CDHF
    !"
Keep PRS private
    !"
Concurrent execution
    !"
Shared data access
    !"
Race Condition
    !"
Need Synchronization
```

Remember:

Threads share the house, but each carries its own PC, Registers and Stack.

NEXT — CHAPTER 4

Next we study:

INTER-PROCESS COMMUNICATION (IPC)

We will cover:

- why separate processes need IPC
- shared memory
- message passing
- pipes
- named vs unnamed pipes where relevant
- sockets/signals at the required GATE depth
- shared memory vs message passing
- synchronization requirement
- blocking vs non-blocking communication
- direct vs indirect communication
- major GATE traps

Then we will enter the extremely important chapters:

Concurrency !' Critical Section !' Synchronization !' Semaphores.

USER

next

ASSISTANT

CHAPTER 4 — INTER-PROCESS COMMUNICATION (IPC) + IMPORTANT

So far:

TEXT

Program
! "
Process
! "
Threads

Threads inside the **same process** can communicate easily because they share memory.

But what about **two different processes**?

TEXT

Process P1	Process P2
Address Space 1	Address Space 2

Normally, they have separate address spaces.

So they need a controlled way to communicate.

That is:

IPC — Inter-Process Communication

1. WHAT IS IPC?

Inter-Process Communication (IPC) refers to mechanisms provided by the operating system that allow processes to exchange data and coordinate their activities.

Two major goals:

1. Communication
2. Synchronization

2. WHY DO PROCESSES NEED IPC?

Suppose:

TEXT

P r o c e s s P 1 ! ' p r o d u c e s s o m e d a t a
P r o c e s s P 2 ! ' n e e d s t h a t d a t a

Examples:

- browser communicating with another service
- shell communicating with another process
- producer process sending data to consumer
- client communicating with server

Because separate processes generally have separate address spaces, they cannot simply access arbitrary memory belonging to each other.

So the OS provides IPC mechanisms.

3. TWO FUNDAMENTAL IPC MODELS

Very important.

The two broad IPC models are:

1. Shared Memory

2. Message Passing

Master diagram:

TEXT



Remember:

IPC = Share or Send

4. SHARED MEMORY

In shared-memory IPC, the OS establishes a memory region that multiple processes can access.

Conceptually:

TEXT



Both processes can:

- read shared data
- potentially write shared data

5. HOW SHARED MEMORY WORKS

Suppose P1 produces:

```
TEXT
x = 100
```

and writes it into shared memory.

Then P2 reads the same region.

```
TEXT
P1
|
| write x=100
| "
+-----+
| Shared Memory |
| x = 100       |
+-----+
| '
| read x
|
P2
```

This can be very efficient.

6. WHY SHARED MEMORY CAN BE FAST

Once the shared region is established, processes may communicate by directly accessing memory rather than sending every item through the kernel as a separate message.

Therefore shared memory can be efficient for large or frequent data transfers.

But there is a major problem.

7. SHARED MEMORY PROBLEM — SYNCHRONIZATION

Suppose:

```
TEXT
Shared variable x = 0
```

P1 executes:

```
TEXT
x = x + 1
```

P2 simultaneously executes:

TEXT

```
x = x + 1
```

Possible incorrect result:

TEXT

1

instead of:

TEXT

2

This is a:

Race Condition

Therefore:

Shared memory provides communication, but synchronization must be handled carefully.

This is why our next major topic is concurrency and synchronization.

& pGATE TRAP

Do not assume:

Shared memory automatically prevents race conditions.

'L False.

It actually makes synchronization particularly important because multiple processes can access the same data.

8. MESSAGE PASSING

Instead of sharing memory, processes can explicitly exchange messages.

Typical conceptual operations:

TEXT

```
send(message)
receive(message)
```

Diagram:

TEXT

```
Process P1
|
| send(data)
! "
```

```
Operating System / Communication Channel
|
| receive(data)
!"
Process P2
```

9. SIMPLE EXAMPLE

P1 sends:

```
TEXT
"Hello"
```

P2 receives:

```
TEXT
"Hello"
```

Conceptually:

```
TEXT
P 1 !' S E N D !' O S / C h a n n e l !' R E C E I V E !' P 2
```

No shared application memory is required between P1 and P2.

10. SHARED MEMORY VS MESSAGE PASSING

Important comparison.

Shared Memory	Message Passing
Processes access common memory	Processes send/receive messages
Usually efficient after setup	More communication-system/kernel involvement
Good for large data transfer	Convenient for smaller/distributed communication
Synchronization often explicit	Communication operations can provide some synchronization behavior
Same-machine use is common	Can also support communication across machines
Race conditions possible on shared data	No shared application data required

Memory shortcut:

Shared Memory = Common Whiteboard

Message Passing = Sending Letters

11. WHICH IS GENERALLY FASTER?

For processes on the same machine and substantial data exchange:

Shared memory can generally be faster after initialization.

Why?

Because the data can be accessed directly from the common memory region.

Message passing can require:

```
TEXT
send
!"
kernel / communication mechanism
!"
receive
```

and sometimes copying.

But exact performance depends on implementation.

For GATE:

Think conceptually: shared memory usually has lower communication overhead after setup.

12. MESSAGE PASSING — DIRECT COMMUNICATION

Processes may identify each other explicitly.

Example:

```
TEXT
send(P2, message)
receive(P1, message)
```

P1 directly names P2.

This is:

Direct Communication

Diagram:

```
TEXT
P1 -----> P2
```

13. INDIRECT COMMUNICATION

Processes communicate through an intermediate object such as:

- mailbox

- port

Conceptually:

TEXT

P 1 !' M a i l b o x M !• P 2

Operations may look conceptually like:

TEXT

```
send(M, message)
receive(M, message)
```

Processes do not necessarily address each other directly.

14. DIRECT VS INDIRECT

Direct	Indirect
Process identifies communication partner	Uses mailbox/port
P 1 !" P 2	P 1 !" M a i l b o x !" P 2
Tight association	More flexible association

Memory trick:

Direct = Person's name

Indirect = Postbox number

15. SYNCHRONOUS VS ASYNCHRONOUS MESSAGE PASSING

Another important classification.

Blocking / Synchronous

A process may have to wait.

Non-blocking / Asynchronous

The process can continue without waiting.

16. BLOCKING SEND

Conceptually:

TEXT

```
send(message)
```

Sender may wait until:

- receiver accepts it
- system safely buffers it

depending on semantics.

17. NON-BLOCKING SEND

Sender sends message and continues immediately.

TEXT

```
P1 sends
```

```
!"
```

```
P1 continues
```

18. BLOCKING RECEIVE

Receiver waits until a message becomes available.

TEXT

```
receive()
```

```
!"
```

```
No message
```

```
!"
```

```
WAIT
```

When message arrives:

TEXT

```
W A I T I N G  ! '   R E A D Y
```

19. NON-BLOCKING RECEIVE

Receiver checks for message.

If none exists, it does not necessarily wait.

It may immediately receive:

TEXT

```
No message / null / status
```

depending on system design.

20. RENDEZVOUS

If both:

- send is blocking
- receive is blocking

communication can behave like a synchronization point.

This is often described conceptually as a:

Rendezvous

Both sides coordinate at the communication event.

21. MESSAGE BUFFERING

Where are messages kept before being received?

A communication link may have a queue.

Conceptually:

TEXT

Sender

!"

+-----+

| Message Queue |

| M1 | M2 | M3 |

+-----+

!"

Receiver

Three conceptual capacity cases are commonly discussed.

22. ZERO CAPACITY

No message can wait in the queue.

Sender must coordinate directly with receiver.

Conceptually:

TEXT

Buffer size = 0

This gives strong synchronization.

23. BOUNDED CAPACITY

Queue can contain at most:

TEXT

n messages

If full:

TEXT

sender may block

until space becomes available.

24. UNBOUNDED CAPACITY

Conceptually, queue size has no practical fixed bound.

Sender normally does not wait because of queue capacity.

Memory Trick

TEXT

```
0 !' M e e t   r e c e i v e r
N !' W a i t   i f   f u l l
" !' N o   f u l l - b u f f e r   w a i t
```

25. IPC MECHANISMS

At GATE level, recognize common IPC mechanisms:

- shared memory
- pipes
- message queues
- sockets
- signals

The official syllabus only says IPC, so understand these primarily from the conceptual perspective.

26. PIPES

A pipe provides a communication channel between processes.

Think:

TEXT

Process P1 ===== PIPE =====> Process P2

One process writes data.

Another reads it.

27. PIPE ANALOGY

Imagine an actual water pipe:

TEXT

P 1 p u t s d a t a i n ! ' = = = = = ! ' P 2 r e c e i v e s d a t a

The pipe transports a stream of data.

28. ORDINARY / UNNAMED PIPE

A traditional unnamed pipe is often used between processes with an existing relationship, such as parent and child.

Conceptually:

TEXT

Parent

|

fork()

|

Child

Parent ===== Pipe ===== Child

29. NAMED PIPE

A named pipe has an identifiable name in the system.

It can allow communication between processes that do not necessarily have a direct parent-child relationship.

Broad distinction:

Unnamed Pipe	Named Pipe
Temporary communication channel	Has a system-visible name
Often related processes	Can support unrelated processes
Common parent-child use	More flexible

& pGATE NOTE

Do not memorize OS-specific implementation details unless the question supplies them.

The useful conceptual distinction is:

Unnamed pipes are commonly tied to related processes; named pipes provide a named communication endpoint.

30. PIPE DIRECTION

A basic pipe is conceptually often treated as:

Unidirectional

TEXT

P 1 ! ' P 2

For two-way communication, systems may use:

- two pipes
- or another full-duplex IPC mechanism

Implementation details vary.

31. MESSAGE QUEUES

A message queue stores messages until the receiving process retrieves them.

TEXT

```
P1
! "
+-----+
| Message Queue |
| M1 M2 M3      |
+-----+
! "
P2
```

Unlike a raw shared-memory region, communication is structured around discrete messages.

32. SOCKETS

Sockets are communication endpoints.

Very important when processes communicate:

- on the same machine
- or across a network

Conceptually:

TEXT

```
Process A
! "
Socket
! "
Network
! "
Socket
! "
Process B
```

Sockets become especially important in Computer Networks.

For OS GATE, know the IPC idea.

33. SIGNALS

A signal is a mechanism used to notify a process that some event occurred.

Conceptually:

```
TEXT
Event
  !"
Signal
  !"
Process notified
```

Examples conceptually include:

- termination notification
- interrupt-like software notifications
- exceptional conditions

Signals usually carry much less data than mechanisms such as shared memory.

Think:

Signal = Notification, not bulk data transfer.

34. IPC COMPARISON

Mechanism	Main Idea
Shared Memory	Share common memory
Message Passing	Send/receive messages
Pipe	Stream communication
Message Queue	Queue discrete messages
Socket	Communication endpoint
Signal	Event notification

35. PRODUCER-CONSUMER PROBLEM

Ø=Very important connection.

Consider two processes:

```
TEXT
Producer
Consumer
```

Producer generates items.

Consumer uses them.

They share a buffer.

```
TEXT
Producer
    !"
+-----+
|  BUFFER  |
+-----+
    !"
Consumer
```

Examples:

- keyboard produces input; application consumes it
- server produces results; client consumes them
- one process generates data; another processes it

36. BOUNDED BUFFER

Suppose buffer capacity is:

```
TEXT
N
```

Producer cannot insert if buffer is full.

Consumer cannot remove if buffer is empty.

So we need:

```
TEXT
Producer waits if FULL
Consumer waits if EMPTY
```

This requires synchronization.

37. WHY IPC LEADS TO SYNCHRONIZATION

Suppose shared buffer:

```
TEXT
[ A ][ B ][   ][   ]
```

Producer is modifying it.

Consumer simultaneously modifies indices.

Without coordination:

- data may be lost
- same item may be consumed twice
- buffer indexes may become inconsistent

Hence:

TEXT

IPC

!"

Shared Resources

!"

Concurrent Access

!"

Synchronization Required

38. COOPERATING VS INDEPENDENT PROCESSES

A process is **independent** if it cannot affect or be affected by other executing processes in the relevant sense.

A **cooperating process** can affect or be affected by another process.

IPC is primarily needed for cooperating processes.

Reasons processes cooperate include:

- information sharing
- computation speedup
- modularity
- convenience

39. IPC AND ADDRESS SPACES

This distinction is fundamental.

Separate Processes

TEXT

P1 Address Space

P2 Address Space

Normally separate.

Need an IPC mechanism.

Threads in One Process

TEXT

T1

T2

T3

!"

Same Process Address Space

Already share memory.

Therefore thread communication is naturally easier, though synchronization remains necessary.

40. GATE-STYLE QUESTION — SHARED MEMORY

Two processes communicate through shared memory.

Which issue must be explicitly addressed if both modify the same shared variable?

A. Compilation B. Synchronization C. Paging only D. Process creation

Answer:

`\boxed{\text{B — Synchronization}}`

Because concurrent modifications can produce race conditions.

41. GATE-STYLE QUESTION — MESSAGE PASSING

If a `receive()` operation waits until a message arrives, it is:

`\boxed{\text{Blocking receive}}`

The process may enter a waiting state.

42. GATE-STYLE QUESTION — MANY PROCESSES

P1 and P2 have separate address spaces but need to exchange a large amount of data repeatedly on the same machine.

A common efficient mechanism is:

`\boxed{\text{Shared Memory}}`

But synchronization is necessary.

43. GATE-STYLE QUESTION — PIPE

A process writes bytes into an ordinary pipe and another process reads from it.

The pipe primarily represents:

`\boxed{\text{A communication channel / byte stream}}`

44. MESSAGE PASSING + PROCESS STATES

Suppose P1 executes:

```
TEXT
receive()
```

using blocking semantics and no message exists.

Then:

TEXT

P 1 : R u n n i n g ! ' W a i t i n g

When a message arrives:

TEXT

W a i t i n g ! ' R e a d y

Notice how this connects to Chapter 2.

45. SHARED MEMORY + THREADS CONNECTION

For threads:

TEXT

T1
T2

same-process address space is already shared.

For processes:

TEXT

P1
P2

OS must explicitly arrange a shared-memory region if shared-memory IPC is desired.

This is an important conceptual distinction.

46. COMMUNICATION VS SYNCHRONIZATION

Do not confuse these.

Communication

Concerned with:

What data is exchanged?

Synchronization

Concerned with:

When and in what order may processes access shared resources?

Example:

TEXT

Producer gives data to Consumer

Communication.

TEXT

Producer must not overwrite a full buffer

Synchronization.

They are related but not identical.

& pGATE TRAPS — IPC

Trap 1

Separate processes normally share the entire address space.

'LFalse.

They normally have separate address spaces.

Trap 2

Shared memory eliminates synchronization problems.

'LFalse.

Shared-memory access can create race conditions.

Trap 3

Message passing requires shared application memory.

'LFalse.

Processes can communicate through send/receive mechanisms.

Trap 4

Blocking receive continues executing normally when there is no message.

'LFalse.

It waits.

Trap 5

After a blocking event finishes:

TEXT

W a i t i n g ! ' R u n n i n g

automatically.

'L

Normally:

TEXT

W a i t i n g ! ' R e a d y

Trap 6

Concurrency and IPC mean the same thing.

'L

IPC = communication between processes.

Concurrency = overlapping execution/progress.

Trap 7

Signals are primarily intended for large bulk-data transfer.

'L

Think:

Signal = notification/event.

Trap 8

Every pipe necessarily supports bidirectional communication in the simplest conceptual model.

'L

A basic pipe is often considered unidirectional.

MASTER COMPARISON — IPC

Feature	Shared Memory	Message Passing
Communication	Memory reads/writes	send / receive
Shared address region	Yes	Not necessary
Data transfer efficiency	Often high	Usually more communication overhead
Synchronization	Explicitly important	May be integrated into operations

Feature	Shared Memory	Message Passing
Large data	Good	Can be less efficient
Distributed systems	Less natural	Very natural
Race conditions	Possible	Shared-memory race avoided

' p WRITE THIS IN YOUR NOTES

IPC

Mechanism for processes to communicate and coordinate.

Main models:

TEXT

IPC

```
% % %   S h a r e d   M e m o r y
% % %   M e s s a g e   P a s s i n g
```

Memory:

Share or Send

Shared Memory

TEXT

```
P 1  !'   S h a r e d   R e g i o n  !•   P 2
```

Advantages:

- fast
- good for large/frequent data

Problem:

Requires synchronization.

Message Passing

TEXT

```
send()
receive()
```

No common application memory required.

Types:

TEXT

Direct
Indirect

Blocking
Non-blocking

Blocking Receive

TEXT

No message
!"
Process waits

Pipes

TEXT

W r i t e r ! ' P I P E ! ' R e a d e r

- ordinary/unnamed
- named

Other IPC

TEXT

M e s s a g e Q u e u e ! ' q u e u e d m e s s a g e s
S o c k e t ! ' c o m m u n i c a t i o n e n d p o i n t
S i g n a l ! ' n o t i f i c a t i o n

Shared vs Message

TEXT

S h a r e d M e m o r y ! ' C o m m o n W h i t e b o a r d
M e s s a g e P a s s i n g ! ' L e t t e r s

MEMORY SHORTCUTS

IPC

S M P Q S S

Shared memory Message passing Pipe Queue Socket Signal

You do not need to force this mnemonic if the concepts themselves are clear.

Better conceptual memory:

Share memory, Send messages, Pipe streams, Queue messages, Socket endpoints, Signal events.

CONCEPT CHAIN

This is extremely important:

TEXT

```
Multiple Processes
      ! "
Need Communication
      ! "
IPC
      ! "
Shared Memory
      ! "
Concurrent Access
      ! "
Race Condition
      ! "
Critical Section
      ! "
Synchronization
```

We have now reached one of the most important parts of Operating Systems.

30-SECOND REVISION

IPC?

!' Communication/coordination between processes.

Two main IPC models?

!' Shared memory and message passing.

Shared memory advantage?

!' Efficient direct memory communication.

Shared memory danger?

!' Race conditions.

Message passing?

!' `send` and `receive`.

Blocking receive?

!' Wait until message arrives.

Direct communication?

!' Processes identify each other.

Indirect?

!' Mailbox/port.

Pipe?

!' Data communication stream.

Socket?

!' Communication endpoint.

Signal?

!' Event notification.

ACTIVE RECALL

Try without looking:

1. Why do different processes need IPC?
2. What are the two fundamental IPC models?
3. Why can shared memory be faster?
4. What major problem arises with shared memory?
5. What is the difference between communication and synchronization?
6. What is blocking receive?
7. Direct vs indirect message passing?
8. What happens if a bounded message queue becomes full?
9. What is the basic purpose of a pipe?
10. Why is a signal different from shared memory?
11. Why do threads usually not require a special mechanism just to share memory?
12. A process blocks on `receive()`. When the message arrives, which state does it enter?

MASTER MEMORY MAP — CHAPTER 4

TEXT

Separate Processes

!"

Need IPC

!"

% % % % % % % % % % % % % % % %

!"

!"

Shared

Message

Memory

Passing

!"

!"

Fast

send/receive

!"

Shared Data

!"

Race Condition

!"

Synchronization

NEXT — CHAPTER 5 Ø=Ý%

Now we enter a **very high-priority GATE area**:

CONCURRENCY + RACE CONDITION + CRITICAL SECTION

Next we will cover:

- what concurrency really means
- atomic vs non-atomic operations
- race conditions
- exactly how `x = x + 1` can fail
- critical sections
- entry/remainder/exit sections
- three requirements of a correct critical-section solution:
 - Mutual Exclusion
 - Progress
 - Bounded Waiting
- Peterson's algorithm
- busy waiting
- spinlocks
- common GATE interleaving questions
- shortcuts for solving shared-variable problems

Then we will be ready for **Mutexes and Semaphores**.

USER
NEXT

USER
NEXT

ASSISTANT

CHAPTER 5 — CONCURRENCY, RACE CONDITION & CRITICAL SECTION Ø=Ý% VERY HIGH PRIORITY

This is one of the most important OS chapters for GATE because it directly leads to:

TEXT
Concurrency
!"
Race Condition
!"
Critical Section
!"
Synchronization
!"
Mutex / Semaphore
!"

Classical Problems
!"
Deadlocks

If this chapter is clear, semaphores become much easier.

1. WHAT IS CONCURRENCY?

Concurrency means:

Multiple processes or threads make progress during overlapping periods of time.

They do not necessarily execute at exactly the same instant.

Example on a single CPU:

TEXT

T i m e !'

P 1 : %^%^%^ %^%^%^
P 2 : %^%^%^ %^%^%^

CPU keeps switching between them.

Both are making progress.

That is **concurrency**.

2. CONCURRENCY VS PARALLELISM

You already saw this with threads, but now it becomes important.

Concurrency

Operations overlap in time.

Possible even with:

TEXT

1 CPU core

through interleaving.

Parallelism

Operations execute literally at the same instant.

Usually:

TEXT

C o r e 1 !' P 1
C o r e 2 !' P 2

Quick memory

Concurrency = Interleaving

Parallelism = Simultaneous

3. WHY DOES CONCURRENCY CREATE PROBLEMS?

Suppose two processes access completely separate data.

No issue.

But suppose both modify:

TEXT

Shared Variable x

Now the final result may depend on:

the order in which their instructions execute

That creates a possible:

Race Condition

4. WHAT IS A RACE CONDITION?

Very important definition.

A race condition occurs when multiple processes/threads concurrently access shared data, at least one modifies it, and the final result depends on the order/timing of execution.

Important keywords:

- shared data
- concurrent access
- modification
- execution order matters
- unpredictable/incorrect result possible

5. SIMPLE EXAMPLE

Suppose:

TEXT

x = 5

Two threads perform:

TEXT

```
T1: x = x + 1
T2: x = x + 1
```

You may think:

TEXT

```
Final x = 7
```

But not necessarily.

Why?

Because:

TEXT

```
x = x + 1
```

is not necessarily one indivisible hardware action.

6. NON-ATOMIC OPERATION

Conceptually:

TEXT

```
x = x + 1
```

can be broken into:

TEXT

```
1. Read x
2. Add 1
3. Write x
```

For T1:

TEXT

```
R1 = x
R1 = R1 + 1
x = R1
```

For T2:

TEXT

```
R2 = x
R2 = R2 + 1
x = R2
```

Now interleaving becomes dangerous.

7. LOST UPDATE EXAMPLE

Initial:

$\boxed{x=5}$

Execution:

TEXT

T1 reads $x = 5$
T2 reads $x = 5$

T1 computes 6
T2 computes 6

T1 writes $x = 6$
T2 writes $x = 6$

Final:

$\boxed{x=6}$

But logically two increments occurred, so expected:

$\boxed{7}$

One update was lost.

This is called a **lost update**.

8. WHAT IS AN ATOMIC OPERATION?

An atomic operation is:

An operation that appears indivisible—it completes entirely or not at all with respect to competing operations.

Think:

TEXT

No other operation can observe it halfway.

Memory trick:

Atomic = All at once

If increment were truly atomic, two increments would correctly produce:

$\boxed{7}$

& pGATE TRAP

Do not assume one high-level statement means one atomic machine instruction.

Example:

```
C
x++;
```

looks like one statement.

But internally it can involve:

```
TEXT
Read
Modify
Write
```

Therefore race conditions can occur.

9. GATE-STYLE INTERLEAVING QUESTION

Suppose:

$\boxed{x=10}$

Processes P1 and P2 each execute:

```
TEXT
x = x + 1
```

If both read 10 before either writes, final value can be:

$\boxed{11}$

instead of:

$\boxed{12}$

This kind of reasoning appears frequently in synchronization questions.

10. ANOTHER CLASSIC EXAMPLE

Suppose:

$\boxed{x=5}$

P1:

```
TEXT
x = x + 1
```

P2:

```
TEXT
x = x - 1
```

Expected final result:

$\boxed{5}$

But consider:

TEXT

P1 reads 5

P2 reads 5

P1 computes 6

P2 computes 4

P1 writes 6

P2 writes 4

Final:

$\boxed{4}$

Another order could produce:

$\boxed{6}$

Correct serial result would be:

$\boxed{5}$

So possible final values can differ because of interleaving.

11. EXAM METHOD FOR INTERLEAVING QUESTIONS

For shared-variable questions:

Step 1

Expand every statement into:

TEXT

READ

COMPUTE

WRITE

Step 2

Assign separate local registers.

Example:

TEXT

P1:

R1 = x

R1 = R1 + 1

x = R1

TEXT

P2:

R2 = x

R2 = R2 - 1

x = R2

Step 3

Preserve the order **inside each process**.

Step 4

Interleave instructions legally.

Step 5

Track each write to the shared variable.

This is much safer than guessing.

12. WHAT IS A CRITICAL SECTION?

Suppose part of the program accesses shared data.

That portion is called the:

Critical Section

Definition:

A critical section is a part of a process/thread where shared data or shared resources are accessed or modified and therefore concurrent execution must be controlled.

Example:

TEXT

```
balance = balance - 100;
```

If multiple threads modify the same bank balance:

TEXT

```
Critical Section
```

needs protection.

13. PROCESS STRUCTURE AROUND CRITICAL SECTION

A process is often conceptually divided into:

TEXT

```
Entry Section
    ! "
Critical Section
    ! "
Exit Section
    ! "
Remainder Section
```

14. ENTRY SECTION

The process requests permission to enter the critical section.

Example conceptually:

```
TEXT
Acquire lock
```

15. CRITICAL SECTION

Shared data/resource is accessed.

Example:

```
TEXT
x = x + 1
```

16. EXIT SECTION

Process releases permission/resource.

Example:

```
TEXT
Release lock
```

17. REMAINDER SECTION

Everything outside the critical section.

Conceptually:

```
TEXT
do {
    ENTRY SECTION

    CRITICAL SECTION

    EXIT SECTION

    REMAINDER SECTION
} while(true);
```

18. CRITICAL SECTION PROBLEM

We need a protocol so that cooperating processes safely share data.

A correct solution must satisfy three famous conditions.

Extremely important.

1. Mutual Exclusion

2. Progress

3. Bounded Waiting

Remember:

M P B

Mnemonic:

"My Process Behaves."

19. MUTUAL EXCLUSION

Definition:

If one process is executing in its critical section, no other competing process may execute in its corresponding critical section at the same time.

Example:

TEXT

P1 in CS

!"

P2 must wait

Not allowed:

TEXT

P1 CS

P2 CS

simultaneously when accessing the protected shared resource.

Memory

Mutual exclusion = One at a time

20. WHY MUTUAL EXCLUSION IS REQUIRED

Suppose shared variable:

$\forall x=5 \forall$

If P1 enters protected section:

TEXT

P1 modifies x

P2 cannot interfere until P1 completes.

Then execution becomes effectively:

TEXT

P1 complete
then P2

or:

TEXT

P2 complete
then P1

Either is safe if operations are intended to be serialized.

21. PROGRESS

This one is often misunderstood.

Definition conceptually:

If no process is inside the critical section and some processes wish to enter, the choice of who enters next cannot be postponed indefinitely by processes that are not interested in entering.

Simpler:

If the critical section is free and someone wants it, eligible processes should eventually be allowed to choose an entrant.

Example of Progress Violation

Suppose:

TEXT

Critical section = empty
P1 wants to enter
P2 does not want to enter

If P2 somehow prevents P1 forever:

Progress is violated.

Memory Trick

Progress = Don't keep an empty room locked.

If nobody is inside and someone wants entry, the system should move forward.

22. BOUNDED WAITING

Definition:

After a process requests entry into the critical section, there must be a bound on the number of times other processes may enter before that process gets its turn.

Simpler:

A process should not wait forever.

This prevents:

Starvation

Example

P1 requests critical section.

But execution keeps allowing:

TEXT

P2

P3

P2

P3

P2

P3

...

forever.

P1 never enters.

Then bounded waiting is violated.

Memory Trick

Bounded Waiting = Your turn must eventually come.

23. MUTUAL EXCLUSION VS PROGRESS VS BOUNDED WAITING

Requirement	Main Question
Mutual Exclusion	Can two enter together?
Progress	If CS is free, can entry decision proceed?
Bounded Waiting	Can someone wait forever?

Perfect shortcut:

TEXT

M E ! ' S a f e t y
P r o g r e s s ! ' M o v e m e n t
B W ! ' F a i r n e s s

24. STARVATION

Starvation means:

A process waits indefinitely because other processes keep getting the required resource or service before it.

Example:

TEXT

P1 waiting
P2 enters
P3 enters
P2 enters
P4 enters
...

P1 keeps being ignored.

This is also called **indefinite postponement**.

25. STARVATIONVS DEADLOCK

Important early distinction.

Starvation

Some process may wait forever while others continue.

Deadlock

A set of processes wait permanently for one another/resources such that none in the set can proceed.

Example starvation:

TEXT

P1 waits
P2, P3 keep running

Example deadlock:

TEXT

```
P1 waits for P2
P2 waits for P1
```

We'll study deadlock deeply later.

26. BUSY WAITING

Suppose a process repeatedly checks:

TEXT

```
while(lock == busy)
{
    // keep checking
}
```

It is waiting, but still consuming CPU instructions.

This is called:

Busy Waiting

Definition:

A process continuously tests a condition while waiting instead of blocking and yielding the CPU.

27. BUSY WAITING EXAMPLE

C

```
while (lock == 1)
{
}
```

The process repeatedly executes the loop.

CPU is occupied doing no useful work.

28. BUSY WAITING VS BLOCKING

Busy Waiting

TEXT

```
Check
Check
Check
Check
Check...
```

Consumes CPU.

Blocking

TEXT

```
Process waits
! "
CPU given to someone else
```

Process sleeps/blocks until relevant event.

Comparison

Busy Waiting	Blocking
Keeps checking	Process sleeps/waits
Uses CPU	CPU can serve others
Good for very short waits in some cases	Better for longer waits
Used with spinlocks	Used with blocking mutex/semaphore implementations

29. SPINLOCK

A spinlock is a lock where the waiting process/thread:

spins continuously until the lock becomes available.

Conceptually:

TEXT

```
while(test_lock() == BUSY)
;
```

Hence:

Spinlock != Busy Waiting

30. WHEN CAN SPINLOCK BE USEFUL?

Suppose waiting time will be extremely short.

Blocking might involve expensive:

TEXT

```
sleep
context switch
```

```
wake up  
context switch
```

Then briefly spinning can sometimes be cheaper.

Particularly relevant in:

- multiprocessor systems
- short critical sections

But excessive spinning wastes CPU.

& **PGATE TRAP**

Busy waiting is not automatically “wrong.”

It has a cost but may be suitable when:

- wait is short
- context-switch overhead would be larger
- another processor can release the lock soon

31. SIMPLE SOFTWARE LOCK — WHY IT CAN FAIL

Suppose:

```
C  
while (lock == 1)  
    ;  
  
lock = 1;  
  
/* critical section */  
  
lock = 0;
```

Looks correct?

Not necessarily.

Suppose:

```
TEXT  
lock = 0
```

P1 checks:

```
TEXT  
lock == 0
```

Before P1 sets it to 1, CPU switches.

P2 checks:

TEXT

```
lock == 0
```

Both think lock is free.

Then both set:

TEXT

```
lock = 1
```

and both may enter.

Mutual exclusion violated.

32. WHY?

Because:

TEXT

```
Check lock
Set lock
```

are separate operations.

They are not atomic.

We therefore need:

- correct software algorithms
 - or atomic hardware instructions
 - or synchronization primitives
-

33. HARDWARE ATOMIC OPERATIONS

Operating systems/processors may provide atomic primitives such as conceptually:

- Test-and-Set
- Compare-and-Swap
- Swap/Exchange

These can help implement locks.

For GATE, the most common classic is:

Test-and-Set

34. TEST-AND-SET

Conceptually:

TEXT

```
TestAndSet(lock)
```

does two things atomically:

1. returns old value of `lock`
2. sets `lock = true`

One indivisible operation.

Pseudo-code:

```
c
boolean TestAndSet(boolean *target)
{
    boolean rv = *target;
    *target = true;
    return rv;
}
```

The important property is:

The entire operation is atomic.

35. LOCK USING TEST-AND-SET

Conceptually:

```
c
while (TestAndSet(&lock))
    ;

/* critical section */

lock = false;
```

If:

TEXT

```
lock = false
```

first process atomically obtains it.

Other processes see:

TEXT

```
true
```

and spin.

Thus mutual exclusion can be achieved.

36. DOES TEST-AND-SET AUTOMATICALLY GUARANTEE BOUNDED WAITING?

No.

Ø=Important GATE trap.

A simple Test-and-Set lock can provide:

Mutual exclusion

but may permit:

Starvation

because the same or other processes may repeatedly acquire the lock before a waiting process.

Therefore:

Mutual exclusion does not automatically imply bounded waiting.

37. PETERSON'S ALGORITHM

Ø=Classic GATE topic.

Peterson's algorithm is a software solution for:

Two processes

It uses:

TEXT
flag[2]
turn

38. IDEA OF flag[]

For:

TEXT
P0
P1

flag[i] = true means:

Process Pi wants to enter the critical section.

Example:

TEXT

```
flag[0] = true
```

means P0 is interested.

39. IDEA OF turn

turn helps resolve conflict when both processes want entry.

If:

TEXT

```
turn = 1
```

P0 effectively gives priority to P1 when both are interested.

40. PETERSON'S ALGORITHM FOR P0

```
C
flag[0] = true;
turn = 1;

while (flag[1] && turn == 1)
    ;

/* critical section */

flag[0] = false;

/* remainder section */
```

For P1:

```
C
flag[1] = true;
turn = 0;

while (flag[0] && turn == 0)
    ;

/* critical section */

flag[1] = false;
```

41. SIMPLE INTUITION

Suppose P0 wants to enter.

It says:

TEXT

```
flag[0] = true
```

Meaning:

"I want the critical section."

Then:

TEXT

```
turn = 1
```

Meaning:

"But if P1 also wants it, I give P1 priority."

P1 does symmetrical logic.

If both race:

the final value of `turn` breaks the tie.

42. WHY DOES PETERSON'S ALGORITHM WORK?

Under its classic theoretical assumptions, it satisfies:

- Mutual Exclusion
- Progress
- Bounded Waiting

So:

MPB '

43. PETERSON'S ALGORITHM LIMITATION

Classic Peterson's algorithm is designed for:

$\lfloor \boxed{2} \rfloor$ processes

Also, practical modern hardware/compiler memory reordering means real systems require proper memory-ordering guarantees; but for GATE, treat Peterson's algorithm according to the standard theoretical model.

44. PETERSON GATE TRAP

Suppose P0 executes:

TEXT

```
flag[0] = true;
turn = 1;
```

Why does it set `turn` to the **other** process?

Because P0 is effectively saying:

"I am interested, but if both of us want to enter, give you priority."

That tie-breaking logic helps guarantee fairness.

45. PROCESS SYNCHRONIZATION

Synchronization means:

Coordinating concurrent processes/threads so that shared resources are accessed in a correct order and race conditions are prevented.

Critical-section solutions are part of synchronization.

Later we will use:

- Mutex
- Semaphore

to implement it more conveniently.

46. SERIALIZATION

If two critical sections cannot execute simultaneously, they become serialized.

Example:

TEXT

```
P1 CS
    ! "
P2 CS
```

or:

TEXT

```
P2 CS
    ! "
P1 CS
```

but not both together.

This is:

Mutual exclusion / serialized access

47. GATE NUMERICAL — TWO INCREMENTS

Initial:

$\boxed{x=0}$

Two processes independently execute once:

TEXT

```
x = x + 1
```

Assume read and write operations may interleave.

Maximum final value:

If operations effectively happen serially:

$\boxed{2}$

Minimum possible final value:

Both read 0 before writing:

TEXT

```
P1 reads 0
P2 reads 0
P1 writes 1
P2 writes 1
```

$\boxed{1}$

So possible final values:

$\boxed{1\text{ or }2}$

48. GATE NUMERICAL — MANY INCREMENTS

Suppose n processes each execute one non-atomic increment on shared x , initially 0.

Maximum:

$\boxed{n}$

because all increments can occur serially.

Minimum under a simple read-modify-write model can be much smaller because of overwritten updates.

But be careful:

Do not apply a memorized minimum formula unless the exact instruction model/interleaving constraints are given.

For GATE, expand the actual read/write operations.

49. CONCURRENT PROGRAM QUESTION SHORTCUT

Whenever you see:

TEXT

```
shared x
multiple processes
x++
x--
```

immediately think:

RMW

Read Modify Write

Mnemonic:

“Race Means RMW.”

This is the first thing to write mentally.

50. CRITICAL SECTION EXAMPLE

Suppose bank account:

\[Balance = 1100]

Two ATMs simultaneously withdraw 1700 .

Without synchronization:

TEXT

```
ATM1 reads 1000
ATM2 reads 1000

ATM1 approves
ATM2 approves

Both update balance
```

Potentially inconsistent behavior.

Correct solution requires critical handling of:

TEXT

```
Check Balance
Modify Balance
```

as one protected sequence.

51. LOCK CONCEPT

A lock can conceptually be:

TEXT

```
Unlocked !' Critical section free
Locked   !' Someone using it
```


Process:

TEXT

```
acquire(lock)

critical section

release(lock)
```

Simple mental model:

TEXT

Door key

Only key-holder enters.

52. MUTEX — PREVIEW

A **mutex** is a mutual-exclusion synchronization primitive.

Conceptually:

TEXT

```
lock()
    !"
Critical Section
    !"
unlock()
```

We'll study it properly with semaphores in the next chapter.

53. PREEMPTION INSIDE CRITICAL SECTION

Suppose P1 enters a critical section and then gets preempted.

Is that automatically wrong?

No.

The important condition is:

Another process must not enter the same protected critical section while P1 still owns the lock/protection.

P1 may temporarily stop executing.

Mutual exclusion can still remain satisfied.

& pGATE TRAP

Mutual exclusion does **not** mean:

"The process can never be preempted."

It means:

“No competing process enters the same protected critical section simultaneously.”

54. CRITICAL SECTION SIZE

Generally we want critical sections to be:

As short as reasonably possible

Why?

Because others may be waiting.

Long critical sections increase:

- waiting
- contention
- reduced parallelism
- potential performance loss

55. SAFETY VS LIVENESS

Useful advanced GATE intuition.

Safety property

Something bad should never happen.

Example:

Two processes should never enter CS together.

That's **Mutual Exclusion**.

Liveness property

Something good should eventually happen.

Examples:

- Progress
- Bounded waiting

Meaning:

Processes should eventually be able to move forward.

56. HOW GATE CAN ASK THIS CHAPTER

Pattern 1 — Shared Variable

Calculate possible values after interleaving.

Pattern 2 — Atomicity

Identify whether operation must be atomic.

Pattern 3 — Critical Section Conditions

Determine whether an algorithm satisfies:

TEXT

Mutual Exclusion
Progress
Bounded Waiting

Pattern 4 — Peterson Algorithm

Trace:

TEXT

flag
turn

Pattern 5 — Test-and-Set

Check:

- mutual exclusion
 - busy waiting
 - starvation/bounded waiting
-

Pattern 6 — Busy Waiting

Determine whether CPU is consumed while waiting.

Pattern 7 — Starvation vs Deadlock

Distinguish indefinite waiting from circular resource deadlock.

& pGATE TRAPS — CHAPTER 5

Trap 1

`x++` is always atomic.

'LFalse.

Trap 2

Concurrency requires multiple processors.

'LFalse.

One CPU can interleave processes.

Trap 3

Race condition occurs simply because multiple processes exist.

'LFalse.

Shared data + concurrent access + relevant modification/order dependence are important.

Trap 4

Mutual exclusion guarantees bounded waiting.

'LFalse.

A process can still starve.

Trap 5

Progress and bounded waiting mean the same thing.

'LFalse.

Progress:

system shouldn't unnecessarily stop choosing an entrant.

Bounded waiting:

a particular requester cannot be postponed forever.

Trap 6

Busy waiting means process is blocked and consumes no CPU.

'LFalse.

It actively checks repeatedly.

Trap 7

Spinlock uses sleeping.

'LUsually false conceptually.

Spinlock != spinning/busy waiting.

Trap 8

Test-and-Set alone guarantees fairness.

'L False.

Simple Test-and-Set locks can cause starvation.

Trap 9

Peterson's classic algorithm is for arbitrary n processes.

'L False.

Classic form:

$\boxed{2}$

Trap 10

A process being preempted inside the critical section automatically violates mutual exclusion.

'L False.

Another process must merely be prevented from entering until ownership is released.

' p WRITE THIS IN YOUR NOTES

Race Condition

Final result depends on execution/interleaving order of concurrent accesses to shared data.

Example:

```
TEXT
x++

= Read
  Modify
  Write
```

Atomic

Indivisible operation.

Mnemonic:

Atomic = All at once

Critical Section

Part of program accessing/modifying shared resource.

Structure:

```
TEXT
Entry
    !"
Critical Section
    !"
Exit
    !"
Remainder
```

3 Requirements

MPB

Mutual Exclusion **P**rogress **B**ounded Waiting

Memory:

My Process Behaves

Mutual Exclusion

```
TEXT
Only one process in protected CS
```

Progress

```
TEXT
CS free + interested process
!' decision cannot be postponed forever
```

Memory:

Don't keep an empty room locked.

Bounded Waiting

```
TEXT
Requester must eventually get turn
```

Prevents starvation.

Busy Waiting

TEXT

```
while(condition)
    ;
```

Consumes CPU.

Spinlock

Lock implemented using busy waiting.

Peterson

For two processes:

TEXT

```
flag[2]
turn
```

Satisfies in classic model:

TEXT

```
Mutual Exclusion
Progress
Bounded Waiting
```

Test-and-Set

Atomic hardware operation.

Simple lock:

TEXT

```
while(TestAndSet(lock))
    ;
```

CS

```
lock = false;
```

Mutual exclusion:

,

Bounded waiting/fairness automatically:

'L Not necessarily.

MASTER COMPARISON

Concept	Meaning
Race Condition	Result depends on timing/interleaving
Atomic Operation	Indivisible operation
Critical Section	Code accessing shared resource
Mutual Exclusion	One in CS at a time
Progress	Entry decision keeps moving
Bounded Waiting	No indefinite postponement
Starvation	One process waits indefinitely
Busy Waiting	Keeps using CPU while waiting
Spinlock	Busy-waiting lock

EXAM SHORTCUTS

Shortcut 1 — Shared variable question

Write:

RMW

Read !' Modify !' Write

Then interleave.

Shortcut 2 — Critical section properties

Ask three questions:

TEXT

Can TWO enter?

!' M u t u a l E x c l u s i o n

Can system get STUCK while CS free?

!' P r o g r e s s

Can ONE wait forever?

!' B o u n d e d W a i t i n g

This is an excellent exam test.

Shortcut 3 — Waiting style

TEXT

```
while(...) checking repeatedly
!'  B U S Y   W A I T I N G

sleep/block until signal
!'  B L O C K I N G
```

30-SECOND REVISION

Concurrency?

!' Overlapping progress.

Race condition?

!' Result depends on interleaving.

Atomic?

!' Indivisible.

Critical section?

!' Shared-resource code.

Three requirements?

!' MPB

Mutual exclusion?

!' One at a time.

Progress?

!' Free CS shouldn't remain unnecessarily inaccessible.

Bounded waiting?

!' No starvation.

Spinlock?

!' Busy waiting.

Peterson?

!' 2-process software solution.

Test-and-Set?

!' Atomic hardware primitive; simple lock may starve.

ACTIVE RECALL

Try these mentally:

1. Why can `x++` cause a race condition?
2. What does atomic mean?
3. What are the four conceptual regions around a critical section?
4. State the three critical-section requirements.

5. Difference between progress and bounded waiting?
6. Which requirement prevents two simultaneous critical-section entries?
7. Which requirement is associated with preventing starvation?
8. What is busy waiting?
9. What is a spinlock?
10. Does Test-and-Set automatically guarantee bounded waiting?
11. How many processes does classic Peterson's algorithm handle?
12. If P1 gets preempted inside CS, is mutual exclusion necessarily violated?
13. Initial `x=0` ; two non-atomic increments occur. What are typical possible final values?

MASTER MEMORY MAP

TEXT

```

Concurrency
    !"
Shared Data
    !"
Read-Modify-Write
    !"
Race Condition
    !"
Critical Section
    !"
Need MPB
    !"
Mutual Exclusion
Progress
Bounded Waiting
    !"
Locks / Semaphores
  
```

Remember the chapter in one line:

"Concurrent shared RMW causes Race; protect the Critical Section using MPB."

NEXT — CHAPTER 6 Ø=Ý%Ø=Ý%

Next is one of the most important GATE OS chapters:

SYNCHRONIZATION — MUTEX & SEMAPHORES

We will cover:

- mutex locks
- semaphore definition
- binary semaphore
- counting semaphore
- `wait()` / `P()` / `down()`
- `signal()` / `V()` / `up()`
- exact meaning of semaphore values

- atomicity
- mutex vs binary semaphore
- busy-waiting vs blocking semaphore
- solving semaphore code
- finding possible output/order
- synchronization vs mutual exclusion
- common GATE traps

Then we will use semaphores to solve:

Producer–Consumer, Readers–Writers and Dining Philosophers.

USER
NEXT

ASSISTANT

CHAPTER 6 — SYNCHRONIZATION: MUTEX & SEMAPHORES 0=Ý%N2P% HIGH PRIORITY

This is one of the highest-value OS topics for GATE.

The core chain is:

TEXT
Shared Data
! "
Race Condition
! "
Critical Section
! "
Need Synchronization
! "
Mutex / Semaphore

If you understand the meaning of `wait()` and `signal()` properly, most semaphore questions become much easier.

1. WHAT IS SYNCHRONIZATION?

Synchronization means:

Coordinating concurrent processes or threads so that shared resources are accessed in a correct and controlled order.

Synchronization solves problems such as:

- two processes modifying the same variable
- producer writing into a full buffer
- consumer reading from an empty buffer

- multiple processes entering a critical section together
- one process needing to wait for another to finish some action

2. MUTUAL EXCLUSION VS SYNCHRONIZATION

These are related but not identical.

Mutual Exclusion

Ensures:

Only one process/thread accesses a protected critical section at a time.

Example:

```
TEXT
P1 in CS
!"
P2 waits
```

Synchronization

Broader concept.

Ensures:

Processes execute in the required order or under required conditions.

Example:

```
TEXT
P1 produces data
!"
P2 consumes data
```

P2 may need to wait until P1 produces something.

That is synchronization, even if the main issue is not simply "one at a time."

Shortcut

Mutex = WHO can enter

Synchronization = WHEN can it proceed

3. MUTEX LOCK

A mutex is a synchronization primitive used mainly for:

Mutual Exclusion

Conceptually:

```
TEXT
lock(mutex)

/* critical section */

unlock(mutex)
```

If another thread already owns the mutex:

```
TEXT
lock(mutex)
    ! "
wait
```

until the mutex becomes available.

4. MUTEX ANALOGY

Imagine one bathroom with one key.

```
TEXT
Key available
    ! "
Person takes key
    ! "
Uses bathroom
    ! "
Returns key
```

Only the key holder can enter.

That is mutex.

Memory:

MUTEX = MUTual EXclusion

The word itself tells you the purpose.

5. BASIC MUTEX STRUCTURE

```
C
lock(m);

/* critical section */

unlock(m);
```

If T1 acquires it:

TEXT

T 1 ! ' L O C K E D

T2 trying to acquire:

TEXT

T 2 ! ' W A I T

After T1 unlocks:

TEXT

T2 may acquire

6. OWNERSHIP — IMPORTANT MUTEX IDEA

A mutex usually has the concept of:

Ownership

The thread/process that locks the mutex is generally expected to unlock it.

Think:

TEXT

T1 locks
T1 unlocks

Not normally:

TEXT

T1 locks
T2 unlocks

This becomes important when comparing mutex with semaphores.

7. SEMAPHORE

Extremely important.

A semaphore is:

A synchronization variable accessed only through atomic operations such as `wait()` and `signal()`.

Semaphore is usually represented by an integer:

TEXT

S

Its operations must be atomic.

8. TWO BASIC SEMAPHORE OPERATIONS

Main operations:

wait(S)

Also called:

- P(S)
- down(S)

signal(S)

Also called:

- V(S)
- up(S)

For GATE, all these names may appear.

Remember:

TEXT

wait = P = down

signal = V = up

MEMORY TRICK

WAIT

Wait wants a resource

SIGNAL

Signal returns/releases a resource

9. BASIC MEANING OF A SEMAPHORE VALUE

Conceptually, if semaphore S counts available instances:

TEXT

S = number of currently available resources

Example:

There are 5 identical printers.

Initialize:

$V[S=5]$

Each process acquiring one printer:

```
TEXT
wait(S)
```

reduces availability.

Each process releasing one:

```
TEXT
signal(S)
```

increases availability.

10. COUNTING SEMAPHORE

A counting semaphore can have values representing more than one available resource.

Example:

```
TEXT
S = 3
```

Three identical resources are available.

Processes:

```
TEXT
P 1   w a i t ( S ) !'   S   =   2
P 2   w a i t ( S ) !'   S   =   1
P 3   w a i t ( S ) !'   S   =   0
```

Now no resource remains.

P4 executes:

```
TEXT
wait(S)
```

and must wait.

11. BINARY SEMAPHORE

A binary semaphore conceptually uses:

```
TEXT
0 or 1
```

Typically:

TEXT

```
1 !'  a v a i l a b l e
0 !'  u n a v a i l a b l e
```

It can be used for mutual exclusion.

Initialize:

TEXT

```
mutex = 1
```

Process:

C

```
wait(mutex);

/* critical section */

signal(mutex);
```

12. BINARY VS COUNTING SEMAPHORE

Binary Semaphore	Counting Semaphore
Usually 0/1	Integer count
One permit/resource	Multiple permits/resources
Useful for mutual exclusion	Useful for resource counting
Similar to lock concept	Represents multiple identical resources

Memory:

Binary = One door

Counting = Parking lot with N spaces

13. SEMAPHORE `wait()` — SIMPLE MODEL

A simplified conceptual version:

C

```
wait(S)
{
    while (S <= 0)
        ;

    S--;
}
```

But this version requires the check-and-decrement operation to be atomic.

Otherwise it has a race condition.

14. WHY `wait()` MUST BE ATOMIC

Suppose:

$\backslash [S=1 \backslash]$

P1 checks:

TEXT

`S > 0`

true.

Before decrementing, CPU switches.

P2 also checks:

TEXT

`S > 0`

true.

Now both decrement and both may enter.

This defeats synchronization.

Therefore:

Semaphore operations themselves must be atomic.

15. `signal()` OPERATION

Conceptually:

```
c
signal(S)
{
    S++;
}
```

In a blocking implementation, it may also wake a waiting process.

Again, this operation must be atomic.

16. BUSY-WAITING SEMAPHORE

A simple semaphore implementation may use:

TEXT

```
while(S <= 0)
    keep checking
```

This causes:

Busy waiting

CPU is consumed during the wait.

17. BLOCKING SEMAPHORE

A better approach for longer waits:

If resource unavailable:

TEXT

```
Process blocks
    !"
Placed in waiting queue
    !"
CPU given to another process
```

When `signal()` occurs:

TEXT

```
Waiting process wakes
    !"
Ready queue
```

This avoids continuously wasting CPU.

18. BLOCKING SEMAPHORE STRUCTURE

Conceptually:

C

```
wait(S)
{
    S--;

    if (S < 0)
    {
        add process to waiting queue;
        block();
    }
}
```

Then:

C

```
signal(S)
{
```

```

S++;

if (S <= 0)
{
    remove one waiting process;
    wakeup(process);
}
}

```

This is a classic textbook representation.

19. VERY IMPORTANT: NEGATIVE SEMAPHORE VALUES

In the blocking implementation above:

If:

$\forall S < 0$

then the magnitude can represent the number of waiting processes.

Example:

$\forall S = -3$

can mean:

3 processes are waiting.

This depends on the exact semaphore definition supplied in the question.

& pGATE TRAP

Do not always assume:

Semaphore can never be negative.

In some textbook blocking definitions:

$\forall S < 0$

is perfectly valid.

Always inspect the given definitions of `wait()` and `signal()`.

20. MUTUAL EXCLUSION USING SEMAPHORE

Initialize:

```

TEXT
mutex = 1

```

Each process:

```
C
wait(mutex);

/* critical section */

signal(mutex);
```

Suppose P1 executes:

```
TEXT
wait(mutex)
```

Then:

```
TEXT
m u t e x :   1   !'   0
```

P1 enters.

P2 executes `wait(mutex)`.

It cannot enter until P1 performs:

```
TEXT
signal(mutex)
```

Thus mutual exclusion is achieved.

21. WHY INITIAL VALUE = 1?

Because exactly:

one process is allowed in the critical section at a time.

Think of semaphore value as available permits.

```
TEXT
1 permit
```

P1 takes it:

```
TEXT
0 permits left
```

P2 must wait.

22. WHAT IF MUTEX SEMAPHORE IS INITIALIZED TO 2?

Then potentially:

TEXT

P1 enters
P2 enters

simultaneously.

Therefore it would not provide single-process mutual exclusion.

For a single critical section:

$\boxed{\text{mutex}=1}$

23. SEMAPHORE FOR ORDERING

Semaphore is not just for mutex.

It can enforce execution order.

Suppose:

P2 must execute statement B only after P1 executes statement A.

Use:

TEXT

S = 0

P1:

C

A;
signal(S);

P2:

C

wait(S);
B;

24. WHY INITIALIZE TO ZERO?

Initially:

TEXT

S = 0

P2 executes:

TEXT

wait(S)

and blocks.

Only after P1 executes:

```
TEXT
signal(S)
```

can P2 proceed.

Therefore:

```
TEXT
A happens before B
```

HUGE GATE SHORTCUT

Whenever you see:

“B must happen after A”

think:

```
TEXT
Semaphore S = 0

A
signal(S)

wait(S)
B
```

Memory:

0 means WAIT for an event.

25. TWO COMMON INITIAL VALUES

This is extremely useful.

Semaphore = 1

Usually think:

Mutual exclusion

```
TEXT
one permit
```

Semaphore = 0

Usually think:

Ordering / event synchronization

TEXT

nothing can proceed until signaled

Memory:

1 = One enters

0 = Wait for signal

26. EXAMPLE — ORDERING

P1:

```
C
printf("A");
signal(S);
```

P2:

```
C
wait(S);
printf("B");
```

Initial:

```
TEXT
S = 0
```

Possible output?

Because P2 cannot print B before signal:

$\boxed{AB}$

B before A is impossible.

27. EXAMPLE WITH FREE STATEMENT

P1:

```
C
printf("A");
signal(S);
printf("C");
```

P2:


```
C
wait(S);
printf("B");
```

Initial:

```
TEXT
S = 0
```

Constraints:

```
TEXT
A before signal
signal before B
A before C
```

So:

```
TEXT
A must be first
```

After signal, B and C can occur in either order.

Possible:

```
TEXT
ABC
ACB
```

Not:

```
TEXT
BAC
```

28. HOW TO SOLVE SEMAPHORE ORDERING QUESTIONS

Do not simulate blindly.

Write a dependency graph.

Example:

```
TEXT
P 1 :  A !'  s i g n a l ( S ) !'  C
P 2 :  w a i t ( S ) !'  B
```

Since `wait(S)` needs signal:

```
TEXT
A !'  s i g n a l !'  w a i t !'  B
```

Therefore:

TEXT

A !' B

Within P1:

TEXT

A !' C

So:

TEXT

A before B
A before C

But B and C may swap.

This dependency method is much faster.

29. SEMAPHORE AS RESOURCE COUNTER

Suppose:

TEXT

S = 4

This can model 4 identical resources.

Every:

TEXT

wait(S)

takes one.

Every:

TEXT

signal(S)

returns one.

If 4 processes have acquired:

TEXT

S = 0

the fifth waits.

30. EXAMPLE — PRINTERS

There are 3 printers.

Initialize:

$\backslash$ [printer=3 $\backslash$]

Each process:

```
c
wait(printer);

/* use printer */

signal(printer);
```

Maximum processes simultaneously using printers:

$\backslash$ $\boxed{3}$ $\backslash$]

31. MUTEX VS BINARY SEMAPHORE

This is a common conceptual question.

They may look similar but are not exactly identical.

Mutex

Usually:

- ownership exists
- locker should unlock
- primarily mutual exclusion

Binary Semaphore

- signaling can be performed by another thread/process
- often used for synchronization as well as mutual exclusion
- conceptually 0/1

Comparison

Mutex	Binary Semaphore
Has ownership concept	Usually no ownership concept
Same owner generally unlocks	Another entity may signal
Mainly mutual exclusion	Mutual exclusion or event signaling
Lock/unlock	Wait/signal

Memory:

Mutex has an owner. Semaphore has a count/signal.

32. WHY OWNERSHIP MATTERS

Suppose T1 waits for work from T2.

A binary semaphore can do:

TEXT

T1: wait(S)

T2: signal(S)

This is perfectly natural.

T2 can wake T1.

A mutex is not designed primarily as an inter-thread notification primitive like this.

33. wait() AND signal() DO NOT MEAN NORMAL FUNCTIONS

Important.

wait(S) means roughly:

Request/decrement semaphore; block if permission unavailable.

signal(S) means roughly:

Release/increment semaphore; possibly wake waiter.

Do not confuse semaphore wait() with Unix parent-process wait() from the process chapter.

Same English word, different concept/context.

& pGATE TRAP

There are two different wait ideas:

Process wait()

Parent waits for child termination.

Semaphore wait(S)

Synchronization operation.

Context tells you which one is meant.

34. SEMAPHORE OPERATION NAMES

You may encounter:

TEXT

P(S)
V(S)

from Dijkstra terminology.

Meaning:

TEXT

P(S) = wait(S)
V(S) = signal(S)

Also:

TEXT

down(S) = wait(S)
up(S) = signal(S)

Write this in your notes.

35. SEMAPHORE TRACE — BASIC

Initial:

$\boxed{S=2}$

Sequence:

TEXT

wait(S)
wait(S)
signal(S)
wait(S)

Assume simple count semantics and no blocking occurs.

Trace:

TEXT

Start S = 2

w	a	i	t	!	'	1
w	a	i	t	!	'	0
s	i	g	n	a	l	!
						1
w	a	i	t	!	'	0

Final:

$\boxed{0}$

36. SEMAPHORE TRACE WITH BLOCKING

Initial:

$\boxed{S=1}$

P1:

```
TEXT
wait(S)
```

Now:

```
TEXT
S = 0
```

P1 uses resource.

P2:

```
TEXT
wait(S)
```

P2 waits.

P1:

```
TEXT
signal(S)
```

P2 becomes eligible to continue.

Remember:

```
TEXT
W a k e u p   ! '   R E A D Y
```

not automatically running.

37. GATE CONNECTION TO PROCESS STATES

Semaphore blocking:

```
TEXT
RUNNING
    ! "
wait(S) when unavailable
    ! "
WAITING
```

When signaled:

```
TEXT
WAITING
    ! "
READY
```

Then scheduler decides when it runs.

38. SEMAPHORE AND BUSY WAITING

Question may ask:

Does semaphore always avoid busy waiting?

Answer:

No.

Depends on implementation.

Spin semaphore

TEXT

wait loops

!' busy waiting.

Blocking semaphore

TEXT

process sleeps

!' no busy waiting while blocked.

39. SPINLOCK VS BLOCKING MUTEX

Spinlock	Blocking Lock
Busy waits	Sleeps/blocks
Consumes CPU	Gives CPU away
Better for very short waits	Better for longer waits
Common in kernel/multiprocessor contexts	Common for ordinary long waits

40. SEMAPHORE SYNCHRONIZATION EXAMPLE

We want:

TEXT

S1 in P1

before:

TEXT

S2 in P2

Use:

```
TEXT
sem = 0
```

P1:

```
C
S1;
signal(sem);
```

P2:

```
C
wait(sem);
S2;
```

Guarantee:

$\neg [S1 \rightarrow S2]$

41. MULTIPLE ORDERING CONSTRAINTS

Suppose:

P1:

```
TEXT
A
signal(S1)
```

P2:

```
TEXT
wait(S1)
B
signal(S2)
```

P3:

```
TEXT
wait(S2)
C
```

Then:

```
TEXT
A !' B !' C
```

A very common synchronization pattern.

42. BARRIER IDEA

Suppose two processes must both finish Phase 1 before either begins Phase 2.

Conceptually:

TEXT

```
P 1   P h a s e 1   % %  
                % !'   B a r r i e r   !'   P h a s e 2  
P 2   P h a s e 1   % %
```

This is called a:

Barrier

Semaphores can be used to construct barriers.

For GATE, understand the concept.

43. DEADLOCK WITH SEMAPHORES — PREVIEW

Synchronization primitives can themselves cause deadlock if used incorrectly.

Example:

P1:

```
C  
wait(S);  
wait(T);
```

P2:

```
C  
wait(T);  
wait(S);
```

Suppose:

TEXT

```
P1 acquires S  
P2 acquires T
```

Then:

TEXT

```
P1 waits for T  
P2 waits for S
```

Neither proceeds.

This is deadlock.

We will study it fully later.

44. SELF-DEADLOCK EXAMPLE

Suppose a non-recursive mutex is acquired twice by the same thread:

```
C
lock(m);
lock(m);
```

Second lock may wait forever for a lock that the same thread already holds.

This can cause self-deadlock depending on mutex semantics.

Important idea:

Locks must be acquired/released carefully.

45. SEMAPHORE CODING PATTERN— MUTEX

Initialize:

```
TEXT
mutex = 1
```

Then:

```
C
wait(mutex);

/* critical section */

signal(mutex);
```

Shortcut:

1 = protection

46. SEMAPHORE CODING PATTERN— EVENT ORDER

Initialize:

```
TEXT
event = 0
```

Then:

```
C
P1:
event_happens;
signal(event);
```

```
P2:
wait(event);
continue;
```

Shortcut:

0 = dependency

47. SEMAPHORE CODING PATTERN— N IDENTICAL RESOURCES

Initialize:

$\forall S=N$

Then:

```
c
wait(S);
/* use one resource */
signal(S);
```

Shortcut:

N = maximum concurrent users

48. EXAM QUESTION — INITIAL VALUE

Suppose at most 5 processes may enter a section simultaneously.

Semaphore initial value should be:

$\forall \boxed{5}$

because there are 5 permits.

49. EXAM QUESTION — STRICT MUTUAL EXCLUSION

At most one process may enter.

Initialize:

$\forall \boxed{1}$

50. EXAM QUESTION — P2 AFTER P1

Need:

TEXT

P1 event before P2

Initialize:

$\boxed{0}$

because P2 must initially block.

51. SEMAPHORE VALUES AND WAITING PROCESSES

Suppose blocking definition:

```
c
wait(S):
    S = S - 1
    if S < 0:
        block
```

If:

$S = -4$

then typically:

$\boxed{4}$

processes are waiting.

Again, only if the question uses this definition.

52. EXAMPLE

Initial:

$S = 2$

Five processes each perform `wait(S)` and none has yet called signal.

Using blocking semantics:

TEXT

```
P 1 : S 2! ' 1
P 2 : S 1! ' 0
P 3 : S 0! ' - 1
P 4 : S - 1! ' - 2
P 5 : S - 2! ' - 3
```

So:

$S = -3$

and:

$\boxed{3 \text{ processes waiting}}$

53. IMPORTANT CONDITION

Do not interpret every negative semaphore exactly this way unless the question defines:

TEXT

```
wait:
S--
if S < 0 block
```

Some models maintain nonnegative counters separately from wait queues.

Always read the algorithm.

54. SEMAPHORE QUESTION — POSSIBLE OUTPUT

Initial:

TEXT

```
S = 0
```

P1:

C

```
printf("A");
signal(S);
printf("B");
```

P2:

C

```
wait(S);
printf("C");
```

Constraints:

TEXT

```
A before B
A before C
```

Possible:

TEXT

```
ABC
ACB
```

Impossible:

TEXT

```
CAB
```

because C needs signal after A.

55. DEPENDENCY GRAPH METHOD

Convert the program into arrows.

From P1:

TEXT

```
A !' s i g n a l !' B
```

From semaphore:

TEXT

```
s i g n a l !' s u c c e s s f u l w a i t !' C
```

Therefore:

TEXT

```
A !' B
A !' C
```

No fixed relation:

TEXT

```
B ? C
```

So they may appear either way.

This is one of the best GATE shortcuts.

56. MULTIPLE SIGNALS

Suppose semaphore initially:

$\backslash [S=0]$

Then two `signal(S)` operations occur before a wait.

Conceptually:

TEXT

```
S = 2
```

Two future waits can pass without blocking.

This is why semaphore signals are remembered via the count.

57. MUTEX DOES NOT "COUNT EVENTS" THE SAME WAY

A mutex represents lock ownership/state.

A counting semaphore can accumulate permits.

For event counting:

TEXT

```
signal  
signal
```

can create multiple permits.

This is one reason semaphores are more general.

58. LOST WAKEUP IDEA

Without proper synchronization, a notification can occur just before another process begins waiting and may be missed in naive designs.

Semaphore counts help avoid this in many patterns because:

TEXT

```
signal(S)
```

can increment the count even if no process is currently waiting.

Then a future `wait(S)` can consume that permit.

Useful conceptual advantage.

59. CRITICAL SECTION WITH WRONG ORDER

Correct:

```
C  
wait(mutex);  
critical_section();  
signal(mutex);
```

Wrong:

```
C  
signal(mutex);  
critical_section();  
wait(mutex);
```

because you release permission before protecting the section.

Simple but sometimes used in statement-based questions.

60. WHAT IF `signal()` IS FORGOTTEN?

Example:

```
C  
wait(mutex);  
  
/* critical section */
```

```
/* signal missing */
```

Process leaves but never releases permit.

Other processes may block indefinitely.

This can result in:

- deadlock-like behavior
- resource permanently unavailable

61. WHAT IF `signal()` OCCURS TWICE?

With a counting semaphore:

```
TEXT
signal
signal
```

may incorrectly increase the resource count.

If used as a binary-style mutex, bad signaling can allow multiple processes in.

Therefore semaphore correctness depends on disciplined usage.

62. SEMAPHORES DO NOT AUTOMATICALLY PREVENT ALL BUGS

A semaphore is only a tool.

Bad ordering can cause:

- race conditions
- deadlocks
- starvation
- incorrect resource counts

So always analyze the code.

63. SEMAPHORE VS TEST-AND-SET

Semaphore	Test-and-Set
Higher-level synchronization primitive	Atomic hardware primitive
Can block or spin	Often used to build spinlocks
Supports counting	Usually lock-state operation
Can enforce ordering	Primarily helps lock implementation

64. SEMAPHORE VS SPINLOCK

Semaphore	Spinlock
Can block	Busy waits
Can count resources	Usually binary lock state
Suitable for longer waits	Suitable for very short waits
General synchronization	Mainly exclusion

65. COMMON GATE PATTERN— RESOURCE COUNT

Suppose:

TEXT

Semaphore S = 8

Every process performs:

TEXT

wait(S)

before entering and `signal(S)` after exiting.

Maximum processes inside simultaneously:

`\boxed{8}`

Very easy marks if you recognize:

Initial count = permits

66. COMMON GATE PATTERN— WAITING COUNT

Initial:

`\[ S=3 ]`

Six processes perform `wait(S)` and none performs `signal()`.

Under negative-count blocking semantics:

TEXT

```
a f t e r   P 1   !'   2
P 2   !'   1
P 3   !'   0
P 4   !'  - 1
P 5   !'  - 2
P 6   !'  - 3
```

Waiting:

`\boxed{3}`

67. COMMON GATE PATTERN— ORDERING

Need:

TEXT

A and B must finish before C

One possible idea:

Use two semaphores:

TEXT

S1=0

S2=0

P1:

C

A;
signal(S1);

P2:

C

B;
signal(S2);

P3:

C

wait(S1);
wait(S2);
C;

Then C runs only after both A and B.

68. COMMON GATE PATTERN— MUTUAL EXCLUSION + CONDITION

Often real synchronization needs two things:

1. protect shared variable
2. wait for some condition

This is why producer-consumer may use:

TEXT

mutex
empty
full

We'll cover this next chapter.

69. HOW GATE CAN ASK SEMAPHORES

Pattern 1

Find correct initial semaphore value.

Pattern 2

Find number of waiting processes.

Pattern 3

Determine possible output orders.

Pattern 4

Identify deadlock.

Pattern 5

Find maximum processes in critical region.

Pattern 6

Compare mutex and semaphore.

Pattern 7

Analyze `wait()` / `signal()` code.

Pattern 8

Determine whether synchronization or mutual exclusion is achieved.

& pGATE TRAPS — SEMAPHORES

Trap 1

`wait()` means always sleep.

'L

It may proceed immediately if permit available.

Trap 2

`signal()` means send a message.

'L

In semaphore terminology, it increments/releases a permit and may wake a waiter.

Trap 3

Semaphore initialized to zero can never be useful.

'L

It is excellent for ordering/event synchronization.

Trap 4

Semaphore initialized to one is always a mutex.

Not automatically.

It can behave as a binary semaphore; mutex has ownership semantics.

Trap 5

Semaphore value can never be negative.

'L Depending on definition.

Trap 6

All semaphore implementations busy-wait.

'L

Blocking implementations exist.

Trap 7

Binary semaphore and mutex are identical in every sense.

'L

Ownership differs.

Trap 8

`signal(S)` must always be called by the same process that performed `wait(S)`.

'L for general semaphores.

That restriction is more associated with mutex ownership.

Trap 9

Mutual exclusion is the only use of semaphore.

'L

Semaphores also enforce ordering and resource counting.

Trap 10

A process woken by `signal()` immediately starts running.

'L

Typically:

TEXT

W A I T I N G ! ' R E A D Y

Scheduler decides when it runs.

' p WRITE THIS IN YOUR NOTES

Semaphore

Integer synchronization variable manipulated only using atomic `wait()` and `signal()` operations.

Aliases:

TEXT

`wait = P = down`
`signal = V = up`

Types

Binary Semaphore

TEXT

0 / 1

Useful for:

- mutual exclusion
- event signaling

Counting Semaphore

TEXT

0...N

Counts multiple identical resources.

Important Initial Values

TEXT

`S = 1`
`! '   o n e   p e r m i t`
`! '   m u t u a l   e x c l u s i o n`

TEXT

```
S = 0
!' initial blocking
!' event / order synchronization
```

TEXT

```
S = N
!' N identical resources
```

Memory:

1 = One enters

0 = Wait for event

N = N permits

Mutex vs Semaphore

TEXT

```
Mutex !' ownership
Semaphore !' count / signaling
```

Blocking Semaphore

TEXT

```
wait:
S--
if S < 0 !' block

signal:
S++
if S <= 0 !' wake one
```

If using this definition:

TEXT

```
S = -k
!' k waiting processes
```

MASTER SEMAPHORE SHORTCUT SHEET

Shortcut 1 — Initial value

Ask:

How many permits initially exist?

TEXT

```
One !' 1
None until event !' 0
N resources !' N
```

Shortcut 2 — Order question

Convert:

TEXT

```
signal !' wait
```

into:

TEXT

```
Before !' After
```

Draw arrows.

Shortcut 3 — Waiting-process question

If definition is:

TEXT

```
S--
if S<0 block
```

then:

TEXT

```
negative magnitude = waiters
```

Shortcut 4 — Mutex

TEXT

```
wait
CS
signal
```

Always look for this sandwich.

MASTER COMPARISON TABLE

Primitive	Main Purpose	Busy Wait?	Count?	Ownership?
Mutex	Mutual exclusion	Depends	No	Yes
Binary Semaphore	Exclusion/signaling	Depends	0/1	Usually no
Counting Semaphore	Resource counting	Depends	Yes	No
Spinlock	Mutual exclusion	Yes	Usually no	Lock holder
Test-and-Set	Atomic primitive	Used in spins	No	No direct ownership semantics

CONCEPT CONNECTION

TEXT

Race Condition
!"
Critical Section
!"
Need Mutual Exclusion
!"
Mutex / Binary Semaphore

But:

TEXT

Process ordering
!"
Need Synchronization
!"
Semaphore initialized 0

And:

TEXT

N identical resources
!"
Counting Semaphore = N

So remember:

Semaphore has 3 major uses

M O R

Mutual exclusion Ordering Resource counting

Memory sentence:

Semaphore maintains MOR.

30-SECOND REVISION

Semaphore?

!' Synchronization variable using atomic wait/signal.

wait?

!' Request/decrement, block if unavailable.

signal?

!' Release/increment, possibly wake waiter.

Binary semaphore?

!' 0/1.

Counting semaphore?

!' Multiple resource permits.

Mutex initial value?

!' 1.

Ordering semaphore initial value?

!' 0.

N resources?

!' N.

Mutex vs binary semaphore?

!' Mutex has ownership.

Blocked process gets signal?

!' Waiting !' Ready.

Negative semaphore?

!' May represent waiters under textbook blocking semantics.

ACTIVE RECALL

Try without looking:

1. What is the difference between synchronization and mutual exclusion?
2. What does `wait(S)` do?
3. What does `signal(S)` do?
4. Why must semaphore operations be atomic?
5. Why is mutex semaphore usually initialized to 1?
6. Why is ordering semaphore usually initialized to 0?
7. If there are 7 identical resources, what should the semaphore initially be?
8. What is the difference between mutex and binary semaphore?
9. Can one process signal a semaphore that another process waited on?
10. If `S = -4` under classic negative-count semantics, what does it indicate?

11. Does a woken process immediately run?
12. Why can incorrect semaphore ordering cause deadlock?

MASTER MEMORY MAP

TEXT

Synchronization

!"

% % % % % % % % % % % % % % % %

!"

Mutex

Semaphore

!"

!"

Ownership

wait/signal

!"

% % % % % % % % % % % % % % % %

!"

!"

!"

S=1

S=0

S=N

!"

!"

!"

Mutex

Order

Resources

One-line memory:

"1 protects, 0 orders, N counts."

NEXT — CHAPTER 7 Ø=Ý%Ø=Ý%Ø=Ý%

Next we use semaphores to solve the **classical synchronization problems**, which are extremely important for GATE:

PRODUCER-CONSUMER, READERS-WRITERS & DINING PHILOSOPHERS

We will cover:

- bounded-buffer producer-consumer
- `mutex`, `empty`, `full`
- why their initial values are `1`, `N`, `0`
- exact order of `wait()` and `signal()`
- what goes wrong if order changes
- reader count
- multiple readers vs single writer
- reader-priority / writer-starvation idea
- dining philosophers
- deadlock in dining philosophers
- methods to avoid deadlock
- GATE semaphore tracing shortcuts

USER

NEXT

CHAPTER 7 — CLASSICAL SYNCHRONIZATION PROBLEMS Ø=Ý%Ø=Ý%Ø=Ý%

This chapter applies semaphores to three classic problems:

1. Producer–Consumer
2. Readers–Writers
3. Dining Philosophers

These are important because GATE may ask you to analyze semaphore values, execution order, deadlock, starvation, or correctness.

PART A — PRODUCER–CONSUMER PROBLEM

1. Basic idea

We have:

TEXT

```
P r o d u c e r   ! '   B u f f e r   ! '   C o n s u m e r
```

The producer creates items.

The consumer removes items.

If the buffer has limited size `N`, we call it a:

Bounded Buffer

Example:

TEXT

```
Buffer size = 4
```

```
[ A ] [ B ] [   ] [   ]
```

Producer inserts.

Consumer removes.

2. What problems must be solved?

Three things must be controlled.

Problem 1 — Producer must not insert into a full buffer

TEXT

```
FULL BUFFER
[A][B][C][D]
```

```
Producer must WAIT
```

Problem 2 — Consumer must not remove from an empty buffer

TEXT

```
EMPTY BUFFER  
[ ][ ][ ][ ]
```

Consumer must WAIT

Problem 3 — Producer and consumer must not corrupt the buffer structure

If both modify shared buffer/index variables simultaneously:

Race condition.

So we need mutual exclusion too.

3. THREE SEMAPHORES

For buffer size:

$\backslash N \backslash$

use:

TEXT

```
mutex = 1  
empty = N  
full = 0
```

Remember this.

4. WHY THESE VALUES?

mutex = 1

Only one process should modify the critical buffer structure at a time.

So:

$\backslash \boxed{\text{mutex}=1} \backslash$

empty = N

Initially all N buffer slots are empty.

So:

$\backslash \boxed{\text{empty}=N} \backslash$

full = 0

Initially there are no produced items.

So:

$\boxed{\text{full}=0}$

MASTER MEMORY TRICK

M E F = 1, N, 0

Mutex = 1 Empty = N Full = 0

Remember:

“One lock, N empty, zero full.”

5. PRODUCER CODE

Classic form:

```
c
while (true)
{
    produce_item();

    wait(empty);
    wait(mutex);

    insert_item();

    signal(mutex);
    signal(full);
}
```

Understand every line.

6. PRODUCER STEP BY STEP

Step 1

TEXT
produce item

This can usually happen outside the critical section.

Step 2

TEXT
wait(empty)

Ask:

Is there at least one free buffer slot?

If no:

TEXT

Producer blocks

Step 3

TEXT

wait(mutex)

Acquire exclusive access to buffer structure.

Step 4

TEXT

insert item

Critical section.

Step 5

TEXT

signal(mutex)

Release buffer lock.

Step 6

TEXT

signal(full)

Announce:

One more full slot now exists.

7. CONSUMER CODE

```
c
while (true)
{
    wait(full);
    wait(mutex);

    remove_item();
```

```
    signal(mutex);
    signal(empty);

    consume_item();
}
```

8. CONSUMER STEP BY STEP

wait(full)

Need an available produced item.

If:

```
TEXT
full = 0
```

consumer waits.

wait(mutex)

Acquire exclusive access to shared buffer.

Remove item

Critical section.

signal(mutex)

Release lock.

signal(empty)

One new free slot now exists.

9. WHY **wait(empty)** BEFORE **wait(mutex)** ?

Very important.

Suppose producer does this wrong:

```
C
wait(mutex);
wait(empty);
```

Now imagine buffer is full.

Producer:

TEXT

```
acquires mutex
!"
waits for empty
```

But who can create an empty slot?

Consumer.

Consumer needs:

TEXT

```
mutex
```

to remove an item.

But producer is holding it.

So:

TEXT

```
Producer waits for empty
Consumer waits for mutex
```

Deadlock.

Therefore correct order:

TEXT

```
Producer:
wait(empty)
wait(mutex)
```

Similarly consumer:

TEXT

```
wait(full)
wait(mutex)
```

First check resource availability.

Then take mutex.

& **PHUGE GATE TRAP**

Wrong:

TEXT

```
wait(mutex);
wait(empty);
```

or

TEXT

```
wait(mutex);  
wait(full);
```

can cause deadlock.

Memory:

Count first, lock second.

10. WHY SIGNAL MUTEX BEFORE FULL/EMPTY?

Typical producer:

TEXT

```
signal(mutex);  
signal(full);
```

Typical consumer:

TEXT

```
signal(mutex);  
signal(empty);
```

This releases the critical section before notifying other processes.

That keeps the critical section short and avoids unnecessary blocking.

11. BUFFER INVARIANT

At all times:

$\boxed{\text{empty} + \text{full} = N}$

assuming no operation is midway through changing conceptual counts.

Example:

Buffer size:

$N=5$

If:

TEXT

```
full = 2
```

then:

$\text{empty} = 3$

because:

$2+3=5$

This is a useful GATE shortcut.

12. EXAMPLE

Buffer size:

\[N=4 \]

Initially:

TEXT

```
empty = 4
full = 0
mutex = 1
```

Producer inserts one item.

After successful operation:

TEXT

```
empty = 3
full = 1
mutex = 1
```

Producer inserts another:

TEXT

```
empty = 2
full = 2
```

Consumer removes one:

TEXT

```
empty = 3
full = 1
```

13. PRODUCER-CONSUMER GATE SHORTCUT

Whenever you see bounded buffer, immediately write:

TEXT

```
mutex = 1
empty = N
full = 0
```

Then remember:

TEXT

```
Producer:
wait(empty)
wait(mutex)
INSERT
signal(mutex)
```

```
signal(full)
```

TEXT

```
Consumer :  
wait(full)  
wait(mutex)  
REMOVE  
signal(mutex)  
signal(empty)
```

PART B — READERS–WRITERS PROBLEM Ø=Ý%

Now suppose a shared database/file exists.

Two types of processes:

TEXT

```
Readers  
Writers
```

14. BASIC REQUIREMENT

Readers only read.

So multiple readers may safely read simultaneously.

But a writer modifies data.

Therefore:

TEXT

```
R e a d e r   +   R e a d e r   ! '   A l l o w e d  
W r i t e r   +   W r i t e r   ! '   N o t   a l l o w e d  
R e a d e r   +   W r i t e r   ! '   N o t   a l l o w e d
```

This is the heart of the problem.

15. MASTER RULE

Many readers OR one writer.

Never both.

Memory:

R can group. W must be alone.

16. WHY MULTIPLE READERS CAN ENTER?

If no one modifies the data:

TEXT

R1 reads
R2 reads
R3 reads

simultaneously, the shared data remains unchanged.

So concurrency is allowed.

17. WHY WRITER NEEDS EXCLUSIVE ACCESS?

Suppose writer changes:

TEXT

b a l a n c e = 1 0 0 ! ' 2 0 0

while reader reads halfway through.

Reader may observe inconsistent state.

So while writer is active:

TEXT

No reader
No other writer

18. COMMON READER-PRIORITY SOLUTION

Use:

TEXT

read_count = 0

mutex = 1

wrt = 1

mutex protects read_count .

wrt protects the shared resource from writers/readers conflict.

19. READER CODE

C

```
wait(mutex);  
read_count++;
```

```
if (read_count == 1)  
    wait(wrt);
```

```
signal(mutex);

/* READING */

wait(mutex);
read_count--;

if (read_count == 0)
    signal(wrt);

signal(mutex);
```

Let's understand it.

20. FIRST READER

Suppose:

```
TEXT
read_count = 0
```

R1 enters:

```
TEXT
read_count = 1
```

Because it is the first reader:

```
TEXT
wait(wrt)
```

This blocks writers.

So:

First reader locks writers out.

21. SECOND READER

R2 enters:

```
TEXT
read_count = 2
```

Does it execute:

```
TEXT
wait(wrt)
```

again?

No.

Because:

```
TEXT
read_count != 1
```

So it joins R1.

Thus multiple readers can read concurrently.

22. LAST READER

Suppose there were 3 readers.

They leave one by one:

```
TEXT
3  !'  2  !'  1  !'  0
```

When:

```
TEXT
read_count == 0
```

the last reader executes:

```
TEXT
signal(wrt)
```

Now a writer may enter.

Memory:

First reader locks. Last reader unlocks.

Ø=important shortcut.

23. WRITER CODE

Very simple:

```
C
wait(wrt);

/* WRITING */

signal(wrt);
```

Only one writer can own `wrt`.

Readers collectively also hold writers out via the first reader.

24. WHY MUTEX PROTECTS `read_count` ?

Suppose R1 and R2 both execute:

```
TEXT
read_count++
```

simultaneously.

That itself can have a race condition.

Therefore:

```
TEXT
read_count
```

must be protected.

Hence:

```
TEXT
mutex
```

25. READER-PRIORITY PROBLEM

In the classic reader-preference solution, new readers may continue entering while writers are waiting.

Example:

```
TEXT
R1 reading
Writer W waiting
R 2  a r r i v e s  !'   j o i n s
R 3  a r r i v e s  !'   j o i n s
R 4  a r r i v e s  !'   j o i n s
...
```

Writer may wait indefinitely.

This is:

Writer Starvation

26. STARVATIONVS DEADLOCK HERE

If writer is waiting but readers continue:

```
TEXT
System still making progress
```

So not deadlock.

Writer alone is postponed indefinitely.

That's:

Starvation

27. WRITER-PRIORITY VARIANT

There are other solutions that prevent new readers from entering when writers are waiting.

These can reduce writer starvation but may instead create reader starvation if unfairly designed.

For GATE, recognize:

- reader-preference
- writer-preference
- fair/no-starvation variants

28. READERS–WRITERS MEMORY MAP

TEXT

Readers:
many allowed together

Writer:
must be alone

And:

TEXT

```
F i r s t   r e a d e r   ! '   w a i t ( w r t )  
  
L a s t   r e a d e r   ! '   s i g n a l ( w r t )
```

That alone solves many conceptual questions.

PART C — DINING PHILOSOPHERS Ø=Ý%

Now the classic deadlock problem.

Suppose 5 philosophers sit around a circular table.

Between each pair is one fork/chopstick.

TEXT

```
      P1  
F1    F2  
  
P5      P2  
F5      F3  
  
P4 --F4-- P3
```


Each philosopher needs:

Two forks to eat.

29. PHILOSOPHER STATES

Each philosopher alternates between:

```
TEXT
THINKING
    ! "
HUNGRY
    ! "
EATING
```

To eat:

```
TEXT
Acquire left fork
Acquire right fork
```

30. SIMPLE SEMAPHORE MODEL

For 5 forks:

```
TEXT
fork[0] = 1
fork[1] = 1
...
fork[4] = 1
```

Each fork is a binary semaphore.

Philosopher `i`:

```
C
wait(left);
wait(right);

/* eat */

signal(right);
signal(left);
```

Looks reasonable.

But it can deadlock.

31. HOW DEADLOCK OCCURS

Suppose all 5 philosophers simultaneously pick up their left fork.

Now:

TEXT

P1 holds left, wants right
P2 holds left, wants right
P3 holds left, wants right
P4 holds left, wants right
P5 holds left, wants right

Each right fork is held by neighbor.

So:

TEXT

P1 waits for P2
P2 waits for P3
P3 waits for P4
P4 waits for P5
P5 waits for P1

Nobody can proceed.

Deadlock.

32. CIRCULAR WAIT

This forms:

TEXT

P 1 !' P 2 !' P 3 !' P 4 !' P 5 !' P 1

This is the famous:

Circular Wait

One of the four deadlock conditions we'll study next.

33. DINING PHILOSOPHERS MEMORY

Everyone takes one !' nobody gets two.

That's the deadlock.

34. SOLUTION 1 — ALLOW AT MOST N" PHILOSOPHERS TO COMPETE

For 5 philosophers, permit at most:

$\lfloor 4 \rfloor$

to attempt fork acquisition simultaneously.

Use semaphore:

TEXT

```
room = 4
```

Then:

```
c
wait(room);

wait(left);
wait(right);

/* eat */

signal(right);
signal(left);

signal(room);
```

Why?

Because if only 4 philosophers compete for 5 forks, at least one philosopher can eventually obtain both.

This breaks the all-hold-one circular deadlock scenario.

35. GENERAL RULE

For:

$\lfloor N \rfloor$

philosophers, allowing at most:

$\lfloor \boxed{N-1} \rfloor$

to try acquiring forks at once can prevent the classic deadlock.

36. SOLUTION 2 — RESOURCE ORDERING

Assign numbers to forks:

TEXT

```
F1 < F2 < F3 < F4 < F5
```

Require every philosopher to acquire resources in the same global order.

Example:

Always acquire lower-numbered fork first.

This removes circular wait.

Because everyone follows the same ordering.

37. SOLUTION 3 — ASYMMETRY

For example:

TEXT

```
Odd philosophers:  
left then right
```

```
Even philosophers:  
right then left
```

Now everyone does not acquire in identical circular order.

This can break circular wait.

38. SOLUTION 4 — ATOMICALLY ACQUIRE BOTH

Allow philosopher to eat only if both forks can be acquired together.

Conceptually:

TEXT

```
if both available:  
    acquire both  
else:  
    wait
```

Then no philosopher holds one fork while waiting indefinitely for the other.

39. DEADLOCK-FREE DOES NOT ALWAYS MEAN STARVATION-FREE

Very important.

A solution may prevent deadlock but still allow one philosopher to be repeatedly denied forks.

So:

TEXT

No deadlock

does not automatically mean:

TEXT

No starvation

40. CLASSICAL PROBLEMS COMPARISON

Problem	Main Issue	Main Semaphores/Idea
Producer–Consumer	Buffer full/empty + race	empty, full, mutex
Readers–Writers	Many readers vs exclusive writer	read_count, mutex, wrt

Problem	Main Issue	Main Semaphores/Idea
Dining Philosophers	Circular resource waiting	Fork semaphores + deadlock prevention

PRODUCER-CONSUMER FORMULA BOX

For buffer size:

$\boxed{N}$

Initial:

$\boxed{\text{mutex}=1}$

$\boxed{\text{empty}=N}$

$\boxed{\text{full}=0}$

And:

$\boxed{\text{empty}+\text{full}=N}$

under stable states.

41. GATE-STYLE EXAMPLE — PRODUCER CONSUMER

Buffer size:

$N=10$

There are currently 6 items.

Then:

$\text{full}=6$

and:

$\text{empty}=10-6=4$

So:

$\boxed{\text{empty}=4, \text{full}=6}$

42. GATE-STYLE EXAMPLE — FULL BUFFER

If:

TEXT

`empty = 0`

what happens when producer executes:

TEXT

`wait(empty)`

Producer blocks.

Because no empty slot exists.

43. GATE-STYLE EXAMPLE — EMPTY BUFFER

If:

```
TEXT
full = 0
```

consumer executes:

```
TEXT
wait(full)
```

Consumer blocks.

44. GATE-STYLE EXAMPLE — READER COUNT

Suppose 5 readers are currently reading.

Then:

```
TEXT
read_count = 5
```

The resource semaphore `wrt` is unavailable to writers.

A sixth reader may be allowed in under reader-preference.

A writer must wait.

45. GATE-STYLE EXAMPLE — FIRST READER

When:

```
TEXT
read_count : 0 !' 1
```

what important operation occurs?

$\boxed{\text{wait}(\text{wrt})}$

The first reader blocks writers.

46. GATE-STYLE EXAMPLE — LAST READER

When:

TEXT

```
read_count : 1 !' 0
```

what happens?

$\boxed{\text{signal(wrt)}}$

Writer may proceed.

47. GATE-STYLE EXAMPLE — DINING PHILOSOPHERS

Five philosophers each acquire left fork first.

All five successfully acquire one fork.

What can happen next?

Each waits for right fork.

Therefore:

$\boxed{\text{Deadlock}}$

48. WHY PRODUCER-CONSUMER NEEDS BOTH MUTEX AND COUNTING SEMAPHORES

A common confusion:

Why not just use mutex ?

Because mutex only prevents simultaneous buffer modification.

It does **not** tell you:

TEXT

```
Is buffer full?  
Is buffer empty?
```

Hence:

TEXT

```
mutex !' protects critical section  
empty / full !' track buffer state
```

49. WHY **empty** AND **full** ARE COUNTING SEMAPHORES

If buffer has many slots, we need counts.

Example:

TEXT

```
empty = 7
```

means 7 free slots.

TEXT

```
full = 3
```

means 3 occupied slots.

Binary semaphore cannot represent all those counts directly.

50. WHY `mutex` IS BINARY

Only one process should modify shared buffer metadata at a time.

So:

TEXT

```
mutex = 0 or 1
```

One permit only.

51. PRODUCER-CONSUMER DEADLOCK TRACE

Wrong producer:

C

```
wait(mutex);  
wait(empty);
```

Suppose:

TEXT

```
buffer full  
empty = 0
```

Producer acquires mutex:

TEXT

```
mutex = 0
```

Then blocks on `empty`.

Consumer tries:

TEXT

```
wait(full)
```

succeeds.

Then:

TEXT

```
wait(mutex)
```

blocks because producer holds mutex.

Now:

TEXT

```
Producer waits for consumer to free slot
```

```
Consumer waits for producer to release mutex
```

Deadlock.

This is a very important reasoning pattern.

52. EXAM SHORTCUT — ACQUIRE ORDER

For producer-consumer:

Availability before Exclusivity

Meaning:

TEXT

```
wait(empty/full)
then
wait(mutex)
```

Memory:

Check seat first, lock room second.

53. RELEASE ORDER

Typical:

TEXT

```
signal(mutex)
then
signal(full/empty)
```

Memory:

Unlock room, then announce availability.

54. READERS-WRITERS EXAM SHORTCUT

Think:

TEXT

First reader = closes gate to writers

Middle readers = simply join

Last reader = opens gate to writers

That's the whole core mechanism.

55. DINING PHILOSOPHERS EXAM SHORTCUT

If everyone:

TEXT

holds one

waits for one

think immediately:

Circular wait !' possible deadlock

Then ask:

Which solution breaks circular wait?

Examples:

- N" admission
- ordered resource acquisition
- asymmetry
- acquire both together

& pGATE TRAPS — CLASSICAL PROBLEMS

Trap 1

Producer-consumer needs only mutex.

'L

Need buffer-count synchronization too.

Trap 2

empty=0 initially.

'L

For an initially empty buffer:

\[empty=N \]

\[full=0 \]

Trap 3

`full=N` initially.

'L

Initially no produced items.

Trap 4

Acquire mutex before checking empty/full.

Dangerous.

Can deadlock.

Trap 5

Multiple readers can never access together.

'L

Usually multiple readers are allowed.

Trap 6

Writer can write while readers are active.

'L

Writer requires exclusive access.

Trap 7

Reader-preference means writer can never proceed.

Not necessarily.

But it **may starve** if readers keep arriving.

Trap 8

Dining philosophers always deadlock.

'L

Naive acquisition can deadlock.

Correct protocols can prevent it.

Trap 9

Deadlock-free automatically means starvation-free.

'L

Different properties.

Trap 10

At most N philosophers should be allowed to compete to avoid deadlock.

That's not enough.

For classic resource-allocation prevention:

$\lfloor \frac{N-1}{2} \rfloor$

is the useful bound.

' p WRITE THIS IN YOUR NOTES

Producer–Consumer

Buffer size N :

TEXT

```
mutex = 1
empty = N
full = 0
```

Producer:

TEXT

```
wait(empty)
wait(mutex)
INSERT
signal(mutex)
signal(full)
```

Consumer:

TEXT

```
wait(full)
wait(mutex)
REMOVE
signal(mutex)
signal(empty)
```

Memory:

1 lock, N empty, 0 full

And:

\[empty+full=N \]

Readers–Writers

Rule:

TEXT

```
Many readers allowed
Only one writer
Reader + writer not allowed
```

Important:

TEXT

```
First reader !' wait ( wrt )
Last reader !' signal ( wrt )
```

mutex protects:

TEXT

```
read_count
```

Reader-priority may cause:

Writer starvation.

Dining Philosophers

Each philosopher needs 2 forks.

Naive:

TEXT

```
take left
take right
```

All may hold one and wait for next:

Deadlock.

Solutions:

TEXT

```
Allow at most N" 1
Global resource ordering
Asymmetric acquisition
Acquire both atomically
```

MASTER MEMORY TRICKS

Producer–Consumer

MEF = 1, N, 0

Mutex = 1 Empty = N Full = 0

Readers–Writers

First locks, last unlocks.

Dining Philosophers

One each !' none eat.

That describes deadlock perfectly.

30-SECOND REVISION

Producer-consumer semaphores?

!' mutex=1, empty=N, full=0

Producer order?

!' empty !' mutex !' insert !' mutex release !' full signal

Consumer order?

!' full !' mutex !' remove !' mutex release !' empty signal

Why empty first?

!' Avoid holding mutex while waiting for space.

Readers?

!' Many can read together.

Writer?

!' Exclusive.

First reader?

!' locks writer.

Last reader?

!' releases writer.

Reader preference problem?

!' Writer starvation.

Dining philosophers deadlock?

! Everyone holds one fork and waits for another.

Simple prevention?

! Let at most N compete.

ACTIVE RECALL

1. For a buffer of size 8, initialize `mutex`, `empty`, and `full`.
2. Why does the producer execute `wait(empty)` before `wait(mutex)`?
3. Why is `mutex` alone insufficient for producer-consumer?
4. If buffer size is 10 and 7 items exist, what are `empty` and `full`?
5. Can two readers read simultaneously?
6. Can two writers write simultaneously?
7. What does the first reader do?
8. What does the last reader do?
9. What starvation can occur in reader-priority?
10. Why does naive dining philosophers deadlock?
11. How does allowing only N philosophers help?
12. Is a deadlock-free solution automatically starvation-free?

MASTER CONNECTION

TEXT

Semaphores
!"
Producer-Consumer
!"
Resource counts + mutex

TEXT

Semaphores
!"
Readers-Writers
!"
Shared reading + exclusive writing

TEXT

Semaphores
!"
Dining Philosophers
!"
Resource acquisition
!"
Deadlock

And that brings us directly to the next major chapter:

NEXT — CHAPTER 8 Ø=Ý%Ø=Ý%Ø=Ý%

DEADLOCKS

We will cover:

- what deadlock actually means
- deadlock vs starvation
- resource-allocation model
- four Coffman conditions
- Resource Allocation Graph
- single vs multiple instances
- cycle implications
- deadlock prevention
- avoidance
- safe state vs unsafe state
- safe sequence
- Banker's Algorithm
- Need matrix
- Available/Allocation/Max
- full GATE numericals
- detection and recovery
- shortcuts for checking safe sequence
- common GATE traps

USER

NEXT

ASSISTANT

CHAPTER 8 — DEADLOCKS Ø=Ý%Ø=Ý%Ø=Ý% OVERVIEW HIGH PRIORITY

Deadlocks are among the most important GATE OS topics because questions can be both:

- conceptual
- numerical

The complete chain is:

TEXT

Processes

!"

Acquire Resources

!"

Hold Some + Wait for Others

!"

Circular Dependency

!"

1. WHAT IS A DEADLOCK?

A deadlock is a situation where:

A set of processes are permanently blocked because each process is waiting for a resource/event that can be caused only by another process in that same set.

Simple example:

TEXT

P1 holds R1
P1 wants R2

P2 holds R2
P2 wants R1

So:

TEXT

P1 waits for P2
P2 waits for P1

Neither can continue.

That is deadlock.

2. REAL-LIFE ANALOGY

Imagine two cars on a one-lane road:

TEXT

C a r A ! ' ! • C a r B

Neither can move forward.

Neither wants/can reverse.

Both wait forever.

That is a deadlock-like situation.

3. RESOURCE MODEL

Processes generally use resources in this sequence:

TEXT

REQUEST
! "

```
USE
    ! "
RELEASE
```

Examples of resources:

- CPU-related resources
- files
- printers
- locks
- semaphores
- memory blocks
- database locks
- I/O devices

Deadlock becomes possible when processes compete for limited resources.

4. SIMPLE DEADLOCK EXAMPLE

Resources:

```
TEXT
R1 = printer
R2 = scanner
```

P1:

```
TEXT
holds printer
waits for scanner
```

P2:

```
TEXT
holds scanner
waits for printer
```

Diagram:

```
TEXT
P 1 ! ' w a n t s R 2
! ' ! "
R1 P2
! ' ! "
% % % % % % % % R 2
```

More simply:

```
TEXT
P 1 ! ' R 2 ! ' P 2 ! ' R 1 ! ' P 1
```

Circular dependency.

5. FOUR NECESSARY DEADLOCK CONDITIONS

0-1% extremely important.

Deadlock can occur only if all four conditions hold simultaneously.

They are called the **Coffman Conditions**:

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

Mnemonic:

M H N C

Memory sentence:

"My Hungry Neighbor Circles."

6. MUTUAL EXCLUSION

At least one resource must be non-shareable.

Meaning:

Only one process can use that resource at a time.

Example:

TEXT
One printer

If P1 is using it:

TEXT
P2 must wait

7. HOLD AND WAIT

A process:

holds at least one resource while waiting for additional resources.

Example:

TEXT

P1 holds R1
P1 waits for R2

Memory:

Hold one, wait for another.

8. NO PREEMPTION

A resource cannot simply be forcibly taken away from a process.

It must normally be:

released voluntarily after use.

Example:

If P1 holds a lock, OS cannot arbitrarily pretend P1 released it unless the system/resource model permits that.

9. CIRCULAR WAIT

There exists a circular chain:

TEXT

P1 waits for P2
P2 waits for P3
P3 waits for P1

More formally:

TEXT

P 1 !' P 2 !' . . . !' P n !' P 1

Each process waits for a resource held by the next.

10. MASTER MEMORY

MHNC

Mutual Exclusion Hold and Wait No Preemption Circular Wait

Remember:

All 4 together !' deadlock becomes possible.

& **pgATE TRAP**

The four Coffman conditions are:

necessary conditions

They do not mean:

If one condition exists, deadlock definitely occurs.

All four must be present for deadlock.

11. BREAK ONE CONDITION = PREVENT DEADLOCK

Since all four are necessary:

If we ensure even one can never hold, deadlock is prevented.

This is the basis of:

Deadlock Prevention

12. DEADLOCK VS STARVATION

Very important.

Deadlock

Processes in a set wait forever because of circular/resource dependency.

Example:

TEXT

P1 waits for P2
P2 waits for P1

No one proceeds.

Starvation

One process waits indefinitely while others may continue progressing.

Example:

TEXT

P1 keeps waiting
P2, P3, P4 continue getting CPU/resource

Comparison

Deadlock	Starvation
Set of processes stuck	Particular process may wait forever
Circular/resource dependency common	Unfair scheduling/allocation
Processes in deadlock cannot progress	Other processes can continue
Requires Coffman conditions	Does not require all Coffman conditions

Memory:

Deadlock = Everybody in cycle stuck

Starvation = Somebody keeps losing

13. RESOURCE ALLOCATION GRAPH — RAG

Ø = Important.

A Resource Allocation Graph represents:

- processes
- resources
- requests
- allocations

Processes are usually represented as circles:

TEXT
(P1)

Resources as squares:

TEXT
[R1]

14. REQUEST EDGE

If process P1 requests R1:

TEXT
P 1 !' R 1

Meaning:

P1 is waiting for R1.

15. ASSIGNMENT EDGE

If R1 is allocated to P1:

TEXT

R 1 !' P 1

Meaning:

P1 currently holds R1.

16. SIMPLE RAG EXAMPLE

TEXT

R 1 !' P 1 !' R 2 !' P 2 !' R 1

Interpretation:

- R1 allocated to P1
- P1 requests R2
- R2 allocated to P2
- P2 requests R1

There is a cycle.

17. CYCLE RULE — SINGLE RESOURCE INSTANCE

Very important.

If every resource type has exactly **one instance**:

Cycle in RAG ![⊖]Deadlock

So:

TEXT

Cycle present

!"

Deadlock definitely exists

for the single-instance case.

18. CYCLE RULE — MULTIPLE RESOURCE INSTANCES

If resource types can have multiple instances:

A cycle is necessary but not sufficient for deadlock.

Meaning:

TEXT

Cycle

does not automatically prove deadlock.

Why?

Because another instance may become available.

& pGATE TRAP

Single instance:

TEXT

Cycle !" Deadlock

Multiple instances:

TEXT

Deadlock !' Cycle

but:

TEXT

Cycle !> necessarily deadlock

This distinction is very important.

19. DEADLOCK HANDLING STRATEGIES

Operating systems can conceptually handle deadlocks in four ways:

1. Ignore
2. Prevention
3. Avoidance
4. Detection and Recovery

Memory:

I P A D

Ignore Prevent Avoid Detect

20. IGNORE DEADLOCK

Some general-purpose systems may effectively adopt an approach often called the:

Ostrich approach

Meaning:

Assume deadlocks are rare enough that full prevention may cost more than dealing with occasional occurrences.

For GATE, know the idea.

21. DEADLOCK PREVENTION

Goal:

Structurally ensure at least one Coffman condition never holds.

So we attack:

TEXT

M
H
N
C

one by one.

22. PREVENT MUTUAL EXCLUSION

If resources are made shareable, mutual exclusion disappears.

Example:

Read-only file:

TEXT

many processes can read

But not all resources are sharable.

A printer cannot realistically print two unrelated jobs at the same physical instant in a simple exclusive model.

So this method is limited.

23. PREVENT HOLD AND WAIT

Require:

A process must request all required resources at once before execution.

Or:

It must release currently held resources before requesting new ones.

Example:

Instead of:

TEXT

Get R1
later request R2

require:

TEXT

Request R1 and R2 together

If both unavailable:

TEXT

wait without holding either

24. PROBLEM WITH BREAKING HOLD AND WAIT

This may cause:

- low resource utilization
- long waiting
- starvation

because a process may wait for all resources even though some are currently free.

25. PREVENT NO PREEMPTION

Allow resources to be taken back under some rules.

Example conceptually:

P1 holds R1 and requests R2.

If R2 unavailable:

TEXT

P1 releases/preempts held resources

then retries later.

But some resources cannot be easily preempted.

26. PREVENT CIRCULAR WAIT

Assign a total ordering to resource types.

Example:

TEXT

R1 < R2 < R3 < R4

Require processes to request resources only in increasing order.

Example allowed:

TEXT

R 1 !' R 3 !' R 4

Not:

TEXT

R 3 !' R 1

This eliminates circular wait.

MEMORY TRICK FOR PREVENTION

TEXT

M !' m a k e s h a r e a b l e
H !' r e q u e s t a l l t o g e t h e r
N !' a l l o w p r e e m p t i o n
C !' o r d e r r e s o u r c e s

27. DEADLOCK PREVENTION VS AVOIDANCE

Very important distinction.

Prevention

Design rules make deadlock structurally impossible by breaking a Coffman condition.

Avoidance

Coffman conditions may exist, but system checks each allocation and grants it only if the system remains in a:

Safe State

Comparison

Prevention	Avoidance
Break a deadlock condition	Carefully grant resources
More restrictive	More dynamic

Prevention	Avoidance
Does not require future max demand in same way	Usually requires knowledge of maximum demand
May reduce utilization	Often better utilization
Example: resource ordering	Banker's Algorithm

28. SAFE STATE

Very important.

A system is in a **safe state** if:

There exists at least one sequence of process completion such that each process can obtain its remaining resources, complete, and release its resources.

Such an order is called a:

Safe Sequence

Example:

TEXT
<P2, P1, P3>

means:

1. P2 can finish
2. releases resources
3. P1 can finish
4. releases resources
5. P3 can finish

29. SAFE STATE DOES NOT MEAN EVERY ORDER WORKS

Important.

A state is safe if:

At least ONE safe sequence exists.

Not:

Every sequence must work.

30. UNSAFE STATE

A state is unsafe if:

No safe sequence exists.

Important:

Unsafe "Deadlocked

An unsafe state means:

Deadlock may occur in the future.

It does not necessarily mean deadlock has already happened.

31. SAFE / UNSAFE / DEADLOCK RELATION

Think:

TEXT

SAFE

!" allocation continues badly

UNSAFE

!" may eventually become

DEADLOCK

Key:

Every deadlocked state is unsafe.

But:

Every unsafe state is not necessarily already deadlocked.

& pGATE TRAP

TEXT

Unsafe = Deadlock

'LFalse.

Correct:

TEXT

Deadlock ", Unsafe

Deadlock is a subset of unsafe states conceptually.

32. BANKER'S ALGORITHM

Extremely important numerical topic.

Banker's Algorithm is a:

Deadlock avoidance algorithm

It grants a resource request only if the resulting system state remains safe.

Why the name?

A banker loans money only if it can still satisfy everyone eventually.

33. DATA STRUCTURES IN BANKER'S ALGORITHM

For each resource type, we use:

Available

Number of currently available instances.

Max

Maximum demand of each process.

Allocation

Resources currently allocated to each process.

Need

Remaining resources each process may still require.

FORMULA BOX

$$\text{Need} = \text{Max} - \text{Allocation}$$

This formula is essential.

34. EXAMPLE OF NEED

Suppose process P1:

TEXT

Max = (7, 5, 3)
Allocation = (0, 1, 0)

Then:

$$\text{Need} = \text{Max} - \text{Allocation}$$

$$= (7-0, 5-1, 3-0)$$

$$\text{Need} = (7, 4, 3)$$

35. SAFETY ALGORITHM — BASIC IDEA

We maintain:

TEXT

Work = Available

Then find a process such that:

$\forall [Need_i \leq Work]$

component-wise.

Meaning:

Currently available resources are enough for that process to finish.

Then pretend it finishes and releases its allocation:

$\forall [Work = Work + Allocation_i]$

Continue.

If all processes can finish:

Safe state.

If at some point none can finish but unfinished processes remain:

Unsafe state.

36. COMPONENT-WISE COMPARISON

Suppose:

TEXT

Need = (1, 2, 1)

Work = (2, 2, 3)

Check each resource:

TEXT

1 "d 2

2 "d 2

1 "d 3

Yes.

So process can finish.

But:

TEXT

Need = (1, 3, 1)
Work = (2, 2, 3)

fails because:

$\nexists 3 > 2$

One resource type is insufficient.

So cannot select that process yet.

37. STANDARD BANKER SAFETY ALGORITHM

Step 1

Calculate:

$\nexists \text{Need} = \text{Max} - \text{Allocation}$

Step 2

Initialize:

TEXT

Work = Available
Finish[i] = false

Step 3

Find some unfinished process P_i satisfying:

$\nexists \text{Need}_i \leq \text{Work}$

Step 4

Pretend P_i finishes:

$\nexists \text{Work} = \text{Work} + \text{Allocation}_i$

Mark:

TEXT

Finish[i] = true

Step 5

Repeat.

Step 6

If all Finish = true:

Safe.

Otherwise:

38. FULL GATE-STYLE EXAMPLE

Suppose we have 3 processes and 3 resource types A, B, C.

Allocation

Process	A	B	C
P0	1	0	1
P1	1	1	0
P2	1	1	1

Max

Process	A	B	C
P0	2	1	1
P1	1	2	1
P2	3	1	1

Available:

$\vee (1,1,1) \vee$

39. CALCULATE NEED

Using:

$\vee [\text{Need} = \text{Max} - \text{Allocation}]$

For P0:

$\vee (2,1,1) - (1,0,1) \vee$

$\vee = (1,1,0) \vee$

For P1:

$\vee (1,2,1) - (1,1,0) \vee$

$\vee = (0,1,1) \vee$

For P2:

$\vee (3,1,1) - (1,1,1) \vee$

$\vee = (2,0,0) \vee$

So:

Process	Need A	Need B	Need C
P0	1	1	0
P1	0	1	1
P2	2	0	0

40. INITIAL WORK

$[\text{Work} = \text{Available} = (1,1,1)]$

Check P0:

$[(1,1,0) \leq (1,1,1)]$

Yes.

So P0 can finish.

Release P0 allocation:

$[\text{Work} = (1,1,1) + (1,0,1)]$

$[= (2,1,2)]$

41. NEXT PROCESS

Check P1:

$[\text{Need}_1 = (0,1,1)]$

$[(0,1,1) \leq (2,1,2)]$

Yes.

P1 finishes.

Release allocation:

$[\text{Work} = (2,1,2) + (1,1,0)]$

$[= (3,2,2)]$

42. P2

Need:

$[(2,0,0)]$

Available Work:

$[(3,2,2)]$

So P2 finishes.

Safe sequence:

$[\boxed{\langle P0, P1, P2 \rangle}]$

Therefore:

$\boxed{\text{System is SAFE}}$

43. THERE MAY BE MULTIPLE SAFE SEQUENCES

In the same state, perhaps P1 could also have completed first.

Then another safe sequence may exist.

Question may ask:

- whether safe
- one safe sequence
- all safe sequences
- number of safe sequences

Be careful about exact wording.

44. BANKER'S EXAM SHORTCUT

Instead of staring at the full matrices:

1. Write `Work = Available`
2. Search for **any row** where:

TEXT

N e e d "d W o r k

3. Cross it out
4. Add its Allocation to Work
5. Repeat

Think:

Need fits !' Finish !' Free allocation

Mnemonic:

N F F

Need fits !' Finish !' Free

45. RESOURCE REQUEST ALGORITHM

Suppose process P_i makes a new request:

Request_i

We first check:

Condition 1

$\neg [Request_i \leq Need_i]$

Otherwise process asked for more than declared maximum.

Condition 2

$\neg [Request_i \leq Available]$

Otherwise it must wait.

Condition 3

Pretend to grant:

TEXT

```
Available = Available - Request
Allocation_i = Allocation_i + Request
Need_i = Need_i - Request
```

Then run safety algorithm.

If state remains safe:

Grant permanently.

If unsafe:

Roll back and make process wait.

46. WHY BANKER'S ALGORITHM IS CALLED AVOIDANCE

It does not wait until deadlock happens.

Instead it asks:

“If I grant this now, can everyone still finish in some order?”

If yes:

TEXT

grant

If no:

TEXT

delay

47. SAFE SEQUENCE EXAMPLE — SINGLE RESOURCE TYPE

Suppose one resource type R has 10 instances.

Processes:

Process	Allocated	Maximum
P1	3	5
P2	2	4
P3	2	7

Total allocated:

$$\{ 3+2+2=7 \}$$

Available:

$$\{ 10-7=3 \}$$

Need:

TEXT

$$P1 = 5 - 3 = 2$$

$$P2 = 4 - 2 = 2$$

$$P3 = 7 - 2 = 5$$

Available = 3.

P1 can finish.

Release 3:

$$\{ \text{Available}=6 \}$$

P2 can finish.

Release 2:

$$\{ \text{Available}=8 \}$$

P3 can finish.

Safe sequence:

$$\{ \boxed{P1, P2, P3} \}$$

48. GATE SHORTCUT FOR ONE RESOURCE TYPE

With one resource:

TEXT

$$\text{Need} = \text{Max} - \text{Alloc}$$

Sort mentally/find a process with:

TEXT

$$\text{Need} \leq \text{Available}$$

After it completes:

TEXT

Available += Alloc

Repeat.

Very fast.

49. MINIMUM RESOURCES TO GUARANTEE NO DEADLOCK

Ø=0%TE numerical pattern.

Suppose:

- n processes
- each process may require at most k identical resources

What is the minimum number of resources that guarantees no deadlock?

Worst case:

Each process holds:

$\lfloor k-1 \rfloor$

resources and waits for one more.

Total held:

$\lfloor n(k-1) \rfloor$

If we have just one additional resource:

$\lfloor \boxed{n(k-1)+1} \rfloor$

then one process can obtain its final needed resource, finish, and release everything.

So:

$\lfloor \boxed{R_{\min}=n(k-1)+1} \rfloor$

50. EXAMPLE

5 processes.

Each may require at most 3 identical resources.

Minimum number to guarantee no deadlock:

$\lfloor 5(3-1)+1 \rfloor$

$\lfloor =10+1 \rfloor$

$\lfloor \boxed{11} \rfloor$

51. GENERALIZED VERSION

Suppose process P_i may require at most:

$\lfloor m_i \rfloor$

identical resources.

To guarantee no deadlock:

$$\sum_{i=1}^m (m_i - 1) + 1 \leq M$$

because worst case each holds one less than maximum.

& #x26; GATE TRAP

Do not blindly use this formula for multiple resource types.

It is mainly for identical instances of a single resource type under the classic assumptions.

52. DEADLOCK DETECTION

Suppose system does **not** avoid unsafe states.

Instead:

1. allocate resources
2. periodically check whether deadlock has occurred

This is:

Deadlock Detection

Then recovery is performed.

53. DETECTION — SINGLE INSTANCE

For single-instance resource types, a:

Wait-For Graph

can be used.

Processes only.

If:

TEXT

P 1 !' P 2

means:

P1 waits for resource held by P2.

Cycle:

TEXT

P 1 !' P 2 !' P 3 !' P 1

means:

$\boxed{\text{Deadlock}}$

54. RESOURCE ALLOCATION GRAPH VS WAIT-FOR GRAPH

RAG contains:

TEXT

Processes + Resources

Wait-for graph contains:

TEXT

Processes only

Example conversion:

TEXT

P 1 !' R 1 !' P 2

becomes:

TEXT

P 1 !' P 2

55. DETECTION WITH MULTIPLE INSTANCES

For multiple instances, an algorithm similar to Banker's safety algorithm can be used.

But instead of **Max/Need**, detection typically works with:

- Available
- Allocation
- current outstanding Request

The key logic remains:

Can some process's outstanding request be satisfied by current Work?

If yes:

TEXT

pretend it finishes
release Allocation

If some processes cannot be marked finished:

those may be deadlocked.

56. DEADLOCK RECOVERY

Once deadlock is detected, options include:

1. Terminate processes

Could terminate:

- all deadlocked processes
- one process at a time until cycle breaks

2. Resource Preemption

Take resources from selected processes if possible.

3. Rollback

Return a process to an earlier safe checkpoint.

57. VICTIM SELECTION

If terminating/preempting, OS may consider:

- process priority
- resources held
- resources still needed
- amount of work completed
- amount of work remaining
- rollback cost
- number of processes affected

Goal:

Minimize recovery cost.

58. STARVATION DURING RECOVERY

If the same process is repeatedly selected as victim and rolled back:

It may starve.

Therefore recovery mechanisms may consider:

- number of previous rollbacks
 - fairness
-

59. PREVENTION VS AVOIDANCE VS DETECTION

Method	Main Idea
Prevention	Break a Coffman condition
Avoidance	Never enter unsafe state
Detection	Allow deadlock possibility, detect afterward
Recovery	Break existing deadlock

Shortcut:

Prevent = Make impossible

Avoid = Be careful

Detect = Find later

Recover = Fix later

60. DEADLOCK AND SCHEDULING

Deadlock is not the same as:

A process simply waiting for CPU.

A Ready process:

TEXT

waiting for CPU

is normal.

A deadlocked process:

TEXT

waiting for resource/event that can never become available because of circular dependency

is fundamentally different.

61. DEADLOCK AND BLOCKED STATE

A deadlocked process is usually blocked/waiting.

But:

Every blocked process is not deadlocked.

Example:

TEXT

P1 waits for disk I/O

Disk completes soon.

Normal blocking.

No deadlock.

62. EXAMPLE — NO DEADLOCK

P1:

TEXT

holds R1
waits R2

P2:

TEXT

holds R2

but P2 does not wait for anything.

P2 can finish and release R2.

Then P1 continues.

So:

No deadlock.

Why?

No circular wait.

63. EXAMPLE — CYCLE WITH MULTIPLE INSTANCES

Suppose R1 has 2 instances.

One instance held by P1, another can later be released by P3.

Even if a graph visually contains a cycle involving P1 and P2, another resource instance may break it.

Therefore:

cycle alone does not prove deadlock for multiple instances.

64. SAFE STATEEXAMPLE

Available:

TEXT

3 resources

P1 needs 2 more.

P2 needs 4 more.

P3 needs 1 more.

Choose P3:

TEXT

Need 1 "d Available 3

P3 finishes, releases its allocation.

Now more become available.

Then P1 may finish.

Then P2.

Safe sequence exists.

So state is safe.

65. UNSAFE STATEEXAMPLE

Suppose:

TEXT

Available = 1

and remaining needs are:

TEXT

P1 needs 2

P2 needs 3

P3 needs 2

No one can finish.

If current processes are not necessarily each holding resources in a circular deadlock yet, state may simply be classified unsafe in Banker's sense.

No safe sequence exists.

66. GATE-STYLE BANKER QUESTION METHOD

When given:

- Allocation
- Max
- Available

Immediately:

Step 1

Compute:

$\backslash [\text{Need} = \text{Max} - \text{Allocation}]$

Step 2

Set:

$\backslash [\text{Work} = \text{Available}]$

Step 3

Circle any row satisfying:

$\backslash [\text{Need} \leq \text{Work}]$

Step 4

Add its Allocation to Work.

Step 5

Cross that row.

Repeat.

Do not recompute Max repeatedly.

67. BANKER'S TABLE FORMAT FOR NOTES

Use:

Process	Allocation	Max	Need
P0	...	...	Max - Alloc
P1	...	...	...

Then underneath:

TEXT

Work = Available

and trace:

TEXT

P2 finishes !' Work = . . .
P0 finishes !' Work = . . .
P1 finishes !' Work = . . .

Much faster than long paragraphs.

68. HOW GATE CAN ASK DEADLOCK

Expect:

Pattern 1

Identify Coffman condition.

Pattern 2

Given a RAG, determine deadlock.

Pattern 3

Cycle reasoning with single/multiple instances.

Pattern 4

Differentiate prevention vs avoidance.

Pattern 5

Find Need matrix.

Pattern 6

Find safe sequence.

Pattern 7

Determine whether a request can be granted.

Pattern 8

Find minimum number of resources preventing deadlock.

Pattern 9

Wait-for graph.

Pattern 10

Deadlock vs starvation / unsafe state.

& pGATE TRAPS — DEADLOCK

Trap 1

A blocked process is automatically deadlocked.

'L False.

Trap 2

A cycle always implies deadlock.

'L Only guaranteed for single-instance resources.

Trap 3

Unsafe state means deadlock has already occurred.

'L False.

Unsafe means deadlock is possible.

Trap 4

Safe state means no process ever waits.

'L False.

Processes may wait temporarily.

Safe only means some completion sequence exists.

Trap 5

Deadlock avoidance breaks a Coffman condition.

'L That's prevention.

Avoidance allows allocations only if resulting state remains safe.

Trap 6

Banker's algorithm detects deadlock after it happens.

'L

Banker's is avoidance.

Trap 7

Need is:

$\backslash [\text{Allocation} - \text{Max}]$

'L Wrong.

Correct:

$\backslash [\boxed{\text{Need} = \text{Max} - \text{Allocation}}]$

Trap 8

For safety check, compare:

TEXT

A l l o c a t i o n " d A v a i l a b l e

'L

Compare:

$\boxed{\text{Need} \leq \text{Work}}$

Trap 9

When a process finishes, add its **Need** to Work.

'L

It releases what it was actually holding:

$\boxed{\text{Work} = \text{Work} + \text{Allocation}}$

Trap 10

All safe sequences must work for state to be safe.

'L

Only one safe sequence is required.

' p WRITE THIS IN YOUR NOTES

Deadlock

Set of processes permanently blocked because each waits for resource/event held/generated by another in same set.

Four Conditions — MHNC

TEXT

M !' M u t u a l E x c l u s i o n
H !' H o l d a n d W a i t
N !' N o P r e e m p t i o n
C !' C i r c u l a r W a i t

Mnemonic:

My Hungry Neighbor Circles

All 4 necessary.

RAG

TEXT

P !' R = r e q u e s t
R !' P = a l l o c a t i o n

Cycle Rule

TEXT

Single instance:

`Cycle !⇒ Deadlock`

Multiple instances:

Cycle does not guarantee deadlock

Handling

TEXT

Ignore

Prevention

Avoidance

Detection + Recovery

Prevention

TEXT

Break any one of MHNC

Examples:

TEXT

`H !' request all resources together`
`C !' impose resource ordering`

Safe State

At least one safe sequence exists.

Unsafe State

No safe sequence.

Important:

TEXT

`Unsafe "⇏ necessarily deadlock`

Banker's Algorithm

$\boxed{\text{Need} = \text{Max} - \text{Allocation}}$

Safety test:

```
TEXT
Work = Available

Find Need[i] "d Work

Then:
Work = Work + Allocation[i]
```

If all processes finish:

Safe.

Resource Request

Check:

$\forall \text{ Request}_i \leq \text{Need}_i$

$\forall \text{ Request}_i \leq \text{Available}$

Pretend grant !' run safety test.

Minimum Identical Resources

For n processes each requiring at most k :

$\lceil \frac{n(k-1)+1}{k} \rceil$

to guarantee no deadlock under classic assumptions.

MASTER MEMORY MAP

TEXT		
Resources	!"	
Processes Request/Hold	!"	
MHNC all present	!"	
Possible Deadlock	!"	
Prevention	Avoidance	Detection
!"		!"
Break MHNC	Safe State	Find Deadlock
		!"
	Banker	Recovery
		!"

EXAM SHORTCUT SHEET

Shortcut 1 — Coffman

MHNC

Shortcut 2 — Cycle

TEXT

```
1 instance !' cycle means deadlock
many instances !' cycle may not mean deadlock
```

Shortcut 3 — Banker

NFF

Need fits !' Finish !' Free allocation

Shortcut 4 — State relation

TEXT

```
Safe !' guaranteed some completion order
Unsafe !' no guaranteed safe order
Deadlock !' already stuck
```

Shortcut 5 — Prevention vs Avoidance

Prevention breaks a condition.

Avoidance checks safety.

30-SECOND REVISION

Deadlock?

!' Processes permanently waiting on each other/resources.

4 conditions?

!' MHNC.

Single-instance cycle?

!' Deadlock.

Multiple-instance cycle?

!' May or may not be deadlock.

Prevention?

!' Break one Coffman condition.

Avoidance?

!' Stay safe.

Banker?

!' Deadlock avoidance.

Need?

\[Max-Allocation \]

Safe?

!' At least one safe sequence.

Unsafe?

!' Not necessarily deadlocked.

When process finishes in safety algorithm?

!' Add its Allocation back to Work.

ACTIVE RECALL

1. State all four Coffman conditions.
2. Why must all four be present for deadlock?
3. What is the difference between starvation and deadlock?
4. What does **P** !'mean in a RAG?
5. What does **R** !'mean?
6. When does a cycle guarantee deadlock?
7. Difference between prevention and avoidance?
8. Define safe state.
9. Is unsafe state equal to deadlock?
10. What is the Banker's formula for **Need** ?
11. What condition must hold before selecting a process in the safety algorithm?
12. What gets added to **Work** when a process finishes?
13. Why does global resource ordering prevent circular wait?
14. For 5 processes needing at most 3 identical resources each, how many resources guarantee no deadlock?

Answer to 14:

\[$5(3-1)+1=\boxed{11}$ \]

NEXT — CHAPTER 9 Ø=Ý%Ø=Ý%Ø=Ý%

Next we enter another major numerical scoring area:

CPU SCHEDULING

We will cover:

- CPU burst and arrival time
- preemptive vs non-preemptive
- FCFS
- SJF
- SRTF
- Priority Scheduling
- Round Robin
- Gantt charts
- completion time
- turnaround time
- waiting time
- response time
- throughput
- CPU utilization
- convoy effect
- starvation
- aging
- time quantum
- full GATE numericals
- fastest methods for solving scheduling questions

USER

NEXT

ASSISTANT

CHAPTER 9 — CPU SCHEDULING OVERVIEW

CPU Scheduling is one of the most important numerical areas in Operating Systems.

The complete chain is:

TEXT

```
Processes arrive
! "
Ready Queue
! "
CPU Scheduler chooses one
! "
Process executes
! "
May finish / block / get preempted
```

The main GATE task is usually:

Given arrival times, burst times, priorities, and an algorithm, calculate execution order and performance metrics.

1. WHY DO WE NEED CPU SCHEDULING?

Suppose:

TEXT

P1
P2
P3
P4

are all ready.

But only one CPU core is available.

Question:

Who should execute first?

The OS uses a **CPU scheduling algorithm**.

2. CPU SCHEDULER

The CPU scheduler selects a process from the:

Ready Queue

and allocates CPU to it.

Remember:

TEXT

R E A D Y ! ' w a i t i n g f o r C P U

So CPU scheduling mainly operates on ready processes.

3. CPU BURST

A process alternates between:

TEXT

CPU Burst
!"
I/O Burst
!"
CPU Burst
!"
I/O Burst

A **CPU burst** is a period during which the process executes on the CPU.

Example:

TEXT

P1:
CPU 5 ms
I/O
CPU 3 ms

4. IMPORTANT INPUT TERMS

Most scheduling numericals give some combination of:

Arrival Time — AT

Time at which process arrives in the ready queue.

Burst Time — BT

CPU execution time required.

Priority

Used by priority scheduling.

Time Quantum

Used by Round Robin.

MEMORY

A B C T W R

We'll use:

- Arrival Time
- Burst Time
- Completion Time
- Turnaround Time
- Waiting Time
- Response Time

This is the core scheduling family.

5. COMPLETION TIME — CT

Completion Time:

Time at which a process finishes execution completely.

Example:

TEXT

P1 starts at 3
runs for 5

Then:

$\boxed{CT = 8}$

6. TURNAROUND TIME — TAT

Formula:

$\boxed{TAT = CT - AT}$

Meaning:

Total time spent in the system from arrival until completion.

7. WAITING TIME — WT

Formula:

$\boxed{WT = TAT - BT}$

Therefore:

$\boxed{WT = CT - AT - BT}$

Meaning:

Total time process spends waiting in the ready queue.

8. RESPONSE TIME — RT

Response Time:

Time from process arrival until it gets CPU for the first time.

Formula:

$\boxed{RT = \text{First Start Time} - AT}$

Important:

Response Time cares only about the FIRST CPU allocation.

9. MASTER FORMULA BOX

$\boxed{TAT = CT - AT}$

$$\boxed{WT = TAT - BT}$$

$$\boxed{RT = \text{First Start} - AT}$$

Memory:

Complete minus Arrive = Turnaround

Turnaround minus Burst = Waiting

10. EXAMPLE

Suppose:

TEXT

AT = 2

BT = 5

First Start = 4

CT = 10

Then:

$$TAT = 10 - 2 = 8$$

$$WT = 8 - 5 = 3$$

$$RT = 4 - 2 = 2$$

11. RESPONSE TIME VS WAITING TIME

Very important.

Response Time

Waiting until first execution only.

Waiting Time

Total waiting over the whole lifetime.

For non-preemptive algorithms, they may be equal in some simple one-burst cases.

For preemptive algorithms, often:

$$RT \neq WT$$

12. PREEMPTIVE VS NON-PREEMPTIVE

Very important.

Non-Preemptive

Once process receives CPU:

It continues until it finishes its CPU burst or blocks voluntarily.

CPU is not forcibly taken away due to arrival of another process.

Examples:

- FCFS
- Non-preemptive SJF
- Non-preemptive Priority

Preemptive

OS can interrupt a running process and give CPU to another.

Examples:

- SRTF
- Preemptive Priority
- Round Robin

Memory

Preempt = Pull CPU away

13. DISPATCHER

The dispatcher performs the actual transfer of CPU control to the selected process.

Tasks include conceptually:

- context switch
- switching execution mode
- jumping to appropriate instruction

Time required is called:

Dispatch latency

14. SCHEDULING PERFORMANCE CRITERIA

Common metrics:

CPU Utilization

Keep CPU busy as much as possible.

Throughput

Processes completed per unit time.

Turnaround Time

Minimize.

Waiting Time

Minimize.

Response Time

Minimize, especially for interactive systems.

15. THROUGHPUT

$$\text{Throughput} = \frac{\text{Number of completed processes}}{\text{Time}}$$

Example:

20 processes finish in 10 seconds:

$$\text{Throughput} = 2 \text{ processes/sec}$$

16. CPU UTILIZATION

Conceptually:

$$\text{CPU Utilization} = \frac{\text{CPU Busy Time}}{\text{Total Time}} \times 100\%$$

Example:

CPU busy for 90 ms out of 100 ms:

$$90\%$$

PART A — FCFS

17. FIRST COME FIRST SERVED

FCFS means:

Process that arrives first gets CPU first.

Think:

TEXT

Queue at ticket counter

First person to arrive is served first.

18. FCFS CHARACTERISTICS

- non-preemptive
- simple
- usually FIFO ready queue
- may have poor average waiting time
- can suffer convoy effect

19. FCFS EXAMPLE 1

Processes:

Process	AT	BT
P1	0	5
P2	1	3
P3	2	1

Execution order:

TEXT

P 1 ! ' P 2 ! ' P 3

Gantt:

TEXT

0 5 8 9
| P1 | P2 | P3 |

20. COMPLETION TIMES

TEXT

P1 = 5
P2 = 8
P3 = 9

21. TURNAROUND TIMES

\[P1: 5-0=5 \]

\[P2: 8-1=7 \]

\[P3: 9-2=7 \]

22. WAITING TIMES

\[P1: 5-5=0 \]

$\backslash [P2: 7-3=4 \backslash]$

$\backslash [P3: 7-1=6 \backslash]$

Average WT:

$\backslash [\frac{0+4+6}{3} = \boxed{\frac{10}{3}} \backslash]$

23. RESPONSE TIMES

First starts:

TEXT

P1 = 0

P2 = 5

P3 = 8

So:

$\backslash [RT_{P1}=0-0=0 \backslash]$

$\backslash [RT_{P2}=5-1=4 \backslash]$

$\backslash [RT_{P3}=8-2=6 \backslash]$

For this non-preemptive example:

$\backslash [RT = WT \backslash]$

24. CONVOY EFFECT

Ø = important.

Suppose one very long CPU-bound process arrives before many short processes.

Example:

TEXT

P1 = 100 ms

P2 = 2 ms

P3 = 1 ms

FCFS:

TEXT

P 1 ! ' P 2 ! ' P 3

Short processes wait behind huge P1.

This is:

Convoy Effect

Memory:

One slow truck delays all cars behind it.

& pGATE TRAP

Convoy effect is strongly associated with:

$\boxed{\text{FCFS}}$

PART B — SJF

25. SHORTEST JOB FIRST

SJF chooses:

Ready process with the smallest CPU burst.

Classic SJF is non-preemptive unless specified otherwise.

26. WHY SJF IS IMPORTANT

SJF minimizes:

Average waiting time

under ideal conditions when exact burst lengths are known.

This is a famous GATE fact.

27. SJF EXAMPLE

All arrive at time 0.

Process	BT
P1	6
P2	2
P3	8
P4	3

Order:

TEXT

P 2 ! ' P 4 ! ' P 1 ! ' P 3

Gantt:

TEXT

0 2 5 11 19
|P2| P4 | P1 | P3 |

Waiting:

TEXT

P2 = 0
P4 = 2
P1 = 5
P3 = 11

Average:

$\lfloor \frac{0+2+5+11}{4} \rfloor = \frac{18}{4} = \boxed{4.5} \rfloor$

28. SJF WITH ARRIVAL TIMES

Very important.

Suppose:

Process	AT	BT
P1	0	7
P2	2	4
P3	4	1

At time 0:

Only P1 has arrived.

So SJF must choose P1.

Because SJF is non-preemptive:

TEXT

P 1 r u n s 0 ! ' 7

At time 7:

P2 and P3 are ready.

Choose shortest:

TEXT

P3 BT=1

Then P2.

Gantt:

TEXT

0 7 8 12
| P1 | P3 | P2 |

& pGATE TRAP

Do not select a process that has:

not arrived yet.

Scheduling algorithms only choose among processes currently in the ready queue.

29. STARVATIONIN SJF

A long job may wait indefinitely if short jobs keep arriving.

This is possible:

Starvation

So SJF has excellent average waiting time but fairness can be poor.

PART C — SRTF

30. SHORTEST REMAINING TIME FIRST

SRTF is:

Preemptive version of SJF

Scheduler chooses process with:

smallest remaining CPU time.

If a new process arrives with shorter remaining time than current process:

current process may be preempted.

31. SRTF EXAMPLE

Processes:

Process	AT	BT
P1	0	8

Process	AT	BT
P2	1	4
P3	2	2

Time 0

Only P1:

TEXT
P1 runs

At time 1:

P1 remaining = 7 P2 burst = 4

Choose P2.

TEXT
0-1 P1

At time 2:

P2 remaining = 3 P3 burst = 2

Choose P3.

TEXT
1-2 P2

P3 runs 2! and finishes.

Then:

TEXT
P2 remaining 3
P1 remaining 7

P2 runs 4! 7 .

Then P1 runs 7! 14 .

Gantt:

TEXT
0 1 2 4 7 14
|P1|P2| P3 | P2 | P1 |

32. COMPLETION TIMES

TEXT
P1 = 14
P2 = 7

P3 = 4

TAT:

$\backslash [P1=14-0=14 \backslash]$

$\backslash [P2=7-1=6 \backslash]$

$\backslash [P3=4-2=2 \backslash]$

WT:

$\backslash [P1=14-8=6 \backslash]$

$\backslash [P2=6-4=2 \backslash]$

$\backslash [P3=2-2=0 \backslash]$

33. RESPONSE TIMES

First starts:

TEXT

P1 = 0

P2 = 1

P3 = 2

So:

TEXT

RT1 = 0

RT2 = 0

RT3 = 0

Notice:

TEXT

W T " ` R T

for P1 and P2.

34. SJF VS SRTF

SJF	SRTF
Non-preemptive	Preemptive
Uses shortest burst among ready jobs	Uses shortest remaining time
Running job normally continues	New shorter job can preempt
Lower context-switch overhead	Potentially more switches

Memory:

SRTF = Shortest Remaining Time, so Reconsider on arrival.

PART D — PRIORITY SCHEDULING

35. PRIORITY SCHEDULING

CPU is allocated based on process priority.

Could be:

- preemptive
- non-preemptive

Very important:

Always check whether smaller number means higher priority or the opposite.

The question must define convention, or you must use stated assumptions.

36. PRIORITY EXAMPLE

Assume:

smaller number = higher priority.

Processes all arrive at 0:

Process	BT	Priority
P1	4	3
P2	3	1
P3	2	2

Order:

TEXT

P 2 ! ' P 3 ! ' P 1

37. PREEMPTIVE PRIORITY

Suppose P1 is executing.

Then P2 arrives with higher priority.

If algorithm is preemptive:

TEXT

P1 gets preempted
P2 runs

If non-preemptive:

TEXT

P1 finishes first

38. PRIORITY STARVATION

Low-priority processes may wait indefinitely if high-priority processes keep arriving.

This is:

Starvation

39. AGING

Solution:

Gradually increase the priority of a process the longer it waits.

This is called:

Aging

Memory:

Older process gets stronger priority.

Aging helps prevent starvation.

PART E — ROUND ROBIN Ø=Ý%Ø=Ý%Ø=Ý%

40. ROUND ROBIN

Round Robin gives each ready process a fixed CPU time called:

Time Quantum / Time Slice

After quantum expires:

- if process finished !' exit
- if not !' preempt and move to back of ready queue

41. ROUND ROBIN IS PREEMPTIVE

Yes.

Because CPU may be taken away when quantum expires.

42. RR EXAMPLE

All arrive at time 0.

Process	BT
P1	5
P2	4
P3	2

Quantum:

$\lfloor q=2 \rfloor$

Initial queue:

TEXT

P1 P2 P3

43. FIRST ROUND

P1 executes 2.

Remaining:

TEXT

P1 = 3

Queue:

TEXT

P2 P3 P1

P2 executes 2.

Remaining:

TEXT

P2 = 2

Queue:

TEXT

P3 P1 P2

P3 executes 2.

Finishes.

Queue:

TEXT

P1 P2

44. SECOND ROUND

P1 executes 2.

Remaining:

TEXT

P1 = 1

P2 executes 2.

Finishes.

P1 executes 1.

Finishes.

Gantt:

TEXT

0 2 4 6 8 10 11
|P1|P2|P3|P1| P2 |P1|

45. COMPLETION TIMES

TEXT

P1 = 11

P2 = 10

P3 = 6

Since all AT=0:

$\forall$ TAT= CT $\forall$

So:

TEXT

P1 = 11

P2 = 10

P3 = 6

Waiting:

$$\backslash [WT_{\{P1\}}=11-5=6 \backslash]$$

$$\backslash [WT_{\{P2\}}=10-4=6 \backslash]$$

$$\backslash [WT_{\{P3\}}=6-2=4 \backslash]$$

46. RESPONSE TIMES IN RR

First start:

TEXT

P1 = 0

P2 = 2

P3 = 4

Therefore:

$$\backslash [RT_{\{P1\}}=0 \backslash]$$

$$\backslash [RT_{\{P2\}}=2 \backslash]$$

$$\backslash [RT_{\{P3\}}=4 \backslash]$$

47. ROUND ROBIN WITH ARRIVAL TIMES

This is where many GATE mistakes occur.

At each event:

- quantum expires
- process finishes
- new process arrives

you must correctly update the ready queue.

Do not blindly rotate initial processes.

48. RR READY-QUEUE RULE

When process uses quantum but does not finish:

It is appended to the end of the ready queue.

Processes that arrive during its execution are also inserted according to their arrival times.

The exact tie handling at an identical timestamp may depend on convention/question wording, so be careful.

49. QUANTUM EFFECT

Very important.

If quantum is very large:

Round Robin approaches:

$\boxed{\text{FCFS}}$

because each process tends to finish before quantum ends.

If quantum is very small:

- better response may result
- context switching increases significantly

So:

Tiny quantum != huge context-switch overhead.

50. RR TRADE-OFF

TEXT

Large q
!"
less switching
but poorer interactive response

TEXT

Small q
!"
better responsiveness
but more context switching

51. CONTEXT SWITCH COUNT

GATE may ask:

Number of context switches in RR.

Be careful:

A change from one process to another counts as a context switch.

Whether initial dispatch or final transition to idle counts depends on wording.

Do not silently include/exclude them.

52. RR QUICK EXAMPLE

Burst:

TEXT

P1=10

Quantum:

$q=3$

Number of CPU slices:

$\lceil \frac{10}{3} \rceil = 4$

Slices:

TEXT

3, 3, 3, 1

This helps estimate scheduling events.

53. NUMBER OF QUANTA

For burst time B and quantum q :

$\boxed{\lceil \frac{B}{q} \rceil}$

CPU allocations needed for that process if ignoring arrival/interleaving complications.

PART F — ALGORITHM COMPARISON

54. MASTER TABLE

Algorithm	Preemptive?	Selection Basis	Main Issue
FCFS	No	Arrival order	Convoy effect
SJF	No	Shortest burst	Starvation
SRTF	Yes	Shortest remaining	Starvation + switches
Priority	Either	Priority	Starvation
Round Robin	Yes	Time quantum	Quantum overhead

55. WHICH MINIMIZES AVG WAITING TIME?

Under ideal known burst times:

$\boxed{SJF}$

This is a famous result.

Its preemptive counterpart SRTF is similarly optimal for minimizing average completion/turnaround style criteria under suitable assumptions, but for standard GATE memory:

56. WHICH IS GOOD FOR INTERACTIVE/TIME-SHARING SYSTEMS?

$\boxed{\text{Round Robin}}$

because every ready process gets CPU periodically.

57. WHICH SUFFERS CONVOY EFFECT?

$\boxed{\text{FCFS}}$

58. WHICH CAN STARVE?

Typically:

- SJF
- SRTF
- Priority

can starve some processes.

Round Robin is generally much fairer if the ready queue operates fairly.

59. AGING IS ASSOCIATED WITH

$\boxed{\text{Priority Scheduling}}$

to prevent starvation.

60. IDLE CPU TIME

Suppose:

TEXT

P1 AT = 3

and no earlier process exists.

CPU from:

TEXT

0 !' 3

is idle.

Gantt:

TEXT

0 3 ...
| IDLE | P1 ...

Do not begin P1 at time 0.

& pGATE TRAP

Arrival Time matters.

If nothing has arrived:

CPU may be idle.

61. FULL NUMERICAL — FCFS WITH IDLE TIME

Process	AT	BT
P1	2	3
P2	4	2
P3	8	1

Gantt:

TEXT

0 2 5 7 8 9
| Idle | P1 | P2 | I | P3 |

CT:

TEXT

P1=5
P2=7
P3=9

TAT:

\[P1=5-2=3 \]

\[P2=7-4=3 \]

\[P3=9-8=1 \]

WT:

\[P1=3-3=0 \]

\[P2=3-2=1 \]

\[P3=1-1=0 \]

62. CPU UTILIZATION EXAMPLE

Busy intervals:

TEXT

```
2! ' 7 = 5 u n i t s
8! ' 9 = 1 u n i t
```

Total busy:

$\backslash [6 \backslash]$

Observed interval:

$\backslash [0! ' 9= 9 \backslash]$

CPU utilization:

$\backslash [\frac{6}{9} \backslash] \times 100 = \boxed{66.67\%} \backslash]$

63. SJF FULL EXAMPLE

Process	AT	BT
P1	0	5
P2	1	2
P3	2	3
P4	3	1

Non-preemptive SJF.

At time 0:

Only P1 available.

Run:

TEXT

```
P 1 : 0! ' 5
```

At 5, all P2,P3,P4 ready.

Shortest P4:

TEXT

```
5! ' 6
```

Then P2:

TEXT

```
6! ' 8
```

Then P3:

TEXT

8! ' 1 1

Gantt:

TEXT

0 5 6 8 11
| P1 | P4 | P2 | P3 |

64. CT

TEXT

P1=5
P2=8
P3=11
P4=6

TAT:

TEXT

P1=5
P2=7
P3=9
P4=3

WT:

TEXT

P1=0
P2=5
P3=6
P4=2

65. SRTF EXAM METHOD

At every process arrival:

1. subtract execution already done from current process
2. compare **remaining times**
3. choose minimum
4. continue until next arrival/completion

Do not compare original burst time after a process has partially executed.

& pGATE TRAP

SRTF uses:

$\lfloor \boxed{\text{Remaining Time}} \rfloor$

not original Burst Time.

66. PRIORITY EXAM METHOD

At every scheduling decision:

1. list only arrived ready processes
2. compare priority
3. use correct convention
4. if preemptive, reconsider on every new arrival
5. if non-preemptive, running process finishes burst

67. ROUND ROBIN EXAM METHOD

Use a literal queue.

Example:

TEXT

Queue: P1 P2 P3

After P1's incomplete quantum:

TEXT

P2 P3 P1

Then add any newly arrived processes appropriately.

Do not solve RR only from intuition.

68. GANTT CHART SHORTCUT

Always create timeline:

TEXT

0	4	7	10
P1	P2	P3	

Then CT is simply:

right end of each process's final execution block.

This prevents many calculation mistakes.

69. AVERAGE TIMES

Average Waiting Time:

$$\boxed{AWT = \frac{\sum WT_i}{n}}$$

Average Turnaround Time:

$$\boxed{ATAT = \frac{\sum TAT_i}{n}}$$

Average Response Time:

$$\boxed{ART = \frac{\sum RT_i}{n}}$$

70. NORMALIZED TURNAROUND TIME

Sometimes scheduling theory may use:

$$\boxed{\text{Normalized TAT} = \frac{TAT}{BT}}$$

Know it if supplied in a question, but basic GATE scheduling focuses heavily on WT/TAT/RT.

71. TURNAROUND TIME INTERPRETATION

TAT contains:

TEXT

Waiting Time + CPU execution time

So:

$$\boxed{TAT = WT + BT}$$

which is equivalent to:

$$\boxed{WT = TAT - BT}$$

72. RESPONSE TIME CAN BE ZERO

If process begins execution exactly when it arrives:

$$\boxed{RT = 0}$$

This frequently occurs for the first scheduled process.

73. WAITING TIME CAN ALSO BE ZERO

If process receives CPU whenever it is ready and never waits in ready queue:

$$\boxed{WT = 0}$$

74. RESPONSE TIME "dWAITING TIME?

For common single-CPU-burst scheduling models:

Response time is the initial waiting portion.

Total waiting time includes initial plus later ready-queue waits.

So often:

$[RT \leq WT]$

But rely on definitions/model given rather than memorizing it as an unrestricted universal law.

75. COMPLETION TIME IS NOT TURNAROUND TIME

Example:

```
TEXT
AT=5
CT=20
```

Then:

$[TAT=20-5=15]$

Not 20.

& pGATE TRAP

Always subtract arrival time.

76. PREEMPTIVE SCHEDULING TRIGGER

Typical preemption events:

- higher-priority process arrives
- shorter remaining job arrives
- RR quantum expires

Not every arrival necessarily causes preemption.

Example SRTF:

Current remaining = 2.

New process burst = 8.

Current continues.

77. TIE BREAKING

If two processes have equal:

- burst
- remaining time
- priority

then scheduler may use:

- FCFS among ties
 - lower PID
 - stated rule
-

Do not invent a tie-breaking rule if question specifies one.

If it doesn't, GATE usually designs the question so ambiguity doesn't affect answer or uses a conventional tie resolution explicitly/implicitly.

78. CONTEXT SWITCH OVERHEAD IN RR

Suppose context-switch cost:

$$c$$

If there are k relevant switches:

$$\text{Total overhead} = k \times c$$

But again, determine exactly what transitions are counted.

79. EFFECTIVE CPU TIME WITH CONTEXT SWITCH

If useful process execution:

$$U$$

and context-switch overhead:

$$O$$

then efficiency conceptually:

$$\boxed{\frac{U}{U+O}}$$

This may appear indirectly.

80. RR QUANTUM AND OVERHEAD EXAMPLE

Suppose each process receives:

$$q = 10\text{ms}$$

context switch:

$$1\text{ms}$$

For each full slice pattern, approximate CPU useful fraction:

$$\frac{10}{10+1} = 90.9\%$$

This is conceptual; exact questions may define whether overhead occurs after every slice.

81. FCFS VS RR WHEN $q \rightarrow \infty$

As:

$$q \rightarrow \infty$$

or simply quantum exceeds all burst times:

$$\boxed{RR \rightarrow FCFS}$$

Very important conceptual result.

82. RR WHEN q IS VERY SMALL

As quantum becomes tiny:

- responsiveness improves
- context switches explode
- overhead rises

So quantum cannot be chosen arbitrarily small.

83. SJF BURST PREDICTION

In real systems, exact future CPU burst is generally unknown.

Textbooks sometimes estimate it using exponential averaging:

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$$

where:

- t_n = actual previous burst
- τ_n = previous prediction
- τ_{n+1} = next prediction
- $0 \leq \alpha \leq 1$

This is worth knowing if it appears in GATE context.

84. EFFECT OF α

If:

$$\alpha = 1$$

then:

$$\tau_{n+1} = t_n$$

Only latest actual burst matters.

If:

$$\alpha = 0$$

then:

$$\tau_{n+1} = \tau_n$$

New actual burst contributes nothing.

85. RESPONSE VS TURNAROUND FOR INTERACTIVE SYSTEMS

Interactive users care about:

How quickly does the system respond initially?

So:

$\boxed{\text{Response Time}}$

is especially important.

Batch systems may care more about:

- throughput
- turnaround time

86. HOW GATE CAN ASK CPU SCHEDULING

Common patterns:

Pattern 1 — Gantt chart

Find execution order.

Pattern 2 — Average waiting time

Very common.

Pattern 3 — Turnaround time

Direct numerical.

Pattern 4 — Response time

Especially with RR/preemptive scheduling.

Pattern 5 — Context switches

RR/SRTF/Priority.

Pattern 6 — Compare algorithms

Conceptual.

Pattern 7 — Starvation / aging

Priority/SJF.

Pattern 8 — Convoy effect

FCFS.

Pattern 9 — Quantum behavior

RR.

Pattern 10 — Burst prediction

SJF theory.

& pGATE TRAPS — CPU SCHEDULING

Trap 1

Use burst time instead of remaining time in SRTF.

'L

Trap 2

Choose shortest future process before it arrives.

'L

Trap 3

Forget CPU idle interval.

'L

Trap 4

Use:

$\lfloor TAT = CT \rfloor$

even when $AT \neq 0$.

'L

Correct:

$\lfloor TAT = CT - AT \rfloor$

Trap 5

Use:

$\lfloor WT = CT - BT \rfloor$

'L if $AT \neq 0$.

Correct:

$\lfloor WT = CT - AT - BT \rfloor$

Trap 6

Response time means total waiting time.

'L

Response only until first execution.

Trap 7

SJF is preemptive.

Not by default.

SJF normally refers to non-preemptive.

SRTF is preemptive.

Trap 8

Priority number 1 is always highest priority.

'L

Convention depends on problem.

Trap 9

Round Robin is non-preemptive.

'L

Quantum expiry causes preemption.

Trap 10

Large quantum makes RR more preemptive.

Actually:

Large quantum makes RR behave more like FCFS.

' p WRITE THIS IN YOUR NOTES

Scheduling Formulas

$\boxed{TAT=CT-AT}$

$\boxed{WT=TAT-BT}$

$\boxed{RT=FirstStart-AT}$

FCFS

TEXT

F i r s t a r r i v a l ! ' f i r s t e x e c u t i o n

- non-preemptive
 - convoy effect
-

SJF

TEXT

Shortest burst among READY processes

- non-preemptive
- minimum average WT ideally
- starvation possible

SRTF

TEXT

Shortest remaining time

- preemptive SJF
- reconsider on arrivals

Priority

TEXT

Highest priority chosen

- preemptive/non-preemptive
- starvation possible
- Aging prevents starvation

Round Robin

TEXT

Each gets quantum q
un finished !' back o f q u e u e

- preemptive
- fair/time-sharing
- large q !' FCFS
- tiny q !' high switching overhead

MASTER MEMORY TRICK

F S S P R

FCFS SJF SRTF Priority Round Robin

Story:

First, Short, Shorter-Remaining, Priority, Rotate.

That tells you the selection logic.

ALGORITHM IDENTIFICATION SHORTCUT

FCFS

Who arrived first?

SJF

Who has shortest total burst among ready?

SRTF

Who has shortest remaining time now?

Priority

Who has best priority?

RR

Who is at front of rotating queue?

NUMERICAL SOLVING TEMPLATE

For every scheduling problem:

GIVEN

Write table:

TEXT

Process | AT | BT | Priority

STEP 1

Draw Gantt chart.

STEP 2

Find CT from final execution point.

STEP 3

$\sqrt{[TAT=CT-AT]}$

STEP 4

$\sqrt{[WT=TAT-BT]}$

STEP 5

$\sqrt{[RT=FirstStart-AT]}$

STEP 6

Calculate averages if requested.

30-SECOND REVISION

AT?

! ' Arrival.

BT?

! ' CPU burst.

CT?

! ' Finish time.

TAT?

$\sqrt{[CT-AT]}$

WT?

$\sqrt{[TAT-BT]}$

RT?

! ' First start " arrival.

FCFS issue?

! ' Convoy effect.

SJF advantage?

! ' Minimum average waiting ideally.

SRTF?

! ' Preemptive SJF.

Priority issue?

! ' Starvation.

Fix starvation?

! ' Aging.

RR?

!' Time quantum.

Large quantum?

!' FCFS-like.

Small quantum?

!' More context switching.

ACTIVE RECALL

1. What is the difference between waiting time and response time?
2. Write formulas for TAT and WT.
3. Why does FCFS suffer convoy effect?
4. Why can SJF cause starvation?
5. Difference between SJF and SRTF?
6. What is aging?
7. Is priority scheduling always preemptive?
8. Why is Round Robin suitable for time sharing?
9. What happens when RR quantum becomes very large?
10. What happens when quantum becomes extremely small?
11. In SRTF, do you compare original or remaining burst?
12. If a process arrives at 4, starts first at 7, finishes at 15, BT=6, find RT, TAT, WT.

Answer:

$RT = 7 - 4 = \boxed{3}$

$TAT = 15 - 4 = \boxed{11}$

$WT = 11 - 6 = \boxed{5}$

MASTER CONNECTION

TEXT

Ready Queue

!"

CPU Scheduler

!"

FCFS / SJF / SRTF / Priority / RR

!"

Gantt Chart

!"

CT

!"

TAT

!"

WT + RT

The single most important calculation chain is:

Gantt !' CT !' TAT !' WT

and separately:

First start !' RT

NEXT — CHAPTER 10 +PØ=Ý%

Next we cover the other scheduling topic in the official syllabus:

I/O / DISK SCHEDULING

We will study:

- disk structure needed for GATE
- tracks, cylinders and seek time
- request queue
- head movement
- FCFS
- SSTF
- SCAN
- C-SCAN
- LOOK
- C-LOOK
- direction-based numericals
- total head movement
- starvation
- unfairness
- elevator analogy
- exact difference between SCAN vs LOOK and C-SCAN vs C-LOOK
- fastest GATE numerical shortcuts.

USER

NEXT

ASSISTANT

CHAPTER 10 — I/O / DISK SCHEDULING +PØ=Ý%

Disk scheduling is the second major scheduling topic in the GATE OS syllabus.

The central idea is simple:

TEXT

Disk Requests
! "
Request Queue

!"
Disk Scheduler
!"
Choose order
!"
Reduce head movement / access delay

Most GATE questions ask you to calculate **total disk-head movement**.

1. WHY DO WE NEED DISK SCHEDULING?

Suppose disk requests arrive for cylinders:

TEXT
98, 183, 37, 122, 14, 124, 65, 67

and the disk head is currently at:

TEXT
53

The disk could serve them in many different orders.

Bad order:

TEXT
5 3 !' 1 8 3 !' 1 4 !' 1 2 4 !' 3 7 . . .

causes a lot of head movement.

Better ordering can reduce movement significantly.

That is why we use:

Disk Scheduling Algorithms

2. BASIC DISK TERMINOLOGY

For GATE, know the basics.

A magnetic disk is conceptually organized into:

- platters
- tracks
- sectors
- cylinders

For disk scheduling, requests are usually represented by:

Cylinder numbers

Example:

TEXT

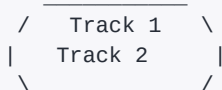
0 ... 199

3. TRACK

A track is a circular path on a disk surface.

Conceptually:

TEXT



4. SECTOR

Each track is divided into smaller units called:

sectors

A sector stores a block of data.

5. CYLINDER

Tracks at the same radius across different platters form a:

Cylinder

Disk scheduling questions often simply give:

TEXT

Cylinder 37
Cylinder 122
...

You usually do not need physical platter geometry to solve them.

6. DISK ACCESS TIME

A simplified disk access time contains components such as:

$\boxed{\text{Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}}$

Sometimes controller/queueing overhead may also exist.

For disk-scheduling questions, the main focus is often:

Seek Time

7. SEEK TIME

Seek time is the time required to move the disk arm/head to the required track/cylinder.

If head moves:

TEXT

50 !' 80

movement:

$|80-50|=30$

cylinders.

If seek time is proportional to head movement, minimizing total movement helps reduce seek time.

8. TOTAL HEAD MOVEMENT

0 = Min numerical quantity.

If service order is:

TEXT

50 !' 70 !' 20 !' 30

total movement:

$|$

70-50	+	20-70	+	30-20
-------	---	-------	---	-------

$|$

$| = 20+50+10$

$\boxed{80}$

Always use absolute difference.

MEMORY

Disk scheduling = Add absolute jumps.

9. MAIN DISK SCHEDULING ALGORITHMS

We will cover:

1. FCFS

2. SSTF
3. SCAN
4. C-SCAN
5. LOOK
6. C-LOOK

Master map:

TEXT
FCFS
SSTF
SCAN
C-SCAN
LOOK
C-LOOK

PART A — FCFS

10. FCFS DISK SCHEDULING

Same idea as CPU FCFS:

Serve requests in arrival order.

Suppose:

TEXT
Queue:
98, 183, 37, 122

Head:

TEXT
53

Order:

TEXT
5 3 !' 9 8 !' 1 8 3 !' 3 7 !' 1 2 2

11. FCFS EXAMPLE

Calculate movement:

$$\lfloor |98-53|=45 \rfloor$$

$$\lfloor |183-98|=85 \rfloor$$

$$\lfloor |37-183|=146 \rfloor$$

$$\lfloor |122-37|=85 \rfloor$$

Total:

$\lfloor 45+85+146+85 \rfloor$

$\lfloor \boxed{361} \rfloor$

12. FCFS PROPERTIES

Advantages:

- simple
- fair by arrival order
- no starvation due to reordering

Disadvantage:

- very large head movement possible
- poor average seek time

MEMORY

FCFS = Follow queue exactly.

PART B — SSTF

13. SHORTEST SEEK TIME FIRST — SSTF

SSTF chooses:

The request closest to the current head position.

At every step:

TEXT

Current Head

!"

Find minimum distance

!"

Serve nearest request

14. SSTF EXAMPLE

Current head:

TEXT

50

Requests:

TEXT

10, 20, 55, 90

Distances:

TEXT

to 10 = 40
to 20 = 30
to 55 = 5
to 90 = 40

Choose:

TEXT

55

Now current head = 55.

Remaining:

TEXT

10, 20, 90

Distances:

TEXT

45, 35, 35

There is a tie between 20 and 90.

Tie-breaking depends on problem convention.

15. SSTF ADVANTAGE

Compared with FCFS, it often reduces:

average head movement / seek time.

16. SSTF DISADVANTAGE

Far-away requests may keep waiting if nearby requests continue arriving.

This can cause:

Starvation

Example:

TEXT

head around 50

new requests continuously:
48, 52, 49, 51...

A request at:

TEXT

190

may wait a long time.

MEMORY

SSTF = Serve nearest.

And:

Nearest can starve farthest.

PART C — SCAN Ø=Ý%

17. SCAN ALGORITHM

SCAN is also called:

Elevator Algorithm

The disk head moves in one direction, servicing requests along the way.

It continues to the physical end of the disk in that direction, then reverses.

Example:

TEXT

0 ----- 199
 !'
 Head

If moving right:

TEXT

! ' ! ' ! ' ! ' e n d
! • ! • ! • ! • ! •

Exactly like an elevator.

18. WHY “ELEVATOR”?

Imagine elevator at floor 5 moving upward.

Requests:

TEXT

7, 9, 2, 3

It first serves:

TEXT

7, 9

then reverses and serves:

TEXT

3, 2

Disk SCAN behaves similarly.

19. SCAN EXAMPLE

Disk cylinders:

TEXT

0 to 199

Head:

TEXT

53

Requests:

TEXT

98, 183, 37, 122, 14, 124, 65, 67

Assume head initially moves:

TEXT

toward higher cylinder numbers

Separate requests:

Right side

TEXT

65, 67, 98, 122, 124, 183

Left side

TEXT

37, 14

SCAN order:

TEXT

5 3 !' 6 5 !' 6 7 !' 9 8 !' 1 2 2 !' 1 2 4 !' 1 8 3 !' 1 9 9
!' 3 7 !' 1 4

Notice:

SCAN goes all the way to physical endpoint 199 before reversing.

20. SCAN MOVEMENT SHORTCUT

Instead of adding every jump, use endpoints.

From:

TEXT

5 3 !' 1 9 9

movement:

$\backslash [199-53=146 \backslash]$

Then:

TEXT

1 9 9 !' 1 4

movement:

$\backslash [199-14=185 \backslash]$

Total:

$\backslash [146+185 \backslash]$

$\backslash \boxed{331} \backslash$

This shortcut works because movement is monotonic in each direction.

21. VERY IMPORTANT SCAN RULE

If moving toward higher cylinders:

TEXT

serve all larger requests
!' go to disk end
!' reverse
!' serve lower requests

If moving toward lower cylinders:

TEXT

```
serve all lower requests
!' go to 0
!' reverse
!' serve larger requests
```

PART D — C-SCAN

22. CIRCULAR SCAN

C-SCAN gives more uniform waiting time.

Head moves in only one service direction.

Example moving upward:

TEXT

```
5 3 !' . . . !' 1 9 9
```

After reaching the end:

TEXT

```
1 9 9 !' 0
```

Then continues moving upward again.

It does **not** serve requests while returning from 199 to 0.

23. C-SCAN VISUAL

TEXT

```
0 ----- 199
```

```
!' !' !' !' !' !' !' !' !' !'
```

At 199:

jump back to 0

```
!' !' !' !' !' a g a i n
```

Think:

Circular elevator

It always serves in the same direction.

24. C-SCAN EXAMPLE

Same requests:

TEXT

98, 183, 37, 122, 14, 124, 65, 67

Head:

TEXT

53

moving toward higher numbers.

Service:

TEXT

5 3 !' 6 5 !' 6 7 !' 9 8 !' 1 2 2 !' 1 2 4 !' 1 8 3 !' 1 9 9

Then jump:

TEXT

1 9 9 !' 0

Then:

TEXT

0 !' 1 4 !' 3 7

Order:

TEXT

65, 67, 98, 122, 124, 183, 14, 37

with endpoint travel included.

25. C-SCAN TOTAL MOVEMENT

Movement:

TEXT

5 3 !' 1 9 9

\[146 \]

Jump:

TEXT

1 9 9 !' 0

\[199 \]

Then:

TEXT

0 ! ' 3 7

\[37 \]

Total:

\[146+199+37 \]

\[\boxed{382} \]

& pGATE TRAP

In most standard head-movement calculations:

The return jump in C-SCAN is counted in total head movement.

Unless the question explicitly defines otherwise.

26. SCAN VS C-SCAN

SCAN	C-SCAN
Services in both directions	Services in one direction
Reverses at endpoint	Jumps to opposite endpoint
Elevator	Circular elevator
Waiting can vary by position	More uniform waiting

MEMORY

SCAN

Go and come back

C-SCAN

Go, jump, go again

PART E — LOOK

27. LOOK ALGORITHM

LOOK is similar to SCAN, but:

Head does NOT go to physical disk end unless there is a request there.

It goes only as far as the last request in the current direction.

Then reverses.

This avoids unnecessary movement.

28. SCAN VS LOOK

Suppose disk ends at:

TEXT

199

but highest request is:

TEXT

183

SCAN

TEXT

. . . !' 1 8 3 !' 1 9 9 !' r e v e r s e

LOOK

TEXT

. . . !' 1 8 3 !' r e v e r s e

So LOOK literally:

looks ahead for the last request.

29. LOOK EXAMPLE

Same data:

Head:

TEXT

53

moving high.

Requests:

TEXT

65, 67, 98, 122, 124, 183

then left:

TEXT

37, 14

LOOK order:

TEXT

5 3 !' 6 5 !' 6 7 !' 9 8 !' 1 2 2 !' 1 2 4 !' 1 8 3
!' 3 7 !' 1 4

No trip to 199.

30. LOOK MOVEMENT

From:

TEXT

5 3 !' 1 8 3

\[130 \]

Then:

TEXT

1 8 3 !' 1 4

\[169 \]

Total:

\[130+169 \]

\[\boxed{299} \]

MEMORY

SCAN sees the wall. LOOK sees the last request.

Excellent exam shortcut.

PART F — C-LOOK

31. C-LOOK

C-LOOK combines:

- circular servicing
- no unnecessary physical-end movement

Head travels in one direction until the last request.

Then jumps directly to the lowest request and continues in same direction.

32. C-LOOK EXAMPLE

Head:

TEXT
53

moving high.

Requests:

TEXT
65, 67, 98, 122, 124, 183

then lower requests:

TEXT
14, 37

Service:

TEXT
5 3 !' 6 5 !' 6 7 !' 9 8 !' 1 2 2 !' 1 2 4 !' 1 8 3

Then jump directly:

TEXT
1 8 3 !' 1 4

Then:

TEXT
1 4 !' 3 7

No travel to:

TEXT
199 or 0

33. C-LOOK MOVEMENT

$\backslash$ 53 !' 183 = 130 $\backslash$

$\backslash$ 183 !' 14 = 169 $\backslash$

$\lfloor 14 \cdot 37 = 23 \rfloor$

Total:

$\lfloor 130 + 169 + 23 \rfloor$

$\lfloor \boxed{322} \rfloor$

34. C-SCAN VS C-LOOK

C-SCAN	C-LOOK
Goes to physical endpoint	Stops at last request
Jumps end to opposite end	Jumps last request to first request
More movement	Usually less movement
Circular servicing	Circular servicing

Memory:

C-SCAN jumps wall-to-wall.

C-LOOK jumps request-to-request.

35. MASTER COMPARISON

Algorithm	Rule	Endpoint?	Direction
FCFS	Arrival order	No special rule	Random
SSTF	Nearest request	No	Depends on nearest
SCAN	Elevator	Physical end	Both directions
C-SCAN	Circular elevator	Physical ends	One service direction
LOOK	SCAN without unnecessary ends	Last request	Both directions
C-LOOK	Circular LOOK	Last request	One service direction

36. THE MOST IMPORTANT DIFFERENCE

Remember these two questions:

Does it go to the disk boundary?

If yes:

TEXT

SCAN / C-SCAN

If no, only goes to last request:

TEXT

LOOK / C-LOOK

Does it service in both directions?

If yes:

TEXT

SCAN / LOOK

If one direction only:

TEXT

C-SCAN / C-LOOK

This 2×2 table solves many MCQs:

	Both Directions	One Direction
Goes to End	SCAN	C-SCAN
Stops at Last Request	LOOK	C-LOOK

Memorize this table.

37. STANDARD NUMERICAL METHOD

Suppose question gives:

- request queue
- current head
- direction
- disk range
- algorithm

Do this:

Step 1

Sort requests.

Example:

TEXT

14, 37, 65, 67, 98, 122, 124, 183

Step 2

Place current head:

TEXT

14, 37 | 53 | 65, 67, 98, 122, 124, 183

Step 3

Check direction.

Example:

TEXT

toward higher

Step 4

Apply algorithm rule.

Step 5

Add absolute movements or use endpoint shortcut.

38. SORTING SHORTCUT

Immediately divide:

TEXT

Requests < Head

Requests > Head

For head 53:

TEXT

Left:

14, 37

Right:

65, 67, 98, 122, 124, 183

This makes SCAN-family questions much easier.

39. SCAN FAMILY SHORTCUT

If moving right:

SCAN

TEXT

H e a d ! ' R i g h t E n d ! ' L e f t m o s t r e q u e s t

LOOK

TEXT

Head !' Highest request !' Lowest request

C-SCAN

TEXT

Head !' Right End !' Left End !' highest low-side request

C-LOOK

TEXT

Head !' Highest request !' Lowest request !' remaining up w

40. FULL COMPARISON USING SAME EXAMPLE

Disk:

TEXT

0-199

Head:

TEXT

53

Queue:

TEXT

98,183,37,122,14,124,65,67

Initial direction:

TEXT

higher cylinders

FCFS

TEXT

5 3 !' 9 8 !' 1 8 3 !' 3 7 !' 1 2 2 !' 1 4 !' 1 2 4 !' 6 5 !' 6 7

Movement:

$\lfloor 640 \rfloor$

Let's verify:

$\lfloor 45+85+146+85+108+110+59+2 \rfloor$

$\lfloor =\boxed{640} \rfloor$

SCAN

TEXT
5 3!' 6 5!' 6 7!' 9 8!' 1 2 2!' 1 2 4!' 1 8 3!' 1 9 9!' 3 7!' 1 4

\[\boxed{331} \]

C-SCAN

TEXT
5 3!' 6 5!' 6 7!' 9 8!' 1 2 2!' 1 2 4!' 1 8 3!' 1 9 9!' 0!' 1 4!' 3 7

\[\boxed{382} \]

LOOK

TEXT
5 3!' 6 5!' 6 7!' 9 8!' 1 2 2!' 1 2 4!' 1 8 3!' 3 7!' 1 4

\[\boxed{299} \]

C-LOOK

TEXT
5 3!' 6 5!' 6 7!' 9 8!' 1 2 2!' 1 2 4!' 1 8 3!' 1 4!' 3 7

\[\boxed{322} \]

SSTF depends on repeated nearest-request choices and tie rules; calculate step-by-step.

41. SSTF FULL TRACE

Same data:

Head:

TEXT
53

Requests:

TEXT
14, 37, 65, 67, 98, 122, 124, 183

Nearest to 53:

TEXT
6 5 !' d i s t a n c e 1 2
3 7 !' d i s t a n c e 1 6

Choose 65.

From 65:

TEXT

6 7 !' 2

Choose 67.

From 67:

TEXT

3 7 !' 3 0
9 8 !' 3 1

Choose 37.

From 37:

TEXT

1 4 !' 2 3
9 8 !' 6 1

Choose 14.

From 14:

TEXT

9 8 !' 8 4

Then:

TEXT

9 8 !' 1 2 2 !' 1 2 4 !' 1 8 3

Order:

TEXT

5 3!' 6 5!' 6 7!' 3 7!' 1 4!' 9 8!' 1 2 2!' 1 2 4!' 1 8 3

Movement:

$\backslash [12+2+30+23+84+24+2+59 \backslash]$

$\backslash [= \boxed{236} \backslash]$

In this fixed queue example, SSTF gives a lower movement than the others.

But remember:

Low movement here does not mean SSTF is always best overall—starvation is possible.

42. GATE TRAP — DIRECTION

For:

- SCAN
- C-SCAN
- LOOK
- C-LOOK

initial direction matters.

If direction is not given, the answer may be ambiguous.

GATE usually supplies enough information or constructs choices appropriately.

Never silently assume right/up unless stated or clearly implied.

43. GATE TRAP — ENDPOINTS

Suppose highest request is 180 and disk ends at 199.

SCAN

goes:

TEXT

1 8 0 ! ' 1 9 9

LOOK

stops:

TEXT

180

This tiny distinction changes total movement.

44. GATE TRAP — CIRCULAR JUMP

In C-SCAN/C-LOOK, the head must physically reposition.

Unless the problem says otherwise:

Include jump distance in head movement.

45. GATE TRAP — SERVICE DURING C-SCAN RETURN

Suppose moving right.

At:

TEXT

199

C-SCAN returns to 0.

Does it serve requests during:

TEXT

1 9 9 !' 0

No.

It treats this as repositioning.

Service resumes in the original direction.

46. GATE TRAP — C-LOOK RETURN

Same concept.

If moving upward:

TEXT

h i g h e s t r e q u e s t !' l o w e s t r e q u e s t

is a circular jump.

Requests are then served upward again.

47. STARVATIONCOMPARISON

FCFS

Generally no starvation from scheduling reordering.

SSTF

Can starve far-away requests.

SCAN

Much less starvation-prone because head eventually sweeps both directions.

C-SCAN

Provides more uniform wait across cylinder positions.

48. WHY C-SCAN HAS MORE UNIFORM WAITING

SCAN favors middle cylinders somewhat because the head passes them in both directions.

C-SCAN serves only in one direction.

Therefore waiting time is more uniform across cylinders.

Think:

Everyone waits for the same circular sweep.

49. SEEK TIME VS HEAD MOVEMENT

If seek time per cylinder is constant:

$$\text{Seek Time} = \text{Movement} \times \text{TimePerCylinder}$$

Example:

Total movement:

$$200$$

Time per cylinder:

$$0.2\text{ms}$$

Then:

$$200 \times 0.2 = \boxed{40\text{ms}}$$

50. AVERAGE SEEK LENGTH

If n requests are serviced and total movement is M :

$$\boxed{\text{Average head movement} = \frac{M}{n}}$$

provided that's what the question defines.

51. ROTATIONAL LATENCY

After reaching the correct track, the desired sector may not yet be under the read/write head.

Disk must rotate.

That waiting is:

Rotational Latency

52. AVERAGE ROTATIONAL LATENCY

If one full rotation takes:

$$T$$

then average rotational latency is:

$$\boxed{\frac{T}{2}}$$

because average wait is half a revolution.

53. RPM FORMULA

If disk rotates at:

$\backslash \text{RPM} \backslash$

then time for one revolution:

$\backslash \boxed{ T_{\text{rev}} = \frac{60}{\text{RPM}} \text{ seconds} } \backslash$

or:

$\backslash \boxed{ T_{\text{rev}} = \frac{60,000}{\text{RPM}} \text{ ms} } \backslash$

54. EXAMPLE

Disk:

$\backslash 7200 \text{ RPM} \backslash$

One revolution:

$\backslash \frac{60}{7200} = 0.008333\text{s} \backslash$

$\backslash = 8.33\text{ms} \backslash$

Average rotational latency:

$\backslash \frac{8.33}{2} = \boxed{4.17\text{ms}} \backslash$

FORMULA BOX — DISK ACCESS

$\backslash \boxed{ \text{Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time} } \backslash$

Average rotation:

$\backslash \boxed{ \text{Rotational Latency}_{\text{avg}} = \frac{1}{2} T_{\text{rotation}} } \backslash$

One revolution:

$\backslash \boxed{ T_{\text{rotation}} = \frac{60}{\text{RPM}} } \backslash$

55. TRANSFER TIME

Once the correct sector is under the head, actual data is transferred.

If one full track has N bytes and takes T seconds per rotation, transfer time for b bytes conceptually:

$\backslash \boxed{ \text{Transfer Time} = \frac{b}{N} \times T } \backslash$

when data is contiguous on that track and assumptions fit.

Use exact formulas supplied in question.

56. DISK SCHEDULING VS CPU SCHEDULING

Do not mix them.

CPU scheduling decides:

Which ready process gets CPU?

Disk scheduling decides:

Which pending disk request should the disk head serve next?

57. HOW GATE CAN ASK

Common patterns:

Pattern 1

Total head movement.

Pattern 2

Compare SCAN/LOOK.

Pattern 3

Identify starvation.

Pattern 4

Find service order.

Pattern 5

Initial direction effects.

Pattern 6

Disk access-time calculation.

Pattern 7

RPM / rotational delay.

& pGATE TRAPS — DISK SCHEDULING

Trap 1

SCAN and LOOK are identical.

'L

SCAN reaches physical end.

LOOK stops at last request.

Trap 2

C-SCAN services on both directions.

'L

Only one service direction.

Trap 3

C-LOOK goes to physical disk boundaries.

'L

It jumps between extreme pending requests.

Trap 4

SSTF always guarantees fairness.

'L

Can starve distant requests.

Trap 5

FCFS means sorted cylinder order.

'L

It means arrival order.

Trap 6

Ignore initial head direction in SCAN-family algorithms.

'L

Direction changes answer.

Trap 7

Head movement is signed difference.

'L

Use absolute distance:

$\backslash$

a-b

$\backslash$

Trap 8

Rotational latency and seek time are the same.

'L

Seek:

move head radially.

Rotation:

wait for sector to rotate under head.

' p WRITE THIS IN YOUR NOTES

Disk Scheduling

Goal:

Reduce head movement / seek time.

Total Head Movement

$\boxed{\sum |Current-Next|}$

FCFS

Arrival order.

Pros:

- simple
- fair

Con:

- high seek movement.

SSTF

Nearest request first.

Pros:

- lower average movement

Con:

Starvation possible.

SCAN

TEXT

Go to disk end
then reverse

Elevator algorithm.

C-SCAN

TEXT

Go to end
jump to other end
continue same direction

LOOK

TEXT

Go only to last request
then reverse

C-LOOK

TEXT

Go to last request
jump to opposite extreme request
continue same direction

MASTER 2×2 TABLE

	Both Directions	One Direction
Physical End	SCAN	C-SCAN
Last Request	LOOK	C-LOOK

Ø=Yes is the most useful memory table in this chapter.

MEMORY TRICKS

SCAN

Touch wall, turn back.

LOOK

Look only until last request.

C-SCAN

Touch wall, teleport to other wall.

C-LOOK

Last request, jump to first request.

EXAM SHORTCUT

For SCAN-family numericals:

1. Sort requests.
2. Split around head.
3. Mark direction.
4. Ask two questions:

Do I touch physical end?

Do I reverse or circular-jump?

That's enough to identify the exact route.

30-SECOND REVISION

FCFS?

! Arrival order.

SSTF?

! Nearest cylinder.

SSTF problem?

! Starvation.

SCAN?

! Both directions + physical ends.

C-SCAN?

! One direction + physical ends.

LOOK?

! Both directions + extreme requests.

C-LOOK?

! ' One direction + extreme requests.

Seek time?

! ' Head movement.

Rotational latency?

! ' Wait for sector rotation.

Average rotational latency?

! ' Half revolution.

ACTIVE RECALL

1. What is the main objective of disk scheduling?
2. How do you calculate head movement between cylinders 40 and 110?
3. Difference between SCAN and LOOK?
4. Difference between C-SCAN and C-LOOK?
5. Which algorithm may starve distant requests?
6. Why is SCAN called elevator algorithm?
7. Does C-SCAN service requests during its return jump?
8. In SCAN, if highest request is 180 but disk ends at 199, where does it reverse?
9. In LOOK, where does it reverse?
10. What are the three main components of disk access time?
11. What is average rotational latency for one rotation time T ?
12. At 6000 RPM, what is one-revolution time?

For 12:

$$T = \frac{60}{6000} = 0.01s = \boxed{10ms}$$

Average rotational latency:

$$\boxed{5ms}$$

MASTER CONNECTION

TEXT

I/O Requests

! "

Disk Queue

! "

FCFS / SSTF / SCAN Family

! "

Head Movement

! "

Seek Time

! "

Disk Access Time

NEXT — CHAPTER 11 Ø=Ý%Ø=Ý%Ø=Ý%

Now we move into another huge GATE scoring area:

MEMORY MANAGEMENT BASICS

Next we will cover:

- logical vs physical address
- address binding
- MMU
- contiguous allocation
- fixed and variable partitioning
- internal fragmentation
- external fragmentation
- first fit
- best fit
- worst fit
- compaction
- swapping
- memory protection
- relocation
- GATE allocation numericals

Then we will enter the most important memory topic:

PAGING + PAGE TABLE + TLB + ADDRESS TRANSLATION

USER

NEXT

ASSISTANT

CHAPTER 11 — MEMORY MANAGEMENT BASICS Ø=Ý%Ø=Ý%Ø=Ý%

Memory management is one of the biggest GATE OS areas because it leads directly to:

TEXT

Logical Address

!"

Physical Address

!"

Contiguous Allocation

!"

Fragmentation

!"

Paging

!"

Page Tables
!"
TLB
!"
Virtual Memory

The key question is:

How does the OS give memory to multiple processes safely and efficiently?

1. WHY MEMORY MANAGEMENT IS NEEDED

Suppose RAM is:

```
TEXT
+-----+
| Operating System |
+-----+
|               |
| Free Memory    |
|               |
+-----+
```

Now several processes arrive:

```
TEXT
P1 = 200 MB
P2 = 100 MB
P3 = 300 MB
```

The OS must decide:

- where each process will be placed
- how much memory it gets
- how processes are isolated
- how addresses are translated
- how free space is reused

That is **Memory Management**.

2. MAIN RESPONSIBILITIES

Memory management includes:

- allocation
- deallocation
- protection
- address translation
- relocation
- sharing
- tracking free/used memory

3. LOGICAL ADDRESS

Very important.

A logical address is:

The address generated by the CPU for a process.

Also commonly called:

Virtual Address

Example:

CPU generates:

```
TEXT
Logical Address = 1200
```

But actual RAM location may be something completely different.

4. PHYSICAL ADDRESS

Physical address is:

The actual address in main memory (RAM).

Example:

```
TEXT
Logical address = 1200
Physical address = 5200
```

Translation is required.

5. LOGICAL VS PHYSICAL

Logical Address	Physical Address
Generated by CPU	Actual RAM address
Seen by process	Used by memory hardware
Also called virtual address	Real memory location
Must often be translated	Final address after translation

Memory:

Logical = Process thinks

Physical = RAM really has

6. MMU — MEMORY MANAGEMENT UNIT

Very important.

The hardware that performs address translation is called:

MMU

Memory Management Unit.

Conceptually:

```
TEXT
CPU
! "
Logical Address
! "
MMU
! "
Physical Address
! "
RAM
```

7. SIMPLE RELOCATION USING BASE REGISTER

Suppose process generates logical address:

[LA]

and process starts physically at:

[Base]

Then:

$$PA = Base + LA$$

Example:

```
TEXT
Base = 5000
Logical Address = 300
```

Then:

$$PA = 5000 + 300$$

$$5300$$

8. LIMIT REGISTER

A limit register can store the valid logical address range/size of the process.

Suppose:

```
TEXT
Base = 5000
Limit = 1000
```

Valid logical addresses:

```
TEXT
0 to 999
```

If process generates:

```
TEXT
LA = 1200
```

invalid.

OS/hardware raises protection exception.

9. BASE + LIMIT IDEA

```
TEXT
if LA < Limit:
    PA = Base + LA
else:
    trap/error
```

This gives:

- relocation
- protection

10. EXAMPLE

Base:

\[4000 \]

Limit:

\[600 \]

Logical address:

\[450 \]

Since:

\[450 < 600 \]

valid.

Physical:

$\lfloor 4000+450=\boxed{4450} \rfloor$

11. INVALID ADDRESS EXAMPLE

Same:

TEXT

Base = 4000

Limit = 600

Logical:

TEXT

700

Since:

$\lfloor 700 \geq 600 \rfloor$

invalid.

No translation should occur.

& pGATE TRAP

Do not compare:

TEXT

Physical address < Limit

The limit usually applies to the process's logical range.

First validate logical address.

Then add base.

12. ADDRESS BINDING

Address binding means:

Associating program addresses with actual memory addresses.

This can conceptually happen at:

1. Compile time
2. Load time
3. Execution time

13. COMPILE-TIME BINDING

If memory location is known in advance:

absolute addresses can be generated.

If starting location changes:

program may need recompilation.

Very restrictive.

14. LOAD-TIME BINDING

Compiler generates relocatable code.

Actual physical location is decided when program is loaded.

If moved later during execution, simple load-time binding is not enough.

15. EXECUTION-TIME BINDING

Address mapping occurs while program executes.

Process can be moved during execution.

Requires hardware support such as:

MMU

This is the common foundation of modern virtual-memory systems.

MEMORY TRICK

C L E

Compile Load Execution

Think:

“Choose Location Eventually.”

16. DYNAMIC RELOCATION

With execution-time address translation:

TEXT

Logical address
+ relocation/base

!"
Physical address

Process does not need to know where it actually resides in RAM.

That is powerful.

17. CONTIGUOUS MEMORY ALLOCATION

In contiguous allocation:

Each process occupies one continuous block of physical memory.

Example:

```
TEXT
+-----+
|  OS   |
+-----+
|  P1   |
|  P1   |
+-----+
|  P2   |
|  P2   |
+-----+
| Free  |
+-----+
```

Each process is stored in one contiguous region.

18. FIXED PARTITIONING

Main memory is divided beforehand into fixed partitions.

Example:

```
TEXT
RAM
+-----+
|  OS   |
+-----+
| 100 MB |
+-----+
| 100 MB |
+-----+
| 100 MB |
+-----+
| 100 MB |
+-----+
```

Each partition can contain one process.

19. FIXED PARTITION PROBLEM

Suppose partition:

[100MB]

Process:

[60MB]

Unused space inside allocated partition:

[100-60=40MB]

This is:

Internal Fragmentation

20. INTERNAL FRAGMENTATION

Not important.

Wasted memory inside an allocated block.

Example:

TEXT

Partition = 100 KB
Process = 72 KB

Internal fragmentation:

[100-72=28KB]

Memory:

Internal = waste Inside

Excellent shortcut.

21. VARIABLE / DYNAMIC PARTITIONING

Instead of fixed partitions, allocate memory according to process size.

Example:

TEXT

P 1 n e e d s 7 0 M B ! ' a l l o c a t e 7 0
P 2 n e e d s 4 0 M B ! ' a l l o c a t e 4 0

This reduces fixed-partition internal waste.

But another problem appears:

External Fragmentation

22. EXTERNAL FRAGMENTATION

Free memory exists, but is split into small non-contiguous holes.

Example:

TEXT

```
+-----+
| P1    |
+-----+
| Free  | 20
+-----+
| P2    |
+-----+
| Free  | 30
+-----+
| P3    |
+-----+
| Free  | 25
+-----+
```

Total free:

$\backslash$ $20+30+25=75$ $\backslash$

Suppose new process needs:

$\backslash$ 60 $\backslash$

Total free memory is enough.

But no single contiguous hole of size 60 exists.

So allocation fails.

That is:

External Fragmentation

MEMORY TRICK

Internal = waste Inside allocated block

External = free holes EXist outside processes

23. INTERNAL VS EXTERNAL FRAGMENTATION

Internal	External
Waste inside allocated region	Free space split into holes

Internal	External
Common in fixed-size allocation	Common in variable contiguous allocation
Allocated block larger than needed	Enough total free memory may exist
Space is allocated but unused	Space is free but scattered

24. EXAMPLE

Memory holes:

TEXT

10 KB
20 KB
5 KB
40 KB

Total:

[75KB]

New process:

[50KB]

Cannot fit.

Why?

Largest contiguous hole:

[40KB]

So external fragmentation.

25. DYNAMIC STORAGE ALLOCATION

Given several free memory holes, which one should receive a new process?

Classical algorithms:

1. First Fit
2. Best Fit
3. Worst Fit

26. FIRST FIT

Choose:

First hole large enough.

Example holes:

TEXT

100, 500, 200, 300, 600

Process:

TEXT

212

Check from start:

TEXT

1 0 0 'L
5 0 0 '

Allocate in 500.

Remaining:

\[500-212=288 \]

27. BEST FIT

Choose:

Smallest hole that is large enough.

Same holes:

TEXT

100, 500, 200, 300, 600

Process:

\[212 \]

Possible:

TEXT

500
300
600

Smallest suitable:

\[300 \]

Remaining:

\[300-212=88 \]

28. WORST FIT

Choose:

Largest available hole.

Same holes:

TEXT

100, 500, 200, 300, 600

Process 212.

Largest:

\[600 \]

Remaining:

\[600-212=388 \]

29. MEMORY TRICK

First Fit

First enough

Best Fit

Tightest enough

Worst Fit

Largest hole

30. FULL ALLOCATION EXAMPLE

Holes:

TEXT

100, 500, 200, 300, 600

Processes:

TEXT

212, 417, 112, 426

Let's solve each algorithm.

31. FIRST FIT

Initial:

TEXT

100, 500, 200, 300, 600

P1 = 212

First fitting hole = 500.

Remaining:

TEXT

100, 288, 200, 300, 600

P2 = 417

First fitting hole = 600.

Remaining:

TEXT

100, 288, 200, 300, 183

P3 = 112

First suitable = 288.

Remaining:

TEXT

100, 176, 200, 300, 183

P4 = 426

No hole large enough.

So P4 cannot be allocated.

32. BEST FIT

Start:

TEXT

100, 500, 200, 300, 600

P1=212

Best = 300.

Remaining 88:

TEXT

100, 500, 200, 88, 600

P2=417

Best = 500.

Remaining 83:

TEXT

100, 83, 200, 88, 600

P3=112

Best = 200.

Remaining 88:

TEXT

100, 83, 88, 88, 600

P4=426

Best = 600.

Remaining:

\[600-426=174 \]

All allocated.

33. WORST FIT

Start:

TEXT

100, 500, 200, 300, 600

P1=212:

Use 600 !' 388.

TEXT

100, 500, 200, 300, 388

P2=417:

Use 500 !' 83.

TEXT

100, 83, 200, 300, 388

P3=112:

Use largest 388 !' 276.

TEXT

100, 83, 200, 300, 276

P4=426:

No hole "e 4 2 6 .

Fails.

34. GATE ALLOCATION METHOD

For each process:

1. Look at current hole sizes.
2. Choose according to algorithm.
3. Subtract process size.
4. Update hole immediately.
5. Move to next process.

Do not reuse original hole sizes after allocation.

& pGATE TRAP

Allocation is sequential.

After each process:

hole sizes change.

This is one of the most common mistakes.

35. WHICH FIT IS ALWAYS BEST?

None.

Do not memorize:

Best Fit is always best.

False.

Best Fit can create many tiny unusable holes.

First Fit often performs reasonably and can be faster.

Worst Fit also does not guarantee better utilization.

GATE may ask conceptual comparisons, so avoid absolute claims.

36. NEXT FIT — OPTIONAL PREREQUISITE

A variation of First Fit is **Next Fit**.

Instead of scanning from the beginning every time:

Start searching from where the previous search ended.

Know the idea if encountered, but the core official focus is memory management rather than memorizing every

allocator variation.

37. COMPACTION

How can external fragmentation be reduced?

One solution:

Move processes so scattered free holes combine into one large hole.

This is:

Compaction

Before:

```
TEXT
P1
Free 10
P2
Free 20
P3
Free 30
```

After:

```
TEXT
P1
P2
P3
Free 60
```

38. COMPACTION REQUIREMENT

Processes must be relocatable.

If process addresses cannot change during execution, moving them becomes difficult.

Therefore dynamic relocation support helps compaction.

39. COMPACTION COST

Compaction requires:

- moving process memory
- updating mappings
- time/CPU overhead

So it can be expensive.

40. DOES COMPACTION SOLVE INTERNAL FRAGMENTATION?

No.

Compaction mainly addresses:

External fragmentation

It joins scattered holes.

It does not shrink unused space trapped inside fixed-size blocks.

41. SWAPPING

Swapping means:

Temporarily moving a process between main memory and secondary storage.

Conceptually:

TEXT

```
R A M ! ' D i s k
D i s k ! ' R A M
```

This can free RAM for other processes.

42. SWAP OUT / SWAP IN

Swap Out

TEXT

```
Process in RAM
! "
Secondary storage
```

Swap In

TEXT

```
Secondary storage
! "
RAM
```

43. WHY SWAPPING?

Suppose RAM is full.

A waiting/inactive process can be moved out.

Then another process can be loaded.

This allows more processes to coexist conceptually than can fit simultaneously in RAM.

Virtual memory later generalizes this idea at page level.

44. SWAPPING COST

Disk is much slower than RAM.

So swapping an entire process can be expensive.

This is one reason modern systems rely heavily on paging/virtual memory.

45. MEMORY PROTECTION

Processes must not access arbitrary memory belonging to:

- another process
- kernel
- protected regions

Mechanisms include conceptually:

- base/limit
- page permissions
- segmentation protection

Protection is essential for system stability/security.

46. RELOCATION

Relocation means:

Adjusting/address-translating a process so it can execute even if placed at different physical memory locations.

Example:

Program logical:

TEXT
0 . . . 999

could physically reside at:

TEXT
5000 . . . 5999

or later:

TEXT
12000 . . . 12999

without changing its logical addresses.

47. DYNAMIC LOADING

A routine is loaded into memory:

only when it is actually needed.

Example:

TEXT

Large error-handling routine

may never execute.

No need to load it immediately.

Advantage:

saves memory.

48. DYNAMIC LINKING

Linking of library routines can be postponed until execution time.

Multiple processes may share common libraries.

This reduces duplicate copies.

Know the broad distinction:

- loading !' bring code into memory
- linking !' connect references to library code

49. CONTIGUOUS ALLOCATION PROTECTION

Suppose process has:

TEXT

Base = 10000

Limit = 4000

Physical range:

TEXT

10000 to 13999

Logical address 3999:

Valid.

Physical:

$\backslash [10000 + 3999 = 13999]$

Logical address 4000:

Invalid if valid range is 0 . . 3999 .

Classic off-by-one trap.

50. GATE-STYLE QUESTION — INTERNAL FRAGMENTATION

Partition size:

[8KB]

Process:

[6.5KB]

Internal fragmentation:

[$8 - 6.5 = \boxed{1.5\text{KB}}$]

51. GATE-STYLE QUESTION — EXTERNAL FRAGMENTATION

Free blocks:

TEXT

20, 40, 10, 30 KB

Total free:

[100KB]

Process asks:

[50KB]

No individual hole "e 5 0 .

Therefore allocation fails due to:

[$\boxed{\text{External Fragmentation}}$]

52. GATE-STYLE FIRST FIT

Holes:

TEXT

100, 300, 250, 500

Request:

[240]

First fitting:

TEXT

300

Remaining:

\[60 \]

53. GATE-STYLE BEST FIT

Same holes.

Suitable:

TEXT
300, 250, 500

Smallest fitting:

\[250 \]

Remaining:

\[10 \]

54. GATE-STYLE WORST FIT

Largest:

\[500 \]

Remaining:

\[260 \]

55. FIXED VS VARIABLE PARTITIONS

Fixed Partition	Variable Partition
Predefined sizes	Created according to process needs
Simple	More flexible
Internal fragmentation common	External fragmentation common
Number of processes bounded by partitions	Depends on available holes

56. WHY PAGING WAS INVENTED

This is the key transition.

With contiguous allocation, process needs:

One continuous block.

External fragmentation becomes a major issue.

What if we divide:

TEXT

P r o c e s s ! ' f i x e d - s i z e p i e c e s
M e m o r y ! ' f i x e d - s i z e p i e c e s

and place process pieces anywhere?

That is:

Paging

Which we study next.

57. PAGING PREVIEW

Logical memory is divided into:

Pages

Physical memory is divided into:

Frames

Page size:

$\lfloor \frac{M}{P} \rfloor$

Frame size.

Example:

TEXT

Process pages:
P0 P1 P2 P3

can be placed:

TEXT

F r a m e	2	!	P 0
F r a m e	8	!	P 1
F r a m e	1	!	P 2
F r a m e	6	!	P 3

No need for contiguous placement.

58. DOES PAGING HAVE EXTERNAL FRAGMENTATION?

Under standard fixed-size paging:

$\boxed{\text{No external fragmentation}}$

because any free frame can hold any page.

But paging can have:

Internal Fragmentation

especially in the last page.

This is a huge GATE point.

59. WHY INTERNAL FRAGMENTATION IN PAGING?

Suppose page size:

$$[4KB]$$

Process size:

$$[10KB]$$

Needs:

$$[\lceil 10/4 \rceil = 3]$$

pages.

Allocated:

$$[3 \times 4 = 12KB]$$

Unused:

$$[12 - 10 = 2KB]$$

Internal fragmentation:

$$[\boxed{2KB}]$$

60. GENERAL PAGING FRAGMENTATION

If process size:

$$[P]$$

page size:

$$[S]$$

number of pages:

$$[\boxed{\left\lceil \frac{P}{S} \right\rceil}]$$

Allocated memory:

$$[\left\lceil \frac{P}{S} \right\rceil S]$$

Internal fragmentation:

$$[\boxed{\left\lceil \frac{P}{S} \right\rceil S - P}]$$

61. MAXIMUM INTERNAL FRAGMENTATION IN LAST PAGE

If page size is:

$$[S]$$

maximum possible unused space in last page is conceptually:

$\lfloor \frac{S-1}{2} \rfloor$ byte

assuming byte-addressable memory and non-empty last page.

Average internal fragmentation often approximated as:

$\frac{S}{2}$

per process under uniform assumptions.

62. MEMORY ALLOCATION SUMMARY

TEXT

Contiguous Allocation

!"

Fixed Partition

!' Internal Fragmentation

Variable Partition

!' External Fragmentation

!"

Compaction

Then:

TEXT

Paging

!' eliminates external fragmentation

!' may have internal fragmentation

This connection is extremely important.

63. HOW GATE CAN ASK THIS CHAPTER

Typical patterns:

Pattern 1

Logical !' physical using base register.

Pattern 2

Check address validity with limit.

Pattern 3

Internal vs external fragmentation.

Pattern 4

First/Best/Worst Fit simulation.

Pattern 5

Compaction.

Pattern 6

Swapping.

Pattern 7

Fixed vs variable partitioning.

Pattern 8

Calculate paging internal fragmentation.

& pGATE TRAPS — MEMORY BASICS

Trap 1

Logical and physical addresses are always identical.

'LFalse.

Trap 2

MMU generates logical address.

'L

CPU generates logical address.

MMU translates it.

Trap 3

External fragmentation means allocated blocks contain unused memory.

'L

That's internal fragmentation.

Trap 4

Compaction solves internal fragmentation.

'L

It mainly addresses external fragmentation.

Trap 5

Best Fit always gives globally best memory utilization.

'LFalse.

Trap 6

First Fit always starts from previous allocation point.

'L

That's Next Fit.

First Fit usually starts from beginning.

Trap 7

Paging suffers external fragmentation.

'L

Standard paging eliminates external fragmentation.

Trap 8

Paging has zero fragmentation.

'L

Internal fragmentation can occur.

Trap 9

Page size and frame size differ.

'L in standard paging.

$\boxed{\text{Page Size} = \text{Frame Size}}$

Trap 10

If total free memory is sufficient, contiguous allocation must succeed.

'L

External fragmentation may prevent it.

' p WRITE THIS IN YOUR NOTES

Logical vs Physical

TEXT

L o g i c a l ! ' g e n e r a t e d b y C P U
P h y s i c a l ! ' a c t u a l R A M l o c a t i o n

MMU:

TEXT

L o g i c a l ! ' M M U ! ' P h y s i c a l

Base/Limit

$\boxed{PA=Base+LA}$

provided:

$LA < Limit$

Fragmentation

Internal

Waste inside allocated block.

Mnemonic:

IN = INside

External

Free memory split into holes.

Mnemonic:

EX = free holes EXist outside

Fixed vs Variable

TEXT

F i x e d ! ' I n t e r n a l f r a g m e n t a t i o n
V a r i a b l e ! ' E x t e r n a l f r a g m e n t a t i o n

Fit Algorithms

TEXT

F i r s t F i t ! ' f i r s t s u f f i c i e n t h o l e
B e s t F i t ! ' s m a l l e s t s u f f i c i e n t h o l e
W o r s t F i t ! ' l a r g e s t h o l e

Compaction

Combine external holes.

Solves/reduces:

TEXT

External fragmentation

Swapping

TEXT

R A M ! " D i s k

Move process out/in.

Paging Preview

TEXT

P r o c e s s ! ' P a g e s
R A M ! ' F r a m e s

$\lfloor \text{Page} \backslash \text{Size} = \text{Frame} \backslash \text{Size} \rfloor$

Paging:

TEXT

No external fragmentation
Possible internal fragmentation

FORMULA BOX

Base relocation

$\lfloor \boxed{PA = \text{Base} + LA} \rfloor$

Number of pages

$\lfloor \boxed{\text{Pages} = \left\lceil \frac{\text{ProcessSize}}{\text{PageSize}} \right\rceil} \rfloor$

Paging internal fragmentation

$\lfloor \boxed{\text{Pages} \times \text{PageSize} - \text{ProcessSize}} \rfloor$

MASTER MEMORY MAP

TEXT

CPU
! "

```

Logical Address
!"
MMU
!"
Physical Address
!"
Memory Allocation
!"
Contiguous
%% Fixed !" Internal Fragmentation
%% Variable !" External Fragmentation
!"
Compaction

```

Then:

```

TEXT
Need better allocation
!"
Paging

```

30-SECOND REVISION

Logical address?

!' CPU-generated.

Physical?

!' actual RAM.

MMU?

!' translation.

Base formula?

\[$PA = Base + LA$ \]

Internal fragmentation?

!' waste inside allocation.

External?

!' scattered holes.

Fixed partitioning?

!' internal fragmentation.

Variable partitioning?

!' external fragmentation.

First Fit?

!' first enough.

Best Fit?

!' tightest enough.

Worst Fit?

! ' biggest hole.

Compaction?

! ' joins external holes.

Paging?

! ' pages + frames; no external fragmentation.

ACTIVE RECALL

1. Who generates logical addresses?
2. What does MMU do?
3. Base = 5000, LA = 250. Find PA.
4. If Limit = 1000, is LA = 1000 valid in the standard 0 . . . 999 model?
5. Difference between internal and external fragmentation?
6. Which fragmentation occurs in fixed partitioning?
7. Which occurs in variable contiguous allocation?
8. What does compaction solve?
9. Difference between First Fit and Best Fit?
10. Does paging have external fragmentation?
11. Why can paging have internal fragmentation?
12. Process size = 18 KB, page size = 4 KB. How many pages?

$\lceil 18/4 \rceil = 5$

Allocated:

20KB

Internal fragmentation:

$20 - 18 = 2\text{KB}$

NEXT — CHAPTER 12

Next is one of the **most important numerical chapters in the entire GATE OS syllabus**:

PAGING + PAGE TABLES + ADDRESS TRANSLATION + TLB

We will cover:

- pages and frames
 - page number + offset
 - physical frame number
 - logical address split
 - number of pages/frames
 - offset bits
 - page-number bits
-

- page-table entries
- page-table size
- PTE size
- address translation
- TLB
- TLB hit/miss
- effective memory access time
- multilevel paging
- inverted page tables
- full GATE numerical shortcuts

Type **NEXT**.

USER

NEXT

ASSISTANT

CHAPTER 12 — PAGING, PAGE TABLES, ADDRESS TRANSLATION & TLB

This is one of the most important numerical chapters in GATE Operating Systems.

The core chain is:

TEXT

```

Logical Address
    ! "
Page Number + Offset
    ! "
Page Table
    ! "
Frame Number + Offset
    ! "
Physical Address
  
```

If you understand this one chain properly, a large number of paging questions become mechanical.

1. WHY PAGING EXISTS

Contiguous allocation had a major problem:

A process required one continuous block of physical memory.

Example:

TEXT

Free memory:

20 KB 30 KB 40 KB

Total:

$\boxed{90\text{KB}}$

Suppose a process needs:

$\boxed{60\text{KB}}$

No single 60 KB block exists.

So allocation fails because of:

External Fragmentation

Paging solves this by allowing different parts of the same process to occupy different places in RAM.

2. WHAT IS PAGING?

Paging divides:

Logical / Virtual Memory

into fixed-size blocks called:

Pages

Physical Memory

into equal-size blocks called:

Frames

And:

$\boxed{\text{Page Size} = \text{Frame Size}}$

3. SIMPLE EXAMPLE

Suppose process has four pages:

TEXT

Page 0

Page 1

Page 2

Page 3

They do not need contiguous physical memory.

Example mapping:

TEXT

```
P a g e   0   !'   F r a m e   5
P a g e   1   !'   F r a m e   2
P a g e   2   !'   F r a m e   8
P a g e   3   !'   F r a m e   1
```

Physical memory could look like:

TEXT

```
F r a m e   0   !'   O t h e r   p r o c e s s
F r a m e   1   !'   P a g e   3
F r a m e   2   !'   P a g e   1
F r a m e   3   !'   F r e e
F r a m e   4   !'   O t h e r   p r o c e s s
F r a m e   5   !'   P a g e   0
F r a m e   6   !'   F r e e
F r a m e   7   !'   O t h e r   p r o c e s s
F r a m e   8   !'   P a g e   2
```

This eliminates the requirement for contiguous physical allocation.

4. PAGE TABLE

How does the OS know:

TEXT

```
P a g e   0   !'   F r a m e   5
P a g e   1   !'   F r a m e   2
...
```

It uses a:

Page Table

A page table stores the mapping:

Page Number !' Frame Number

Example:

Page	Frame
0	5
1	2
2	8
3	1

MEMORY TRICK

Page is virtual. Frame is physical.

Or:

P !' F

Page table converts:

TEXT

P a g e !' F r a m e

5. LOGICAL ADDRESS STRUCTURE

A logical address is divided into:

TEXT

```
+-----+-----+
| Page Number | Offset |
+-----+-----+
```

The page number tells:

Which page?

The offset tells:

Which byte/location inside that page?

6. WHY OFFSET DOES NOT CHANGE

Suppose logical address points to:

TEXT

Page 3, Offset 100

Page 3 is stored in Frame 7.

Then physical address becomes:

TEXT

Frame 7, Offset 100

Notice:

Offset remains exactly the same.

Only:

TEXT

P a g e N u m b e r ! ' F r a m e N u m b e r

changes.

This is one of the most important paging rules.

7. ADDRESS TRANSLATION

Suppose:

TEXT

Logical Address
= Page 2, Offset 300

Page table:

TEXT

P a g e 2 ! ' F r a m e 5

Then:

TEXT

Physical Address
= Frame 5, Offset 300

8. PHYSICAL ADDRESS FORMULA

If:

- frame number = $\lfloor f \rfloor$
- page/frame size = $\lfloor S \rfloor$
- offset = $\lfloor d \rfloor$

then:

$\lfloor \boxed{PA = f \times S + d} \rfloor$

9. EXAMPLE

Page size:

$\lfloor 1024 \rfloor$ bytes

Logical address corresponds to:

TEXT

Page = 3
Offset = 100

Page table:

TEXT

P a g e 3 ! ' F r a m e 7

Physical address:

$$\lfloor 7 \times 1024 + 100 \rfloor$$

$$\lfloor = 7168 + 100 \rfloor$$

$$\lfloor \boxed{7268} \rfloor$$

10. CONVERTING A LOGICAL ADDRESS INTO PAGE + OFFSET

Suppose page size:

$$\lfloor 1024 \rfloor$$

Logical address:

$$\lfloor 2500 \rfloor$$

Then:

$$\lfloor \text{PageNumber} = \left\lfloor \frac{2500}{1024} \right\rfloor \rfloor$$

$$\lfloor = \boxed{2} \rfloor$$

Offset:

$$\lfloor 2500 \bmod 1024 \rfloor$$

$$\lfloor = \boxed{452} \rfloor$$

So:

TEXT

Logical Address 2500

! ' P a g e 2 , O f f s e t 4 5 2

FORMULA BOX

For logical address **LA** and page size **S**:

$$\lfloor \boxed{\text{PageNumber} = \left\lfloor \frac{\text{LA}}{S} \right\rfloor} \rfloor$$

$$\lfloor \boxed{\text{Offset} = \text{LA} \bmod S} \rfloor$$

Then page table gives frame $\lfloor f \rfloor$:

$$\lfloor \boxed{\text{PA} = fS + \text{Offset}} \rfloor$$

11. FULL TRANSLATION EXAMPLE

Given:

TEXT

Page size = 1 KB = 1024 bytes
Logical Address = 2500

Page table:

Page	Frame
0	4
1	7
2	3
3	8

Step 1 — Page Number

$\lfloor 2500/1024 \rfloor$

Page:

$\lfloor \boxed{2} \rfloor$

Step 2 — Offset

$\lfloor 2500 - 2(1024) \rfloor$

$\lfloor = 2500 - 2048 \rfloor$

$\lfloor = \boxed{452} \rfloor$

Step 3 — Page Table

TEXT

Page 2 !' Frame 3

Step 4 — Physical Address

$\lfloor 3(1024) + 452 \rfloor$

$\lfloor = 3072 + 452 \rfloor$

$\lfloor \boxed{3524} \rfloor$

12. POWER-OF-TWO PAGE SIZES

GATE frequently gives:

TEXT

1 KB
2 KB
4 KB
8 KB
...

These are powers of two.

Remember:

$1\text{KB} = 2^{10} \text{ bytes}$

$2\text{KB} = 2^{11}$

$4\text{KB} = 2^{12}$

$8\text{KB} = 2^{13}$

This lets you find **offset bits** instantly.

13. OFFSET BITS

If page size is:

2^n bytes

then:

$\boxed{\text{Offset Bits} = n}$

Example:

Page size:

$4\text{KB} = 4096 = 2^{12}$

Therefore:

$\boxed{\text{Offset} = 12 \text{ bits}}$

MEMORY SHORTCUT

Page size exponent = Offset bits

Examples:

TEXT

1	K	B	!	'	1	0
2	K	B	!	'	1	1
4	K	B	!	'	1	2
8	K	B	!	'	1	3
16	K	B	!	'	1	4

14. LOGICAL ADDRESS BITS

Suppose logical address space:

2^{32} bytes

Then logical address length:

$\boxed{32 \text{ bits}}$

If page size:

2^{12} bytes

Offset:

$[12 \text{ bits}]$

Therefore page number bits:

$[32-12]$

$[20 \text{ bits}]$

15. NUMBER OF VIRTUAL PAGES

With:

- logical address space = (2^m) bytes
- page size = (2^n) bytes

number of pages:

$\frac{2^m}{2^n}$

$[2^{m-n}]$

Equivalent to:

Number of page-number combinations.

16. EXAMPLE

Virtual address space:

$[4\text{GB} = 2^{32} \text{ bytes}]$

Page size:

$[4\text{KB} = 2^{12}]$

Number of pages:

$[2^{32-12}]$

$[2^{20}]$

That is:

$[1,048,576]$

pages.

17. PHYSICAL ADDRESS BITS

Suppose physical memory:

$[1\text{GB} = 2^{30} \text{ bytes}]$

Then physical addresses need:

$[30 \text{ bits}]$

If page size:

$$4KB = 2^{12}$$

offset bits:

$$12$$

Frame number bits:

$$30-12$$

$$\boxed{18}$$

18. NUMBER OF FRAMES

Physical memory:

$$2^p$$

Page/frame size:

$$2^n$$

Number of frames:

$$\boxed{2^{p-n}}$$

Example:

$$1GB/4KB = 2^{30}/2^{12} = \boxed{2^{18}}$$

frames.

19. MASTER ADDRESS SPLIT

Suppose:

TEXT

Virtual address = 32 bits

Page size = 4 KB = 2^{12}

Logical:

TEXT

```
+-----+-----+
| Page No. 20 bits | Offset 12 |
+-----+-----+
```

If physical memory:

TEXT

1 GB = 2^{30}

Physical:

TEXT

```
+-----+-----+
| Frame No. 18 bits | Offset 12 |
+-----+-----+
```

+-----+-----+

Notice:

Offset bits are the same in logical and physical addresses.

& pGATE TRAP

Do not calculate physical offset separately.

Since:

$\lfloor \text{PageSize} = \text{FrameSize} \rfloor$

the offset is unchanged.

20. NUMBER OF PAGE TABLE ENTRIES

Normally:

One page-table entry for every virtual page.

So:

$\lfloor \boxed{\text{\#PTEs} = \text{\#VirtualPages}} \rfloor$

If there are:

$\lfloor 2^{20} \rfloor$

virtual pages:

$\lfloor \boxed{2^{20}} \text{ PTEs} \rfloor$

21. PAGE TABLE ENTRY — PTE

Each page-table entry may contain:

- frame number
- valid/present bit
- protection bits
- dirty/modified bit
- referenced/accessed bit
- other control information

The exact fields depend on architecture.

For GATE, page-table size questions often tell you:

PTE size = 4 bytes

or require minimum bits from physical frame count.

22. PAGE TABLE SIZE

Very common numerical.

Formula:

$$\boxed{\text{PageTableSize} = \text{NumberOfPages} \times \text{PTEsize}}$$

23. EXAMPLE

Virtual address:

$$32 \text{ bits}$$

Page size:

$$4 \text{ KB}$$

PTE:

$$4 \text{ bytes}$$

Number pages:

$$2^{32} / 2^{12} = 2^{20}$$

Page table size:

$$2^{20} \times 4$$

$$= 2^{20} \times 2^2$$

$$= 2^{22} \text{ bytes}$$

$$\boxed{4 \text{ MB}}$$

EXAM SHORTCUT

For page-table-size questions:

Step 1

$$\text{Pages} = \frac{\text{VirtualAddressSpace}}{\text{PageSize}}$$

Step 2

$$\text{PageTableSize} = \text{Pages} \times \text{PTEsize}$$

That's it.

24. MINIMUM PTE BITS

Suppose physical memory:

$$2^{30} \text{ bytes}$$

Page size:

$$2^{12}$$

Frames:

$$\lfloor 2^{18} \rfloor$$

Minimum bits needed just to identify a frame:

$$\lfloor \boxed{18 \text{ bits}} \rfloor$$

But a real PTE may be larger because of:

- valid bit
- dirty bit
- permissions
- alignment
- architecture design

So carefully read whether question asks:

minimum frame-number bits

or:

actual PTE size.

25. EXAMPLE — MINIMUM PTE SIZE

Physical memory:

$$\lfloor 4\text{GB} = 2^{32} \rfloor$$

Page size:

$$\lfloor 4\text{KB} = 2^{12} \rfloor$$

Number frames:

$$\lfloor 2^{20} \rfloor$$

So minimum frame number:

$$\lfloor 20 \text{ bits} \rfloor$$

If question says add:

- 1 valid bit
- 1 dirty bit

minimum:

$$\lfloor 22 \text{ bits} \rfloor$$

But actual storage might need rounding/alignment if question specifies bytes.

26. PAGE TABLE MEMORY PROBLEM

Suppose every process needs a large single-level page table.

For:

2^{20}

entries $\times$ 4 bytes:

4MB

per process.

For 100 processes:

400MB

just for page tables.

This motivates:

Multilevel Paging

PART B — TLB

27. PROBLEM WITH PAGE TABLE LOOKUP

Without TLB, CPU wants one memory item.

But first:

1. Access page table in memory
2. Find frame number
3. Access actual data in memory

So one logical memory reference can require:

$2 \times \text{memory accesses}$

for a single-level page table.

28. EXAMPLE

Memory access time:

100ns

No TLB.

Page-table lookup:

100ns

Data access:

100ns

Total:

200ns

Ignoring other overhead.

29. TLB — TRANSLATION LOOKASIDE BUFFER

Very important.

TLB is:

A small, fast associative cache containing recently used page-table entries.

Conceptually:

```
TEXT
CPU
!"
Virtual Page Number
!"
TLB
!"
Hit?
%%   Y e s   !'   G e t   F r a m e
%%   N o     !'   C o n s u l t   P a g e   T a b l e
```

30. WHY TLB?

Page-table lookup in RAM is slow.

TLB stores frequently/recently used mappings:

```
TEXT
P a g e   !'   F r a m e
```

Example:

```
TEXT
P a g e   5   !'   F r a m e   2 0
P a g e   8   !'   F r a m e   4
P a g e   1 1   !'   F r a m e   1 8
```

If required page is already in TLB:

No extra RAM access is needed for the page table.

31. TLB HIT

If page mapping is found in TLB:

```
TEXT
TLB HIT
```

Then:

1. TLB lookup
2. One memory access for actual data

32. TLB MISS

If mapping is not in TLB:

TEXT
TLB MISS

Then:

1. TLB lookup
2. Page-table memory access
3. Actual memory access

assuming the page itself is present in memory.

& pTLB miss is **not** the same as page fault.

More on that shortly.

33. TLB HIT RATIO

If:

$\{ h \}$

is TLB hit ratio:

Example:

$\{ h=0.95 \}$

means:

95% translations found in TLB.

Miss ratio:

$\{ 1-h \}$

So:

$\{ 0.05 \}$

34. EFFECTIVE ACCESS TIME — BASIC MODEL

Suppose:

- TLB lookup time = $\{ \epsilon \}$
- memory access time = $\{ m \}$
- hit ratio = $\{ h \}$

On hit:

$\{ \epsilon + m \}$

On miss:

$$\sqrt[\epsilon+2m]{}$$

Therefore:

$$\sqrt[\boxed{EAT=h(\epsilon+m)+(1-h)(\epsilon+2m)}]{}$$

35. SIMPLIFYING

$$\sqrt[EAT = h\epsilon + hm + (1-h)\epsilon + 2m(1-h)]{}$$

$$\sqrt[\epsilon + m[h+2(1-h)]]{}$$

$$\sqrt[\boxed{\epsilon + m(2-h)}]{}$$

for this specific model.

But memorize the weighted form rather than only the simplified expression because GATE may change assumptions.

36. FULL TLB EXAMPLE

Given:

TEXT

TLB lookup = 10 ns
Memory access = 100 ns
Hit ratio = 0.9

Hit time:

$$\sqrt[10+100=110ns]{}$$

Miss time:

$$\sqrt[10+100+100=210ns]{}$$

Effective:

$$\sqrt[0.9(110)+0.1(210)]{}$$

$$\sqrt[=99+21]{}$$

$$\sqrt[\boxed{120ns}]{}$$

37. ALTERNATIVE TLB MODEL

Some questions assume TLB lookup happens in parallel with part of cache/memory lookup or ignore TLB lookup time.

Then formulas change.

Example if TLB lookup time is negligible:

Hit:

$$\sqrt[m]{}$$

Miss:

$$\sqrt[2m]{}$$

Then:

$$EAT = hm + (1-h)2m$$

Always obey the exact model given.

& pGATE TRAP

Never blindly memorize one EAT formula.

Instead:

Hit cost × Hit probability + Miss cost × Miss probability

Universal shortcut:

$$EAT = \sum \text{Probability} \times \text{Cost}$$

This works for TLB, cache, page faults, and many other GATE questions.

38. TLB ASSOCIATIVE LOOKUP

A normal page table is indexed by page number.

TLB usually uses associative lookup:

Search entries by page number very quickly.

This is why TLB is also called an associative memory/cache conceptually.

39. TLB ENTRY

Typical entry contains:

TEXT

Virtual Page Number
Physical Frame Number
Protection/control bits

In systems with multiple processes, address-space identifiers may also be used.

40. TLB CONTEXT SWITCH ISSUE

Different processes can use the same virtual page number but map it to different frames.

Example:

TEXT

P 1	p a g e	5	!	'	f r a m e	2 0
P 2	p a g e	5	!	'	f r a m e	1 0 0

So on context switch, TLB mappings must be handled carefully.

Possible approaches include:

- flushing relevant TLB entries
- using address-space identifiers / tags

Conceptual GATE point:

TLB entries must not incorrectly mix address spaces.

41. TLB MISS VS PAGE FAULT Ø=Ý%

This is extremely important.

TLB Miss

Translation not found in TLB.

But page table may say:

TEXT
Page is in RAM

So simply obtain mapping from page table.

Page Fault

Required page itself is:

not currently in physical memory.

Then OS must bring it from disk/backing storage.

Comparison

TLB Miss	Page Fault
Mapping absent from TLB	Page absent from RAM
Page may still be in RAM	Must obtain page from secondary storage
Relatively cheap	Very expensive
Translation issue	Residency issue

Memory:

TLB miss = map missing

Page fault = page missing

42. VALID / PRESENT BIT

Page-table entries often contain a bit telling whether the page is currently valid/present.

Conceptually:

TEXT

```
P r e s e n t   =   1   ! '   p a g e   i s   i n   R A M
P r e s e n t   =   0   ! '   n o t   c u r r e n t l y   r e s i d e n t   /   i n v a l i d   d e p e n d i n
```

If referenced while absent:

Page fault.

Virtual memory chapter will cover this deeply.

PART C — MULTILEVEL PAGING Ø=Ý%

43. WHY SINGLE-LEVEL PAGE TABLE CAN BE WASTEFUL

Example:

32-bit virtual address.

Page size:

[4KB]

Pages:

[2^{20}]

PTE size:

[4 bytes]

Page table:

[4MB]

Even a tiny process may not use most of its virtual address space.

But the flat page table may still be huge.

Multilevel paging avoids allocating every lower-level table unless needed.

44. TWO-LEVEL PAGING

Instead of:

TEXT

```
P a g e   N u m b e r   ! '   h u g e   p a g e   t a b l e
```

split page number:

TEXT

```
+-----+-----+-----+
| Level 1 | Level 2 | Offset |
+-----+-----+-----+
```

First-level index selects a second-level page table.

Second-level index selects actual PTE.

45. ADDRESS TRANSLATION

Conceptually:

TEXT

```
Virtual Address
    !"
p 1 !' Outer Page Table
    !"
address of inner table
    !"
p 2 !' Inner Page Table
    !"
Frame Number
    !"
Frame + Offset
```

46. MEMORY ACCESSES WITHOUT TLB

For two-level paging:

1. Read outer table
2. Read inner table
3. Access actual data

So:

$\boxed{3}$ memory accesses

without TLB.

For k -level paging:

$\boxed{k+1}$ memory accesses

without TLB.

47. MULTILEVEL PAGING GATE SHORTCUT

If page table has k levels:

translation accesses = k

plus:

one actual memory access

Therefore:

$\boxed{k+1}$

memory accesses.

48. TWO-LEVEL ADDRESS SPLIT EXAMPLE

Suppose:

TEXT

32-bit virtual address
4 KB pages

Offset:

$\boxed{12}$

Page-number bits:

$\boxed{20}$

Suppose split equally:

TEXT

10 bits + 10 bits

Then:

TEXT

+-----+-----+-----+		
L1:10	L2:10	Offset:12
+-----+-----+-----+		

Each level indexes:

$\boxed{2^{10}=1024}$

entries.

49. DERIVING LEVEL BITS USING PAGE TABLE PAGE SIZE

Ø=100 important GATE numerical pattern.

Suppose:

TEXT

Page size = 4 KB
PTE size = 4 bytes

How many PTEs fit in one page-table page?

$\boxed{\frac{4096}{4}=1024}$

$$\lceil \frac{2^{10}}{2^p} \rceil$$

Therefore one page-table level can index:

$$\lceil \frac{10}{p} \rceil \text{ bits}$$

at that level.

This is a powerful shortcut.

50. GENERAL FORMULA

If page-table page size is:

$$2^p$$

and each PTE size is:

$$2^e$$

then entries per page table:

$$2^{p-e}$$

Therefore index bits per full page-table level:

$$\boxed{p-e}$$

51. EXAMPLE

Page size:

$$8\text{KB} = 2^{13}$$

PTE size:

$$8 \text{ bytes} = 2^3$$

Entries:

$$2^{13-3} = 2^{10}$$

So:

$$\boxed{10 \text{ index bits}}$$

per page-table page.

52. HOW MANY PAGE TABLE LEVELS?

Suppose:

TEXT

48-bit virtual address

4 KB page !' 12 offset bits

PTE = 8 bytes

Page table fits in one 4KB page

Each page-table page:

$$\lfloor 4096/8=512=2^9 \rfloor$$

So each full level handles:

$$\lfloor 9 \rfloor \text{ bits}$$

Virtual page number bits:

$$\lfloor 48-12=36 \rfloor$$

Need:

$$\lfloor 36/9=4 \rfloor$$

levels.

Therefore:

$$\boxed{4 \text{ levels}}$$

53. IF BITS DO NOT DIVIDE EXACTLY

Suppose virtual page number has 38 bits.

Each lower level indexes 9 bits.

Then:

TEXT

$$9 + 9 + 9 + 9 = 36$$

Need another top level for remaining:

$$\lfloor 2 \rfloor$$

bits.

So:

$$\lfloor 5 \rfloor$$

levels, with top-level index using only 2 bits.

54. WHY MULTILEVEL PAGING SAVES MEMORY

Suppose a process uses only a small portion of its huge virtual space.

Only the necessary lower-level page tables are allocated.

Unused regions:

TEXT

No lower-level page table needed

So page-table memory is saved.

55. COST OF MULTILEVEL PAGING

Advantage:

smaller page-table memory footprint.

Disadvantage:

more translation accesses.

TLB helps compensate for this.

PART D — INVERTED PAGE TABLE

56. NORMAL PAGE TABLE

Traditional page table has:

one entry per virtual page, per process.

Huge virtual spaces mean huge tables.

57. INVERTED PAGE TABLE

An inverted page table instead has approximately:

one entry per physical frame.

So table size depends mainly on:

physical memory

rather than total virtual pages.

58. BASIC IDEA

Normal:

TEXT

V i r t u a l P a g e ! ' P h y s i c a l F r a m e

Inverted:

TEXT

For each physical frame:
Which process + virtual page occupies it?

Example:

Frame	Process	Virtual Page
0	P2	5
1	P1	10
2	P3	7

59. ADVANTAGE

Potentially much smaller page-table memory for huge virtual address spaces.

60. DISADVANTAGE

Translation is more complex because you cannot simply directly index by virtual page number in the same straightforward way.

Hashing/TLB techniques may be used.

For GATE:

Know the conceptual tradeoff.

PART E — PAGE SIZE TRADE-OFFS

61. SMALL PAGE SIZE

Advantages:

- less internal fragmentation
- finer-grained memory allocation

Disadvantages:

- more pages
- larger page tables
- more page-management overhead

62. LARGE PAGE SIZE

Advantages:

- fewer pages
- smaller page table
- efficient large transfers / TLB coverage benefits

Disadvantages:

- more internal fragmentation
- may load unused data

MASTER TRADE-OFF

TEXT

Small pages

!' Less internal fragmentation

!' Bigger page table

Large pages

!' More internal fragmentation

!' Smaller page table

Ø=Very important.

63. EXPECTED INTERNAL FRAGMENTATION

Under simple uniform assumptions, average unused space in the last page is approximately:

$\lfloor \boxed{\frac{\text{PageSize}}{2}} \rfloor$

So larger pages generally increase expected internal fragmentation.

64. PTE SIZE AND PAGE SIZE CONNECTION

Increasing page size:

TEXT

Virtual pages decrease

Therefore:

TEXT

Number of PTEs decreases

Thus:

Page table becomes smaller.

This is often tested conceptually.

65. FULL GATE NUMERICAL — PAGE TABLE SIZE

Given:

TEXT

Virtual address = 36 bits

Page size = 8 KB

PTE size = 8 bytes

Step 1 — Offset

$$\lfloor 8\text{KB}=2^{\{13\}} \rfloor$$

Offset:

$$\lfloor 13 \rfloor$$

Step 2 — Page number bits

$$\lfloor 36-13=23 \rfloor$$

Step 3 — Number pages

$$\lfloor 2^{\{23\}} \rfloor$$

Step 4 — Page table size

$$\lfloor 2^{\{23\}} \times 2^3 \rfloor$$

$$\lfloor =2^{\{26\}} \rfloor$$

bytes

$$\lfloor =\boxed{64\text{MB}} \rfloor$$

66. FULL GATE NUMERICAL — FRAME BITS

Physical memory:

$$\lfloor 8\text{GB}=2^{\{33\}} \rfloor$$

Page size:

$$\lfloor 8\text{KB}=2^{\{13\}} \rfloor$$

Frame number bits:

$$\lfloor 33-13 \rfloor$$

$$\lfloor \boxed{20} \rfloor$$

Number frames:

$$\lfloor \boxed{2^{\{20\}}} \rfloor$$

67. FULL GATE NUMERICAL — ADDRESS TRANSLATION

Page size:

$$\lfloor 2\text{KB}=2048 \rfloor$$

Logical address:

$$\lfloor 7000 \rfloor$$

Page number:

$$\lfloor \left\lfloor 7000/2048 \right\rfloor = 3 \rfloor$$

Offset:

$$\lfloor 7000-3(2048) \rfloor$$

$$\lfloor =7000-6144 \rfloor$$

$$\lfloor =856 \rfloor$$

Suppose:

TEXT

P a g e 3 ! ' F r a m e 9

Physical:

$$\lfloor 9(2048)+856 \rfloor$$

$$\lfloor =18432+856 \rfloor$$

$$\lfloor \boxed{19288} \rfloor$$

68. FULL GATE NUMERICAL — TLB

Memory:

$$\lfloor 80\text{ns} \rfloor$$

TLB:

$$\lfloor 10\text{ns} \rfloor$$

Hit ratio:

$$\lfloor 95\%=0.95 \rfloor$$

Hit:

$$\lfloor 10+80=90 \rfloor$$

Miss:

$$\lfloor 10+80+80=170 \rfloor$$

EAT:

$$\lfloor 0.95(90)+0.05(170) \rfloor$$

$$\lfloor =85.5+8.5 \rfloor$$

$$\lfloor \boxed{94\text{ns}} \rfloor$$

69. PAGE TABLE STORED IN MEMORY

Classic fact:

Without a TLB, if a single-level page table is stored in main memory:

One memory reference effectively requires two memory accesses.

One for:

TEXT

PTE

one for:

TEXT

Actual word/data

This doubles memory access time approximately.

70. TLB HIT IMPROVES THIS

On a TLB hit:

TEXT

Translation comes from TLB

Then only one main-memory access is needed for the actual data.

Hence TLB significantly reduces effective access time.

71. PAGE TABLE BASE REGISTER — PTBR

The OS/hardware needs to know where the page table begins.

A register can contain:

Page Table Base Register (PTBR)

Conceptually:

TEXT

P T B R ! ' b a s e a d d r e s s o f p a g e t a b l e

Then page number indexes into that table.

72. PAGE TABLE LENGTH REGISTER — PTLR

Some conceptual architectures also use:

Page Table Length Register

to indicate table size / valid range.

For GATE, recognize these names if they appear.

73. PAGE TABLE ENTRY ADDRESS

Suppose:

- page table base = PTBR
- page number = p
- each PTE size = E

Then address of PTE:

$$\boxed{\text{PTBR} + pE}$$

This can appear in address-calculation questions.

74. EXAMPLE

PTBR:

$$10000$$

PTE size:

$$4$$

Page number:

$$7$$

PTE address:

$$10000 + 7(4)$$

$$= \boxed{10028}$$

Then read PTE to obtain frame.

75. PAGE OFFSET MASK IDEA

If page size:

$$2^{12}$$

lower 12 address bits are:

Offset.

Upper bits:

Page number.

So with power-of-two sizes, you don't need division conceptually.

Binary split gives the answer directly.

76. EXAMPLE IN BINARY

Suppose 16-bit logical addresses.

Page size:

$\lceil 256 = 2^8 \rceil$

Then:

TEXT

```
+-----+-----+
| Page 8 | Offset 8 |
+-----+-----+
```

Logical address:

TEXT

0x1234

Upper 8 bits:

TEXT

0x12

Page.

Lower 8:

TEXT

0x34

Offset.

77. PAGE NUMBER BITS VS NUMBER OF PAGES

If page-number field is:

$\lceil p \rceil$ bits

then number pages:

$\lceil 2^p \rceil$

Likewise frame-number bits:

$\lceil f \rceil$

number frames:

$\lceil 2^f \rceil$

78. PAGE TABLE SHARING

Pages containing common read-only code can potentially be shared among processes.

Example:

TEXT

Shared library code

can map into multiple virtual address spaces.

Each process still has private writable data where appropriate.

This improves memory utilization.

79. PROTECTION BITS

A PTE may specify:

- Read
- Write
- Execute

Example:

TEXT

```
C o d e   p a g e   ! '   R e a d   +   E x e c u t e
D a t a   p a g e   ! '   R e a d   +   W r i t e
```

Trying forbidden operation causes protection fault/exception.

80. DIRTY / MODIFIED BIT

Dirty bit indicates:

Page has been modified since it was loaded.

Why useful?

If page must be evicted:

Clean page

May not need writing back if disk copy is unchanged.

Dirty page

Must usually be written back.

This becomes important in virtual memory/page replacement.

81. REFERENCE / ACCESSED BIT

Indicates:

Page has been recently accessed.

Can help page-replacement algorithms approximate usage.

We'll see this later.

82. VALID VS PRESENT TERMINOLOGY

Different architectures/textbooks use slightly different meanings.

A bit may indicate:

- mapping valid
- page belongs to process
- page currently resident

Read exact problem wording.

For GATE, do not assume all architectures name bits identically.

83. PAGE TABLE VS TLB

Page Table	TLB
Complete/large mapping structure	Small cache of mappings
Usually in memory	Very fast hardware cache
Many entries	Few entries
Page! Frame mappings	Recently used Page! Frame mappings
Slower	Faster

Memory:

Page table = book

TLB = bookmark

Excellent shortcut.

84. TLB HIT DOES NOT MEAN CACHE HIT

TLB caches:

address translations.

CPU cache caches:

actual data/instructions.

These are different structures.

Possible situation:

TEXT

TLB hit
Cache miss

or:

TEXT

TLB miss
Cache hit after translation mechanisms

depending on architecture.

Do not mix them.

85. THREE DIFFERENT "MISSES"

Keep these separate:

TLB Miss

Translation not cached.

Cache Miss

Data not in CPU cache.

Page Fault

Page not in RAM.

Memory trick:

TLB = translation

Cache = data

Page fault = RAM residency

86. MULTILEVEL PAGING ACCESS WITH TLB

Suppose 3-level page table.

Without TLB:

$\sqrt[4]{}$

main-memory accesses:

TEXT

L1
L2
L3
Data

With TLB hit:

TEXT

TLB already provides final frame

so usually:

one data memory access after TLB lookup.

This is why TLB becomes even more valuable with multilevel paging.

87. EAT FOR MULTILEVEL PAGING

Suppose:

- k page-table levels
- memory time = m
- TLB time = ϵ
- hit = h

Hit:

$\epsilon + m$

Miss:

$\epsilon + (k+1)m$

Therefore:

$$EAT = h(\epsilon + m) + (1-h)(\epsilon + (k+1)m)$$

under the simple serial model.

88. EXAMPLE — 2-LEVEL TLB

TLB:

10ns

Memory:

100ns

Hit:

90%

Two-level paging.

Hit cost:

$10 + 100 = 110$

Miss cost:

TEXT

TLB

+ Level 1

+ Level 2

+ Data

$$\lfloor 10 + 3(100) = 310 \rfloor$$

EAT:

$$\lfloor 0.9(110) + 0.1(310) \rfloor$$

$$\lfloor 99 + 31 \rfloor$$

$$\lfloor \boxed{130\text{ns}} \rfloor$$

89. GATE QUESTION — PAGE SIZE EFFECT

If page size increases while address space remains fixed:

Number of pages

decreases.

Number of PTEs

decreases.

Page table size

generally decreases.

Offset bits

increase.

Internal fragmentation

generally increases.

Ø=Very common conceptual pattern.

90. PAGE SIZE TRADE-OFF TABLE

Larger Pages	Smaller Pages
Fewer pages	More pages
Smaller page table	Larger page table
More internal fragmentation	Less internal fragmentation
Larger offset	Smaller offset
Better TLB coverage per entry	Finer allocation granularity

91. TLB REACH

Useful advanced concept:

$\boxed{\text{TLB Reach} = \text{NumberOfTLBEntries} \times \text{PageSize}}$

It tells how much virtual memory can be covered by TLB entries at once.

Example:

TEXT

```
TLB entries = 128
Page size = 4 KB
```

$128 \times 4\text{KB} = \boxed{512\text{KB}}$

Larger pages increase TLB reach.

92. PAGE TABLE SIZE QUESTION — FAST METHOD

Suppose:

TEXT

```
VA = 40 bits
Page = 4 KB
PTE = 8 bytes
```

Immediately:

TEXT

```
Offset = 12
V P N   b i t s   =   4 0 "   1 2   =   2 8
Pages = 2^28
PTE = 2^3 bytes
```

Therefore table:

$2^{28+3} = 2^{31}$

$\boxed{2\text{GB}}$

No long arithmetic needed.

93. LOG BASE 2 SHORTCUTS

Useful values:

TEXT

```
1 KB = 2^10
1 MB = 2^20
1 GB = 2^30
1 TB = 2^40

4 KB = 2^12
8 KB = 2^13
16 KB = 2^14
```

64 KB = 2^{16}

4 bytes = 2^2

8 bytes = 2^3

Keep these in your notes.

94. ADDRESS SPACE VS PHYSICAL MEMORY

Do not assume:

$\{ \text{VirtualAddressSpace} = \text{PhysicalMemory} \}$

They can differ.

Example:

TEXT

Virtual address = 64-bit

Physical address = 48-bit

This is perfectly possible.

Then:

- VPN bits derived from virtual address
- frame bits derived from physical address

Do not mix them.

95. GATE TRAP — PAGE TABLE INDEX

Page table is indexed using:

$\{ \boxed{\text{Virtual Page Number}} \}$

not offset.

Offset bypasses translation and is copied unchanged.

96. GATE TRAP — PAGE TABLE ENTRIES

Do not use number of physical frames to calculate number of entries in a conventional process page table.

Normal page table entries depend on:

virtual pages.

Physical frames determine:

frame-number bits.

This is a classic confusion.

97. GATE TRAP — OFFSET BITS

Offset depends only on:

$\lfloor \boxed{\text{PageSize}} \rfloor$

Not:

- RAM size
 - virtual address size
 - number of processes
-

98. GATE TRAP — FRAME NUMBER

Frame number depends on:

physical memory size and frame size.

Page number depends on:

virtual address space and page size.

Keep these completely separate.

99. GATE TRAP — PAGE TABLE SIZE

If PTE size isn't a whole number of bytes theoretically, don't automatically round unless storage format/alignment requires it or question says so.

Read wording carefully.

100. GATE TRAP — TLB MISS

TLB miss:

'L does not automatically require disk access.

If page is present:

TEXT

Consult page table
Update TLB
Continue

Disk is involved only if there is a:

page fault.

101. HOW GATE CAN ASK THIS CHAPTER

Expect:

Pattern 1 — Address split

Find:

- page bits
- offset bits
- frame bits

Pattern 2 — Number of pages/frames

Use powers of two.

Pattern 3 — Page table size

Very common.

Pattern 4 — Logical-to-physical translation

Use page table.

Pattern 5 — TLB EAT

Weighted average.

Pattern 6 — Multilevel page table

Determine number of levels/index bits.

Pattern 7 — Page size trade-offs

Conceptual MCQ/MSQ.

Pattern 8 — PTE bit calculation

Physical frames + control bits.

Pattern 9 — TLB miss vs page fault

Conceptual.

& MASTER GATE TRAPS — CHAPTER 12

Trap 1

Page size " frame size.

'L

Standard paging:

$\boxed{\text{PageSize} = \text{FrameSize}}$

Trap 2

Offset changes during translation.

'L

Offset remains unchanged.

Trap 3

Number of page-table entries = number of frames.

'L

Normal page table:

$\lfloor \frac{\text{\#PTE}}{\text{\#VirtualPages}} \rfloor$

Trap 4

Frame-number bits come from virtual address space.

'L

They come from physical memory.

Trap 5

Page-number bits come from physical memory.

'L

They come from virtual address space.

Trap 6

TLB miss = page fault.

'L

Trap 7

Page-table lookup is free.

'L

Without TLB it requires memory access.

Trap 8

Single-level page-table access needs one total memory access.

'L

Normally:

TEXT

1 for PTE + 1 for data = 2

Trap 9

Multilevel paging always makes memory access faster.

'L

It saves page-table memory but may require more accesses.

Trap 10

Larger pages always better.

'L

Trade-off with internal fragmentation.

'p WRITE THIS IN YOUR NOTES

Paging

TEXT

L o g i c a l M e m o r y ! ' P a g e s
P h y s i c a l M e m o r y ! ' F r a m e s

$\lfloor \frac{\text{Logical Memory}}{\text{Frame Size}} \rfloor$

Page table:

TEXT

P a g e ! ' F r a m e

Logical Address

TEXT

+-----+-----+
| Page No. | Offset |
+-----+-----+

Physical:

TEXT

+-----+-----+
| Frame No. | Offset |
+-----+-----+

Offset unchanged.

Translation Formulas

$$\boxed{\text{Page} = \left\lfloor \frac{\text{LA}}{\text{PageSize}} \right\rfloor}$$

$$\boxed{\text{Offset} = \text{LA} \bmod \text{PageSize}}$$

$$\boxed{\text{PA} = \text{Frame} \times \text{PageSize} + \text{Offset}}$$

Bit Formulas

If:

$$\boxed{\text{PageSize} = 2^n}$$

then:

$$\boxed{\text{OffsetBits} = n}$$

If $\text{VA} = m$ bits:

$$\boxed{\text{PageBits} = m - n}$$

If $\text{PA} = p$ bits:

$$\boxed{\text{FrameBits} = p - n}$$

Pages / Frames

$$\boxed{\text{Pages} = \frac{\text{VirtualAddressSpace}}{\text{PageSize}}}$$

$$\boxed{\text{Frames} = \frac{\text{PhysicalMemory}}{\text{PageSize}}}$$

Page Table Size

$$\boxed{\text{PTSize} = \text{NumberOfVirtualPages} \times \text{PTEsize}}$$

TLB

Fast cache of page-table entries.

Hit:

TEXT

T L B ! ' F r a m e ! ' M e m o r y

Miss:

TEXT

T L B m i s s ! ' P a g e T a b l e ! ' F r a m e ! ' M e m o r y

Basic EAT:

$$\boxed{h(TLB+M)+(1-h)(TLB+2M)}$$

for a single-level table under serial lookup assumptions.

Memory trick:

Weighted average: probability × cost

TLB Miss vs Page Fault

TEXT

T L B m i s s ! ' m a p p i n g a b s e n t f r o m T L B
P a g e f a u l t ! ' p a g e a b s e n t f r o m R A M

Multilevel Paging

If k page-table levels and no TLB:

$$\boxed{(k+1) \text{ memory accesses}}$$

Index bits per page-table page:

$$\boxed{\log_2 \left(\frac{\text{Page Size}}{\text{PTE size}} \right)}$$

MASTER SHORTCUT SHEET

Shortcut 1

Page size exponent = offset bits

Shortcut 2

VA " Offset = Page bits

Shortcut 3

PA " Offset = Frame bits

Shortcut 4

VPN count × PTE bytes = Page-table size

Shortcut 5

Page! Frame offset stays

Shortcut 6

TLB EAT = probability × path cost

MASTER MEMORY MAP

TEXT

Virtual Address

! "

+-----+-----+

| Page Number | Offset |

+-----+-----+

! "

TLB

hit ! " miss

! " Page Table

! "

Frame Number

! "

+-----+-----+

| Frame Number | Offset |

+-----+-----+

! "

Physical Memory

30-SECOND REVISION

Page?

! Virtual block.

Frame?

! Physical block.

Page size?

! Frame size.

Offset bits?

! log₂(page size).

Page bits?

! VA bits " offset.

Frame bits?

! PA bits " offset.

PTE count?

! number of virtual pages.

Page table size?

! ' pages $\times$ PTE size.

TLB?

! ' fast translation cache.

TLB miss?

! ' mapping missing from TLB.

Page fault?

! ' page missing from RAM.

Two-level without TLB?

! ' 3 memory accesses.

Larger page?

! ' smaller page table, more internal fragmentation.

ACTIVE RECALL

Try these without looking:

1. Why is page size equal to frame size?
2. In paging, which part of the logical address remains unchanged?
3. A 32-bit VA with 4 KB pages has how many offset bits?
4. How many page-number bits?
5. How many virtual pages?
6. Physical memory = 1 GB and page size = 4 KB. How many frame bits?
7. What is the difference between TLB miss and page fault?
8. If a single-level page table is in RAM and no TLB exists, how many memory accesses are needed for one data reference?
9. A 3-level page table without TLB requires how many memory accesses?
10. Why does multilevel paging save memory?
11. Page size = 4 KB and PTE = 4 bytes. How many entries fit in one page-table page?
12. Why do larger pages reduce page-table size?

Answers for the key numericals:

Q3

$$\lfloor 4\text{KB} = 2^{12} \rfloor$$

$$\lfloor \boxed{12 \text{ offset bits}} \rfloor$$

Q4

$$\lfloor 32 - 12 = \boxed{20} \rfloor$$

Q5

$$\lfloor \boxed{2^{20}} \rfloor$$

Q6

$1\text{GB} = 2^{30}$

$30 - 12 = \boxed{18 \text{ frame bits}}$

Q9

$3 + 1 = \boxed{4}$

Q11

$4096/4 = 1024 = \boxed{2^{10}}$

NEXT — CHAPTER 13 +PØ=Ý%

Next we cover:

SEGMENTATION+ PAGING VS SEGMENTATION

We will study:

- what a segment is
- logical view of program
- segment number + offset
- segment table
- base and limit
- address translation
- validity checking
- internal vs external fragmentation
- sharing and protection
- paging vs segmentation
- paged segmentation / segmented paging idea where relevant
- GATE numericals and traps

After that we enter:

VIRTUAL MEMORY + DEMAND PAGING + PAGE FAULTS+ PAGE REPLACEMENT

USER

NEXT

ASSISTANT

CHAPTER 13 — SEGMENTATION+ PAGING VS SEGMENTATION+PØ=Ý%

Segmentation is another important memory-management technique.

The key idea is:

Paging divides memory into equal-sized blocks based on physical convenience. Segmentation divides a program according to its logical structure.

Think:

TEXT

Program

```
% % %   C o d e
% % %   D a t a
% % %   S t a c k
% % %   F u n c t i o n s
% % %   A r r a y s
```

Each logical part can become a **segment**.

1. WHAT IS SEGMENTATION?

Segmentation divides a process into:

Variable-sized logical units called segments.

Examples:

- code segment
- data segment
- stack segment
- heap segment
- function/module
- array/table

Unlike paging:

$\boxed{\text{Segments need not have equal size}}$

2. SIMPLE INTUITION

Suppose a book has:

TEXT

```
C h a p t e r   0   !'   2 0   p a g e s
C h a p t e r   1   !'   5 0   p a g e s
C h a p t e r   2   !'   1 0   p a g e s
C h a p t e r   3   !'   3 5   p a g e s
```

Chapters naturally have different sizes.

Segmentation treats a program similarly.

TEXT

```
S e g m e n t   0   ! '   M a i n   p r o g r a m
S e g m e n t   1   ! '   F u n c t i o n   A
S e g m e n t   2   ! '   A r r a y
S e g m e n t   3   ! '   S t a c k
```

Each segment may have a different length.

3. WHY SEGMENTATION?

A programmer naturally thinks about:

TEXT

```
Code
Stack
Data
Functions
Objects
Modules
```

not:

TEXT

```
Page 0
Page 1
Page 2
```

Segmentation reflects this logical program organization.

This helps with:

- protection
- sharing
- modularity
- logical memory organization

4. LOGICAL ADDRESS IN SEGMENTATION

A logical address consists of:

TEXT

```
+-----+-----+
| Segment Number |   Offset   |
+-----+-----+
```

Written as:

$\boxed{(s,d)}$

where:

- (s) = segment number
- (d) = displacement/offset within segment

5. EXAMPLE

Logical address:

$\mathbb{V}[(2,500)]$

means:

Segment 2, offset 500.

We need to determine:

1. where Segment 2 begins in physical memory
2. whether offset 500 lies within Segment 2

This information comes from the:

Segment Table

6. SEGMENT TABLE

Each segment-table entry contains mainly:

Base

Starting physical address of the segment.

Limit

Length/size of the segment.

Example:

Segment	Base	Limit
0	1000	400
1	5000	1000
2	9000	700

7. ADDRESS TRANSLATION

For logical address:

$\mathbb{V}[(s,d)]$

Find:

TEXT

Base[s]

Limit[s]

First check:

$\boxed{d < \text{Limit}[s]}$

If valid:

$\boxed{PA = \text{Base}[s] + d}$

If invalid:

trap / segmentation fault-like protection exception.

MASTER FORMULA

$\boxed{\text{If } d < \text{Limit}[s], \quad PA = \text{Base}[s] + d}$

Otherwise:

$\boxed{\text{Invalid address}}$

8. EXAMPLE

Segment table:

Segment	Base	Limit
0	1000	600
1	4000	1200
2	8000	500

Logical address:

$(1, 300)$

Check:

$300 < 1200$

Valid.

Physical:

$4000 + 300$

$\boxed{4300}$

9. INVALID EXAMPLE

Logical address:

$(2, 700)$

For Segment 2:

$\text{Limit} = 500$

Check:

$\lfloor 700 < 500 \rfloor$

False.

Therefore:

$\lfloor \boxed{\text{Invalid}} \rfloor$

Do **not** calculate:

$\lfloor 8000 + 700 \rfloor$

because validation comes first.

& pGATE TRAP

Always:

FIRST

$\lfloor \text{Offset} < \text{Limit} \rfloor$

THEN

$\lfloor \text{Base} + \text{Offset} \rfloor$

Do not translate an invalid address.

10. SEGMENT SIZE IS VARIABLE

Example:

TEXT

Segment 0 = 4 KB
Segment 1 = 15 KB
Segment 2 = 2 KB
Segment 3 = 20 KB

No requirement that:

TEXT

Segment 0 size = Segment 1 size

This is the major difference from paging.

11. PHYSICAL PLACEMENT

Segments can be placed at different locations in RAM.

Example:

TEXT

Physical Memory

```

+-----+
| Segment 2 |
+-----+
| Free      |
+-----+
| Segment 0 |
+-----+
| Another Proc |
+-----+
| Segment 1 |
+-----+

```

Segments of one process need not be adjacent.

But:

Each individual segment is ordinarily stored contiguously in basic segmentation.

12. FRAGMENTATION IN SEGMENTATION

Because segments are:

- variable-sized
- allocated into variable-sized holes

basic segmentation can suffer from:

External Fragmentation

This resembles variable-partition allocation.

13. WHY EXTERNAL FRAGMENTATION?

Suppose free holes:

```

TEXT
20 KB
10 KB
30 KB

```

Total:

[60KB]

New segment requires:

[40KB]

No single contiguous 40-KB hole exists.

Allocation fails.

Therefore:

External fragmentation.

14. DOES SEGMENTATION HAVE INTERNAL FRAGMENTATION?

Pure variable-sized segmentation generally avoids the fixed-block internal fragmentation characteristic of paging.

But the primary GATE distinction is:

TEXT

Paging !' Internal fragmentation possible
Segmentation !' External fragmentation possible

MEMORY TRICK

Paging fixes sizes !' waste can remain inside.

Segmentation varies sizes !' holes appear outside.

15. PAGING VS SEGMENTATION Ø=Ý%

Paging	Segmentation
Fixed-size pages	Variable-size segments
Physical-memory-oriented mechanism	Logical program-oriented
Page number + offset	Segment number + offset
Page table	Segment table
Page !' frame	Segment !' base + limit
No external fragmentation	External fragmentation possible
Internal fragmentation possible	Little/no classic internal fragmentation
Programmer usually doesn't see pages	Segments reflect logical program units
Easy physical allocation	Better logical protection/sharing

16. MEMORY SHORTCUT

Paging

Equal pieces

Segmentation

Meaningful pieces

17. PAGE TABLE VS SEGMENT TABLE

Page Table Entry

Main mapping:

TEXT

P a g e ! ' F r a m e

Because every page has the same size.

Segment Table Entry

Needs:

TEXT

Base
Limit

because segments have different sizes.

This distinction is important.

18. WHY DOES SEGMENT TABLE NEED LIMIT?

Page sizes are fixed.

If:

TEXT

P a g e s i z e = 4 K B

the offset range is already known:

TEXT

0 . . . 4095

But segments can have different sizes.

Example:

TEXT

Segment 1 = 300 bytes
Segment 2 = 9000 bytes

Therefore each segment requires its own:

Limit.

19. SEGMENT NUMBER VALIDATION

Suppose the process has:

TEXT

5 segments

Segment numbers:

TEXT

0, 1, 2, 3, 4

Logical address:

TEXT

(7, 100)

is invalid even before offset checking.

Systems may maintain a:

Segment Table Length Register (STLR)

to determine valid segment numbers.

20. SEGMENT TABLE BASE REGISTER — STBR

Conceptually:

STBR stores the starting address of the segment table.

Similar idea to:

TEXT

P T B R ! ' P a g e T a b l e B a s e
S T B R ! ' S e g m e n t T a b l e B a s e

21. SEGMENT TABLE LENGTH REGISTER — STLR

STLR stores:

number/length of valid segment-table entries.

If:

$\lfloor s \geq \text{STLR} \rfloor$

segment number is invalid.

Then if segment exists, check:

$\lfloor d < \text{Limit}[s] \rfloor$

22. COMPLETE SEGMENT TRANSLATION

Logical:

$\lfloor (s, d) \rfloor$

Check 1

$\lfloor s < \text{STLR} \rfloor$

Check 2

$\lfloor d < \text{Limit}[s] \rfloor$

Translate

$\lfloor \text{PA} = \text{Base}[s] + d \rfloor$

That's the complete conceptual procedure.

23. FULL EXAMPLE

Suppose:

TEXT
STLR = 4

Segment table:

Segment	Base	Limit
0	2000	500
1	6000	900
2	10000	400
3	15000	1200

Logical address:

$\lfloor (3, 1000) \rfloor$

Check:

$\lfloor 3 < 4 \rfloor$

valid segment.

Then:

$\lfloor 1000 < 1200 \rfloor$

valid offset.

Physical:

$\lfloor 15000 + 1000 \rfloor$

$\lfloor \boxed{16000} \rfloor$

24. ANOTHER EXAMPLE

Address:

$\lfloor (2, 450) \rfloor$

Segment 2 limit:

$\lfloor 400 \rfloor$

Since:

$\lfloor 450 \geq 400 \rfloor$

$\lfloor \boxed{\text{Invalid}} \rfloor$

25. PROTECTION USING SEGMENTS

Segmentation is useful because protection can be defined per logical unit.

Example:

TEXT

```
Code Segment !' Read + Execute
Data Segment !' Read + Write
Stack Segment !' Read + Write
```

This aligns naturally with program structure.

26. SHARING USING SEGMENTS

Suppose two processes use the same read-only library.

They could share:

TEXT

Library Code Segment

while having separate:

TEXT

Data
Stack
Heap

So segmentation naturally supports:

27. SEGMENTATIONEXAMPLE — FUNCTION SHARING

Process P1:

TEXT

```
Segment 0 !' Main
Segment 1 !' Shared Math Library
Segment 2 !' Data
```

Process P2:

TEXT

```
Segment 0 !' Main
Segment 1 !' Same Shared Math Library
Segment 2 !' Data
```

Their segment tables can point the shared-library segment to the same physical memory.

28. SEGMENTATIONAND GROWTH

Logical units such as:

- stack
- heap

may grow dynamically.

Segmentation makes this conceptually natural because they are distinct logical regions.

But physical contiguous expansion may be difficult if neighboring memory is occupied.

This contributes to allocation complexity.

29. COMPACTION

Since segmentation suffers external fragmentation, compaction may help.

Before:

TEXT

```
S1
Free
S2
Free
S3
Free
```

After:

TEXT

S1
S2
S3
Large Free Hole

But moving segments is expensive.

30. WHY PAGING BECAME ATTRACTIVE

Paging avoids needing each segment/process region to find a variable-sized contiguous hole.

Any free frame can hold a page.

Thus:

$\boxed{\text{No external fragmentation}}$

in standard paging.

This makes physical memory allocation easier.

31. BUT PAGING LOSES SOME LOGICAL STRUCTURE

A 4-KB page may contain arbitrary pieces based purely on addresses.

The OS sees:

TEXT

Page 0
Page 1
Page 2

rather than naturally:

TEXT

Function A
Stack
Array

Segmentation preserves logical boundaries better.

32. PAGED SEGMENTATION

Can we combine both?

Yes.

A system can:

1. divide program into logical segments
2. divide each segment into pages

Conceptually:

TEXT

```
Program
!"
Segments
!"
Each Segment divided into Pages
!"
Pages placed in Frames
```

This is often called:

Segmentation with Paging / Paged Segmentation

33. WHY COMBINE THEM?

Segmentation gives:

- logical organization
- protection
- sharing

Paging gives:

- non-contiguous physical allocation
- eliminates external fragmentation at the page-allocation level

So combination tries to obtain benefits of both.

34. LOGICAL ADDRESS IN PAGED SEGMENTATION

Conceptually:

TEXT

```
+-----+-----+-----+
| Segment | Page   | Offset |
+-----+-----+-----+
```

Steps:

1. Segment number selects segment-table entry.
2. Entry points to that segment's page table.
3. Page number selects a PTE.
4. PTE gives frame.
5. Offset remains unchanged.

35. TRANSLATION PATH

TEXT

```
Logical Address
(Segment, Page, Offset)
```

Segment Table	!"
Page Table for that Segment	!"
Frame Number	!"
Frame + Offset	!"

36. MEMORY ACCESSES

Without TLB/cache assistance, paged segmentation can require:

1. segment-table lookup
2. page-table lookup
3. actual data access

So potentially:

$\boxed{3 \text{ memory accesses}}$

in a basic implementation.

Exact architectures vary.

37. SEGMENTED PAGING TERMINOLOGY

Textbook terminology can vary:

- paging with segmentation
- segmentation with paging
- paged segmentation

Do not obsess over labels.

Focus on address structure and translation path given in the question.

38. PAGING VS SEGMENTATION— FRAGMENTATION

This is extremely important:

Paging

Fixed-size blocks.

$\boxed{\text{Internal fragmentation}}$

possible.

Segmentation

Variable-size contiguous segments.

$\boxed{\text{External fragmentation}}$

possible.

Paged Segmentation

Segments themselves are paged.

Physical page allocation removes classic external fragmentation of whole segments, but page-level internal fragmentation can remain.

39. GATE-STYLE QUESTION

Which technique has variable-sized logical units?

A. Paging B. Segmentation

Answer:

$\boxed{\text{Segmentation}}$

40. GATE-STYLE QUESTION

Which table entry naturally requires both Base and Limit?

$\boxed{\text{Segment table entry}}$

because each segment can differ in size/location.

41. GATE-STYLE NUMERICAL

Segment table:

Segment	Base	Limit
0	1000	400
1	5000	600
2	9000	800

Find physical address for:

$(2, 700)$

Check:

$700 < 800$

Valid.

Then:

$9000 + 700 = \boxed{9700}$

42. GATE-STYLE INVALID ADDRESS

Same table.

Address:

$\sqrt{(1,650)}$

Limit:

$\sqrt{600}$

Since:

$\sqrt{650 \geq 600}$

$\sqrt{\boxed{\text{Invalid}}}$

43. GATE-STYLE FRAGMENTATION QUESTION

Which statements are correct?

1. Paging can have internal fragmentation.
2. Paging has external fragmentation in standard allocation.
3. Segmentation can have external fragmentation.
4. Segment sizes are fixed.

Correct:

$\sqrt{\boxed{1, 3}}$

44. GATE-STYLE CONCEPT QUESTION

Which better reflects the programmer's logical view of a program?

$\sqrt{\boxed{\text{Segmentation}}}$

because logical entities such as code/data/functions can become segments.

45. GATE-STYLE PROTECTION QUESTION

Why can segmentation provide natural protection?

Because permissions can be assigned to logical units.

Example:

TEXT

C	o	d	e	!	'	R	X
D	a	t	a	!	'	R	W

46. OFFSET IN PAGING VS SEGMENTATION

In both mechanisms an offset exists, but meanings differ.

Paging Offset

Offset within a fixed-size page.

Maximum offset determined by page size.

Segmentation Offset

Offset within a variable-sized segment.

Must be compared to that segment's limit.

47. ADDRESS SPLIT DIFFERENCE

Paging:

TEXT

Page Number | Fixed-size Offset

Segmentation:

TEXT

Segment Number | Variable-valid-range Offset

The offset field may have a fixed bit width in hardware representation, but validity depends on each segment's limit.

48. PAGING ALLOCATION

Physical memory is divided beforehand into equal frames.

Any page fits any free frame.

Thus allocation is straightforward.

49. SEGMENTATION ALLOCATION

Need to find a hole large enough for each variable-sized segment.

Therefore algorithms conceptually resemble:

- First Fit
- Best Fit
- Worst Fit

and external fragmentation may occur.

50. PAGE TABLE SIZE VS SEGMENT TABLE SIZE

Suppose virtual address space contains:

2^{20}

pages.

Page table may have:

2^{20}

entries.

But a process may have only:

TEXT

5 logical segments

Then segment table needs roughly:

TEXT

5 entries

So segmentation tables can be smaller in entry count.

However segment allocation/management is more complex.

51. SHARING DIFFERENCE

Paging can share pages.

Segmentation can share entire logical segments.

Conceptually, segment-level sharing is often easier to reason about because:

logical module boundaries are preserved.

52. PROTECTION DIFFERENCE

Paging:

Permissions can be assigned page-by-page.

Segmentation:

Permissions naturally align with logical units.

Both can provide protection; segmentation's logical structure is the key distinction.

53. INTERNAL FRAGMENTATIONEXAMPLE — PAGING

Process:

[18KB]

Page size:

[4KB]

Pages needed:

[5]

Allocated:

[20KB]

Internal waste:

[2KB]

54. EXTERNAL FRAGMENTATIONEXAMPLE — SEGMENTATION

Free holes:

```
TEXT
15 KB
20 KB
30 KB
```

Total:

\[65KB \]

Segment:

\[40KB \]

No single hole fits.

Therefore:

\[\boxed{\text{External fragmentation}} \]

55. COMPARISON — MEMORY VIEW

Paging

```
TEXT
Virtual:

[P0][P1][P2][P3]
```

All equal.

Segmentation

```
TEXT
[Code-----]
[Data--]
[Stack----]
[Array-----]
```

Different sizes.

This visual is worth remembering.

56. PAGING IS TRANSPARENT TO PROGRAMMER

The programmer normally does not explicitly say:

“Put this in page 7.”

Paging is a system-level mechanism.

Segmentation can correspond much more directly to programmer/compiler-visible logical regions.

57. SEGMENTATION FAULT IDEA

If a process accesses beyond segment limit:

TEXT

Offset "e Limit

hardware generates an exception.

This is conceptually related to protection faults often described as segmentation faults in common systems, although real modern OS behavior can involve paging and other mechanisms too.

For GATE:

Limit violation !' invalid segment access.

58. MULTIPLE SEGMENTS CAN OVERLAP PHYSICALLY?

Normally separate writable segments should map according to system protection rules.

But multiple logical segment-table entries may intentionally refer to shared physical memory for controlled sharing.

So physical sharing is possible if designed.

59. RELOCATION

Because segment table stores base addresses, a segment can conceptually be moved to a new physical location by changing:

TEXT

Base

rather than changing every logical reference.

This enables dynamic relocation.

60. SEGMENT TABLE ENTRY

A useful conceptual entry:

TEXT

Base

Limit

Protection

Sharing information

Compare page PTE:

TEXT

Frame number
Valid/present
Protection
Dirty
Reference

61. HOW GATE CAN ASK SEGMENTATION

Common patterns:

Pattern 1

Given segment table, translate address.

Pattern 2

Determine whether address is valid.

Pattern 3

Paging vs segmentation comparison.

Pattern 4

Internal vs external fragmentation.

Pattern 5

Base/limit meaning.

Pattern 6

Logical program structure.

Pattern 7

Combined paging + segmentation translation.

& pGATE TRAPS — SEGMENTATION

Trap 1

All segments are equal size.

'L

Segments are variable-sized.

Trap 2

Segmentation eliminates external fragmentation.

'L

Basic segmentation can suffer external fragmentation.

Trap 3

Segment table stores frame number only.

'L

It requires at least:

TEXT

Base + Limit

conceptually.

Trap 4

For address (s, d) , immediately calculate $\text{Base} + d$.

'L

First check:

$\lceil d < \text{Limit} \rceil$

Trap 5

Paging reflects logical program structure better than segmentation.

'L

Segmentation does.

Trap 6

Paging and segmentation both use fixed-size logical units.

'L

Only paging uses fixed-size pages.

Trap 7

Paging suffers classic external fragmentation.

'L

Standard paging does not.

Trap 8

Segmentation always has internal fragmentation.

'L

Its classic major issue is external fragmentation.

Trap 9

Page table and segment table contain identical information.

'L

Page table:

TEXT

P a g e ! ' F r a m e

Segment table:

TEXT

S e g m e n t ! ' B a s e + L i m i t

Trap 10

Combining segmentation and paging still requires each whole segment to occupy one contiguous physical block.

'L

If segments are paged, pages can occupy separate frames.

' p WRITE THIS IN YOUR NOTES

Segmentation

Memory management technique dividing a process into variable-sized logical segments.

Examples:

TEXT

Code
Data
Stack
Functions
Arrays

Logical address:

$\lfloor \boxed{(\text{Segment}, \text{Offset})} \rfloor$

Segment Table

Each entry:

TEXT

Base
Limit

Translation:

$\lfloor \boxed{\text{Offset} < \text{Limit}} \rfloor$

then:

$\lfloor \boxed{\text{PA} = \text{Base} + \text{Offset}} \rfloor$

Fragmentation

TEXT

Paging !' Internal fragmentation
Segmentation !' External fragmentation

Memory:

Pages are equal !' inside waste.

Segments vary !' outside holes.

PAGING VS SEGMENTATION

Paging	Segmentation
Fixed size	Variable size
Pages	Logical segments
Page table	Segment table
Page !' Frame	Base+Limit
Internal fragmentation	External fragmentation
Physical organization	Logical organization

Paged Segmentation

TEXT

Process
! "
Segments
! "
Pages
! "

Logical address:

TEXT

Segment | Page | Offset

MASTER SHORTCUT

When given a segmentation numerical:

Step 1

Find segment entry.

Step 2

Check:

$\backslash$ Offset < Limit $\backslash$

Step 3

If valid:

$\backslash$ PA = Base + Offset $\backslash$

Memory:

Check limit before adding base.

30-SECOND REVISION

Segment?

!' Variable-sized logical unit.

Logical address?

!' Segment + offset.

Segment table?

!' Base + Limit.

Validation?

$\backslash$ Offset < Limit $\backslash$

Translation?

$\backslash$ Base + Offset $\backslash$

Paging fragmentation?

!' Internal.

Segmentation fragmentation?

!' External.

Which reflects programmer's view?

!' Segmentation.

Paged segmentation?

!' Segments divided into pages.

ACTIVE RECALL

Try without looking:

1. What is the fundamental difference between a page and a segment?
2. Why does each segment-table entry need a limit?
3. Which fragmentation is associated with segmentation?
4. Which fragmentation is associated with paging?
5. Segment base = 5000, limit = 800, offset = 700. Find physical address.
6. Same values, offset = 900. What happens?
7. Which mechanism better represents code/data/stack as separate logical units?
8. Why can segmentation support natural protection?
9. What is the logical address format in paged segmentation?
10. Does basic segmentation require each segment to be physically contiguous?

For Q5:

$\lfloor 700 < 800 \rfloor$

$\lfloor 5000 + 700 = \boxed{5700} \rfloor$

For Q6:

$\lfloor 900 \geq 800 \rfloor$

$\lfloor \boxed{\text{Invalid}} \rfloor$

MASTER MEMORY MAP

TEXT

```
Program
  !"
Logical Components
  !"
Segments
  !"
Segment Table
  !"
Base + Limit
  !"
Physical Memory
```

Compared with:

TEXT

Virtual Memory
! "
Fixed Pages
! "
Page Table
! "
Frames

One-line memory:

"Paging cuts equally; segmentation cuts meaningfully."

NEXT — CHAPTER 14 Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

Now we enter one of the most important areas in all of GATE OS:

VIRTUAL MEMORY + DEMAND PAGING + PAGE FAULTS

Next we will cover:

- what virtual memory really means
- why a process can be larger than RAM
- demand paging
- locality of reference
- page fault
- exact page-fault handling sequence
- valid/present bits
- pure demand paging
- effective access time with page faults
- why page faults are extremely expensive
- copy-on-write idea where relevant
- memory-mapped concepts only to required depth
- GATE numericals and traps

Then immediately after that:

PAGE REPLACEMENT — FIFO, OPTIMAL, LRU, BELADY'S ANOMALY

USER

NEXT

ASSISTANT

CHAPTER 14 — VIRTUAL MEMORY + DEMAND PAGING + PAGE FAULTS

Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

Virtual memory is one of the most important GATE OS topics.

The core idea is:

TEXT

Program may be very large
!"
Not all of it is needed at once
!"
Keep only required pages in RAM
!"
Remaining pages stay on disk
!"
Bring page when needed

This is **Virtual Memory**.

1. WHAT IS VIRTUAL MEMORY?

Virtual memory is a memory-management technique that allows a process to execute even when:

The entire process is not currently present in physical memory.

Only the actively required parts need to be in RAM.

The rest can remain on secondary storage.

2. WHY DO WE NEED VIRTUAL MEMORY?

Suppose:

TEXT

RAM = 4 GB

A process has a virtual address space of:

TEXT

8 GB

Does the entire 8 GB need to be loaded into RAM before execution?

No.

The process may only actively use:

TEXT

500 MB

at a particular time.

So:

TEXT

```
N e e d e d   p a g e s   ! '   R A M
U n u s e d   p a g e s   ! '   D i s k
```

This lets programs have an address space larger than physical memory.

3. SIMPLE ANALOGY

Imagine studying from a huge library.

You don't bring every book to your desk.

You keep:

TEXT

```
B o o k s   c u r r e n t l y   n e e d e d   ! '   d e s k
O t h e r   b o o k s   ! '   s h e l v e s
```

RAM is your desk.

Disk is the library shelf.

Virtual memory lets you work as though many books are available, even though only a few are physically on the desk.

4. VIRTUAL VS PHYSICAL MEMORY

TEXT

```
Virtual Memory
= address space seen by process
```

```
Physical Memory
= actual RAM
```

Virtual memory can be:

$\boxed{\text{larger than physical memory}}$

This is a key idea.

5. HOW PAGING SUPPORTS VIRTUAL MEMORY

Remember:

TEXT

```
V i r t u a l   m e m o r y   ! '   P a g e s
P h y s i c a l   m e m o r y   ! '   F r a m e s
```

With virtual memory, not every virtual page must have a physical frame at the same time.

Example:

Virtual Page	Status
P0	Frame 3
P1	On disk
P2	Frame 8
P3	On disk
P4	Frame 1

Only some pages are resident.

6. DEMAND PAGING

Very important.

Demand paging means:

Load a page into physical memory only when the process actually references it.

Memory shortcut:

Demand Paging = Bring on Demand

No need to load every page initially.

7. PURE DEMAND PAGING

In pure demand paging:

Initially, no pages may be loaded.

Pages are brought in only when first referenced.

Therefore process execution may initially generate several page faults.

8. WHAT IS A PAGE FAULT?

A page fault occurs when:

A process references a virtual page that is not currently present in physical memory.

Example:

TEXT

CPU references Page 7

!"

Page Table says:

```
Present = 0
!
PAGE FAULT
```

The OS must obtain the page from secondary storage.

9. PAGE FAULT IS NOT NECESSARILY AN ERROR

Very important.

A page fault can be a **normal virtual-memory event**.

Example:

```
TEXT
Valid virtual page
but currently on disk
```

! page fault.

The OS loads it and execution continues.

But if the address itself is invalid:

```
TEXT
Process references memory outside its legal address space
```

then the OS may terminate the process or raise an invalid-memory exception.

So:

Page not resident "illegal address.

10. VALID PAGE VS PRESENT PAGE

Suppose process owns Page 5.

But Page 5 is currently on disk.

Then:

```
TEXT
Valid?  YES
Present? NO
```

Referencing it causes a page fault.

Suppose process references Page 999 which is outside its address space:

```
TEXT
Valid? NO
```

That's an invalid reference, not a normal demand-page retrieval.

11. PAGE FAULT HANDLING — COMPLETE SEQUENCE

Extremely important.

Suppose CPU references a missing page.

Typical sequence:

TEXT

1. CPU generates virtual address
!"
2. Page-table entry checked
!"
3. Page absent
!"
4. Page-fault trap to OS
!"
5. OS verifies reference is legal
!"
6. Find a free frame
!"
7. If no free frame !' choose victim page
!"
8. If victim dirty !' write it to disk
!"
9. Read required page from disk
!"
10. Update page table / TLB
!"
11. Restart interrupted instruction

This sequence is worth remembering.

12. WHY RESTART THE INSTRUCTION?

Suppose instruction:

TEXT

LOAD R1, [address]

causes a page fault.

The instruction could not complete.

After the page is loaded:

The same instruction is restarted.

This is why CPU/OS must preserve process state correctly.

13. PAGE FAULT STATE TRANSITION

Suppose P1 is running.

Page fault requires disk I/O.

Then:

```
TEXT
RUNNING
    ! "
WAITING / BLOCKED
```

While page is loaded, CPU can run another process.

When disk I/O completes:

```
TEXT
WAITING
    ! "
READY
```

Later scheduler gives CPU again.

14. LOCALITY OF REFERENCE

Virtual memory works efficiently because programs exhibit:

Locality

Meaning:

Programs tend to access a relatively small portion of their address space repeatedly during a period.

Two important forms:

1. Temporal locality
2. Spatial locality

15. TEMPORAL LOCALITY

If something was accessed recently:

It is likely to be accessed again soon.

Example:

```
c
for (...) {
    sum = sum + x;
}
```

Variables/instructions inside the loop are repeatedly accessed.

Memory:

Temporal = Time

Recent in time != likely again.

16. SPATIAL LOCALITY

If one memory location is accessed:

Nearby locations are likely to be accessed soon.

Example:

Sequential array traversal:

```
c
a[0], a[1], a[2], a[3]...
```

Memory:

Spatial = Space

Nearby addresses != likely soon.

17. WHY LOCALITY HELPS PAGING

Suppose process has:

```
TEXT
1000 pages
```

But during the next second it repeatedly uses only:

```
TEXT
20 pages
```

Then keeping those 20 pages in RAM is enough for good performance.

This is why:

Entire program does not need to stay in RAM.

18. RESIDENT SET

The set of a process's pages currently in physical memory is its:

Resident Set

Example:

TEXT

Process P has pages:
0, 1, 2, 3, 4, 5, 6, 7

Currently in RAM:
1, 3, 4, 7

Resident set:

$\{1, 3, 4, 7\}$

19. PAGE HIT

If requested page is already in physical memory:

TEXT

Page present
! "
Normal memory access

This can be thought of as a page hit.

No page fault occurs.

20. PAGE FAULT RATE

If:

p

is page-fault probability:

Example:

$p = 10^{-5}$

means roughly one page fault per:

100,000

memory references on average.

Page faults must be rare because they are extremely expensive.

21. WHY PAGE FAULTS ARE EXPENSIVE

RAM access:

TEXT

~nanoseconds

Disk/SSD access:

TEXT

microseconds to milliseconds

That's many orders of magnitude slower.

So even a tiny page-fault probability can dramatically increase effective memory access time.

22. EFFECTIVE ACCESS TIME WITH PAGE FAULTS

Suppose:

- memory access time = m
- page-fault probability = p
- page-fault service time = F

Then simplified:

$$\boxed{EAT = (1-p)m + pF}$$

More precisely, if a fault path also includes restarted memory access, the exact formula may include that too.

GATE rule:

Use the costs exactly as defined in the question.

23. EXAMPLE

Memory access:

$$100\text{ns}$$

Page fault service:

$$10\text{ms}$$

Fault probability:

$$10^{-6}$$

Convert:

$$10\text{ms} = 10,000,000\text{ns}$$

Then:

$$EAT = (1 - 10^{-6})(100) + (10^{-6})(10,000,000)$$

Approximately:

$$100 + 10$$

$$\boxed{110\text{ns}}$$

A one-in-a-million page-fault rate increased access time by about 10%.

That shows how expensive faults are.

24. UNIVERSAL EAT SHORTCUT

For any multi-path access:

$$EAT = \sum (\text{Probability}) \times \text{Cost}$$

This works for:

- TLB hit/miss
- cache hit/miss
- page faults
- combinations

Don't memorize twenty formulas.

Memorize:

Probability × Path Cost

25. PAGE-FAULTSERVICE TIME COMPONENTS

A page fault may involve:

1. trap overhead
2. find page location
3. choose/free frame
4. possibly write victim
5. disk read
6. update page table
7. restart process

Disk I/O dominates.

26. CLEAN VS DIRTY VICTIM

Suppose memory is full and a page must be evicted.

Clean page

Not modified.

Disk already contains correct copy.

Can often simply discard it.

Dirty page

Modified since loading.

Must usually:

TEXT

Write victim to disk

before reusing frame.

Therefore dirty-page replacement is more expensive.

27. DIRTY BIT

Page-table entry may contain:

Dirty / Modified bit

Meaning:

TEXT

```
0 !'  p a g e  n o t  m o d i f i e d
1 !'  p a g e  m o d i f i e d
```

If dirty:

write back before eviction.

28. REFERENCE BIT

Page table may also contain:

Reference / Accessed bit

Set when page is accessed.

Useful in replacement algorithms to estimate recent use.

29. WHAT IF FREE FRAME EXISTS?

Page fault:

TEXT

```
Missing page
! "
Free frame available
! "
Load page there
```

No replacement needed.

30. WHAT IF NO FREE FRAME EXISTS?

Then we need:

Page Replacement

The OS chooses a victim page to remove.

This leads to algorithms:

- FIFO
- Optimal
- LRU
- others

Next chapter focuses on these.

31. DEMAND PAGING REQUIREMENTS

Demand paging needs mechanisms for:

- detecting page absence
 - trapping to OS
 - secondary storage/backing store
 - free-frame allocation
 - page replacement when necessary
 - restarting faulting instruction
-

32. BACKING STORE / SWAP SPACE

Pages not currently in RAM may be stored in:

- executable/file storage
- swap space
- other backing storage

Conceptually:

TEXT

Virtual Pages

!"

RAM OR Disk

33. SWAPPING VS PAGING

Earlier:

Swapping

Often conceptualized as moving an entire process.

Demand Paging

Moves individual pages.

Therefore paging gives much finer-grained movement.

34. PREPAGING

Opposite idea to strict demand paging.

Instead of loading only the faulting page:

Load additional pages believed likely to be used soon.

This may exploit spatial locality.

But if predictions are wrong:

unnecessary I/O occurs.

For GATE, understand the idea; demand paging remains the central concept.

35. COPY-ON-WRITE — COW

+Useful GATE-relevant extension.

Suppose parent creates child using `fork()`.

Naively copying every page immediately would be expensive.

Instead:

TEXT

```
Parent pages
           ! "
Shared read-only
           ! '
Child pages
```

Both initially share the same physical pages.

Only when one process writes to a page:

TEXT

```
Write occurs
           ! "
Make private copy
```

This is:

Copy-on-Write

36. WHY COPY-ON-WRITE?

It avoids unnecessary copying.

Example:

Parent forks child.

Child soon calls:

TEXT

`exec()`

If we had copied every parent page before `exec`:

huge wasted work.

COW delays copying until actually needed.

37. COW MEMORY TRICK

Read together, Copy when Writing.

That's the whole concept.

38. PAGE FAULT CAN OCCUR EVEN WHEN MEMORY IS NOT FULL

Yes.

Suppose required page is absent but free frames exist.

That's still a page fault.

Memory full determines whether replacement is necessary, not whether fault occurs.

& PAGE TRAP

Page fault:

required page absent.

Page replacement:

needed only when fault occurs and no suitable free frame exists.

These are different.

39. MINOR VS MAJOR PAGE FAULT — OPTIONAL IDEA

Real systems sometimes distinguish:

- minor/soft page fault
- major/hard page fault

depending on whether disk I/O is actually required.

For standard GATE theory, focus on:

page absent !' OS fault handling.

Do not overcomplicate unless question explicitly brings such terminology.

40. NUMBER OF PAGE FAULTS

Given a page-reference string:

TEXT

1, 2, 3, 1, 4, 2, ...

and a fixed number of frames, page faults depend on:

- initial frame contents
- number of frames
- replacement algorithm

This is the central numerical task in the next chapter.

41. COMPULSORY / FIRST-REFERENCE FAULTS

The first time a page is referenced, if it was not initially loaded:

It must fault.

This is often called:

- compulsory fault
- cold-start fault

Example:

Reference string:

TEXT

1, 2, 3

with empty frames.

The first references to 1, 2, 3 all fault.

42. REFERENCE STRING

A sequence of virtual page references is called:

Page Reference String

Example:

TEXT

7 0 1 2 0 3 0 4

Replacement algorithms process this sequence.

43. FRAME COUNT

Suppose process has:

TEXT

3 frames

At most three of its pages can be resident at once, assuming a fixed allocation of three frames.

Reference string may contain many more distinct pages.

Replacement becomes necessary.

44. FIXED VS VARIABLE FRAME ALLOCATION

Fixed allocation

Process receives a fixed number of frames.

Variable allocation

Number of frames assigned to process can change.

This becomes relevant to page replacement and thrashing.

45. LOCAL VS GLOBAL REPLACEMENT

Important concept.

Local Replacement

A process may replace only:

one of its own allocated frames.

Global Replacement

A process may select a victim:

from frames allocated to other processes too.

46. LOCAL VS GLOBAL

Local	Global
Victim from same process	Victim may belong to another process
More predictable isolation	More flexible utilization
One process less able to disrupt others	Processes influence each other's resident sets

This becomes useful in thrashing discussions.

47. DEMAND PAGING AND MULTIPROGRAMMING

Virtual memory allows more processes to be kept logically active because:

each needs only some pages in RAM.

This can increase multiprogramming.

But if too many processes compete for too few frames:

page-fault rate explodes.

That condition is called:

Thrashing

We will cover it after replacement algorithms.

48. PAGE FAULT FREQUENCY

As a process gets more frames, its page-fault rate generally decreases.

But not every replacement algorithm behaves monotonically.

Important exception:

FIFO can exhibit Belady's anomaly.

We'll study this next.

49. DEMAND PAGING WITH TLB

Complete conceptual flow:

```

TEXT
CPU produces virtual address
                        !"
TLB lookup
    !" hit             !" miss
frame    Page table
                        !"

```

```
present?
! " y e s      ! " n o
frame    PAGE FAULT
```

So there are multiple levels of "miss":

```
TEXT
TLB miss
Page-table check
Possible page fault
```

50. TLB MISS VS PAGE FAULT— AGAIN

If:

```
TEXT
TLB miss
Page present in RAM
```

then:

no disk access.

If:

```
TEXT
TLB miss
Page absent in RAM
```

then:

page fault.

So:

$\boxed{\text{TLB miss does not imply page fault}}$

51. PAGE FAULT WITHOUT TLB MISS?

Conceptually, if a valid TLB entry claims the page is present, then normal systems maintain consistency so a stale invalid mapping should not be used. When residency changes, corresponding TLB entries must be invalidated/updated.

For GATE:

Treat TLB/page-table state as coherent unless question states otherwise.

52. EFFECTIVE ACCESS TIME WITH TLB + PAGE FAULTS

Suppose:

- TLB hit ratio (h)
- page-fault probability (p)
- TLB time (t)
- memory time (m)
- page-fault service (F)

Do not rush into a memorized formula.

Draw paths:

TEXT

No page fault

% % T L B h i t

% % T L B m i s s

Page fault

% % f a u l t s e r v i c e

Then:

$EAT = \sum P(\text{path}) \times \text{Cost}(\text{path})$

GATE loves such weighted-average questions.

53. EXAMPLE — MAXIMUM ACCEPTABLE PAGE FAULT RATE

Memory access:

100ns

Page fault:

$10\text{ms} = 10^7\text{ns}$

Suppose desired EAT d :

200ns

Using:

$EAT = (1-p)100 + p(10^7)$

Require:

$100(1-p) + 10^7p \leq 200$

$100 + p(10^7 - 100) \leq 200$

$p \leq \frac{100}{9,999,900}$

Approximately:

$p \leq 10^{-5}$

So page-fault probability must be extremely low.

54. WHY A SMALL FAULT RATE MATTERS

Suppose page fault is:

$\sim 100,000 \times$

times slower than RAM.

Even:

$\sim 0.01\%$

faults can have a serious performance impact.

Memory:

Rare faults, huge cost.

55. DEMAND PAGING ADVANTAGES

- only required pages loaded
- less physical memory needed per process
- larger programs can execute
- more processes can coexist
- faster initial loading in many cases
- better memory utilization

56. DEMAND PAGING DISADVANTAGES

- page-fault overhead
- disk I/O cost
- replacement complexity
- thrashing possible
- page tables / TLB management needed

57. VIRTUAL MEMORY DOES NOT CREATE REAL RAM

Important conceptual trap.

Virtual memory makes the **address space appear larger**, but if the working set greatly exceeds available physical memory:

performance collapses due to page faults.

It does not magically provide RAM-speed memory.

58. WORKING SET — PREVIEW

A process usually actively uses some subset of pages during a recent time window.

That set is called:

Working Set

If enough frames exist for the working set:

low page-fault rate.

If not:

frequent faults !' thrashing.

We'll cover this properly later.

59. PAGE FAULT EXAMPLE

Frames initially empty:

TEXT

3 frames

Reference:

TEXT

1

Page 1 absent.

Fault:

TEXT

[1][-][-]

Reference:

TEXT

2

Fault:

TEXT

[1][2][-]

Reference:

TEXT

1

Page already resident.

No fault.

So for:

TEXT

1, 2, 1

faults:

$\lfloor \boxed{2} \rfloor$

No replacement algorithm was even needed because free frames existed.

60. PAGE FAULT VS REPLACEMENT EXAMPLE

Three frames:

TEXT

[1][2][3]

Reference page:

TEXT

4

Page 4 absent.

So:

Page fault.

No free frame.

Therefore:

Page replacement required.

We must choose one of:

TEXT

1, 2, 3

as victim.

Which one?

Depends on algorithm.

61. OPTIMAL VICTIM IDEA — PREVIEW

Replace page that:

will not be used for the longest time in the future.

This minimizes page faults theoretically.

But OS usually cannot know the future.

That's:

Optimal Page Replacement

62. FIFO PREVIEW

Replace page that:

entered memory earliest.

Think:

oldest resident page leaves first.

63. LRU PREVIEW

Replace page that:

has not been used for the longest time in the past.

Think:

least recently used.

These three form our next chapter.

64. HOW GATE CAN ASK VIRTUAL MEMORY

Common patterns:

Pattern 1

Define demand paging / virtual memory.

Pattern 2

TLB miss vs page fault.

Pattern 3

Page-fault handling sequence.

Pattern 4

Effective access time with page faults.

Pattern 5

Dirty bit / reference bit.

Pattern 6

Copy-on-write.

Pattern 7

Local vs global replacement.

Pattern 8

Locality.

Pattern 9

Identify why virtual memory works efficiently.

& pGATE TRAPS — VIRTUAL MEMORY

Trap 1

Virtual memory means all pages are in RAM.

'L

Only some need to be resident.

Trap 2

Page fault means invalid memory reference.

'L

A valid page may simply be absent.

Trap 3

Every page fault requires page replacement.

'L

If a free frame exists, no victim needed.

Trap 4

TLB miss means page fault.

'L

Trap 5

Page fault handling is cheap.

'L

Usually extremely expensive because of storage I/O.

Trap 6

Dirty page can simply be discarded.

'L

Usually must be written back.

Trap 7

Temporal locality means nearby addresses.

'L

That's spatial locality.

Temporal:

recently used item likely reused.

Trap 8

Spatial locality means recently used item likely reused.

'L

That's temporal.

Trap 9

Copy-on-write copies all pages at `fork()` immediately.

'L

It delays copying until a write occurs.

Trap 10

Virtual address space cannot exceed RAM.

'L

It can.

'p WRITE THIS IN YOUR NOTES

Virtual Memory

Allows process execution without requiring the entire process to be in physical memory.

TEXT

Virtual Pages

! "

Some in RAM

Some on Disk

Demand Paging

Load page only when referenced.

Mnemonic:

Bring on Demand

Page Fault

Referenced valid page is not currently in RAM.

Sequence:

TEXT

Reference page

! "

Not present

! "

Trap OS

! "

Validate

! "

Find frame / replace victim

! "

Read page from disk

! "

Update page table

! "

Restart instruction

Locality

Temporal

Recently used !' likely used again.

Spatial

Nearby address !' likely used soon.

Dirty Bit

TEXT

```
0 !'  u n c h a n g e d
1 !'  m o d i f i e d
```

Dirty victim:

usually write back before replacement.

EAT with Page Fault

Simplified:

$$EAT = (1-p)m + pF$$

Better universal rule:

$$EAT = \sum \text{Probability} \times \text{PathCost}$$

TLB Miss vs Page Fault

TEXT

```
T L B   m i s s !'   t r a n s l a t i o n   n o t   c a c h e d
P a g e   f a u l t !'   p a g e   n o t   r e s i d e n t
```

Copy-on-Write

Parent and child initially share pages; copy only when one writes.

Memory:

Read together, Copy when Writing.

Local vs Global Replacement

TEXT

```
L o c a l !'   r e p l a c e   o w n   f r a m e
G l o b a l !'   m a y   r e p l a c e   a n o t h e r   p r o c e s s ' s   f r a m e
```

MASTER MEMORY MAP

TEXT

```
Virtual Address Space
    Pages
Demand Paging
Page referenced?
    Memory      Page Fault
    Free frame?
    Load      Replace
    Victim
    FIFO / Optimal / LRU
```

EXAM SHORTCUTS

Shortcut 1

If page absent:

Fault first.

Then ask:

Free frame or replacement?

Shortcut 2

Locality:

TEXT

```
Temporal = Time
Spatial = Space
```

Shortcut 3

Dirty victim:

Modified !' Write

Shortcut 4

EAT:

Probability × Cost

Shortcut 5

Don't confuse:

TEXT

TLB miss = translation issue
Page fault = residency issue

30-SECOND REVISION

Virtual memory?

! Process need not be fully in RAM.

Demand paging?

! Load page when referenced.

Page fault?

! Valid page absent from RAM.

Fault always replacement?

! No.

Dirty page?

! Modified; write back before eviction.

Temporal locality?

! recent ! reuse.

Spatial locality?

! nearby ! soon.

COW?

! copy only when write occurs.

TLB miss?

! mapping absent from TLB.

Page fault?

! page absent from RAM.

Main performance danger?

! page-fault cost / thrashing.

ACTIVE RECALL

1. Why can a process be larger than physical memory?
2. What is demand paging?
3. What exactly causes a page fault?
4. Is every page fault an invalid memory access?
5. What happens if a page fault occurs and a free frame exists?
6. What if no free frame exists?
7. Difference between temporal and spatial locality?
8. Why must dirty pages usually be written back?
9. What does Copy-on-Write optimize?
10. Difference between TLB miss and page fault?
11. What does local replacement mean?
12. Why must page-fault probability be very small?

NEXT — CHAPTER 15 Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

Next is the main virtual-memory numerical chapter:

PAGE REPLACEMENT ALGORITHMS

We will cover:

- FIFO
- Optimal / MIN
- LRU
- exact frame-by-frame tracing
- page-fault counting
- page-hit counting
- Belady's anomaly
- stack algorithms
- why OPT is theoretical
- why LRU approximates OPT
- reference bits / Second Chance where useful
- how frame count affects faults
- fastest GATE method for long reference strings
- full tricky numerical examples and traps.

USER

NEXT

ASSISTANT

CHAPTER 15 — PAGE REPLACEMENT ALGORITHMS Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

This is one of the most important numerical chapters in GATE Operating Systems.

The situation is:

```
TEXT
Page Reference
    !"
Page not in RAM
    !"
PAGE FAULT
    !"
Any free frame?
    !"  Y e s          !"  N o
Load page      Choose victim
                !"
                PAGE REPLACEMENT
```

The three most important algorithms are:

1. FIFO
2. Optimal / OPT / MIN
3. LRU

1. WHAT IS PAGE REPLACEMENT?

Suppose a process has only 3 frames:

```
TEXT
[1] [2] [3]
```

and now references:

```
TEXT
Page 4
```

Page 4 is not present.

So page fault occurs.

But there is no free frame.

The OS must remove one of:

```
TEXT
1, 2, 3
```

The selected page is called the:

Victim Page

A page replacement algorithm decides which page becomes the victim.

2. MAIN GOAL

The primary goal is:

Minimize the number of page faults.

Because page faults are extremely expensive.

3. PAGE HIT VS PAGE FAULT

Suppose frames:

TEXT

[1] [2] [3]

Reference:

TEXT

2

2 is already present.

Therefore:

$\boxed{\text{Page Hit}}$

No replacement.

Reference:

TEXT

4

4 absent.

Therefore:

$\boxed{\text{Page Fault}}$

4. BASIC NUMERICAL METHOD

Given:

TEXT

Reference String:

7 0 1 2 0 3 0 4

and:

TEXT

Frames = 3

create a table:

Reference	F1	F2	F3	Hit/Fault
7	7	-	-	F
0	7	0	-	F
...	...	...	...	...

Do not calculate mentally for long strings.

PART A — FIFO Ø=Ý%

5. FIFO — FIRST IN FIRST OUT

FIFO replaces:

The page that has been in memory the longest.

In other words:

First page loaded !' first page removed.

Memory trick:

FIFO = Oldest arrival leaves first.

6. FIFO IS NOT BASED ON RECENT USE

Suppose pages entered in this order:

TEXT

1 !' 2 !' 3

Even if page 1 was used just one moment ago, FIFO may still replace page 1 because:

It entered memory first.

This is a major distinction from LRU.

7. FIFO EXAMPLE

Reference string:

TEXT

1 2 3 4 1 2 5

Frames:

\[3 \]

Initially empty.

8. STEP-BY-STEP FIFO

Reference 1

Fault.

TEXT

[1][-][-]

Faults = 1.

Reference 2

Fault.

TEXT

[1][2][-]

Faults = 2.

Reference 3

Fault.

TEXT

[1][2][3]

Faults = 3.

FIFO order:

TEXT

1 o l d e s t ! ' 2 ! ' 3 n e w e s t

9. REFERENCE 4

4 absent.

No free frame.

Oldest page:

TEXT

1

Replace 1.

TEXT

[4][2][3]

Faults = 4.

FIFO age order now:

TEXT

2 !' 3 !' 4

10. REFERENCE 1

1 absent.

Oldest:

TEXT

2

Replace 2.

TEXT

[4][1][3]

Faults = 5.

Age order:

TEXT

3 !' 4 !' 1

11. REFERENCE 2

2 absent.

Oldest:

TEXT

3

Replace 3.

TEXT

[4][1][2]

Faults = 6.

12. REFERENCE 5

5 absent.

Oldest:

```
TEXT
4
```

Replace 4.

```
TEXT
[5][1][2]
```

Faults = 7.

So:

```
\[ \boxed{FIFO\ Faults=7} \]
```

13. FIFO QUEUE SHORTCUT

Instead of tracking ages manually, keep a queue:

```
TEXT
Front !' oldest
Rear  !' newest
```

Example:

```
TEXT
1 2 3
```

When 4 faults:

```
TEXT
remove 1
append 4
```

Queue:

```
TEXT
2 3 4
```

14. IMPORTANT FIFO RULE ON HIT

Suppose:

```
TEXT
Queue = 1 2 3
```

Reference:

TEXT

1

Hit.

Does 1 move to the back?

'L No.

FIFO does not care about access recency.

Queue remains:

TEXT

1 2 3

This is a huge GATE trap.

PART B — OPTIMAL / OPT / MIN Ø=Ý%Ø=Ý%

15. OPTIMAL PAGE REPLACEMENT

Optimal replaces:

The page whose next use is farthest in the future.

If a page will never be used again:

Replace that page.

Also called:

- OPT
- Optimal
- MIN

16. MEMORY TRICK

OPT looks into the future.

Ask:

“Among current pages, which one do I need latest?”

That becomes victim.

17. WHY OPTIMAL?

It gives:

Minimum possible page faults for a given reference string and number of frames.

Therefore it is used as a benchmark.

18. WHY CAN'T REAL OS USE PERFECT OPT?

Because OS generally does not know exact future page references.

So OPT is mainly:

- theoretical
- benchmark
- useful for exam calculations

19. OPT EXAMPLE

Reference:

TEXT

1 2 3 4 1 2 5 1 2 3 4 5

Frames:

\[3 \]

First three:

TEXT

1	!	'	F
2	!	'	F
3	!	'	F

Frames:

TEXT

[1][2][3]

Faults = 3.

20. REFERENCE 4

Current pages:

TEXT

1, 2, 3

Future sequence after 4:

TEXT

1 2 5 1 2 3 4 5

Next uses:

TEXT

1 !' s o o n
2 !' s o o n
3 !' l a t e r

So 3 is used farthest in future.

Replace 3:

TEXT

[1][2][4]

Faults = 4.

21. REFERENCES 1 AND 2

Both hits.

Frames remain:

TEXT

[1][2][4]

22. REFERENCE 5

Current pages:

TEXT

1, 2, 4

Future after 5:

TEXT

1 2 3 4 5

Next uses:

TEXT

1 !' f i r s t
2 !' s e c o n d
4 !' f o u r t h

Replace 4.

Frames:

TEXT

[1][2][5]

Faults = 5.

23. REFERENCE 1

Hit.

Reference 2:

Hit.

Reference 3:

Absent.

Current:

TEXT

1, 2, 5

Future after 3:

TEXT

4 5

Next use:

TEXT

1 !' n e v e r
2 !' n e v e r
5 !' l a t e r

Either 1 or 2 may be replaced because neither is used again.

Replace 1 for example:

TEXT

[3][2][5]

Faults = 6.

24. REFERENCE 4

4 absent.

Current:

TEXT

3, 2, 5

Future:

3 and 2 never used again.

Replace either.

Faults = 7.

Reference 5 then hits.

So total:

$\boxed{\text{OPT Faults}=7}$

for this reference string.

25. OPT EXAM SHORTCUT

At a fault:

Step 1

Look at current pages.

Step 2

Scan reference string to the right.

Step 3

Find next use of each current page.

Step 4

Replace:

never used again

or, if all reappear:

the one appearing farthest away.

PART C — LRU $\emptyset = \text{YES}$ $\emptyset = \text{YES}$ $\emptyset = \text{YES}$

26. LEAST RECENTLY USED

LRU replaces:

The page that has not been used for the longest time in the past.

Memory:

LRU looks backward.

Compare:

TEXT

```
O P T !'   f u t u r e
L R U !'   p a s t
F I F O !'  a r r i v a l   t i m e
```

This 3-way distinction is essential.

27. LRU INTUITION

If page hasn't been used for a long time:

Maybe it won't be needed soon.

This exploits:

Temporal Locality

Remember:

Recently used pages are likely to be used again.

28. LRU EXAMPLE

Reference:

TEXT

```
1 2 3 1 4 5
```

Frames:

\[3 \]

First:

TEXT

```
1  !'  F
2  !'  F
3  !'  F
```

Frames:

TEXT

```
[1][2][3]
```

Faults = 3.

29. REFERENCE 1

Hit.

But unlike FIFO:

LRU recency order changes.

Recent usage order:

TEXT

2 oldest

3

1 newest

30. REFERENCE 4

4 absent.

Need victim.

Among:

TEXT

1, 2, 3

when were they last used?

TEXT

1 !' j u s t u s e d

3 !' m o r e r e c e n t l y

2 !' l o n g e s t a g o

Replace:

TEXT

2

Frames:

TEXT

[1][4][3]

Faults = 4.

31. REFERENCE 5

Current pages:

TEXT

1, 4, 3

Past uses:

TEXT

```
3 !' o l d
1 !' m o r e   r e c e n t
4 !' m o s t   r e c e n t
```

Replace 3.

Frames:

TEXT

[1][4][5]

Faults = 5.

Thus:

$\boxed{\text{LRU} \setminus \text{Faults}=5}$

32. LRU HIT RULE

On every hit:

Update recency.

Example recency queue:

TEXT

```
o l d e s t !' 1 2 3 !• N e w e s t
```

Reference 1.

Now:

TEXT

```
o l d e s t !' 2 3 1 !• N e w e s t
```

This is unlike FIFO.

33. LRU STACK METHOD

A fast way is to keep:

TEXT

```
L e a s t   R e c e n t !'   M o s t   R e c e n t
```

Example:

TEXT

1 2 3

Reference 2:

TEXT

1 3 2

Reference 4:

Remove least recent:

TEXT

1

Then:

TEXT

3 2 4

This makes LRU fast to solve.

34. FIFO VS LRU

Suppose page 1 entered first but was just accessed.

FIFO

Still considers 1 oldest by arrival.

LRU

Considers 1 recently used.

So they may choose different victims.

MASTER 3-WAY MEMORY TABLE

Algorithm	Looks At	Victim
FIFO	Arrival history	Oldest loaded
LRU	Past usage	Least recently used
OPT	Future usage	Farthest future use

Memory sentence:

FIFO = Arrival, LRU = Past, OPT = Future.

PART D — BELADY'S ANOMALY Ø=Ý%Ø=Ý%Ø=Ý%

35. WHAT SHOULD WE EXPECT?

Normally, if we increase number of frames:

TEXT

3 ! ' 4

we would expect:

Same or fewer page faults.

But with FIFO, something strange can happen.

More frames can produce:

More page faults.

This is:

Belady's Anomaly

36. CLASSIC REFERENCE STRING

A famous example:

TEXT

1 2 3 4 1 2 5 1 2 3 4 5

With FIFO:

3 Frames

Page faults:

\[\boxed{9} \]

4 Frames

Page faults:

\[\boxed{10} \]

So:

\[Frames \uparrow \]

but:

\[Faults \uparrow \]

That's Belady's anomaly.

37. WHY IS IT CALLED ANOMALY?

Because intuition says:

More RAM should not make things worse.

But FIFO can violate that intuition.

38. DOES LRU SHOW BELADY'S ANOMALY?

$\boxed{\text{No}}$

Does OPT?

$\boxed{\text{No}}$

Why?

They are:

Stack Algorithms

39. STACK ALGORITHM

A replacement algorithm has the stack property if:

The set of pages in memory with (n) frames is always a subset of the pages in memory with $(n+1)$ frames, at corresponding points in the reference string.

LRU and OPT have this property.

Therefore:

Increasing frames cannot increase page faults.

40. FIFO IS NOT A STACK ALGORITHM

Therefore FIFO can exhibit:

Belady's anomaly.

Memory:

TEXT

```
F I F O ! '   a n o m a l y   p o s s i b l e
L R U ! '   n o   a n o m a l y
O P T ! '   n o   a n o m a l y
```

41. GATE MEMORY TRICK

F can Fail with More Frames

FIFO:

Frames !' can make Faults !'

LRU/OPT:

stack algorithms !' no Belady.

PART E — FULL COMPARATIVEEXAMPLE

Reference string:

TEXT

7 0 1 2 0 3 0 4 2 3 0 3 2

Frames:

\[3 \]

Let's compare.

42. FIFO TRACE

We track the FIFO queue.

Start:

TEXT

[]

7:

TEXT

[7] F

0:

TEXT

[7, 0] F

1:

TEXT

[7, 0, 1] F

2:

replace 7:

TEXT

[0, 1, 2] F

0:

hit.

3:

replace 0:

TEXT

[1, 2, 3] F

0:

replace 1:

TEXT

[2, 3, 0] F

4:

replace 2:

TEXT

[3, 0, 4] F

2:

replace 3:

TEXT

[0, 4, 2] F

3:

replace 0:

TEXT

[4, 2, 3] F

0:

replace 4:

TEXT

[2, 3, 0] F

3:

hit.

2:

hit.

Total faults:

$\boxed{10}$

43. LRU TRACE

Reference:

TEXT

7 0 1 2 0 3 0 4 2 3 0 3 2

Use recency order:

TEXT

o l d e s t ! ' n e w e s t

7:

TEXT

7 F

0:

TEXT

7 0 F

1:

TEXT

7 0 1 F

2:

remove 7:

TEXT

0 1 2 F

0 hit:

TEXT

1 2 0

3 fault:

remove 1:

TEXT

2 0 3

0 hit:

TEXT

2 3 0

4 fault:

remove 2:

TEXT

3 0 4

2 fault:

remove 3:

TEXT

0 4 2

3 fault:

remove 0:

TEXT

4 2 3

0 fault:

remove 4:

TEXT

2 3 0

3 hit:

TEXT

2 0 3

2 hit:

TEXT

0 3 2

Total faults:

$\boxed{9}$

44. OPT TRACE — STRATEGY

For OPT, don't maintain recency.

At each fault when full:

Look right.

Example at the first replacement:

Frames after:

```
TEXT
7, 0, 1
```

Reference 2 arrives.

Future:

```
TEXT
0 3 0 4 2 3 0 3 2
```

Next use:

```
TEXT
7 !'   n e v e r
0 !'   s o o n
1 !'   n e v e r
```

Either 7 or 1 may be replaced.

Continue this approach.

OPT must produce:

no more page faults than any other replacement algorithm for the same reference/frame configuration.

45. HIT COUNT

If total references:

$\lfloor N \rfloor$

and page faults:

$\lfloor F \rfloor$

assuming every reference is either hit or fault:

$\lfloor \boxed{\text{Hits} = N - F} \rfloor$

Example:

13 references.

Faults = 9.

Hits:

$\lfloor 13 - 9 = \boxed{4} \rfloor$

46. HIT RATIO

$\lfloor \boxed{\text{HitRatio} = \frac{\text{Hits}}{\text{TotalReferences}}} \rfloor$

Page fault ratio:

$$\boxed{\text{FaultRatio} = \frac{\text{Faults}}{\text{TotalReferences}}}$$

And:

$$\text{HitRatio} + \text{FaultRatio} = 1$$

PART F — SECOND CHANCE / CLOCK +P

The official syllabus says virtual memory, so FIFO/LRU/OPT are the central algorithms. But Second Chance is a useful GATE-relevant extension.

47. PROBLEM WITH FIFO

FIFO may remove a page that is:

heavily/recently used

just because it arrived long ago.

Can we improve FIFO cheaply?

Yes.

Use a:

Reference Bit

48. SECOND CHANCE ALGORITHM

Pages remain in FIFO-like circular order.

Each has reference bit:

TEXT

$R = 0 \text{ or } 1$

When page is accessed:

TEXT

$R = 1$

When choosing victim:

If oldest page has $R=0$

Replace it.

If $R=1$

Give it a second chance:

TEXT

```
R = 0
move past it
```

continue searching.

49. MEMORY TRICK

Referenced recently? Give one more chance.

Hence:

Second Chance

50. CLOCK ALGORITHM

Second Chance is often implemented using a circular list and pointer resembling a clock hand.

TEXT

```
P 1  !'   P 2  !'   P 3  !'   P 4
! '                                ! ''
% % % % % % % % % % % % % % % %
```

Clock hand scans reference bits.

This is often called:

Clock Algorithm

51. SECOND CHANCE VS LRU

Second Chance approximates LRU but with much less overhead.

It doesn't maintain exact access order.

It uses:

one reference bit

to estimate recency.

PART G — IMPLEMENTING LRU

52. WHY EXACT LRU IS EXPENSIVE

To know exact least-recently-used page, OS would need to track every memory reference.

Possible conceptual methods:

- timestamps/counters
- stack/list updates

This can be expensive.

Real systems therefore often use approximations.

53. COUNTER IMPLEMENTATION

Each page could store:

time of last access.

Victim:

TEXT

minimum timestamp

But updating timestamps on every reference is costly.

54. STACK IMPLEMENTATION

Whenever page accessed:

move it to top of stack.

Bottom:

LRU victim.

Again, frequent list updates cost overhead.

55. APPROXIMATE LRU

Reference bits can approximate usage.

Algorithms include:

- Second Chance
- Clock
- aging/reference-bit schemes

For GATE, know the core idea rather than every OS-specific implementation.

PART H — FRAME COUNT EFFECT

56. WHAT HAPPENS WHEN FRAMES INCREASE?

For OPT and LRU:

$\boxed{\text{Faults never increase}}$

For FIFO:

$\boxed{\text{Faults may increase}}$

due to Belady's anomaly.

57. MINIMUM POSSIBLE FAULTS

Suppose reference string contains:

$\boxed{D}$

distinct pages and frames are at least:

$\boxed{D}$

starting empty.

Then every distinct page faults once, after which all references hit.

So:

$\boxed{\text{Minimum compulsory faults} = D}$

under pure initial empty loading.

58. EXAMPLE

Reference:

TEXT

1 2 3 1 2 3 1 2

Distinct pages:

$\boxed{3}$

If frames "e 3 :

Faults:

$\boxed{3}$

No replacement needed.

59. ONE FRAME

With only one frame, every time reference changes to a different page:

fault.

Example:

TEXT

1 2 1 2 1

Faults:

\[5 \]

because every reference differs from current resident page.

60. CONSECUTIVE SAME PAGE

With one frame:

TEXT

1 1 1 2 2

Faults:

TEXT

1	!	f	a	u	l	t
1	!	h	i	t		
1	!	h	i	t		
2	!	f	a	u	l	t
2	!	h	i	t		

Total:

\[\boxed{2} \]

PART I — GATE NUMERICAL STRATEGY

61. DO NOT USE SAME METHOD FOR ALL THREE

FIFO

Track:

arrival order.

LRU

Track:

recent-use order.

OPT

Track:

future next use.

This one distinction prevents most mistakes.

62. FIFO SHORTCUT TABLE

Maintain:

TEXT

Oldest !' Newest

On fault:

TEXT

remove oldest
append new

On hit:

TEXT

DO NOTHING to order

63. LRU SHORTCUT TABLE

Maintain:

TEXT

Least Recent !' Most Recent

On hit:

TEXT

move referenced page to newest

On fault:

TEXT

remove least recent
append new

64. OPT SHORTCUT

At every replacement:

Write next-use positions.

Example:

Current:

TEXT

2 3 5

Future:

TEXT

1 5 4 2 ...

Next use:

TEXT

2 !' l a t e r
3 !' n e v e r
5 !' s o o n

Victim:

\[\boxed{3} \]

65. FULL NUMERICAL TEMPLATE

Given:

TEXT

Reference String = ...
Frames = k
Algorithm = ...

Create:

Ref	Frames	Hit/Fault	Victim
...	...	...	...

Then count only:

TEXT

F

at end.

This avoids recount errors.

66. INITIAL EMPTY FRAMES

Unless stated otherwise, GATE questions commonly assume:

Frames initially empty.

But always read the question.

If initial contents are supplied, use them.

67. PAGE REPLACEMENT ONLY WHEN FULL

If page is absent but a free frame exists:

Put it there.

Do not evict another page unnecessarily.

This is a common beginner mistake.

68. DIRTY PAGE AND REPLACEMENT COST

Two replacement algorithms might have same page fault count but different write-back costs.

If victim is dirty:

extra disk write.

Some advanced algorithms consider both:

- recency
- dirty/reference bits

But basic FIFO/LRU/OPT page-fault counting typically ignores write-back cost unless specified.

69. OPT AS LOWER BOUND

For a fixed reference string and frame count:

$$\boxed{\text{Faults}_{\text{OPT}}} \leq \text{Faults}_{\text{any realizable algorithm}}$$

So if your calculated LRU has fewer faults than OPT:

You made a mistake.

Excellent checking shortcut.

70. FIFO CAN SOMETIMES BEAT LRU ON A PARTICULAR STRING

LRU is not guaranteed to have fewer faults than FIFO for every individual reference string.

What is guaranteed?

- OPT is minimum.
- LRU has stack property.
- FIFO can exhibit Belady anomaly.

Do not memorize:

LRU always beats FIFO.

That is too strong.

71. OPT VS LRU INTUITION

OPT:

Perfect knowledge of future.

LRU:

Uses recent past as prediction of future.

Why reasonable?

Because of:

Temporal locality.

72. LRU AND LOCALITY

If page was used recently:

It may be used again soon.

So LRU tries to keep recent pages.

This makes it a practical approximation to OPT conceptually.

73. FIFO DOES NOT USE LOCALITY

FIFO only cares when page entered memory.

Therefore it may replace:

a heavily used old page.

This contributes to its weaker behavior.

74. BELADY QUESTION PATTERN

GATE may ask:

Which algorithms can show Belady's anomaly?

Standard answer:

$\boxed{\text{FIFO}}$

Among FIFO/LRU/OPT.

LRU and OPT cannot due to stack property.

75. STACK PROPERTY QUESTION

If algorithm is stack-based:

Increasing frames:

$n \rightarrow n+1$

cannot increase page faults.

Examples:

$\boxed{\text{LRU, OPT}}$

76. PAGE-FAULTCURVE

Conceptually:

For LRU/OPT:

TEXT

F r a m e s !'
F a u l t s ! " o r s t a y s a m e

For FIFO:

TEXT

F r a m e s !'
F a u l t s m a y s o m e t i m e s !'

77. GATE-STYLE MINI EXAMPLE

Reference:

TEXT

1 2 3 1 4

Frames = 3.

FIFO

1 F 2 F 3 F 1 H 4 F !' oldest is 1

Faults:

$\boxed{4}$

Victim:

\[1 \]

Even though 1 was just used.

LRU

At reference 1:

1 becomes most recent.

When 4 arrives, least recent is:

\[2 \]

So victim:

\[\boxed{2} \]

This perfectly illustrates FIFO vs LRU.

78. GATE-STYLE MINI OPT EXAMPLE

Frames:

TEXT

1 2 3

Future:

TEXT

2 4 1 5 3

Page 4 faults.

Next uses:

TEXT

1	!	p	o	s	i	t	i	o	n	3
2	!	p	o	s	i	t	i	o	n	1
3	!	p	o	s	i	t	i	o	n	5

Farthest:

TEXT

3

Replace:

\[\boxed{3} \]

79. PAGE FAULT COUNT CHECK

For initially empty frames:

Fault count must be at least:

number of distinct pages that actually need loading before any reuse can happen, subject to capacity.

More precisely, each first reference to a distinct page faults in pure demand paging.

If your count is less than number of distinct pages:

definitely wrong.

80. REFERENCE STRING LENGTH

Suppose:

$[N=20]$

references.

Then:

$[0 \leq \text{Hits} \leq 20]$

and:

$[\text{Faults} + \text{Hits} = 20]$

Use this to verify counting.

PART J — PAGE REPLACEMENT VS THRASHING

81. WHAT IF PROCESS GETS TOO FEW FRAMES?

It repeatedly replaces pages that it soon needs again.

Pattern:

TEXT

```
N e e d   A !'   e v i c t   B
N e e d   B !'   e v i c t   A
N e e d   A !'   e v i c t   B
...
```

Page fault after page fault.

This may lead to:

Thrashing

82. WHY REPLACEMENT ALGORITHM ALONE MAY NOT SOLVE THRASHING

Even perfect replacement cannot help much if:

process's active working set is larger than available frames.

The real problem is:

insufficient physical frames.

This leads to our next chapter.

83. HOW GATE CAN ASK PAGE REPLACEMENT

Common patterns:

Pattern 1

Count FIFO faults.

Pattern 2

Count LRU faults.

Pattern 3

Count OPT faults.

Pattern 4

Determine victim page.

Pattern 5

Belady anomaly.

Pattern 6

Identify stack algorithm.

Pattern 7

Compare hit/fault ratios.

Pattern 8

Frame-count effect.

Pattern 9

Reference-bit / Second Chance concept.

& pGATE TRAPS — PAGE REPLACEMENT

Trap 1

FIFO updates order on every hit.

'L

Hit does not change arrival order.

Trap 2

LRU replaces oldest loaded page.

'L

LRU replaces least recently accessed.

Trap 3

OPT replaces least recently used page.

'L

OPT looks into the future.

Trap 4

More frames always reduce FIFO faults.

'L

Belady's anomaly.

Trap 5

LRU can exhibit Belady's anomaly.

'L

LRU is a stack algorithm.

Trap 6

OPT can exhibit Belady's anomaly.

'L

Trap 7

Every page fault requires replacement.

'L

Only if no free frame exists.

Trap 8

LRU always gives fewer faults than FIFO.

'L Not for every individual reference string.

Trap 9

OPT is practical because OS knows future references.

'L

Future is not generally known.

Trap 10

Second Chance is exact LRU.

'L

It is an approximation using reference information.

' p WRITE THIS IN YOUR NOTES

Page Replacement

Occurs when:

TEXT

Page fault

+

No free frame

Goal:

Minimize page faults.

FIFO

Replace oldest loaded page.

TEXT

H i t ! ' o r d e r u n c h a n g e d
F a u l t ! ' r e m o v e o l d e s t

Problem:

Belady's anomaly possible.

OPT

Replace page used farthest in future.

If page never appears again:

excellent victim.

Property:

$\boxed{\text{Minimum possible faults}}$

But:

not implementable perfectly in practice.

LRU

Replace page not used for longest time in past.

On hit:

update recency.

Based on:

Temporal locality.

MASTER TABLE

FIFO	LRU	OPT
Oldest arrival	Oldest usage	Farthest future use
Looks at entry time	Looks backward	Looks forward
Belady possible	No Belady	No Belady
Not stack	Stack	Stack

BELADY'S ANOMALY

More frames != more faults.

Possible with:

$\boxed{\text{FIFO}}$

Not with:

$\boxed{\text{LRU, OPT}}$

SECOND CHANCE

FIFO + reference bit.

TEXT

```
R = 0 !' replace
R = 1 !' clear bit, give second chance
```

HIT / FAULT FORMULAS

$$\boxed{\text{Hits} = \text{TotalReferences} - \text{Faults}}$$

$$\boxed{\text{HitRatio} = \frac{\text{Hits}}{\text{TotalReferences}}}$$

$$\boxed{\text{FaultRatio} = \frac{\text{Faults}}{\text{TotalReferences}}}$$

MASTER MEMORY TRICK

A P F

- FIFO !' Arrival
- LRU !' Past
- OPT !' Future

Or simply:

FIFO remembers entry. LRU remembers history. OPT knows destiny.

EXAM SHORTCUTS

FIFO

Queue:

TEXT

```
O l d e s t !' N e w e s t
```

Hit:

Don't move.

LRU

Queue:

TEXT

L e a s t r e c e n t ! ' M o s t r e c e n t

Hit:

Move to newest.

OPT

At fault:

Look right and replace page whose next reference is farthest.

30-SECOND REVISION

Replacement needed when?

!' Fault + no free frame.

FIFO victim?

!' Oldest loaded.

LRU victim?

!' Least recently used.

OPT victim?

!' Farthest next use.

Who knows future?

!' OPT.

Who may have Belady anomaly?

!' FIFO.

Stack algorithms?

!' LRU and OPT.

Second Chance?

!' FIFO + reference bit.

LRU based on?

!' Temporal locality.

ACTIVE RECALL

1. What are the two conditions needed before page replacement occurs?
2. What exactly does FIFO track?
3. Does a FIFO hit change replacement order?
4. What does LRU track?

5. What does OPT inspect?
6. Why can't a real OS perfectly implement OPT?
7. What is Belady's anomaly?
8. Which of FIFO/LRU/OPT can exhibit it?
9. What is a stack algorithm?
10. Why does LRU approximate OPT reasonably well?
11. What does Second Chance add to FIFO?
12. If total references = 20 and faults = 7, how many hits?

[$20 - 7 = \boxed{13}$]

MASTER CONNECTION

TEXT

Demand Paging

!"

Page Fault

!"

No Free Frame

!"

Page Replacement

!"

FIFO / LRU / OPT

!"

Too Few Frames?

!"

Repeated Faults

!"

THRASHING

NEXT — CHAPTER 16 Ø=Ý%Ø=Ý%

Next we study:

THRASHING + WORKING SET + PAGE-FAULT FREQUENCY

We will cover:

- what thrashing actually means
- why CPU utilization can decrease when multiprogramming increases
- working-set model
- locality and working-set window
- page-fault frequency
- local vs global replacement connection
- frame allocation
- minimum frames
- prepaging idea
- GATE conceptual traps

After that we enter the final major syllabus area:

FILE SYSTEMS

USER

NEXT

ASSISTANT

CHAPTER 16 — THRASHING + WORKING SET + PAGE-FAULT FREQUENCY

This chapter completes the core **Virtual Memory** portion of the GATE syllabus.

The central chain is:

TEXT

```
Too few frames
    !"
Frequent page faults
    !"
More disk I/O
    !"
Processes spend time waiting
    !"
CPU utilization drops
    !"
THRASHING
```

1. WHAT IS THRASHING?

Thrashing is a condition in which:

A process or system spends most of its time servicing page faults rather than doing useful execution.

In simple language:

Pages keep entering and leaving RAM repeatedly.

Example:

TEXT

```
Need Page A
!"
A loaded, B removed

Need Page B
!"
B loaded, A removed

Need Page A again
!"
```

A loaded, B removed
...

This repeated swapping of pages causes extremely poor performance.

2. WHY DOES THRASHING OCCUR?

The major reason is:

A process has too few frames to hold the pages it is actively using.

Suppose a program actively needs:

TEXT

Pages = {1, 2, 3, 4, 5}

but receives only:

TEXT

3 frames

Then:

TEXT

Use 1,2,3

Need 4 !' remove something

Need 5 !' remove something

Need removed page again !' page fault

Faults happen repeatedly.

3. SIMPLE INTUITION

Imagine studying using 5 textbooks.

Your desk can hold only 2 books.

You repeatedly do:

TEXT

Bring Book A

Remove Book B

Need B

Remove C

Need C

Remove A

You spend more time moving books than studying.

That is exactly the idea of thrashing.

4. THRASHING AND PAGE FAULT RATE

Normally:

TEXT

Page-fault rate = low

During thrashing:

TEXT

Page-fault rate = extremely high

Therefore:

High page-fault rate is the key symptom of thrashing.

5. WHY IS THRASHING SO EXPENSIVE?

A normal memory access may take:

TEXT

nanoseconds

A page fault can require storage I/O:

TEXT

microseconds / milliseconds

So:

TEXT

Memory access !' fast
Page fault !' enormously slower

A high page-fault rate destroys performance.

6. CONNECTION WITH CPU UTILIZATION

This is a famous GATE concept.

Suppose too many processes are placed in memory.

Each gets fewer frames.

Then:

TEXT

Frames per process !"
!"

```
P a g e   f a u l t s   !'
!'"
Processes wait for disk
!'"
Ready queue may shrink
!'"
CPU may become idle
!'"
C P U   u t i l i z a t i o n   !"
```

7. THE DANGEROUS OS RESPONSE

Suppose CPU utilization drops.

OS may incorrectly think:

"CPU is idle because there aren't enough processes."

So it increases:

Degree of Multiprogramming

Meaning:

Load even more processes.

But now:

```
TEXT
More processes
!'"
Even fewer frames per process
!'"
Even more page faults
!'"
Even lower CPU utilization
```

This creates a vicious cycle.

8. THRASHING CYCLE

Important diagram:

```
TEXT
M u l t i p r o g r a m m i n g   !'
!'"
F r a m e s / p r o c e s s   !'"
!'"
P a g e   f a u l t s   !'
!'"
D i s k   I / O   !'
!'"
C P U   u t i l i z a t i o n   !"
```

!"
OS may add more processes
!"
Thrashing becomes worse

9. DEGREE OF MULTIPROGRAMMING

Definition:

Number of processes kept in memory / actively participating in execution.

Initially, increasing multiprogramming can improve CPU utilization.

Why?

While one process waits for I/O:

another can execute.

But beyond a certain point:

insufficient frames cause thrashing.

10. CPU UTILIZATION CURVE

Conceptually:



Initially:

TEXT

More processes !' CPU utilization !'

After threshold:

TEXT

More processes !' CPU utilization !"

because of thrashing.

& pGATE TRAP

Do not assume:

Increasing multiprogramming always increases CPU utilization.

'LFalse.

After memory pressure becomes too high:

Thrashing may reduce CPU utilization.

11. HOW DO WE PREVENT THRASHING?

Two important approaches:

1. Working Set Model
2. Page Fault Frequency (PFF)

Both try to answer:

Does each process have enough frames?

PART A — WORKING SET MODEL Ø=Ý%Ø=Ý%

12. WHAT IS A WORKING SET?

A process doesn't normally use all of its pages equally at every moment.

During a period it repeatedly uses a smaller group.

That group is its:

Working Set

Definition:

The working set of a process is the set of pages referenced during a recent window of execution.

13. EXAMPLE

Reference sequence:

TEXT
1 2 3 2 1 4 1 2 3 ...

The process may currently be actively using:

TEXT

{1, 2, 3, 4}

These pages constitute its working set for an appropriate recent window.

14. WHY DOES WORKING SET EXIST?

Because of:

Locality of Reference

Programs execute in phases.

Example:

TEXT

```
P h a s e   1   !'   i n i t i a l i z a t i o n   c o d e
P h a s e   2   !'   l o o p   o v e r   a r r a y
P h a s e   3   !'   s o r t   d a t a
P h a s e   4   !'   w r i t e   o u t p u t
```

Each phase uses a particular subset of pages.

So:

TEXT

Locality
!"
Working Set

15. WORKING SET WINDOW "

We define a window:

$[\Delta]$

representing a recent number of memory references or time interval.

Working set:

$\boxed{WSS_i(\Delta)}$

for process (P_i) .

It contains all distinct pages referenced in the last (Δ) references/time units.

16. SIMPLE WORKING SET EXAMPLE

Reference sequence:

TEXT

... 1 2 3 2 4

Suppose:

$\Delta=4$

Last 4 references:

TEXT

2 3 2 4

Distinct pages:

TEXT

{2, 3, 4}

So working-set size:

$\boxed{3}$

17. ANOTHER EXAMPLE

Reference string:

TEXT

1 2 3 4 2 1 5 2

At the final reference, let:

$\Delta=5$

Last five references:

TEXT

4, 2, 1, 5, 2

Distinct pages:

TEXT

{1, 2, 4, 5}

Therefore:

$\boxed{WSS=4}$

18. WORKING SET SIZE

Working Set Size:

$\boxed{WSS_i = \text{number of distinct pages in recent window}}$

A process ideally needs roughly enough frames to hold its active working set.

If:

TEXT

F r a m e s " e W o r k i n g S e t

page faults should generally be manageable.

If:

TEXT

Frames < Working Set

process may thrash.

19. TOTAL DEMAND FOR FRAMES

Suppose there are processes:

TEXT

P₁, P₂, ..., P_n

with working-set sizes:

$\{ WSS_1, WSS_2, \dots, WSS_n \}$

Total frame demand:

$\boxed{D = \sum_i WSS_i}$

20. WORKING SET DECISION

Suppose total physical frames available:

M

If:

$D \leq M$

there are enough frames to support all working sets.

If:

$D > M$

not enough frames exist.

Then OS may need to:

suspend/swap out one or more processes

to reduce degree of multiprogramming.

21. EXAMPLE

Available frames:

$\lfloor 20 \rfloor$

Processes:

TEXT

P1 WSS = 8

P2 WSS = 7

P3 WSS = 9

Total:

$\lfloor D=8+7+9=24 \rfloor$

Since:

$\lfloor 24 > 20 \rfloor$

not enough memory exists for all active working sets.

Potential solution:

suspend one process.

22. WHY SUSPENDING A PROCESS HELPS

Suppose P3 is suspended.

Its 9 frames can be redistributed.

Now:

$\lfloor 8+7=15 \rfloor$

frames needed.

Available:

$\lfloor 20 \rfloor$

Remaining processes can have enough frames.

Result:

TEXT

P a g e f a u l t s !"
C P U u t i l i z a t i o n m a y !'

Counterintuitive but important:

Reducing multiprogramming can improve performance during thrashing.

23. WORKING SET AND LOCALITY

Working set changes over time.

Example:

TEXT

Time 1:

{1, 2, 3}

Time 2:

{2, 3, 4}

Time 3:

{7, 8, 9, 10}

This reflects changes in program locality.

So memory allocation should ideally adapt.

24. EFFECT OF "

Choice of working-set window matters.

"too small

May not capture full current locality.

Working set underestimated.

"too large

May include several old localities.

Working set overestimated.

So:

"should approximate the locality window.

MEMORY TRICK

Small "forgets too much.

Large "remembers too much.

PART B — PAGE FAULT FREQUENCY $\emptyset = \dot{Y}\%$

25. PAGE FAULT FREQUENCY — PFF

Instead of explicitly calculating working sets, we can monitor:

Page-fault rate

This is the Page-Fault Frequency approach.

26. BASIC IDEA

Define:

TEXT

Upper Page-Fault Threshold
Lower Page-Fault Threshold

Then:

If fault rate too high

Process likely has too few frames.

Give it more frames.

If fault rate too low

Process may have more frames than necessary.

Take some frames away.

27. PFF DIAGRAM

TEXT

Page Fault
Frequency

^
| T o o H i g h ! ' G i v e m o r e f r a m e s
---|----- Upper Threshold
|
| D e s i r e d r a n g e
---|----- Lower Threshold
| T o o L o w ! ' R e m o v e s o m e f r a m e s
+-----> Time

28. MEMORY RULE

TEXT

F a u l t r a t e H I G H ! ' F r a m e s L O W ! ' G i v e m o r e
F a u l t r a t e L O W ! ' F r a m e s p r o b a b l y e x t r a ! ' T a k e s o m e

Easy memory:

High faults? Feed frames.

29. WHAT IF NO FREE FRAMES EXIST?

Suppose:

TEXT

P1 fault rate too high

but RAM has no free frames.

Then OS may need to:

suspend another process

and release its frames.

Again:

Reduce multiprogramming.

30. WORKING SET VS PFF

Working Set	Page-Fault Frequency
Tracks recently used pages	Tracks fault rate
Estimates frame requirement directly	Adjusts frames from observed behavior
Based on locality window "	Based on upper/lower thresholds
More explicit locality model	Simpler dynamic feedback

Both aim to:

prevent thrashing.

PART C — FRAME ALLOCATION

31. FRAME ALLOCATION

Physical memory contains a limited number of frames.

OS must decide:

How many frames should each process receive?

Approaches include:

- equal allocation
- proportional allocation
- priority-based allocation

32. EQUAL ALLOCATION

Suppose:

$\lfloor m \rfloor$

available frames and:

$\lfloor n \rfloor$

processes.

Each gets approximately:

$\lfloor \boxed{\frac{m}{n}} \rfloor$

frames.

Example:

60 frames, 3 processes:

$\lfloor 20 \rfloor$

each.

Simple but ignores process sizes.

33. PROPORTIONAL ALLOCATION

Larger processes receive more frames.

Suppose process sizes:

$\lfloor s_1, s_2, \dots, s_n \rfloor$

Total:

$\lfloor S = \sum s_i \rfloor$

For process $\lfloor (P_i) \rfloor$:

$\lfloor \boxed{a_i = \frac{s_i}{S} \times m} \rfloor$

where $\lfloor (m) \rfloor$ is total frames available.

34. EXAMPLE

Two processes:

TEXT

P1 size = 10 pages

P2 size = 30 pages

Available frames:

$\lfloor 20 \rfloor$

Total size:

$\lfloor 40 \rfloor$

P1:

$\lfloor \frac{10}{40} \rfloor \times 20 = 5$

P2:

$\lfloor \frac{30}{40} \rfloor \times 20 = 15$

So:

TEXT

```
P 1 !' 5 f r a m e s
P 2 !' 1 5 f r a m e s
```

35. PRIORITY ALLOCATION

Higher-priority processes may receive:

more frames

or may be allowed to replace lower-priority pages depending on system policy.

For GATE, know the idea.

PART D — MINIMUM NUMBER OF FRAMES

36. WHY DOES A PROCESS NEED A MINIMUM NUMBER OF FRAMES?

Some machine instructions may access multiple pages.

Example conceptually:

TEXT

```
Instruction itself
+
source operand
+
destination operand
```

If all lie on separate pages, the process may require several frames just to execute the instruction correctly without pathological repeated faults.

Therefore:

Minimum frames can depend on machine architecture/instruction format.

37. SIMPLE EXAMPLE

An instruction spans two pages and accesses data on two additional pages.

Potentially:

TEXT

2 instruction pages
+
2 data pages
=
4 pages

So process may need at least enough frames to support such execution.

The exact answer depends on the architecture described.

& pGATE TRAP

There is no universal:

“Every process needs exactly X minimum frames.”

It depends on instruction/addressing behavior.

PART E — LOCAL VS GLOBAL REPLACEMENT

38. LOCAL REPLACEMENT

Process P_i can replace only:

pages currently in P_i 's own frames.

Example:

TEXT

P_1 frames: 1, 2, 3
 P_2 frames: 4, 5, 6

If P_1 faults:

victim comes from 1,2,3.

39. GLOBAL REPLACEMENT

Process can potentially select a victim from:

any eligible frame in the system.

P_1 might take a frame from P_2 .

This makes frame allocation dynamic.

40. THRASHING AND GLOBAL REPLACEMENT

Global replacement can create cascading effects.

Example:

TEXT

```
P1 starts faulting
!"
takes frames from P2
!"
P2 now has fewer frames
!"
P2 starts faulting
!"
takes frames elsewhere
```

Thrashing can spread.

41. LOCAL REPLACEMENT ADVANTAGE

One process's fault behavior is more isolated.

P1 cannot steal P2's frames.

This can make performance more predictable.

42. GLOBAL REPLACEMENT ADVANTAGE

Memory can be used more flexibly.

Processes with immediate demand can receive more frames.

But:

less isolation.

PART F — WHY CPU UTILIZATION FALLS

43. STEP-BY-STEP

Suppose P1 faults.

It waits for disk:

TEXT

```
P 1  !'   W a i t i n g
```

CPU runs P2.

P2 faults.

TEXT

P 2 !' W a i t i n g

CPU runs P3.

P3 faults.

Eventually:

TEXT

P1 waiting on disk
P2 waiting on disk
P3 waiting on disk

Ready queue may become empty.

CPU:

TEXT

IDLE

even though system is extremely busy doing paging I/O.

This is thrashing.

44. DISK UTILIZATION DURING THRASHING

During thrashing:

TEXT

P a g i n g d i s k a c t i v i t y !'
C P U u s e f u l w o r k !"

So you may observe:

- high paging I/O
- low CPU utilization
- low throughput
- poor response time

45. THROUGHPUT EFFECT

Recall:

$\text{Throughput} = \frac{\text{CompletedProcesses}}{\text{Time}}$

Thrashing causes processes to progress very slowly.

Therefore:

$\boxed{\text{Throughput decreases}}$

46. RESPONSE TIME EFFECT

Page faults repeatedly suspend processes.

Therefore:

$\boxed{\text{Response Time increases}}$

47. PAGE-FAULTRATE EFFECT

Obviously:

$\boxed{\text{Page Fault Rate increases}}$

So thrashing generally means:

TEXT

```
F a u l t   r a t e   !'  
D i s k   I / O   !'  
C P U   u t i l i z a t i o n   !"  
T h r o u g h p u t   !"  
R e s p o n s e   t i m e   !'
```

This is a useful GATE summary.

PART G — WORKING SET NUMERICAL

48. EXAMPLE

Reference string:

TEXT

1 2 3 2 4 1 2 5 2 1

At the final reference, let:

$\Delta=6$

Last six references:

TEXT

4 1 2 5 2 1

Distinct pages:

TEXT

{1, 2, 4, 5}

Therefore:

$\boxed{WSS=4}$

49. ANOTHER EXAMPLE

Reference:

TEXT

1 2 1 3 4 2 3 5

At final point:

$\Delta = 4$

Last 4:

TEXT

4 2 3 5

All distinct.

So:

$\boxed{WSS=4}$

50. TOTAL WORKING SET EXAMPLE

Suppose:

TEXT

P1 WSS = 4

P2 WSS = 6

P3 WSS = 3

Then:

$D = 4 + 6 + 3$

$\boxed{13}$

If RAM provides:

16

frames:

enough under this working-set estimate.

If RAM provides:

10

frames:

not enough.

One process may need suspension.

PART H — PAGE-FAULT FREQUENCY EXAMPLE

51. EXAMPLE

Suppose target range:

TEXT

Lower threshold = 2%
Upper threshold = 8%

P1:

TEXT

Fault rate = 12%

What should OS consider?

$[12\% > 8\%]$

So:

Give P1 more frames.

52. ANOTHER EXAMPLE

P2:

TEXT

Fault rate = 0.5%

Since:

$[0.5\% < 2\%]$

P2 may have excess frames.

OS may reclaim some.

53. WHAT IF P1 HAS HIGH FAULT RATE EVEN AFTER MORE FRAMES?

Possible reasons:

- locality changed
- working set expanded
- insufficient total memory
- process needs substantially more frames

If system cannot satisfy demand:

suspend a process / reduce multiprogramming.

PART I — PREPAGING

54. PREPAGING

Demand paging loads pages:

only after they are referenced.

This may cause several initial page faults.

Prepaging tries to load:

multiple pages expected to be needed soon.

Example:

If Page 10 faults, system might also bring:

TEXT

Page 11

Page 12

because of spatial locality.

55. ADVANTAGE

Can reduce future page faults if prediction is good.

56. DISADVANTAGE

If extra pages are never used:

unnecessary I/O + memory usage.

So prepaging is a tradeoff.

PART J — PAGE SIZE AND THRASHING

57. LARGE PAGES

Each page brings more data.

Potential benefit:

more neighboring data loaded due to spatial locality.

But:

- more unused data may be loaded
- internal fragmentation increases
- fewer distinct frames fit in RAM

So page size has complex effects.

58. SMALL PAGES

Better granularity.

But:

- page table larger
- potentially more page-management/TLB pressure
- more faults may be needed to cover same locality

Again, tradeoff.

PART K — GATE CONCEPTUAL QUESTION

59. QUESTION

System exhibits:

- very high page-fault rate
- heavy paging disk activity
- low CPU utilization

Most likely condition?

$\boxed{\text{Thrashing}}$

60. QUESTION

What should the OS generally do during thrashing?

A. Increase multiprogramming B. Reduce multiprogramming C. Reduce frames per process D. Increase page faults

Correct:

$\boxed{B}$

61. QUESTION

A process has working-set size 8 but receives 4 frames.

Likely effect?

Frequent replacement and page faults.

Potential:

$\boxed{\text{Thrashing}}$

62. QUESTION

Under Page Fault Frequency:

High page-fault rate means:

$\boxed{\text{Allocate more frames}}$

if possible.

63. QUESTION

Low page-fault rate means:

Process may have more frames than necessary.

Some can potentially be reclaimed.

64. QUESTION

Which concept explains why working-set model works?

$\boxed{\text{Locality of Reference}}$

65. QUESTION

Does every process have a fixed working set forever?

'LNo.

Working sets change as program moves between localities.

PART L — THRASHING VS DEADLOCK

66. DO NOT CONFUSE THEM

Thrashing

Processes are still progressing, but extremely slowly because of frequent page faults.

Deadlock

Processes cannot progress because of circular/resource waiting.

Comparison

Thrashing	Deadlock
Memory-management problem	Resource/synchronization problem
Frequent paging	Circular waiting
Progress very slow	No progress for deadlocked set
Page-fault rate high	Resource dependency
Reduce multiprogramming/add frames	Break deadlock conditions/recover

Memory:

Thrashing = working badly.

Deadlock = not working at all.

PART M — THRASHING VS STARVATION

67. STARVATION

One process waits indefinitely because others repeatedly get resources/service.

Thrashing

Processes repeatedly page in/out due to insufficient resident memory.

Different problem.

PART N — FRAME ALLOCATION & PAGE REPLACEMENT CONNECTION

68. IMPORTANT CHAIN

```

TEXT
Frame Allocation
    !"
Too few frames?
    !"
  R e p l a c e m e n t   f r e q u e n c y   !'
    !"
  P a g e   f a u l t s   !'
    !"
Working set doesn't fit
    !"
Thrashing

```

So page replacement and frame allocation cannot be considered completely independently.

69. OPT CANNOT SAVE A PROCESS WITH FAR TOO FEW FRAMES

Even Optimal replacement minimizes faults.

But suppose process repeatedly needs:

TEXT

1 2 3 4 1 2 3 4...

and only has:

TEXT

2 frames

Faults remain frequent.

The fundamental problem is:

resident capacity is too small for the locality.

70. GATE TRAP — BETTER REPLACEMENT "THRASHING CURE

Changing FIFO to LRU may reduce faults.

But if the process's working set does not fit:

thrashing may continue.

The real solution often requires:

- more frames
- fewer active processes

PART O — WORKING SET vs RESIDENT SET

71. RESIDENT SET

Pages currently physically in RAM.

Working Set

Pages recently/currently actively used.

They are not necessarily exactly the same.

Ideal situation:

TEXT

Working Set "⊆" Resident Set

If important working-set pages are missing:

page faults occur.

72. EXAMPLE

Working set:

TEXT

{1, 2, 3, 4}

Resident set:

TEXT

{1, 2, 5, 6}

Missing active pages:

TEXT

3, 4

Likely faults soon.

PART P — HOW GATE CAN ASK THIS CHAPTER

Common patterns:

Pattern 1

Identify thrashing from symptoms.

Pattern 2

Multiprogramming vs CPU utilization curve.

Pattern 3

Working-set calculation from reference string.

Pattern 4

Total working-set demand.

Pattern 5

Decide whether process should receive more frames.

Pattern 6

PFF upper/lower threshold.

Pattern 7

Local vs global replacement.

Pattern 8

What to do when total working-set demand exceeds available frames.

Pattern 9

Distinguish thrashing from deadlock/starvation.

& pGATE TRAPS — THRASHING

Trap 1

High CPU utilization indicates thrashing.

'L

Thrashing typically causes:

[CPU\ utilization \downarrow]

Trap 2

Increasing multiprogramming always solves low CPU utilization.

'L

During thrashing it makes things worse.

Trap 3

High page-fault rate means process has too many frames.

Usually opposite:

likely too few frames.

Trap 4

Working set means every page belonging to the process.

'L

Only pages used in a recent locality/window.

Trap 5

Working set is constant.

'L

It changes with program behavior.

Trap 6

If total working-set demand exceeds RAM, add more processes.

'L

Usually reduce multiprogramming.

Trap 7

Global replacement completely isolates processes.

'L

Processes can affect each other's frame allocations.

Trap 8

Local replacement lets P1 replace P2's pages.

'L

Local = own frames only.

Trap 9

PFF high !' remove frames.

'L

High faults !' give more frames.

Trap 10

Thrashing and deadlock are identical.

'L

' p WRITE THIS IN YOUR NOTES

Thrashing

System spends more time paging than doing useful execution.

Symptoms:

TEXT

```
P a g e   f a u l t s   !'
P a g i n g   I / O   !'
C P U   u t i l i z a t i o n   !"
T h r o u g h p u t   !"
```


R e s p o n s e t i m e !'

Cause:

Too few frames for active working sets.

Multiprogramming

TEXT

Initially:

M u l t i p r o g r a m m i n g !' !' C P U u t i l i z a t i o n !'

Too much:

F r a m e s / p r o c e s s !"

!' P a g e f a u l t s !'

!' T h r a s h i n g

!' C P U u t i l i z a t i o n !"

Working Set

Distinct pages referenced during recent window Δ .

$\lfloor \boxed{WSS_i(\Delta)} \rfloor$

Total demand:

$\lfloor \boxed{D = \sum_i WSS_i} \rfloor$

If:

$\lfloor D > M \rfloor$

available memory is insufficient.

Action:

Reduce multiprogramming / suspend process.

Page Fault Frequency

TEXT

Fault rate HIGH

!' G i v e m o r e f r a m e s

Fault rate LOW

!' R e c l a i m s o m e f r a m e s

Memory:

High faults? Feed frames.

Local vs Global

TEXT

L o c a l ! ' r e p l a c e o w n p a g e s

G l o b a l ! ' v i c t i m m a y b e l o n g t o a n o t h e r p r o c e s s

Proportional Allocation

$$a_i = \frac{s_i}{\sum s_j} \times m$$

MASTER MEMORY MAP

TEXT

Locality

!"

Working Set

!"

Need enough Frames

!"

If Frames insufficient

!"

P a g e F a u l t s !'

!"

P a g i n g I / O !'

!"

C P U U t i l i z a t i o n !"

!"

THRASHING

!"

Reduce Multiprogramming

MASTER SHORTCUT

Remember:

F P C

Frames !" Page faults !" CPU utilization !"

That's thrashing.

And cure:

F !' / P !"

30-SECOND REVISION

Thrashing?

!' Excessive paging.

Main cause?

!' Working set doesn't fit in allocated frames.

Page faults during thrashing?

!' High.

CPU utilization?

!' Low.

Should OS add more processes?

!' No.

Working set?

!' Recently used distinct pages.

Total demand?

$\sum WSS_i$

Demand > frames?

!' suspend/reduce processes.

PFF high?

!' more frames.

PFF low?

!' frames may be reclaimed.

Local replacement?

!' own frames only.

Global?

!' system-wide victims possible.

ACTIVE RECALL

1. Define thrashing.
2. Why does CPU utilization fall during thrashing?
3. Why can increasing multiprogramming make thrashing worse?
4. What is a working set?
5. What does Δ represent?
6. If $P1 \text{ WSS}=5$, $P2 \text{ WSS}=7$, $P3 \text{ WSS}=4$, what is total demand?

7. If only 13 frames exist in Q6, what should the OS consider doing?
8. Under PFF, what does high page-fault rate imply?
9. Difference between local and global replacement?
10. Does LRU alone guarantee elimination of thrashing?
11. Difference between working set and resident set?
12. Why is locality fundamental to virtual memory?

For Q6:

$\lceil 5+7+4 \rceil = \boxed{16}$

Since 16 frames are needed but only 13 exist:

memory demand exceeds availability.

The OS should consider reducing the degree of multiprogramming.

VIRTUAL MEMORY — COMPLETE CONNECTION

You have now completed the main virtual-memory chain:

```

TEXT
Paging
    !"
Page Table
    !"
TLB
    !"
Virtual Memory
    !"
Demand Paging
    !"
Page Fault
    !"
Page Replacement
    !"
FIFO / OPT / LRU
    !"
Too Few Frames
    !"
Thrashing
    !"
Working Set / PFF
  
```

This is a major portion of GATE Operating Systems.

NEXT — CHAPTER 17 Ø=Ý%Ø=Ý%Ø=Ý%

Now we enter the final major topic explicitly listed in your syllabus:

FILE SYSTEMS

Next we will cover:

- what a file system is

- file attributes and operations
- file descriptors
- file access methods
- directory structures
- absolute vs relative paths
- file allocation:
 - contiguous
 - linked
 - indexed
- FAT concept
- inode concept
- direct and indirect blocks
- maximum file-size numericals
- free-space management
- bitmaps
- hard links vs symbolic links
- deletion semantics
- file-system GATE traps and shortcuts.

USER

next

ASSISTANT

CHAPTER 17 — FILE SYSTEMS Ø=Ý%Ø=Ý%Ø=Ý%

This is the final major topic in your official Operating Systems syllabus.

The central idea is:

TEXT

```
User creates data
      ! "
Files
      ! "
Directories organize files
      ! "
File System manages storage
      ! "
Disk blocks store actual contents
```

For GATE, the highest-value parts are:

- file attributes and operations
- directory structures
- access methods
- file allocation methods

- FAT
- inode
- direct/indirect block numericals
- free-space management
- hard link vs symbolic link

1. WHAT IS A FILE?

A file is:

A named collection of related information stored on secondary storage.

Examples:

TEXT

notes.txt
photo.jpg
program.c
database.db

Unlike RAM contents, files are generally:

persistent

They remain after process termination or system restart.

2. WHAT IS A FILE SYSTEM?

A file system is the OS component responsible for:

- creating files
- deleting files
- storing file contents
- locating files
- organizing directories
- managing disk blocks
- permissions/protection
- maintaining metadata
- tracking free disk space

Simple memory:

File system = OS manager for persistent storage.

3. FILE ATTRIBUTES / METADATA

A file usually has information such as:

- name
- identifier
- type
- size
- location
- owner
- permissions
- timestamps
- other status information

This information is called:

Metadata

Important:

Metadata describes the file; it is not necessarily the file's actual user data.

4. FILE OPERATIONS

Common operations:

TEXT

```
Create
Open
Read
Write
Seek/Reposition
Close
Delete
Truncate
```

Memory trick:

C O R W S C D

You do not need to memorize the letters exactly; understand what each operation does.

5. OPEN()

Why is `open()` needed?

Suppose a process wants to repeatedly read a file.

Instead of searching directory structures and permissions from scratch for every read:

TEXT

```
open(file)
! "
OS checks metadata
! "
```

```
creates/uses open-file information
!"
returns handle/file descriptor
```

Then the process uses that handle.

6. FILE DESCRIPTOR

In Unix-like terminology, a file descriptor is:

A small process-visible integer referring to an open file or I/O object.

Example:

```
C
fd = open("data.txt", ...);
```

`fd` might be:

```
TEXT
3
```

Then:

```
TEXT
read(fd, ...)
write(fd, ...)
```

use that open-file reference.

7. FILE DESCRIPTOR VS FILE NAME

File name:

```
TEXT
report.txt
```

is used to locate/open the file.

File descriptor:

```
TEXT
3
```

is used by the process after opening.

Memory:

Name finds it; descriptor uses it.

8. FILE POINTER / OFFSET

An open file often has a current position:

```
TEXT
0 1 2 3 4 5 ...
           !'
current offset
```

After reading bytes, the offset advances.

`seek()` changes this position.

This becomes important for sequential/random access.

PART A — FILE ACCESS METHODS

9. SEQUENTIAL ACCESS

Data is accessed in order:

```
TEXT
Record 1
    !"
Record 2
    !"
Record 3
    !"
Record 4
```

Operations conceptually:

```
TEXT
read next
write next
rewind
```

Suitable for:

- logs
- streams
- tape-like processing

10. DIRECT / RANDOM ACCESS

Any block/record can be accessed directly.

Example:

```
TEXT
Read block 100
```

without reading blocks 1–99 first.

Typical disks support this efficiently.

11. SEQUENTIAL VS DIRECT

Sequential	Direct
Access in order	Jump to arbitrary location
Simple	Flexible
Good for streaming	Good for databases/random reads
next style	block/offset based

PART B — DIRECTORY STRUCTURES

12. WHAT IS A DIRECTORY?

A directory organizes file names and their associated file-system information.

Example:

TEXT

```
/
% % %   h o m e
%       % % %   n o t e s . t x t
%       % % %   p h o t o . j p g
% % %   e t c
```

A directory is not simply "the file contents."

It helps map:

TEXT

```
N a m e ! '   f i l e   m e t a d a t a / o b j e c t
```

13. SINGLE-LEVEL DIRECTORY

All files exist in one directory.

TEXT

```
Directory
% % %   a . t x t
% % %   b . t x t
% % %   c . t x t
```

Problem:

- name conflicts

- poor organization
- not suitable for many users/files

14. TWO-LEVEL DIRECTORY

Separate directory per user.

TEXT

Master

```
% % %   U s e r   A
%       % % %   x . t x t
%       % % %   y . t x t
% % %   U s e r   B
%       % % %   x . t x t
%       % % %   z . t x t
```

Now both users may have:

TEXT

x.txt

without conflict.

15. TREE-STRUCTURED DIRECTORY

Most familiar model:

TEXT

```
/
% % %   h o m e
%       % % %   u s e r 1
%       %       % % %   d o c s
%       %       % % %   p i c s
%       % % %   u s e r 2
% % %   v a r
```

Allows:

- hierarchical organization
- directories inside directories
- absolute and relative paths

16. ABSOLUTE PATH

Absolute path starts from root.

Example:

TEXT

/home/user/docs/a.txt

It identifies location independently of current working directory.

17. RELATIVE PATH

Relative path starts from current directory.

If current directory:

```
TEXT
/home/user
```

then:

```
TEXT
docs/a.txt
```

is a relative path.

Memory:

Absolute starts at root. Relative starts here.

18. ACYCLIC GRAPH DIRECTORY

Allows sharing of files/subdirectories.

Example:

```
TEXT
User A % %
          % !' Shared Project
User B % %
```

But cycles are disallowed.

This permits sharing without creating infinite traversal loops.

19. GENERAL GRAPH DIRECTORY

Cycles may exist.

Example:

```
TEXT
A !' B !' C !' A
```

Then file-system traversal/deletion becomes more complicated.

May require:

- cycle detection
- garbage collection
- reference counts carefully handled

PART C — FILE ALLOCATION METHODS Ø=Ý%Ø=Ý%Ø=Ý%

The file has blocks:

```
TEXT
B0 B1 B2 B3 B4
```

Question:

Where should these blocks be placed on disk?

Three classic methods:

1. Contiguous Allocation
 2. Linked Allocation
 3. Indexed Allocation
-

20. CONTIGUOUS ALLOCATION

All file blocks occupy consecutive disk blocks.

Example:

```
TEXT
File A:
Block 20
Block 21
Block 22
Block 23
Block 24
```

Directory needs:

```
TEXT
Start block = 20
Length = 5
```

21. CONTIGUOUS ADDRESS CALCULATION

If:

```
TEXT
Start = S
Logical block = i
```

physical disk block:

$\lfloor \boxed{S+i} \rfloor$

Example:

Start block:

\[100 \]

Logical block:

\[7 \]

Physical:

\[\boxed{107} \]

Very efficient.

22. CONTIGUOUS ALLOCATION ADVANTAGES

- excellent sequential access
- excellent direct access
- simple metadata
- low seek/head movement for sequential blocks
- good locality

23. CONTIGUOUS ALLOCATION DISADVANTAGES

Major problems:

- external fragmentation
- difficult file growth
- need to know/estimate file size
- may require relocation

Suppose file occupies:

TEXT
20–29

and block 30 belongs to another file.

If file needs one more block:

Cannot simply extend contiguously.

24. EXTENT-BASED ALLOCATION

Modern file systems may use:

Extents

An extent is a contiguous run of blocks.

Instead of requiring one contiguous region for entire file:

TEXT

Extent 1 = blocks 100-199
Extent 2 = blocks 500-549

This combines good sequential performance with more flexible growth.

For GATE, know the idea if encountered.

25. LINKED ALLOCATION

Each file block may be anywhere.

Every block stores pointer to next block.

Example:

TEXT

9 !' 1 6 !' 1 !' 1 0 !' E O F

Directory stores:

TEXT

First block = 9

Sometimes last block too.

26. LINKED ALLOCATION ADVANTAGE

No need for contiguous free space.

Therefore:

no external fragmentation due to requiring one contiguous file extent in the classical sense.

Files can grow easily:

TEXT

add another free block
update pointer

27. LINKED ALLOCATION DISADVANTAGES

- poor random/direct access
- pointer overhead
- pointer corruption can lose remainder of file
- blocks may be scattered !' more seeks

To access the 100th block:

TEXT

```
follow block 1
!' b l o c k   2
!' . . .
!' b l o c k   1 0 0
```

So random access is slow.

28. LINKED ALLOCATION POINTER OVERHEAD

Suppose:

TEXT

```
Block size = 1024 bytes
Pointer = 4 bytes
```

Then actual data per block:

$\lfloor 1024 - 4 \rfloor$

$\lfloor \boxed{1020} \text{ bytes} \rfloor$

if pointer is stored inside each data block.

Useful numerical pattern.

29. FAT — FILE ALLOCATION TABLE

FAT improves classic linked allocation.

Instead of storing next-block pointer inside each data block:

Store links in a central table.

Example:

Block	Next
9	16
16	1
1	10
10	EOF

File chain:

TEXT

```
9 !' 1 6 !' 1 !' 1 0
```


30. WHY FAT HELPS

The FAT can be cached in memory.

Then traversing block pointers doesn't require reading every file block merely to discover the next location.

Data blocks themselves can contain more pure data.

31. FAT STILL HAS LINKED-ALLOCATION CHARACTERISTICS

It still represents a chain:

TEXT

```
b l o c k ! '   n e x t   b l o c k ! '   n e x t   b l o c k
```

So file blocks need not be contiguous.

32. INDEXED ALLOCATION

Each file has an:

Index block

containing pointers to the file's data blocks.

Example:

TEXT

Index Block

```
% % %   p o i n t e r ! '   b l o c k   9
% % %   p o i n t e r ! '   b l o c k   1 6
% % %   p o i n t e r ! '   b l o c k   1
% % %   p o i n t e r ! '   b l o c k   1 0
```

File data can be scattered.

33. INDEXED ALLOCATION ADVANTAGES

- supports direct/random access
- no need for entire file to be contiguous
- no external fragmentation from contiguous-file requirement
- easy growth as long as index structure has capacity

34. INDEXED ALLOCATION DISADVANTAGES

- index-block overhead
- small files may waste index space
- very large files may require:
 - linked index blocks

- multilevel indexing
- indirect blocks

This leads directly to:

INODES

35. MASTER ALLOCATION COMPARISON

Method	Random Access	Growth	Main Problem
Contiguous	Excellent	Difficult	External fragmentation/growth
Linked	Poor	Easy	Pointer traversal
Indexed	Good	Good	Index overhead

Memory:

Contiguous = Fast but rigid

Linked = Flexible but slow random access

Indexed = Pointer table gives direct access

PART D — INODE Ø=Ý%Ø=Ý%Ø=Ý%

36. WHAT IS AN INODE?

In Unix-like file systems, an inode stores:

File metadata plus addresses/pointers to file data blocks.

An inode generally stores things like:

- file type
- permissions
- owner
- size
- timestamps
- link count
- data block pointers

Important:

The filename itself is typically stored in a directory entry, not as the core inode identity.

37. DIRECTORY ENTRY + INODE

Conceptually:

```
TEXT
Directory Entry
"notes.txt"
    !"
inode number
    !"
Inode
    !"
metadata + data block pointers
```

This is extremely important for understanding links.

38. INODE NUMBER

The inode has an identifier:

```
TEXT
inode #1234
```

A directory may map:

```
TEXT
n o t e s . t x t ! '   i n o d e   1 2 3 4
```

Another name could also map to:

```
TEXT
r e p o r t . t x t ! '   i n o d e   1 2 3 4
```

That would be a hard-link situation.

39. INODE DATAPOINTERS

A classic inode structure can include:

- direct pointers
- single indirect pointer
- double indirect pointer
- triple indirect pointer

Conceptual structure:

```
TEXT
inode
% % %   d i r e c t ! '   d a t a
% % %   d i r e c t ! '   d a t a
% % %   . . .
```

```

% % %   s i n g l e   i n d i r e c t   ! '   p o i n t e r   b l o c k   ! '   d a t a
% % %   d o u b l e   i n d i r e c t   ! '   p o i n t e r   b l o c k   ! '   p o i n t e r   b l o c k s   ! '
% % %   t r i p l e   i n d i r e c t   ! '   . . .

```

This is a very important GATE numerical area.

40. DIRECT POINTER

A direct pointer points directly to a data block.

Example:

TEXT

```
i n o d e   d i r e c t [ 0 ]   ! '   D a t a   B l o c k   1 0 0
```

One pointer gives one data block.

If there are:

$\lfloor D \rfloor$

direct pointers:

$\lfloor D \rfloor$

data blocks can be addressed directly.

41. SINGLE INDIRECT POINTER

The inode points to an indirect block.

That block contains many data-block addresses.

TEXT

```

inode
  ! "
Indirect Block
% ! '   D a t a   1
% ! '   D a t a   2
% ! '   D a t a   3
...

```

42. POINTERS PER BLOCK

If:

- block size = $\lfloor B \rfloor$ bytes
- pointer size = $\lfloor P \rfloor$ bytes

then number of pointers in one indirect block:

$\lfloor \boxed{\frac{B}{P}} \rfloor$

This formula is essential.

43. EXAMPLE

Block size:

$[4KB=4096 \text{ bytes}]$

Pointer size:

$[4 \text{ bytes}]$

Pointers per indirect block:

$[\frac{4096}{4} = \boxed{1024}]$

So a single indirect pointer can reference:

$[1024]$

data blocks.

44. SINGLE INDIRECT CAPACITY

If each data block size is (B) , and each indirect block stores (N) pointers:

$[\boxed{\text{SingleIndirectData} = N \times B}]$

Example:

$[1024 \times 4KB]$

$[=4096KB]$

$[\boxed{4MB}]$

45. DOUBLE INDIRECT

The inode points to:

TEXT

Level-1 pointer block

!"

many Level-2 pointer blocks

!"

data blocks

If each pointer block contains:

$[N]$

pointers, double indirect can address:

$[\boxed{N^2}]$

data blocks.

46. TRIPLE INDIRECT

Similarly:

$$\boxed{N^3}$$

data blocks.

47. MASTER INODE CAPACITY FORMULA

Suppose inode has:

- (D) direct pointers
- one single indirect
- one double indirect
- one triple indirect

and:

$$N = \frac{\text{BlockSize}}{\text{PointerSize}}$$

Then maximum number of data blocks:

$$\boxed{D + N + N^2 + N^3}$$

Maximum file size:

$$\boxed{(D + N + N^2 + N^3) \times \text{BlockSize}}$$

assuming all these pointer types exist and ignoring other implementation limits.

~~Don't~~ Memorize the structure, not just numbers.

48. FULL GATE NUMERICAL

Suppose:

- Block size = 4 KB
- Pointer size = 4 bytes
- inode has:
 - 12 direct
 - 1 single indirect
 - 1 double indirect
 - 1 triple indirect

First:

$$N = \frac{4096}{4} = 1024 = 2^{10}$$

Data blocks:

$$12 + 1024 + 1024^2 + 1024^3$$

Maximum file size:

$$(12 + 1024 + 1024^2 + 1024^3) \times 4\text{KB}$$

49. BREAK IT INTO LEVELS

Direct

$$[12 \times 4\text{KB} = \boxed{48\text{KB}}]$$

Single indirect

$$[1024 \times 4\text{KB} = \boxed{4\text{MB}}]$$

Double indirect

$$[1024^2 \times 4\text{KB}]$$

$$[= 2^{20} \times 2^{12} = 2^{32} \text{ bytes}]$$

$$[\boxed{4\text{GB}}]$$

Triple indirect

$$[1024^3 \times 4\text{KB}]$$

$$[= 2^{30} \times 2^{12} = 2^{42} \text{ bytes}]$$

$$[\boxed{4\text{TB}}]$$

approximately, using binary units.

50. TOTAL MAX FILE SIZE

Approximately:

TEXT

4 TB
+ 4 GB
+ 4 MB
+ 48 KB

The exact sum can be required, but the powers-of-two structure is more important.

51. GATE SHORTCUT FOR INDIRECT BLOCKS

If:

TEXT

Block = 2^b bytes
Pointer = 2^p bytes

then pointers per block:

$$[\boxed{2^{b-p}}]$$

Suppose 4KB block:

$$[2^{12}]$$

4-byte pointer:

$$[2^2]$$

Then:

$$\lfloor 2^{\{12-2\}} = 2^{\{10\}} \rfloor$$

pointers.

Much faster than decimal division.

52. FINDING WHICH POINTER LEVEL CONTAINS A FILE BLOCK

Suppose 12 direct pointers and 1024 pointers per indirect block.

Logical data block numbers:

TEXT

0–11

use direct pointers.

Next:

TEXT

1 2 t h r o u g h 1 2 + 1 0 2 4 " 1

use single indirect.

Then:

double indirect.

GATE may ask which level is needed for a given byte/block offset.

53. EXAMPLE

12 direct, $\lfloor N=1024 \rfloor$.

What structure accesses logical file block 500?

Since:

$$\lfloor 500 \geq 12 \rfloor$$

but:

$$\lfloor 500 < 12 + 1024 \rfloor$$

it lies under:

$$\lfloor \boxed{\text{Single indirect}} \rfloor$$

54. EXAMPLE — BLOCK 2000

Direct + single capacity:

$$\lfloor 12 + 1024 = 1036 \rfloor$$

Block 2000 is beyond that.

Therefore:

$\boxed{\text{Double indirect}}$

55. DATABLOCK VS INDEX BLOCK

Important:

Indirect blocks store:

pointers

not actual file user data.

So when calculating disk space, distinguish:

- data blocks
- pointer/index blocks

Some GATE questions ask total blocks occupied including indirect blocks.

56. NUMBER OF INDIRECT BLOCKS — EXAMPLE

Suppose:

- block size = 4KB
- pointer = 4B
- 1024 pointers per indirect block
- file needs 2000 data blocks
- inode has 12 direct pointers

Remaining after direct:

$2000 - 12 = 1988$

Single indirect handles:

1024

Remaining:

$1988 - 1024 = 964$

Need double indirect.

Under double indirect:

- one top-level double-indirect block
- one lower indirect block can hold 1024 pointers, enough for 964 data blocks

So pointer blocks needed:

TEXT

```
1 single-indirect block
1 double-indirect top block
1 double-indirect lower block
```

Total pointer blocks:

\boxed{3}

This style of question is common.

PART E — HARD LINKS VS SYMBOLIC LINKS

57. HARD LINK

A hard link is another directory entry pointing to:

the same inode/file object.

Example:

TEXT

```
a . t x t   % %  
              % ! '   i n o d e   1 0 0   ! '   D a t a   B l o c k s  
b . t x t   % %
```

Both names refer to the same underlying file.

58. HARD LINK CONSEQUENCE

If `a.txt` is deleted:

TEXT

`b.txt`

can still access the file.

Why?

Because inode/data remain while another hard link exists.

This is tracked through a:

link count.

59. LINK COUNT

Suppose:

TEXT

```
a . t x t   ! '   i n o d e   1 0 0  
b . t x t   ! '   i n o d e   1 0 0
```

Link count:

\boxed{2}

Delete `a.txt`:

\[2\rightarrow1 \]

File data remains.

Delete `b.txt`:

\[1\rightarrow0 \]

Then, once no relevant open references remain and filesystem rules permit:

inode/data can be freed.

60. SYMBOLIC LINK / SOFT LINK

A symbolic link is a separate file-like object containing/referenceing:

a pathname to another file.

Conceptually:

```
TEXT
s h o r t c u t ! '   " / h o m e / u s e r / a . t x t "
```

It does not simply become another name for the same inode in the same way a hard link does.

61. DELETE TARGET OF SYMBOLIC LINK

Suppose:

```
TEXT
l i n k ! '   t a r g e t . t x t
```

Delete:

```
TEXT
t a r g e t . t x t
```

Now symbolic link remains but points to missing path.

This is:

Dangling symbolic link

62. HARD VS SYMBOLIC LINK

Hard Link	Symbolic Link
Points to same inode/object	Stores/refers to pathname

Hard Link	Symbolic Link
Same underlying file	Separate link object
Target removal doesn't break other hard link	Target removal can make symlink dangling
Link count increases	Target inode link count generally not increased the same way
Typically restricted across file systems	Can often cross file-system boundaries
Directories often restricted for hard-linking	Can point to directories

For GATE, focus on the conceptual distinction.

MEMORY

Hard link = same file, another name.

Symbolic link = shortcut/path.

63. OPEN FILE + UNLINK — IMPORTANT UNIX IDEA

Suppose process opens a file.

Then another process deletes/unlinks its directory name.

If the file is still open:

The underlying inode/data may remain until the final open reference is closed.

So deleting a filename does not always immediately destroy the data.

Important conceptual idea:

TEXT

Directory references
+
Open-file references

can affect lifetime.

PART F — FREE-SPACE MANAGEMENT

64. WHY TRACK FREE SPACE?

The file system must know:

Which disk blocks are unused?

When a file grows, it needs free blocks.

Classical methods include:

- bitmap/bit vector
- linked free list
- grouping
- counting

65. BITMAP / BIT VECTOR

Use one bit per disk block.

Example:

TEXT

```
Block: 0 1 2 3 4 5 6 7
Bits:  1 0 0 1 1 0 1 0
```

The meaning of 0/1 as free/allocated depends on convention.

Always check question wording.

66. BITMAP SIZE

$\mathcal{O}(N)$ common numerical.

If disk has:

$\lfloor N \rfloor$

blocks, bitmap needs:

$\lfloor N \rfloor$

bits.

In bytes:

$\lfloor \boxed{\frac{N}{8}} \rfloor$

assuming exactly one bit per block.

67. EXAMPLE

Disk size:

$\lfloor 1\text{GB} = 2^{30} \rfloor$ bytes

Block size:

$\lfloor 4\text{KB} = 2^{12} \rfloor$

Number of blocks:

$\lfloor 2^{30-12} = 2^{18} \rfloor$

Bitmap bits:

$\lfloor 2^{18} \rfloor$

Bitmap bytes:

$\lfloor 2^{18}/8 = 2^{15} \rfloor$

$\lfloor \boxed{32\text{KB}} \rfloor$

68. BITMAP ADVANTAGES

- compact
- easy to find consecutive free blocks using bit operations
- fixed predictable size

69. FREE LIST

Free blocks are linked:

TEXT

5 !' 2 0 !' 1 8 !' 7 !' . . .

Need a free block?

Take one from list.

Simple.

But finding a large contiguous run can be less efficient than bitmap-based scanning.

70. GROUPING

A free block may store addresses of several other free blocks.

One of those can point to the next group.

This reduces long one-pointer-at-a-time traversal.

Know conceptually if mentioned.

71. COUNTING

If free blocks often appear consecutively:

Store:

TEXT

Starting block + Count

Example:

TEXT

start = 100

count = 50

means:

```
TEXT
100-149 free
```

Efficient for runs/extents.

PART G — FILE ALLOCATION NUMERICALS

72. CONTIGUOUS ALLOCATION EXAMPLE

File starts at block:

$\lfloor 500 \rfloor$

Length:

$\lfloor 20 \rfloor$

Blocks:

```
TEXT
500 ... 519
```

Logical block 13 maps to:

$\lfloor 500+13 = \boxed{513} \rfloor$

assuming zero-based logical block numbering.

73. BYTE OFFSET TO FILE BLOCK

Block size:

$\lfloor 4\text{KB}=4096 \rfloor$

Byte offset:

$\lfloor 10000 \rfloor$

Logical block:

$\lfloor \left\lfloor \frac{10000}{4096} \right\rfloor = \boxed{2} \rfloor$

Offset inside block:

$\lfloor 10000 \bmod 4096 \rfloor$

$\lfloor =1808 \rfloor$

So:

```
TEXT
File block 2, offset 1808
```

This is analogous to paging calculations.

74. FILE SIZE TO NUMBER OF BLOCKS

If file size:

$[F]$

block size:

$[B]$

data blocks needed:

$\lceil \frac{F}{B} \rceil$

Same ceiling idea as pages.

75. EXAMPLE

File size:

$[10KB]$

Block size:

$[4KB]$

Blocks needed:

$\lceil 10/4 \rceil = 3$

Allocated capacity:

$[12KB]$

Possible last-block internal waste:

$[2KB]$

76. FILE-SYSTEM INTERNAL FRAGMENTATION

Fixed-size disk blocks can waste space in the final block.

Example:

TEXT

Block size = 4KB

File size = 6KB

Requires two blocks:

$[8KB]$

Unused:

$[2KB]$

This is analogous to paging's last-page internal fragmentation.

77. AVERAGE LAST-BLOCK WASTE

Under simple uniform assumptions:

$$\lfloor \boxed{\frac{\text{BlockSize}}{2}} \rfloor$$

average internal fragmentation per file.

Same intuition as page size.

PART H — DISK BLOCK SIZE TRADE-OFF

78. LARGE BLOCK SIZE

Advantages:

- fewer block pointers
- better sequential throughput
- smaller metadata structures

Disadvantages:

- more internal fragmentation
 - small files waste more space
-

79. SMALL BLOCK SIZE

Advantages:

- less internal fragmentation
- finer allocation

Disadvantages:

- more metadata/pointers
- potentially more I/O operations
- larger allocation tables/index structures

This resembles the page-size tradeoff.

PART I — FILE PROTECTION

80. FILE PERMISSIONS

File system controls who can:

- read
- write
- execute
- delete/modify metadata

Unix-style permissions often use:

TEXT

```
r = read
w = write
x = execute
```

for:

- owner
- group
- others

You do not need deep Unix administration; understand file protection conceptually.

81. ACCESS CONTROL LIST — ACL

An ACL can associate a file with specific permissions for users/groups.

Conceptually:

TEXT

```
A l i c e  ! '   r e a d / w r i t e
B o b      ! '   r e a d
G u e s t  ! '   n o n e
```

This gives finer-grained control.

PART J — FILE SYSTEM LAYERS

82. CONCEPTUAL LAYERING

A file-system stack can be viewed as:

TEXT

```
Applications
! "
Logical File System
! "
File Organization Module
! "
Basic File System
! "
I/O Control
! "
Storage Device
```

You mainly need the idea that high-level file operations eventually become block I/O.

83. LOGICAL FILE SYSTEM

Deals with concepts such as:

- directories

- metadata
- protection
- file control information

84. FILE ORGANIZATION MODULE

Deals with:

- logical block !' physical block mapping
- allocation
- free-space management

85. BASIC FILE SYSTEM

Issues generic block read/write requests to device-level system.

This layered view may appear in conceptual questions.

PART K — MOUNTING

86. FILE-SYSTEM MOUNTING

Mounting means:

Making a file system accessible at a point in the existing directory hierarchy.

Example conceptually:

TEXT

```
/
% % %   m n t
      % % %   u s b
```

After mounting a USB file system at `/mnt/usb`, its files become accessible there.

For GATE, know the concept.

PART L — IMPORTANT INODE NUMERICAL Ø=Ý%Ø=Ý%

87. EXAMPLE

Suppose:

TEXT

```
Block size = 1 KB
Pointer size = 4 bytes
```

Then:

$$N = \frac{1024}{4} = 256$$

An inode has:

TEXT

```
10 direct
1 single indirect
1 double indirect
```

Maximum data blocks:

$$10 + 256 + 256^2$$

$$= 10 + 256 + 65536$$

$$= \boxed{65802}$$

Maximum file size:

$$65802 \times 1\text{KB}$$

$$\boxed{65802\text{KB}}$$

approximately:

$$64.26\text{MB}$$

88. POWER-OF-TWO METHOD

Since:

TEXT

```
1 KB = 2^10
4 bytes = 2^2
```

Pointers/block:

$$2^{10-2} = 2^8 = 256$$

Double indirect:

$$2^{16}$$

data blocks.

This makes calculations much faster.

89. TRIPLE INDIRECT GENERAL FORMULA

With:

$$N$$

pointers per pointer block:

TEXT

```
Direct !' 1 data block per pointer
Single !' N blocks
```

```
D o u b l e ! '   N 2   b l o c k s
T r i p l e ! '   N 3   b l o c k s
```

Memory:

Each indirect level multiplies by N again.

90. NUMBER OF DISK ACCESSES WITH INODE POINTERS

Suppose inode already resides in memory.

Direct block

May need:

```
TEXT
1 disk access for data
```

Single indirect

```
TEXT
1 for indirect pointer block
+ 1 for data
= 2
```

Double indirect

```
TEXT
2 pointer-block accesses
+ 1 data
= 3
```

Triple indirect

```
TEXT
3 pointer-block accesses
+ 1 data
= 4
```

assuming none of the intermediate blocks are cached.

91. GATE TRAP — CACHE MATTERS

If inode or indirect blocks are already cached in memory:

Disk-access count decreases.

Always read what is assumed resident/cached.

PART M — HARD LINK GATE QUESTIONS

92. HARD LINK EXAMPLE

Initial:

TEXT

```
A !' i n o d e 5 0
```

Link count:

\[1 \]

Create hard link B:

TEXT

```
A % %  
% !' i n o d e 5 0  
B % %
```

Link count:

\[2 \]

Delete A:

\[LinkCount=1 \]

File remains accessible through B.

93. SYMBOLIC LINK EXAMPLE

TEXT

```
A !' i n o d e 5 0  
  
S !' p a t h n a m e " A "
```

Delete A.

Now S points to:

TEXT

```
nonexistent A
```

So:

\[\boxed{\text{Dangling symlink}} \]

94. WHY HARD LINKS TYPICALLY CANNOT CROSS FILE SYSTEMS

A hard link identifies an inode within a particular file system.

Another file system has a separate inode namespace.

Symbolic links store pathnames, so they can refer across mounted file systems.

For GATE, conceptual distinction is enough.

PART N — FILE ALLOCATION COMPARISON TRAPS

95. WHICH SUPPORTS BEST RANDOM ACCESS?

Among classical forms:

$\boxed{\text{Contiguous}}$

is extremely efficient because:

$\text{PhysicalBlock} = \text{Start} + \text{LogicalBlock}$

Indexed also supports direct access well.

Linked is poor for arbitrary random access.

96. WHICH GROWS MOST NATURALLY?

Linked allocation:

easy to append another free block.

Indexed:

also flexible if index capacity available.

Contiguous:

difficult because next block may be occupied.

97. WHICH HAS EXTERNAL FRAGMENTATION?

Classic contiguous allocation can suffer:

$\boxed{\text{External fragmentation}}$

Linked and indexed do not require one contiguous file region.

98. WHICH HAS POINTER OVERHEAD INSIDE EACH DATABLOCK?

Classic linked allocation.

Indexed allocation stores pointers primarily in index blocks instead.

99. WHICH REQUIRES INDEX BLOCK?

$\boxed{\text{Indexed allocation}}$

100. WHICH DOES FAT REPRESENT?

Conceptually:

$\boxed{\text{Linked allocation using a central table}}$

PART O — HOW GATE CAN ASK FILE SYSTEMS

Common patterns:

Pattern 1

Compare contiguous/linked/indexed allocation.

Pattern 2

Calculate file block location.

Pattern 3

Calculate FAT/pointeroverhead.

Pattern 4

Inode direct/indirect maximum file size.

Pattern 5

How many indirect blocks are needed?

Pattern 6

Hard link vs symbolic link.

Pattern 7

Bitmap size.

Pattern 8

Number of data blocks for a file.

Pattern 9

Directory/path concepts.

Pattern 10

File metadata/inode conceptual questions.

& pGATE TRAPS — FILE SYSTEMS

Trap 1

File name is necessarily stored inside the inode.

'L

Typically directory maps a filename to inode number.

Trap 2

Hard links create independent copies of file data.

'L

They refer to the same underlying inode/file.

Trap 3

Deleting one hard link always deletes file data immediately.

'L

Other hard links/open references may remain.

Trap 4

Deleting symlink target automatically deletes symlink too.

'L

The symlink may remain dangling.

Trap 5

Contiguous allocation is poor for direct access.

'L

It is excellent for direct access.

Trap 6

Linked allocation gives efficient arbitrary random access.

'L

Generally poor.

Trap 7

Indexed allocation requires file blocks to be contiguous.

'L

They may be scattered.

Trap 8

FAT stores file data itself.

'L

It mainly stores allocation/link information.

Trap 9

An indirect block contains user file data.

'L

It contains block pointers.

Trap 10

Bitmap of N disk blocks needs N bytes.

'L

It needs:

$\lceil N \rceil$ bits

or:

$\lceil \frac{N}{8} \rceil$ bytes

Trap 11

Double indirect addresses $\lceil 2N \rceil$ blocks.

'L

It addresses:

$\lceil N^2 \rceil$

Trap 12

Triple indirect addresses $\lceil 3N \rceil$ blocks.

'L

It addresses:

$\lceil N^3 \rceil$

'p WRITE THIS IN YOUR NOTES

File System

OS mechanism for organizing and managing persistent files/directories on secondary storage.

File Operations

TEXT

Create
Open
Read
Write
Seek
Close
Delete

Access Methods

TEXT

Sequential !' in order
Direct !' arbitrary block / offset

File Allocation

Contiguous

TEXT

Start + Length

Pros:

- fast sequential
- fast random

Cons:

- external fragmentation
- difficult growth

Linked

TEXT

Block !' next block !' next . . .

Pros:

- easy growth

- no need for contiguous region

Cons:

- poor random access
- pointer overhead

Indexed

TEXT

I n d e x B l o c k ! ' p o i n t e r s t o d a t a b l o c k s

Pros:

- direct access
- flexible placement

Cons:

- index overhead

FAT

Linked allocation with next-block pointers stored in a central table.

Inode

Stores:

TEXT

Metadata

+

Block pointers

Directory:

TEXT

F i l e n a m e ! ' i n o d e n u m b e r

INDIRECT FORMULAS

$\boxed{N = \frac{\text{BlockSize}}{\text{PointerSize}}}$

Single indirect:

$\boxed{N}$

data blocks.

Double:

$\boxed{N^2}$

Triple:

$$\boxed{N^3}$$

If D direct pointers:

$$\boxed{\text{TotalDataBlocks} = D + N + N^2 + N^3}$$

Maximum file size:

$$\boxed{\text{TotalDataBlocks} \times \text{BlockSize}}$$

FREE-SPACE BITMAP

Number of disk blocks:

$$\boxed{\frac{\text{DiskSize}}{\text{BlockSize}}}$$

Bitmap size:

$$\boxed{\frac{\text{NumberOfBlocks}}{8} \text{ bytes}}$$

HARD VS SYMBOLIC LINK

TEXT

Hard link

! ' another name for same inode

Symbolic link

! ' pathname - based shortcut

Delete original:

TEXT

Hard link ! ' still works

Symlink ! ' may dangle

MASTER ALLOCATION MEMORY TABLE

Method	Memory Trick
Contiguous	Together
Linked	Chain
Indexed	Pointer list
FAT	Chain stored in table
inode	Direct + indirect hierarchy

MASTER INODE MEMORY TRICK

Think:

```
TEXT
DIRECT
    ! "
one_jump

SINGLE
    ! "
N

DOUBLE
    ! "
N^2

TRIPLE
    ! "
N^3
```

Each additional indirect level:

multiplies capacity by $\backslash(N)$.

30-SECOND REVISION

File system?

!' Manages persistent files/directories/storage.

Sequential access?

!' In order.

Direct access?

!' Arbitrary position.

Contiguous allocation?

!' consecutive blocks; fast but rigid.

Linked?

!' block chain; easy growth but bad random access.

Indexed?

!' index block of pointers.

FAT?

!' linked allocation pointers in table.

inode?

!' metadata + block pointers.

Pointers per indirect block?

$\backslash \text{BlockSize/PointerSize} \backslash$

Double indirect?

$\lfloor N^2 \rfloor$

Triple?

$\lfloor N^3 \rfloor$

Hard link?

! same inode.

Symlink?

! path shortcut.

Bitmap?

! one bit per block.

ACTIVE RECALL

1. What is the difference between file data and metadata?
2. What does `open()` conceptually provide to a process?
3. Difference between sequential and direct access?
4. Why is contiguous allocation fast?
5. Why is file growth difficult with contiguous allocation?
6. Why is linked allocation poor for random access?
7. How does FAT differ from ordinary linked allocation?
8. What does an inode store?
9. Does an inode normally need the filename to identify the file object?
10. If block size is 4 KB and pointer size is 8 bytes, how many pointers fit in an indirect block?
11. If $\lfloor N=512 \rfloor$, how many data blocks can a double-indirect pointer address?
12. What happens to a hard-linked file when one filename is deleted but another remains?
13. What happens to a symlink if its target is deleted?
14. A disk has $\lfloor 2^{30} \rfloor$ bytes with block size $\lfloor 2^{12} \rfloor$. How many bits are required for a one-bit-per-block bitmap?

Answers to key numericals:

Q10

$\lfloor 4096/8 = \boxed{512} \rfloor$

Q11

$\lfloor 512^2 = \boxed{262144} \rfloor$

Q14

Blocks:

$\lfloor 2^{30-12} = 2^{18} \rfloor$

Bitmap:

$\lfloor \boxed{2^{18}} \text{ bits} \rfloor$

which is:

[2^{15} \text{ bytes} = \boxed{32\text{KB}}]

MASTER CONNECTION — FILE SYSTEMS

TEXT

```
File Name
    !"
Directory
    !"
inode / metadata
    !"
Block Mapping
    !"
Contiguous / Linked / Indexed
    !"
Disk Blocks
```

And free storage is tracked through:

TEXT

```
Bitmap / Free List / Counting
```

NEXT — CHAPTER 18 Ø=Ý%Ø=Ý%

Next I'll do the **final deep file-system numerical chapter**:

INODES + FILE ALLOCATION + FREE-SPACE NUMERICALS

We will focus almost entirely on GATE problem solving:

- maximum file size
- direct/single/double/triple indirect blocks
- locating a byte offset inside inode structures
- number of index blocks required
- number of disk accesses
- FATsize
- bitmap size
- file-block internal fragmentation
- hard-link/inode-count questions
- tricky GATE-style mixed numericals

After that, we can do the **full Operating Systems revision + high-yield formula sheet + GATE traps + test**.

USER

NEXT

CHAPTER 18 — INODE + FILE ALLOCATION + FREE-SPACE NUMERICALS

This chapter is mainly for **GATE problem solving**.

The highest-value formula chain is:

TEXT

Block Size

!"

Pointer Size

!"

Pointers per Indirect Block

!"

Direct / Single / Double / Triple Capacity

!"

Maximum File Size

The master formula is:

$$\boxed{N = \frac{\text{Block Size}}{\text{Pointer Size}}}$$

where N = number of block addresses stored in one indirect block.

PART A — DIRECT + INDIRECT POINTER CAPACITY

1. Direct pointers

Suppose an inode contains:

$$D$$

direct pointers.

Each direct pointer addresses exactly:

$$1$$

data block.

So direct capacity:

$$\boxed{D \text{ data blocks}}$$

and data capacity:

$$\boxed{D \times B}$$

where B = block size.

2. Single indirect pointer

A single-indirect pointer points to one block containing block addresses.

If:

$$N = \frac{B}{P}$$

then single-indirect capacity:

$$\boxed{N \times \text{data blocks}}$$

and file-data capacity:

$$\boxed{N \times B}$$

3. Double indirect

Structure:

```
TEXT
inode
    !
Level-1 indirect block
    !
N Level-2 indirect blocks
    !
N data pointers each
```

Therefore:

$$\boxed{N^2 \times \text{data blocks}}$$

Capacity:

$$\boxed{N^2 B}$$

4. Triple indirect

Similarly:

$$\boxed{N^3 \times \text{data blocks}}$$

Capacity:

$$\boxed{N^3 B}$$

MASTER MEMORY

```
TEXT
Direct ! D
Single ! N
Double ! N^2
Triple ! N^3
```

Memory trick:

Each extra indirect level multiplies by N.

5. Maximum file size formula

Suppose inode contains:

- $\backslash(D\backslash)$ direct pointers
- 1 single indirect
- 1 double indirect
- 1 triple indirect

Then maximum data blocks:

$$\backslash \boxed{D+N+N^2+N^3} \backslash$$

Maximum file size:

$$\backslash \boxed{(D+N+N^2+N^3)\times B} \backslash$$

6. GATE NUMERICAL — STANDARD TYPE

Given:

TEXT

```
Block size = 4 KB
Pointer size = 4 bytes
12 direct pointers
1 single
1 double
1 triple
```

First convert:

$$\backslash 4KB=4096=2^{12}\backslash \text{ bytes } \backslash$$

Pointer:

$$\backslash 4=2^2\backslash \text{ bytes } \backslash$$

Therefore:

$$\backslash N=\frac{2^{12}}{2^2}=2^{10}=1024 \backslash$$

7. Direct capacity

$$\backslash 12\times 4KB \backslash$$

$$\backslash =\boxed{48KB} \backslash$$

8. Single-indirect capacity

$$\backslash 1024\times 4KB \backslash$$

$$\backslash =2^{10}\times 2^{12}=2^{22}\backslash \text{ bytes } \backslash$$

$$\backslash =\boxed{4MB} \backslash$$

9. Double-indirect capacity

$$\lfloor 1024^2 \times 4\text{KB} \rfloor$$

$$\lfloor = 2^{20} \times 2^{12} = 2^{32} \rfloor$$

$$\lfloor = \boxed{4\text{GB}} \rfloor$$

10. Triple-indirect capacity

$$\lfloor 1024^3 \times 4\text{KB} \rfloor$$

$$\lfloor = 2^{30} \times 2^{12} = 2^{42} \rfloor$$

$$\lfloor = \boxed{4\text{TB}} \rfloor$$

So maximum file size is:

$$\lfloor \boxed{48\text{KB} + 4\text{MB} + 4\text{GB} + 4\text{TB}} \rfloor$$

approximately.

EXAM SHORTCUT

If values are powers of two, do not perform decimal multiplication.

Use exponents.

Example:

TEXT

$$\text{Block} = 2^{12}$$

$$\text{Pointer} = 2^2$$

Then:

$$\lfloor N = 2^{10} \rfloor$$

Immediately:

TEXT

S i n g l e ! ' 2¹ p b l o c k s

D o u b l e ! ' 2² p b l o c k s

T r i p l e ! ' 2³ p b l o c k s

Then multiply each by block size $\lfloor 2^{12} \rfloor$.

PART B — FIND WHICH POINTER IS USED

This is another common GATE pattern.

Suppose:

TEXT

D = 12 direct pointers

N = 1024 pointers per indirect block

Using **zero-based file block numbering**:

Direct pointers handle logical file blocks:

$\boxed{0 \text{ through } 11}$

That's 12 blocks.

11. Single-indirect range

After 12 direct blocks, single indirect handles next 1024 blocks.

Start:

12

End:

$12 + 1024 - 1$

$\boxed{1035}$

So:

$\boxed{12 \text{ to } 1035}$

uses single indirect.

12. Double-indirect range

Double indirect begins at:

1036

and contains:

$1024^2 = 1,048,576$

blocks.

End:

$1036 + 1,048,576 - 1$

$\boxed{1,049,611}$

Therefore:

TEXT

```
0 - 11 !' Direct
12 - 1035 !' Single indirect
1036 - 1,049,611 !' Double indirect
```

Everything after that, up to triple capacity:

Triple indirect.

13. QUICK EXAMPLE

Logical file block:

`\[ 500 \]`

Since:

`\[ 12\le500\le1035 \]`

it uses:

`\[ \boxed{\text{Single indirect}} \]`

14. BLOCK 2000

Since:

`\[ 2000>1035 \]`

but well inside double-indirect range:

`\[ \boxed{\text{Double indirect}} \]`

15. GATE SHORTCUT — CUMULATIVE CAPACITY

Write:

TEXT

Direct boundary = D

Single boundary = $D + N$

Double boundary = $D + N + N^2$

Triple boundary = $D + N + N^2 + N^3$

For zero-based block b :

- if $(b < D)$!' direct
- if $(b < D + N)$!' single
- if $(b < D + N + N^2)$!' double
- otherwise !' triple, if within maximum

This is faster than listing ranges.

PART C — BYTE OFFSET !' BLOCK NUMBER

Suppose file byte offset:

`\[ X \]`

and block size:

`\[ B \]`

Then logical block number:

$\lceil \boxed{\left\lfloor \frac{X}{B} \right\rfloor} \rceil$

Offset within block:

$\boxed{X \bmod B}$

This is exactly like page-number + page-offset calculation.

16. EXAMPLE

Block size:

$\boxed{4KB=4096}$

Byte offset:

$\boxed{9000}$

Logical block:

$\left\lfloor \frac{9000}{4096} \right\rfloor = 2$

Offset:

$9000 - 8192 = \boxed{808}$

So:

TEXT

File block 2
Offset 808

Since block 2 is among first 12 blocks:

direct pointer.

17. IMPORTANT BYTE-OFFSET TRAP

Suppose question asks:

byte number 4096

with zero-based byte numbering.

Block 0 contains:

TEXT

bytes 0–4095

So byte 4096 belongs to:

$\boxed{\text{block 1}}$

Do not treat it as the last byte of block 0.

PART D — LOCATING A BLOCK INSIDE SINGLE INDIRECT

Suppose:

TEXT

$D=12$

$N=1024$

Find logical block:

$\lfloor 500 \rfloor$

Single-indirect relative index:

$\lfloor 500-D \rfloor$

$\lfloor 500-12=488 \rfloor$

Therefore use:

entry 488 of the single-indirect block

assuming zero-based indexing.

18. SINGLE-INDIRECT INDEX FORMULA

For logical block $\lfloor b \rfloor$:

$\lfloor \boxed{\text{Index} = b - D} \rfloor$

provided:

$\lfloor D \rfloor \leq b < D + N \rfloor$

PART E — LOCATING A BLOCK INSIDE DOUBLE INDIRECT $\emptyset = \text{Ý\%}$

Suppose logical block lies in double-indirect area.

First remove blocks addressed by direct + single:

$\lfloor r = b - (D + N) \rfloor$

This is the block's relative position inside double-indirect space.

Then:

$\lfloor \boxed{\text{FirstLevelIndex} = \left\lfloor \frac{r}{N} \right\rfloor} \rfloor$

$\lfloor \boxed{\text{SecondLevelIndex} = r \bmod N} \rfloor$

19. EXAMPLE

Given:

TEXT

D = 12
N = 1024
Logical block = 2000

Remove direct + single:

$\lfloor r=2000-(12+1024) \rfloor$

$\lfloor =2000-1036 \rfloor$

$\lfloor =964 \rfloor$

Then:

$\lfloor \text{FirstLevel} = \lfloor \lfloor 964/1024 \rfloor \rfloor = 0 \rfloor$

$\lfloor \text{SecondLevel} = 964 \rfloor$

So access:

TEXT

Double-indirect pointer
! "
Entry 0 of level-1
! "
Entry 964 of level-2
! "
Data block

20. ANOTHER DOUBLE-INDIRECT EXAMPLE

Logical block:

$\lfloor 3000 \rfloor$

Relative:

$\lfloor r=3000-1036=1964 \rfloor$

First level:

$\lfloor 1964/1024=1 \rfloor$

integer quotient:

$\lfloor \boxed{1} \rfloor$

Second level:

$\lfloor 1964 \bmod 1024 = 940 \rfloor$

So:

TEXT

L1 index = 1
L2 index = 940

MEMORY

Double indirect = express relative block number in:

quotient + remainder using N

Very similar to converting an address into page + offset.

PART F — TRIPLE INDIRECT INDEXING

For a block in triple-indirect region:

Remove all earlier capacity:

$$\lfloor r = b - (D + N + N^2) \rfloor$$

Then:

$$\lfloor i_1 = \left\lfloor \frac{r}{N^2} \right\rfloor \rfloor$$

Remaining:

$$\lfloor r_1 = r \bmod N^2 \rfloor$$

Then:

$$\lfloor i_2 = \left\lfloor \frac{r_1}{N} \right\rfloor \rfloor$$

$$\lfloor i_3 = r_1 \bmod N \rfloor$$

This gives three indirect indices.

For most GATE questions, knowing the structure is sufficient unless explicit indexing is asked.

PART G — NUMBER OF POINTER / INDEX BLOCKS REQUIRED $\emptyset = \text{Ý}\% \emptyset = \text{Ý}\%$

This is a highly testable numerical type.

Suppose:

TEXT

Block size = 4KB

Pointer = 4 bytes

N = 1024

D = 12 direct pointers

File contains 2000 data blocks

How many indirect pointer blocks are required?

21. Direct portion

First 12 data blocks use direct pointers.

Remaining:

$$\lfloor 2000 - 12 = 1988 \rfloor$$

22. Single indirect

Single-indirect block can point to:

$$\lfloor 1024 \rfloor$$

data blocks.

Remaining:

$$\lfloor 1988-1024=964 \rfloor$$

Pointer blocks used so far:

$$\lfloor 1 \rfloor$$

for the single-indirect block.

23. Double indirect

964 remaining data blocks.

Need one top-level double-indirect block.

Then lower-level indirect blocks needed:

$$\lfloor \lceil \frac{964}{1024} \rceil \rfloor = 1$$

Therefore double-indirect metadata blocks:

$$\lfloor 1+1=2 \rfloor$$

Total pointer blocks:

$$\lfloor 1+2=3 \rfloor$$

24. MORE DIFFICULT EXAMPLE

Suppose file has:

$$\lfloor 5000 \rfloor$$

data blocks.

Again:

TEXT

D=12

N=1024

After direct:

$$\lfloor 5000-12=4988 \rfloor$$

After single:

$$\lfloor 4988-1024=3964 \rfloor$$

Now double-indirect data remains:

$\lfloor 3964 \rfloor$

Number of lower-level indirect blocks:

$\left\lceil \frac{3964}{1024} \right\rceil$

$\boxed{4}$

Double-indirect pointer blocks:

TEXT

```
1 top-level
+
4 lower-level
=
5
```

Plus one single-indirect block:

$\boxed{6}$ pointer blocks

Total disk blocks occupied:

$5000 \text{ data} + 6 \text{ pointer} = \boxed{5006}$

assuming inode itself is accounted separately/not counted.

25. GENERAL DOUBLE-INDIRECT METADATA FORMULA

Suppose $\lfloor R \rfloor$ data blocks must be represented through double indirect.

Lower indirect blocks required:

$\left\lceil \frac{R}{N} \right\rceil$

Plus:

$\boxed{1}$

double-indirect top block.

Thus:

$\boxed{1 + \left\lceil \frac{R}{N} \right\rceil}$

pointer blocks for that double-indirect portion.

PART H — NUMBER OF DISK ACCESSES $\Theta = \%$

Assume inode is already in main memory and no pointer blocks/data blocks are cached.

Direct

TEXT

```
i n o d e ! ' d a t a b l o c k
```

Disk accesses:

$\boxed{1}$

26. Single indirect

```
TEXT
inode
! "
single-indirect block
! "
data
```

Disk accesses:

\[\boxed{2} \]

27. Double indirect

```
TEXT
inode
! "
L1 block
! "
L2 block
! "
data
```

Disk accesses:

\[\boxed{3} \]

28. Triple indirect

```
TEXT
inode
! "
L1
! "
L2
! "
L3
! "
data
```

Disk accesses:

\[\boxed{4} \]

MASTER SHORTCUT

If inode is already in RAM:

TEXT

```
D i r e c t !' 1
S i n g l e !' 2
D o u b l e !' 3
T r i p l e !' 4
```

Memory:

Indirect level + 1 data access

29. WHAT IF INODE IS NOT IN MEMORY?

Then first fetch inode.

So add:

$\backslash 1 \backslash$

extra disk access.

Example double indirect:

TEXT

```
inode read = 1
L1 = 1
L2 = 1
data = 1
```

Total:

$\backslash \boxed{4} \backslash$

30. WHAT IF INDIRECT BLOCKS ARE CACHED?

Then accesses decrease.

Example:

Double-indirect path:

- inode in memory
- first-level block cached
- second-level not cached
- data not cached

Disk accesses:

TEXT

```
S e c o n d - l e v e l   p o i n t e r   b l o c k !' 1
D a t a !' 1
```

Total:

$\backslash \boxed{2} \backslash$

Always read the caching assumptions.

PART I — MAXIMUM FILE SIZE WITH ONLY SOME POINTER TYPES

Do not automatically use:

$$\lfloor D+N+N^2+N^3 \rfloor$$

unless all four pointer classes exist.

31. EXAMPLE

Inode:

TEXT

```
10 direct
1 single indirect
1 double indirect
No triple indirect
```

Then:

$$\lfloor \boxed{\text{MaxBlocks}=10+N+N^2} \rfloor$$

Not $\lfloor (N^3) \rfloor$.

32. MULTIPLE SINGLE INDIRECT POINTERS

Suppose inode has:

TEXT

```
12 direct
2 single indirect
1 double indirect
```

Then:

$$\lfloor \boxed{\text{MaxBlocks}=12+2N+N^2} \rfloor$$

Always multiply each pointer type by its count.

GENERAL FORM

If:

- $\lfloor (D) \rfloor$ direct
- $\lfloor (S) \rfloor$ single-indirect pointers
- $\lfloor (Q) \rfloor$ double-indirect pointers
- $\lfloor (T) \rfloor$ triple-indirect pointers

then:

$$\lfloor \boxed{\text{MaxBlocks}=D+SN+QN^2+TN^3} \rfloor$$

PART J — FAT SIZE NUMERICALS Ø=Ý%Ø=Ý%

FAT has approximately one entry per disk block/cluster.

So:

$$\boxed{\text{FATSize} = \text{NumberOfEntries} \times \text{EntrySize}}$$

33. EXAMPLE

Disk:

$$1\text{GB} = 2^{30} \text{ bytes}$$

Block size:

$$4\text{KB} = 2^{12}$$

Number blocks:

$$2^{30-12} = 2^{18}$$

Suppose FAT entry:

$$4 \text{ bytes} = 2^2$$

FAT size:

$$2^{18} \times 2^2 = 2^{20}$$

$$\boxed{1\text{MB}}$$

34. FAT MINIMUM ENTRY BITS

Suppose disk contains:

$$2^{18}$$

blocks.

To identify a block, at least:

$$18$$

bits are needed.

But FAT entries may also need special values such as:

- free
- EOF
- bad block
- reserved

So real entry size may need additional encodings.

If GATE asks:

minimum bits purely to address a block

then use:

$$\lfloor \log_2(\text{\#blocks}) \rfloor$$

If actual FAT entry size is given, use given value.

35. FATEXAM TRAP

Do not calculate FAT entries using:

number of files.

FAT is associated with storage allocation units/blocks/clusters.

So entries are based on:

$$\lfloor \boxed{\text{Number of addressable allocation units}} \rfloor$$

PART K — BITMAP NUMERICALS Ø=Ý%

For one bit per disk block:

$$\lfloor \boxed{\text{BitmapBits} = \text{NumberOfBlocks}} \rfloor$$

Number blocks:

$$\lfloor \boxed{\frac{\text{DiskSize}}{\text{BlockSize}}} \rfloor$$

Therefore:

$$\lfloor \boxed{\text{BitmapBytes} = \frac{\text{DiskSize}}{\text{BlockSize} \times 8}} \rfloor$$

when quantities divide cleanly.

36. EXAMPLE

Disk:

$$\lfloor 8\text{GB} = 2^{\{33\}} \rfloor$$

Block:

$$\lfloor 4\text{KB} = 2^{\{12\}} \rfloor$$

Number of blocks:

$$\lfloor 2^{\{21\}} \rfloor$$

Bitmap size:

$$\lfloor 2^{\{21\}} \backslash \text{bits} \rfloor$$

In bytes:

$$\lfloor 2^{\{18\}} \backslash \text{bytes} \rfloor$$

$$\lfloor = \boxed{256\text{KB}} \rfloor$$

37. ANOTHER EXAMPLE

Disk:

$$\lfloor 1\text{TB} = 2^{40} \rfloor$$

Block:

$$\lfloor 4\text{KB} = 2^{12} \rfloor$$

Blocks:

$$\lfloor 2^{28} \rfloor$$

Bitmap:

$$\lfloor 2^{28} \rfloor \text{ bits}$$

Bytes:

$$\lfloor 2^{25} \rfloor$$

$$\lfloor \boxed{32\text{MB}} \rfloor$$

EXAM SHORTCUT

When using powers of two:

TEXT

Disk exponent "Block exponent = Number - of - block bits / e
Then subtract 3 to convert bitmap bits !' bytes

Example:

$$\lfloor 40 - 12 = 28 \rfloor$$

bitmap:

$$\lfloor 2^{28} \rfloor \text{ bits}$$

bytes:

$$\lfloor 2^{28-3} = 2^{25} \rfloor$$

PART L — FILE BLOCK INTERNAL FRAGMENTATION

Suppose block size:

$$\lfloor B \rfloor$$

file size:

$$\lfloor F \rfloor$$

Blocks required:

$$\lfloor \boxed{\left\lceil \frac{F}{B} \right\rceil} \rfloor$$

Allocated:

$$\lfloor \boxed{\left\lceil \frac{F}{B} \right\rceil B} \rfloor$$

Unused space:

$$\left\lceil \frac{F}{B} \right\rceil B - F$$

38. EXAMPLE

File:

$$18\text{KB}$$

Block:

$$4\text{KB}$$

Blocks:

$$\lceil 18/4 \rceil = 5$$

Allocated:

$$20\text{KB}$$

Internal fragmentation:

$$2\text{KB}$$

39. EXACT MULTIPLE CASE

File:

$$16\text{KB}$$

Block:

$$4\text{KB}$$

Blocks:

$$4$$

Waste:

$$16 - 16 = 0$$

Do not automatically assume every file wastes half a block.

$$B/2$$

is only an average estimate under specific assumptions.

PART M — CONTIGUOUS ALLOCATION NUMERICALS

Suppose file begins at disk block:

$$S$$

Then logical file block i :

$$S+i$$

40. EXAMPLE

Start:

$\lfloor 500 \rfloor$

Logical block:

$\lfloor 25 \rfloor$

Physical:

$\lfloor 500+25 = \boxed{525} \rfloor$

assuming zero-based logical numbering.

41. BYTE OFFSET IN CONTIGUOUS FILE

Block size:

$\lfloor 4\text{KB} \rfloor$

File begins physical block:

$\lfloor 100 \rfloor$

Byte offset:

$\lfloor 10,000 \rfloor$

Logical block:

$\lfloor 2 \rfloor$

offset inside block:

$\lfloor 1808 \rfloor$

Physical block:

$\lfloor 100+2 = \boxed{102} \rfloor$

Therefore location:

TEXT

Physical block 102

byte offset 1808 inside that block

PART N — LINKED ALLOCATION NUMERICALS

Suppose block size:

$\lfloor 1\text{KB}=1024\text{B} \rfloor$

Pointer stored inside each block:

$\lfloor 4\text{B} \rfloor$

Actual file-data bytes per block:

$\lfloor 1024-4 = \boxed{1020} \rfloor$

42. NUMBER OF LINKED BLOCKS

File size:

$$\lfloor 10,000B \rfloor$$

Usable bytes/block:

$$\lfloor 1020 \rfloor$$

Blocks:

$$\lfloor \lceil \frac{10000}{1020} \rceil \rfloor$$

Since:

$$\lfloor 1020 \times 9 = 9180 \rfloor$$

not enough.

$$\lfloor 1020 \times 10 = 10200 \rfloor$$

Therefore:

$$\lfloor \boxed{10 \text{ blocks}} \rfloor$$

& pGATE TRAP

For linked allocation where pointer is stored **inside the same block**, use:

$$\lfloor \text{BlockSize} - \text{PointerSize} \rfloor$$

as data capacity.

But if question uses FAT where pointers are stored separately:

Full data block may be usable for file data.

Read architecture carefully.

PART O — INDEXED ALLOCATION NUMERICALS

Suppose one index block contains:

$$\lfloor N \rfloor$$

addresses.

Then one-level indexed allocation can address at most:

$$\lfloor N \rfloor$$

data blocks.

Maximum file data:

$$\lfloor \boxed{N \times \text{BlockSize}} \rfloor$$

43. EXAMPLE

Block:

$\lfloor 1\text{KB} \rfloor$

Pointer:

$\lfloor 4\text{B} \rfloor$

Entries:

$\lfloor 1024/4=256 \rfloor$

Maximum file size with one index block:

$\lfloor 256 \times 1\text{KB} \rfloor$

$\lfloor \boxed{256\text{KB}} \rfloor$

excluding the index block itself from file-data capacity.

44. TOTAL SPACE CONSUMED

A maximum-size 256-KB file in this scheme requires:

TEXT

```
256 data blocks
+
1 index block
```

Total disk blocks:

$\lfloor \boxed{257} \rfloor$

Total physical disk space consumed:

$\lfloor 257\text{KB} \rfloor$

even though user file data is only:

$\lfloor 256\text{KB} \rfloor$

PART P — HARD LINK / INODE COUNT QUESTIONS Ø=Ý%

Suppose:

TEXT

```
a !' i n o d e 5 0
```

Initial hard-link count:

$\lfloor 1 \rfloor$

Create:

TEXT

```
b = hard link to a
```

Now:

$\boxed{\text{LinkCount}=2}$

Create another:

TEXT

c

Now:

$\boxed{3}$

45. DELETE ONE HARD LINK

Delete:

TEXT

a

Link count:

$\rightarrow 2$

The file still exists through:

TEXT

b

c

46. WHEN IS INODE RECLAIMED?

Conceptually, when:

- no directory hard links remain, and
- no relevant open references require it to remain

then inode/data can be reclaimed.

For simplified GATE questions:

link count becomes zero !' file can be deleted.

47. SYMBOLIC LINK EFFECT ON LINK COUNT

Suppose:

TEXT

A !' i n o d e 5 0

Create symbolic link:

TEXT

S ! ' " A "

S is its own file object/inode.

It does not normally increment A's hard-link count.

So if target A had link count 1:

creating symlink does not make it 2 in the hard-link sense.

48. DELETE TARGET WITH SYMLINK

Delete A.

Target inode may be removed when references permit.

The symlink:

TEXT

S

may remain.

But resolving it fails.

That's:

$\boxed{\text{Dangling symbolic link}}$

PART Q — MIXED GATE NUMERICAL Ø=Ý%Ø=Ý%Ø=Ý%

Suppose:

- Disk block size = 2 KB
- Pointer size = 4 bytes
- inode has 8 direct pointers
- 1 single indirect
- 1 double indirect

Find maximum file size.

49. Step 1 — pointers per block

$2\text{KB} = 2048 = 2^{11}$

Pointer:

$4 = 2^2$

Therefore:

$N = 2^{11-2} = 2^9 = 512$

50. Data blocks

Direct:

$$\lfloor 8 \rfloor$$

Single:

$$\lfloor 512 \rfloor$$

Double:

$$\lfloor 512^2 = 262144 \rfloor$$

Total:

$$\lfloor 8 + 512 + 262144 \rfloor$$

$$\lfloor = \boxed{262664} \rfloor$$

blocks.

51. Maximum file size

Each:

$$\lfloor 2\text{KB} \rfloor$$

So:

$$\lfloor 262664 \times 2\text{KB} \rfloor$$

$$\lfloor = \boxed{525328\text{KB}} \rfloor$$

approximately:

$$\lfloor 513.02\text{MB} \rfloor$$

using binary-KB/MB conventions.

For an exam with answer choices, using powers of two may be faster.

PART R — DISK-ACCESS NUMERICAL

Same inode:

TEXT

```
8 direct
1 single
1 double
```

Suppose required logical file block is:

$$\lfloor 700 \rfloor$$

Direct + single capacity:

$$\lfloor 8 + 512 = 520 \rfloor$$

Since:

$\lfloor 700 \geq 520 \rfloor$

it's double indirect.

If inode is already in memory and no indirect blocks cached:

$\lfloor \boxed{\text{disk accesses}} \rfloor$

1. first indirect level
2. second indirect level
3. data block

PART S — FIND EXACT DOUBLE-INDIRECT INDICES

For block:

$\lfloor 700 \rfloor$

Remove direct + single:

$\lfloor r = 700 - 520 = 180 \rfloor$

Since:

$\lfloor N = 512 \rfloor$

First-level index:

$\lfloor \lfloor 180 / 512 \rfloor = 0 \rfloor$

Second-level:

$\lfloor 180 \bmod 512 = 180 \rfloor$

So:

TEXT

Double L1[0]

! "

L2[180]

! "

Data

PART T — BITMAP VS FAT SPACE

Suppose:

- Disk blocks = 2^{20}
- FAT entry = 4 bytes

Bitmap

$\lfloor 2^{20} \rfloor$ bits

$\lfloor = 2^{17} \rfloor$ bytes

$\lfloor = \boxed{128\text{KB}} \rfloor$

FAT

$\lfloor 2^{20} \times 4 = 2^{22} \rfloor$

`\boxed{4MB}`

So bitmap is much smaller for simple allocation-status tracking.

But FAT contains more information than one free/used bit.

52. IMPORTANT CONCEPT

Bitmap answers:

Is this block free/allocated?

FAT can tell:

What is the next block in this file's chain?

So don't compare only by size as though they perform identical tasks.

PART U — DIRECTORY ENTRY VS INODE

Very important GATE distinction:

```
TEXT
Filename
    !"
Directory Entry
    !"
Inode Number
    !"
Inode
    !"
Metadata + Data Pointers
```

Therefore:

- renaming a file generally changes directory-entry name
- inode can remain the same
- hard links create more names to same inode

53. RENAME EXAMPLE

Before:

```
TEXT
a b c . t x t !'   i n o d e   1 0 0
```

Rename:

```
TEXT
x y z . t x t !'   i n o d e   1 0 0
```

The file's underlying inode/data need not change merely because the name changed.

PART V — FILE DESCRIPTOR CONNECTION

After:

```
TEXT
open("xyz.txt")
```

process receives an FD.

Conceptually:

```
TEXT
FD
!"
Per-process open-file information
!"
System open-file/inode information
!"
File
```

Therefore:

File descriptor is not the inode number.

They are different concepts.

& pGATE TRAPS — INODE & FILE NUMERICALS

Trap 1

Single indirect stores:

$\backslash[1\backslash]$

data block.

'L

It stores pointers to:

$\backslash[N\backslash]$

data blocks.

Trap 2

Double indirect gives:

$\backslash[2N\backslash]$

blocks.

'L

$\boxed{N^2}$

Trap 3

Triple indirect gives:

$3N$

blocks.

'L

$\boxed{N^3}$

Trap 4

Index blocks count as file user-data blocks.

'L

They are metadata/pointer blocks.

Trap 5

Maximum file size should include bytes used by indirect pointer blocks.

'L

Maximum **user file size** counts data blocks, not metadata blocks.

Unless question asks total disk space occupied.

Trap 6

For file block 12 with 12 direct pointers, use direct pointer 12.

'L if numbering is zero-based.

Direct blocks are:

$0 \dots 11$

Block 12 is first single-indirect block.

Trap 7

Hard link creates a new inode containing copied data.

'L

Same inode.

Trap 8

Symbolic link and hard link both increase target link count identically.

'L

Trap 9

Bitmap requires one byte per disk block.

'L

Usually one bit per block.

Trap 10

FATsize = number of files × FATentry size.

'L

It depends on allocation units/blocks.

Trap 11

Linked allocation always has full block available for user data.

Not if pointer is stored inside the block.

Check the design.

Trap 12

Byte offset divided by block size should use ceiling.

'L

To locate block:

$\lfloor \text{floor division} \rfloor$

To find number of blocks needed for file size:

$\lceil \text{ceiling} \rceil$

This distinction is extremely important.

' p WRITE THIS IN YOUR NOTES

INODE NUMERICAL FORMULAS

Let:

$B = \text{block size}$

$P = \text{pointer size}$

$N = B/P$

Then:

TEXT

Direct = D blocks
Single = N
Double = N^2
Triple = N^3

Maximum:

$$\lfloor (D+N+N^2+N^3)B \rfloor$$

if one pointer of each indirect type exists.

FILE OFFSET

For byte offset $\lfloor X \rfloor$:

$$\lfloor \text{LogicalBlock} = \left\lfloor \frac{X}{B} \right\rfloor \rfloor$$

$$\lfloor \text{Offset} = X \bmod B \rfloor$$

FILE BLOCK RANGES

Zero-based:

TEXT

Direct:
 $0 \dots D-1$

TEXT

Single:
 $D \dots D+N-1$

TEXT

Double:
 $D+N \dots D+N+N^2-1$

DOUBLE INDIRECT INDEX

$$\lfloor r = b - (D+N) \rfloor$$

$$\lfloor i_1 = \left\lfloor \frac{r}{N} \right\rfloor \rfloor$$

$$\lfloor i_2 = r \bmod N \rfloor$$

DISK ACCESSSES

If inode already in memory and nothing else cached:

TEXT

```
D i r e c t ! ' 1
S i n g l e ! ' 2
D o u b l e ! ' 3
T r i p l e ! ' 4
```

FAT

$\boxed{\text{FATSize} = \frac{\text{DiskSize}}{\text{BlockSize}} \times \text{FATEntrySize}}$

BITMAP

$\boxed{\text{Blocks} = \frac{\text{DiskSize}}{\text{BlockSize}}}$

$\boxed{\text{BitmapBits} = \text{Blocks}}$

$\boxed{\text{BitmapBytes} = \text{Blocks} / 8}$

FILE BLOCKS REQUIRED

$\boxed{\lceil \frac{\text{FileSize}}{\text{BlockSize}} \rceil}$

Internal waste:

$\boxed{\text{Blocks} \times \text{BlockSize} - \text{FileSize}}$

MASTER GATE SHORTCUTS

Shortcut 1 — inode

D, N, N², N³

Shortcut 2 — powers of two

If:

TEXT

```
B l o c k   =   2 G
P o i n t e r =   2 V
```

then:

$\boxed{N = 2^{\{b-p\}}}$

Shortcut 3 — locate byte

Floor finds block; remainder finds byte inside.

Shortcut 4 — blocks required

Ceiling finds number of blocks.

Shortcut 5 — bitmap

One block != one bit.

Shortcut 6 — hard link

More names, same inode.

Shortcut 7 — symlink

Separate shortcut storing/referring to path.

30-SECOND REVISION

Pointers per block?

$\lfloor B/P \rfloor$

Single indirect?

$\lfloor N \rfloor$

Double?

$\lfloor N^2 \rfloor$

Triple?

$\lfloor N^3 \rfloor$

Maximum file size?

!= data blocks $\times$ block size.

Direct block disk access?

!= 1 if inode cached.

Double indirect?

!= 3 if nothing else cached.

Bitmap?

!= 1 bit per disk block.

FAT entries?

!= roughly one per allocation unit.

Hard link?

!' same inode.

Symlink?

!' pathname-based separate object.

Locate byte offset?

!' floor division + modulus.

Blocks needed for whole file?

!' ceiling division.

ACTIVE RECALL

Try these yourself:

1. Block size = 4 KB, pointer size = 8 bytes. Find N .
2. If $N=512$, how many data blocks does double indirect address?
3. If inode has 10 direct pointers, which logical blocks are direct under zero-based numbering?
4. With $D=10$, $N=512$, what is the first double-indirect logical block?
5. If inode is in RAM, how many uncached disk accesses are required for triple indirect?
6. Disk = 16 GB, block = 4 KB. How many disk blocks exist?
7. For Q6, what is bitmap size?
8. A file is 25 KB and block size is 4 KB. How many blocks are needed?
9. What is its final-block internal fragmentation?
10. What happens to inode link count when a new hard link is created?
11. Does a symbolic link normally share the same inode as its target?
12. Why must pointer/index blocks not be included when computing maximum **user data file size**?

Answers to key numericals:

Q1

$\lfloor 4\text{KB}=4096 \rfloor$

$\lfloor 4096/8 = \boxed{512} \rfloor$

Q2

$\lfloor 512^2 = \boxed{262144} \rfloor$

Q3

$\lfloor \boxed{0}\text{ through }9 \rfloor$

Q4

Single handles:

$\lfloor 10\text{ through }10+512-1=521 \rfloor$

So first double-indirect block:

$\lfloor \boxed{522} \rfloor$

Q5

$\boxed{4}$

Q6

$16\text{GB} = 2^{34}$

$4\text{KB} = 2^{12}$

$2^{34-12} = \boxed{2^{22} \text{ blocks}}$

Q7

$2^{22} \text{ bits} = 2^{19} \text{ bytes} = \boxed{512\text{KB}}$

Q8

$\lceil 25/4 \rceil = \boxed{7}$

Q9

Allocated:

$7 \times 4 = 28\text{KB}$

Waste:

$28 - 25 = \boxed{3\text{KB}}$

OPERATING SYSTEMS SYLLABUS — CONTENT COVERAGE COMPLETE '

You have now covered the complete chain from your official syllabus:

TEXT

SYSTEM CALLS

!"

PROCESSES

!"

THREADS

!"

IPC

!"

CONCURRENCY

!"

SYNCHRONIZATION

!"

CLASSICAL SYNCHRONIZATION

!"

DEADLOCKS

!"

CPU SCHEDULING

!"

I/O / DISK SCHEDULING

!"

MEMORY MANAGEMENT

!"

PAGING + TLB

!"

SEGMENTATION

!"

VIRTUAL MEMORY

!"

PAGE REPLACEMENT

! "
THRASHING
! "
FILE SYSTEMS
! "
INODE / FILE ALLOCATION NUMERICALS

NEXT — CHAPTER 19: FINAL GATE OS REVISION Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

The next section should **not introduce another major syllabus topic**. It will consolidate everything into an exam-ready revision pack:

- complete OS formula sheet
- all important definitions in one place
- most important GATE comparison tables
- all algorithms and when to use them
- highest-frequency numerical patterns
- semaphore shortcuts
- deadlock + Banker shortcuts
- CPU scheduling formulas
- disk scheduling shortcuts
- paging/TLB formulas
- virtual-memory + replacement shortcuts
- inode/file-system formulas
- **50+ dangerous GATE traps**
- last-minute mnemonics
- topic priority ranking
- active-recall checklist
- final mixed GATE-style test.

USER
NEXT

ASSISTANT

CHAPTER 19 — FINAL GATE OPERATING SYSTEMS REVISION PACK Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

You have completed the full official syllabus. This chapter is for **compression, revision, traps, formulas, mnemonics, and exam recall**.

Use this before PYQs and again during final revision.

1. COMPLETE OS MASTER MAP

TEXT

OPERATING SYSTEMS

```
%
% % %   S y s t e m   C a l l s
% % %   P r o c e s s e s
%       % % %   S t a t e s
%       % % %   P C B
%       % % %   C o n t e x t   S w i t c h
%       % % %   f o r k   /   e x e c   /   w a i t
%       % % %   Z o m b i e   /   O r p h a n
%
% % %   T h r e a d s
%       % % %   S h a r e d   v s   P r i v a t e
%       % % %   U s e r   v s   K e r n e l   T h r e a d s
%       % % %   M a p p i n g   M o d e l s
%
% % %   I P C
%       % % %   S h a r e d   M e m o r y
%       % % %   M e s s a g e   P a s s i n g
%       % % %   P i p e s
%       % % %   Q u e u e s
%       % % %   S o c k e t s
%
% % %   C o n c u r r e n c y
%       % % %   R a c e   C o n d i t i o n
%       % % %   C r i t i c a l   S e c t i o n
%       % % %   A t o m i c i t y
%       % % %   B u s y   W a i t i n g
%
% % %   S y n c h r o n i z a t i o n
%       % % %   M u t e x
%       % % %   S e m a p h o r e
%       % % %   P r o d u c e r - C o n s u m e r
%       % % %   R e a d e r s - W r i t e r s
%       % % %   D i n i n g   P h i l o s o p h e r s
%
% % %   D e a d   l o c k s
%       % % %   C o f f m a n   C o n d i t i o n s
%       % % %   R A G
%       % % %   P r e v e n t i o n
%       % % %   A v o i d a n c e
%       % % %   B a n k e r ' s   A l g o r i t h m
%       % % %   D e t e c t i o n   +   R e c o v e r y
%
% % %   C P U   S c h e d u l i n g
%       % % %   F C F S
%       % % %   S J F
%       % % %   S R T F
%       % % %   P r i o r i t y
%       % % %   R o u n d   R o b i n
%
% % %   D i s k   S c h e d u l i n g
%       % % %   F C F S
%       % % %   S S T F
%       % % %   S C A N
%       % % %   C - S C A N
%       % % %   L O O K
%       % % %   C - L O O K
%
% % %   M e m o r y   M a n a g e m e n t
%       % % %   B a s e / L i m i t
%       % % %   F r a g m e n t a t i o n
%       % % %   F i r s t / B e s t / W o r s t   F i t
%       % % %   P a g i n g
%       % % %   T L B
%       % % %   S e g m e n t a t i o n
```

```

%
% % %   V i r t u a l   M e m o r y
%       % % %   D e m a n d   P a g i n g
%       % % %   P a g e   F a u l t
%       % % %   F I F O   /   L R U   /   O P T
%       % % %   B e l a d y ' s   A n o m a l y
%       % % %   T h r a s h i n g
%
% % %   F i l e   S y s t e m s
%       % % %   A l l o c a t i o n   M e t h o d s
%       % % %   F A T
%       % % %   i n o d e
%       % % %   D i r e c t / I n d i r e c t   B l o c k s
%       % % %   L i n k s
%       % % %   F r e e - S p a c e   M a n a g e m e n t

```

$$\emptyset = Y\% \emptyset = Y\% \text{ Waste} + Y\%$$

1. Processes + `fork()`
2. Synchronization + Semaphores
3. Deadlocks + Banker's Algorithm
4. CPU Scheduling
5. Paging + Page Tables + TLB
6. Virtual Memory + Page Replacement
7. inode/file-system numericals

$0 = Y\% \text{ of } Y\% \text{ of } Y\% \text{ Important}$

8. Threads
9. Critical section
10. Producer–Consumer
11. Readers–Writers
12. Disk scheduling
13. Fragmentation + allocation
14. Segmentation

Important Concepts

- 15. IPC
- 16. System calls
- 17. File access/directory concepts
- 18. Thrashing/working set

3. PROGRAM VS PROCESS

TEXT

```
P r o g r a m ! '   p a s s i v e   c o d e
P r o c e s s ! '   p r o g r a m   i n   e x e c u t i o n
```

Memory:

Program is stored. Process is running.

4. PROCESS STATES

TEXT

NEW

!"

READY

!"

RUNNING

!"

TERMINATED

From Running:

TEXT

```
R u n n i n g ! '   R e a d y
= Preemption
```

TEXT

```
R u n n i n g ! '   W a i t i n g
= waits for I/O/event
```

TEXT

```
W a i t i n g ! '   R e a d y
= event completes
```

Critical distinction:

Ready = waiting for CPU.

Blocked = waiting for something other than CPU.

5. PCB

PCB contains process information such as:

- PID
- state

- program counter
- registers
- scheduling information
- memory-management information
- I/O information

Memory:

PCB = OS's record of a process.

6. CONTEXT SWITCH

TEXT

Save P1
!"
Load P2

Context switching:

- useful for multitasking
- consumes overhead
- does not perform useful user work itself

7. MODE SWITCH VS CONTEXT SWITCH

Mode Switch	Context Switch
User !" Kernel	P 1 !" P 2
Same process may continue	Execution context changes
Usually lighter	Generally more work

8. `fork()` Ø=Ý%

One unconditional successful `fork()` executed by all existing processes doubles process count.

For $\backslash(n)$ such forks:

$\backslash \boxed{\text{Total}=2^n}$

New children:

$\backslash \boxed{2^{n-1}}$

But only use this when **all processes reach every fork.**

9. `fork()` RETURNS

TEXT

```
C h i l d ! ' 0
P a r e n t ! ' p o s i t i v e c h i l d P I D
F a i l u r e ! ' n e g a t i v e
```

10. exec()

TEXT

```
f o r k ! ' c r e a t e s n e w p r o c e s s
e x e c ! ' r e p l a c e s c u r r e n t p r o c e s s p r o g r a m
```

Memory:

Fork grows family. Exec changes job.

11. ZOMBIE VS ORPHAN

Zombie

Child terminated but parent has not collected exit status.

Dead child, record remains.

Orphan

Parent dies while child continues.

Living child, parent gone.

PART B — THREADS

12. THREAD SHARING

Threads of same process share:

CDHF

- Code
- Data
- Heap
- Files/resources

Each thread has private:

PRS

- Program Counter
- Registers
- Stack

13. PROCESS VS THREAD

Process	Thread
Separate address space	Shares process address space
Heavy	Lighter
IPC needed for data sharing	Shared memory naturally available
Context switch generally heavier	Usually cheaper

14. THREAD MODELS

TEXT

Many-to-One
One-to-One
Many-to-Many

Many-to-One

- many user threads share one kernel thread
- one blocking call may block whole process
- no true multicore parallelism

One-to-One

- each user thread gets kernel thread
- true parallelism possible
- more overhead

PART C — IPC

15. IPC MASTER RULE

Two major approaches:

TEXT

Shared Memory
Message Passing

Memory:

IPC = Share or Send

16. SHARED MEMORY VS MESSAGE PASSING

Shared Memory	Message Passing
Common memory region	send/receive
Usually faster after setup	More kernel involvement
Needs synchronization	Natural isolation
Same-machine use common	Works well distributed too

PART D — CONCURRENCY

17. RACE CONDITION

Race condition occurs when:

multiple processes/threads access shared data concurrently, at least one modifies it, and result depends on execution order.

Classic:

TEXT

```
x = x + 1
```

is really:

TEXT

```
Read  
Modify  
Write
```

Memory:

RMW

18. CRITICAL SECTION REQUIREMENTS

MPB

1. Mutual Exclusion
2. Progress
3. Bounded Waiting

Memory:

19. MEANING

Mutual Exclusion

At most one competing process in CS.

Progress

If CS free and processes want entry, decision cannot be postponed forever.

Bounded Waiting

A requesting process eventually gets its turn.

20. STARVATIONVS DEADLOCK

TEXT

Starvation

! ' one may wait forever while others progress

TEXT

Deadlock

! ' involved processes cannot progress

21. BUSY WAITING

Process continuously tests:

TEXT

```
while(lock)
    ;
```

Consumes CPU.

Blocking instead puts process to sleep/waiting state.

PART E — SEMAPHORES Ø=Ý%Ø=Ý%Ø=Ý%

22. SEMAPHORE OPERATIONS

TEXT

```
wait = P = down
signal = V = up
```

Semaphore operations must be:

$\boxed{\text{Atomic}}$

23. THREE MAGIC INITIAL VALUES

Mutual Exclusion

$\boxed{\text{mutex}=1}$

Event Ordering

$\boxed{s=0}$

N identical resources

$\boxed{s=N}$

Master memory:

1 protects, 0 orders, N counts.

24. MUTEX PATTERN

```
C
wait(mutex);

/* critical section */

signal(mutex);
```

25. ORDERING PATTERN

Need:

```
TEXT
A before B
```

Use:

```
TEXT
s = 0
```

P1:

```
C
A;
signal(s);
```

P2:

```
C
wait(s);
B;
```

Therefore:

$\{ A \rightarrow B \}$

26. BLOCKING SEMAPHORE CONVENTION

Classic:

```
TEXT
wait:
S--
if S < 0 !' b l o c k
```

```
TEXT
signal:
S++
if S <= 0 !' w a k e o n e
```

Under this definition:

$\{ S = -k \}$

can mean:

$\{ \boxed{k} \text{ waiting processes} \}$

Follow the exact pseudocode given in GATE.

PART F — PRODUCER CONSUMER

27. MAGIC VALUES

For buffer size $\{N\}$:

$\{ \boxed{\text{mutex}=1} \}$

$\{ \boxed{\text{empty}=N} \}$

$\{ \boxed{\text{full}=0} \}$

Memory:

One lock, N empty, zero full.

28. PRODUCER

TEXT

```
wait(empty)
wait(mutex)
```

INSERT

```
signal(mutex)
signal(full)
```

29. CONSUMER

TEXT

```
wait(full)
wait(mutex)
```

REMOVE

```
signal(mutex)
signal(empty)
```

30. WHY COUNT BEFORE MUTEX?

Wrong:

TEXT

```
wait(mutex)
wait(empty)
```

may deadlock if buffer is full.

Memory:

Availability before exclusivity.

31. BUFFER RELATION

At stable points:

$\boxed{\text{empty} + \text{full} = N}$

PART G — READERS WRITERS

32. MASTER RULE

Many readers OR one writer.

TEXT

```
Reader + Reader '
Reader + Writer 'L
Writer + Writer 'L
```

33. READER PREFERENCE

TEXT

```
First reader !' wait(wrt)
Last reader !' signal(wrt)
```

Memory:

First locks, last unlocks.

Risk:

$\boxed{\text{Writer starvation}}$

PART H — DINING PHILOSOPHERS

34. DEADLOCK

If all philosophers acquire one fork and wait for another:

TEXT

Everyone holds one
Everyone wants one

!' circular wait.

35. PREVENTION

Examples:

- permit at most $(N-1)$ philosophers
- global resource ordering
- asymmetric acquisition
- acquire both forks atomically

PART I — DEADLOCKS $\emptyset = \dot{Y}\% \emptyset = \dot{Y}\% \emptyset = \dot{Y}\%$

36. FOUR COFFMAN CONDITIONS

MHNC

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

Memory:

My Hungry Neighbor Circles

All four are necessary.

37. RAG EDGES

TEXT

P !' R
= request

TEXT

R !' P
= allocation

38. CYCLE RULE

One instance of each resource

$\neg [\text{Cycle} \wedge \text{Deadlock}]$

Multiple resource instances

Cycle:

necessary but not sufficient.

39. PREVENTION

Break one Coffman condition.

TEXT

Mutual exclusion !' make resource shareable where possible
Hold-and-wait !' request all together
No preemption !' allow resource preemption
Circular wait !' impose total resource ordering

40. PREVENTION VS AVOIDANCE

Prevention

structurally break deadlock condition.

Avoidance

grant resources only if state remains safe.

Example:

$\boxed{\text{Banker's Algorithm}}$

41. SAFE STATE

Safe state:

At least one safe completion sequence exists.

Unsafe state:

No safe sequence exists.

Critical:

$\boxed{\text{Unsafe} \neq \text{Deadlocked}}$

But:

$\boxed{\text{Deadlocked} \rightarrow \text{Unsafe}}$

42. BANKER'S MASTER FORMULA

$\boxed{\text{Need} = \text{Max} - \text{Allocation}}$

43. BANKER SAFETY METHOD

Start:

$\boxed{\text{Work} = \text{Available}}$

Find any process where:

$\boxed{\text{Need}_i \leq \text{Work}}$

Then:

$\boxed{\text{Work} = \text{Work} + \text{Allocation}_i}$

Repeat.

If all finish:

SAFE.

Memory:

NFF

Need Fits !' Finish !' Free

44. NEW RESOURCE REQUEST

Check:

$\lceil \text{Request}_i \rceil \leq \text{Need}_i$

and:

$\lceil \text{Request}_i \rceil \leq \text{Available}$

Then pretend to grant and test safety.

45. MINIMUM IDENTICAL RESOURCES

For n processes each requiring at most k identical resources:

$\boxed{n(k-1)+1}$

guarantees no deadlock under classic assumptions.

General:

$\boxed{\sum_i (m_i - 1) + 1}$

for one identical resource type.

PART J — CPU SCHEDULING Ø=Ý%Ø=Ý%Ø=Ý%

46. MASTER FORMULAS

$\boxed{TAT = CT - AT}$

$\boxed{WT = TAT - BT}$

$\boxed{RT = \text{FirstStart} - AT}$

47. MEMORY

TEXT

AT = Arrival Time
BT = Burst Time
CT = Completion Time
TAT = Turnaround Time
WT = Waiting Time
RT = Response Time

48. GANTT METHOD

Always:

1. draw Gantt chart
2. find CT
3. calculate TAT
4. calculate WT
5. calculate RT separately

Memory:

Gantt !' CT !' TAT!' WT

49. ALGORITHM TABLE

Algorithm	Type	Rule	Problem
FCFS	Non-preemptive	first arrival	convoy
SJF	Non-preemptive	shortest burst	starvation
SRTF	Preemptive	shortest remaining	starvation
Priority	Either	best priority	starvation
RR	Preemptive	quantum rotation	switch overhead

50. FCFS

Associated with:

$\boxed{\text{Convoy Effect}}$

51. SJF

Classic property:

Minimizes average waiting time when burst times are known under standard assumptions.

52. SRTF

Preemptive SJF.

Always compare:

$\boxed{\text{Remaining Time}}$

not original burst.

53. PRIORITY

Check whether:

TEXT

smaller number = higher priority

or reverse.

Never assume silently.

Starvation remedy:

$\boxed{\text{Aging}}$

54. ROUND ROBIN

Quantum:

q

Large:

$RR \rightarrow FCFS$

Tiny:

many context switches.

Number of slices for burst B :

$\boxed{\left\lceil \frac{B}{q} \right\rceil}$

55. THROUGHPUT

$\boxed{\text{Throughput} = \frac{\text{Completed Processes}}{\text{Time}}}$

CPU utilization:

$\boxed{\frac{\text{BusyTime}}{\text{TotalTime}} \times 100}$

PART K — DISK SCHEDULING

56. HEAD MOVEMENT

$\boxed{\text{Total} = \sum | \text{Next} - \text{Current} |}$

57. MASTER 2x2 TABLE

	Both Directions	One Direction
Goes to physical endpoint	SCAN	C-SCAN
Stops at extreme request	LOOK	C-LOOK

Ø=Memorize.

58. ALGORITHMS

FCFS

Arrival order.

SSTF

Nearest cylinder.

Risk:

$\boxed{\text{Starvation}}$

SCAN

Touch wall, reverse.

C-SCAN

Touch wall, jump to other wall, same service direction.

LOOK

Reverse at last request, not physical wall.

C-LOOK

Jump extreme-request to opposite extreme-request.

59. DISK ACCESS FORMULA

$\boxed{\text{AccessTime} = \text{SeekTime} + \text{RotationalLatency} + \text{TransferTime}}$

At RPM:

$\boxed{\text{RotationTime} = \frac{60}{\text{RPM}} \text{ seconds}}$

Average rotational latency:

$\boxed{\frac{\text{RotationTime}}{2}}$

PART L — MEMORY MANAGEMENT

60. LOGICAL VS PHYSICAL

TEXT

Logical = CPU-generated
Physical = RAM address

MMU:

TEXT

L o g i c a l ! ' M M U ! ' P h y s i c a l

61. BASE/LIMIT

Valid if:

$\lceil LA < Limit \rceil$

Then:

$\lceil \boxed{PA = Base + LA} \rceil$

62. FRAGMENTATION

Internal

Waste **inside** allocated block.

Memory:

IN = INside

External

Free memory exists as scattered holes.

63. FIXED VS VARIABLE

TEXT

Fixed partition
! ' I n t e r n a l f r a g m e n t a t i o n

TEXT

Variable contiguous allocation
! ' E x t e r n a l f r a g m e n t a t i o n

64. FIT ALGORITHMS

First Fit

First sufficient hole.

Best Fit

Smallest sufficient hole.

Worst Fit

Largest hole.

Never say Best Fit is always globally best.

65. COMPACTION

Combines scattered holes.

Helps:

$\boxed{\text{External fragmentation}}$

PART M — PAGING Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

66. CORE RULE

TEXT

V i r t u a l m e m o r y ! ' P a g e s
P h y s i c a l m e m o r y ! ' F r a m e s

$\boxed{\text{PageSize=FrameSize}}$

67. LOGICAL ADDRESS

TEXT

Page Number | Offset

Physical:

TEXT

Frame Number | Offset

Offset:

$\boxed{\text{unchanged}}$

68. TRANSLATION

For logical address $\backslash(LA\backslash)$, page size $\backslash(S\backslash)$:

$$\lfloor \boxed{\text{Page} = \left\lfloor \frac{LA}{S} \right\rfloor} \rfloor$$

$$\lfloor \boxed{\text{Offset} = LA \bmod S} \rfloor$$

If page maps to frame $\lfloor f \rfloor$:

$$\lfloor \boxed{PA = fS + \text{Offset}} \rfloor$$

69. OFFSET BITS

If page size:

$$\lfloor 2^n \rfloor$$

bytes:

$$\lfloor \boxed{\text{OffsetBits} = n} \rfloor$$

70. ADDRESS BITS

If virtual address = $\lfloor m \rfloor$ bits:

$$\lfloor \boxed{\text{PageBits} = m - n} \rfloor$$

If physical address = $\lfloor p \rfloor$ bits:

$$\lfloor \boxed{\text{FrameBits} = p - n} \rfloor$$

71. PAGES / FRAMES

$$\lfloor \boxed{\text{VirtualPages} = \frac{\text{VirtualAddressSpace}}{\text{PageSize}}} \rfloor$$

$$\lfloor \boxed{\text{PhysicalFrames} = \frac{\text{PhysicalMemory}}{\text{PageSize}}} \rfloor$$

72. PAGE TABLE SIZE

$$\lfloor \boxed{\text{PTSize} = \text{NumberOfVirtualPages} \times \text{PTEsize}} \rfloor$$

Normal page table:

one entry per virtual page.

73. MULTILEVEL PAGING

With $\lfloor k \rfloor$ levels and no TLB:

$$\lfloor \boxed{k + 1 \text{ memory accesses}} \rfloor$$

74. INDEX BITS PER PAGE-TABLE LEVEL

$$\lfloor \boxed{\log_2 \left(\frac{\text{PageSize}}{\text{PTEsize}} \right)} \rfloor$$

if each page-table level itself occupies one page.

PART N — TLB Ø=Ý%Ø=Ý%Ø=Ý%

75. TLB

Fast cache of recent Page !' Frame translations.

Memory:

Page table = book. TLB = bookmark.

76. HIT/MISS

TLB Hit

Translation found.

TLB Miss

Translation must be fetched from page table.

Important:

$\boxed{\text{TLB miss} \neq \text{Page fault}}$

77. BASIC EAT

Let:

- TLB time = t
- memory = m
- hit ratio = h

Hit:

$t+m$

Miss:

$t+2m$

Thus:

$\boxed{EAT = h(t+m) + (1-h)(t+2m)}$

under standard serial lookup assumptions.

Universal rule:

$\boxed{\sum \text{Probability} \times \text{Cost}}$

78. PAGE SIZE TRADE-OFF

Large pages:

- fewer pages
- smaller page table
- greater internal fragmentation
- larger TLB reach

Small pages:

- larger page table
- lower internal fragmentation

PART O — SEGMENTATION

79. SEGMENT ADDRESS

$\lfloor \boxed{(\text{Segment}, \text{Offset})} \rfloor$

Segment table stores:

TEXT
Base
Limit

Check:

$\lfloor \text{Offset} < \text{Limit} \rfloor$

Then:

$\lfloor \boxed{PA = \text{Base} + \text{Offset}} \rfloor$

80. PAGING VS SEGMENTATION

Paging	Segmentation
Fixed size	Variable size
Physical organization	Logical organization
Internal fragmentation	External fragmentation
Page Frame	Base+Limit

Memory:

Paging cuts equally. Segmentation cuts meaningfully.

PART P — VIRTUAL MEMORY

81. DEMAND PAGING

Bring page only when demanded.

82. PAGE FAULT

Occurs when:

Valid referenced page is absent from RAM.

Fault " necessarily illegal access.

83. FAULT HANDLING

TEXT

Reference

!"

Not present

!"

Trap OS

!"

Validate

!"

Find free frame / victim

!"

Load page

!"

Update page table

!"

Restart instruction

84. TLB MISS VS PAGE FAULT

TEXT

TLB miss

= map missing

TEXT

Page fault

= page missing from RAM

85. LOCALITY

Temporal

Recently used !' likely reused.

Spatial

Nearby address !' likely used soon.

Memory:

Temporal = Time

Spatial = Space

86. DIRTY BIT

TEXT

```
0 !' u n c h a n g e d
1 !' m o d i f i e d
```

Dirty victim usually must be written back.

PART Q — PAGE REPLACEMENT Ø=Ý%Ø=Ý%Ø=Ý%

87. MASTER TABLE

FIFO	LRU	OPT
Oldest arrival	Least recently used	Farthest future use
Entry history	Past usage	Future usage
Belady possible	No Belady	No Belady
Not stack	Stack	Stack

88. FIFO

On hit:

Do not change order.

This is a major trap.

89. LRU

On hit:

Update recency.

Tracks:

TEXT

L e a s t R e c e n t ! ' M o s t R e c e n t

90. OPT

At fault:

look to future and replace page whose next use is farthest.

If never used again:

excellent victim.

OPT has minimum possible faults.

91. BELADY'S ANOMALY

More frames cause more faults.

Possible with:

$\boxed{\text{FIFO}}$

Not with:

$\boxed{\text{LRU}, \text{OPT}}$

92. SECOND CHANCE

FIFO + reference bit.

TEXT

R = 0 ! ' r e p l a c e
R = 1 ! ' c l e a r a n d s k i p

Approximation of LRU.

PART R — THRASHING

93. THRASHING

TEXT

F r a m e s / p r o c e s s ! "
P a g e f a u l t s ! '
D i s k p a g i n g ! '
C P U u t i l i z a t i o n ! "

Memory:

FPC

- Frames !“
- Page faults !‘
- CPU utilization !“

94. CURE

- give more frames
- reduce degree of multiprogramming

Counterintuitive:

During thrashing, fewer active processes may improve CPU utilization.

95. WORKING SET

Pages referenced in recent window Δ .

For process i :

WSS_i

Total demand:

$D = \sum WSS_i$

If:

$D > M$

physical frames insufficient.

Reduce multiprogramming.

96. PAGE FAULT FREQUENCY

High PFF:

give more frames.

Low PFF:

reclaim frames.

Memory:

High faults? Feed frames.

PART 5 — FILE SYSTEMS

97. ALLOCATION METHODS

Contiguous	Linked	Indexed
Consecutive blocks	Block chain	Index block
Great random access	Poor random access	Good random access
Growth hard	Growth easy	Growth flexible
External fragmentation	Pointer overhead	Index overhead

98. CONTIGUOUS

Physical block:

$\lfloor \boxed{\text{Start} + \text{LogicalBlock}} \rfloor$

99. FAT

Linked allocation where next-block information is stored in a central table.

FAT size:

$\lfloor \boxed{\# \text{Blocks} \times \text{FATEntrySize}} \rfloor$

100. INODE

Conceptually stores:

- metadata
- direct pointers
- indirect pointers

Directory:

TEXT

Filename !' inode number

Important:

Filename is generally a directory-entry concept, not the core inode identity.

101. INDIRECT POINTER FORMULAS $\emptyset = \text{Y}\% \emptyset = \text{Y}\% \emptyset = \text{Y}\%$

Let:

$\backslash B = \text{block size} \backslash$

$\backslash P = \text{pointer size} \backslash$

Then:

$\backslash \boxed{N = B/P} \backslash$

Direct

$\backslash D \backslash$

blocks.

Single

$\backslash N \backslash$

Double

$\backslash N^2 \backslash$

Triple

$\backslash N^3 \backslash$

102. MAX FILE SIZE

If inode contains one single, one double, one triple:

$\backslash \boxed{(D + N + N^2 + N^3) \times B} \backslash$

103. DISK ACCESSES

If inode is already in RAM and nothing else cached:

TEXT

D i r e c t	!	1
S i n g l e	!	2
D o u b l e	!	3
T r i p l e	!	4

104. FILE OFFSET

For byte offset $\backslash(X)\backslash$:

$\backslash \boxed{\text{Block} = \lfloor X/B \rfloor} \backslash$

$\backslash \boxed{\text{Offset} = X \bmod B} \backslash$

Remember:

Floor locates a byte. Ceiling allocates a file.

105. FILE BLOCKS NEEDED

$$\lceil \frac{\text{FileSize}}{\text{BlockSize}} \rceil$$

Internal waste:

$$\text{Blocks} \times \text{BlockSize} - \text{FileSize}$$

106. BITMAP

One bit per disk block.

$$\text{BitmapBits} = \frac{\text{DiskSize}}{\text{BlockSize}}$$

In bytes:

$$\text{BitmapBytes} = \frac{\text{DiskSize}}{8 \times \text{BlockSize}}$$

107. HARD LINK VS SYMBOLIC LINK

Hard Link

Another directory name for same inode.

Symbolic Link

Separate object storing/referring to pathname.

Delete original:

TEXT

```
Hard link !' still works  
Symlink !' may become dangling
```

PART T — MASTER FORMULA SHEET

CPU Scheduling

$$\text{TAT} = \text{CT} - \text{AT}$$

$$\text{WT} = \text{TAT} - \text{BT}$$

$$\text{RT} = \text{FirstStart} - \text{AT}$$

Throughput

$$\frac{\text{Completed}}{\text{Time}}$$

CPU Utilization

$$\boxed{\frac{\text{Busy}}{\text{Total}} \times 100}$$

Deadlock Banker

$$\boxed{\text{Need} = \text{Max} - \text{Allocation}}$$

$$\boxed{\text{Need}_i \leq \text{Work}}$$

$$\boxed{\text{Work} = \text{Work} + \text{Allocation}_i}$$

Minimum identical resources

$$\boxed{n(k-1)+1}$$

Disk Movement

$$\boxed{\sum |\text{Next} - \text{Current}|}$$

Rotation

$$\boxed{T_{\text{rev}} = \frac{60}{\text{RPM}}}$$

$$\boxed{\text{AvgRotLatency} = \frac{T_{\text{rev}}^2}{2}}$$

Base relocation

$$\boxed{PA = \text{Base} + LA}$$

if:

$$\boxed{LA < \text{Limit}}$$

Paging

$$\boxed{\text{Page} = \lfloor LA / S \rfloor}$$

$$\boxed{\text{Offset} = LA \bmod S}$$

$$\boxed{PA = \text{Frame} \times S + \text{Offset}}$$

Page-table size

$$\boxed{PTSize = \#Pages \times PTESize}$$

TLB

$$\boxed{EAT = \sum \text{Probability} \times \text{Cost}}$$

Page fault

Simplified:

$$\boxed{EAT = (1-p)m + pF}$$

inode

$$\boxed{N = \frac{\text{BlockSize}}{\text{PointerSize}}}$$

$$\boxed{\text{MaxBlocks} = D + N + N^2 + N^3}$$

Bitmap

$$\boxed{\text{BitmapBits} = \frac{\text{DiskSize}}{\text{BlockSize}}}$$

PART U — 50 DANGEROUS GATE TRAPS

1. Ready "" blocked.
2. Mode switch "" context switch.
3. `exec()` does not create another process.
4. Do not blindly use (2^n) for conditional forks.
5. Threads share heap but not stack.
6. Concurrency "" parallelism.
7. `x++` is not necessarily atomic.
8. Mutual exclusion alone does not guarantee fairness.
9. Test-and-Set does not necessarily guarantee bounded waiting.
10. Simple semaphore `wait()` may busy-wait or block depending implementation.
11. Semaphore value may be negative under some definitions.
12. Binary semaphore "" mutex in every conceptual sense.
13. Semaphore `wait()` "" Unix parent `wait()`.
14. Producer must check `empty` before acquiring mutex in classical solution.
15. Consumer must check `full` before mutex.
16. Reader-priority can starve writers.
17. Deadlock-free "" starvation-free.
18. All four Coffman conditions must coexist for deadlock.
19. RAG cycle guarantees deadlock only for single-instance case.
20. Unsafe "" already deadlocked.
21. Banker is avoidance, not prevention.
22. `Need = Max - Allocation`, not reverse.
23. Safety algorithm compares Need with Work, not Allocation.
24. Finishing process releases Allocation, not Need.
25. SJF normally means non-preemptive.

26. SRTF compares remaining burst.
27. Never schedule a process before it arrives.
28. Don't forget CPU idle intervals.
29. $(TAT \neq CT)$ when $AT = 0$.
30. Response time $\neq$ total waiting time.
31. Priority convention may reverse.
32. Large RR quantum makes RR FCFS-like.
33. SCAN reaches physical endpoint; LOOK doesn't.
34. C-SCAN services only one direction.
35. SSTF may starve distant requests.
36. Internal fragmentation $\neq$ external fragmentation.
37. Compaction helps external, not internal fragmentation.
38. Paging has no classic external fragmentation but can have internal.
39. Page size = frame size.
40. Offset remains unchanged in paging.
41. Normal page-table entries correspond to virtual pages, not frames.
42. TLB miss $\neq$ page fault.
43. Multilevel paging saves table memory but increases translation accesses.
44. Segmentation uses base + limit.
45. Check segment offset before adding base.
46. FIFO hit does not change FIFO order.
47. Belady's anomaly can occur with FIFO.
48. More frames cannot increase LRU/OPT faults.
49. inode indirect blocks contain pointers, not user data.
50. Bitmap is one **bit**, not one byte, per block.

PART V — ULTRA-SHORT MNEMONICS

Process states

Ready waits CPU; Blocked waits event.

Thread sharing

CDHF shared, PRS private

IPC

Share or Send

Critical section

MPB

Semaphore

1 protects, 0 orders, N counts

Producer–Consumer

MEF = 1, N, 0

Readers–Writers

First locks, last unlocks

Dining Philosophers

Everyone takes one !' nobody gets two

Deadlock

MHNC

Banker

NFF — Need Fits, Finish, Free

Scheduling

Gantt !' CT !' TAT !' WT

Schedulers

First, Short, Shorter Remaining, Priority, Rotate

Disk

SCAN wall, LOOK request

Fragmentation

IN = inside

Paging

Page! Frame offset stays

TLB

Book vs Bookmark

Locality

Temporal = Time, Spatial = Space

Page replacement

FIFO Arrival, LRU Past, OPT Future

Thrashing

Frames ! " Faults ! " ,CPU ! "

inode

D, N, N^2 , N^3

PART W — 30-QUESTION FINAL ACTIVE-RECALL TEST

Do these without looking at answers.

Q1

What is the difference between a program and a process?

Q2

Which process state waits for CPU?

Q3

What information is stored in PCB?

Q4

Three unconditional fork calls produce how many total processes?

Q5

What do threads of one process share?

Q6

What is a race condition?

Q7

State MPB.

Q8

What should semaphore initial value be for mutual exclusion?

Q9

For bounded buffer size 10, initialize `mutex`, `empty`, `full`.

Q10

What is the writer-starvation risk in reader-preference?

Q11

State all four Coffman conditions.

Q12

When does a RAG cycle guarantee deadlock?

Q13

What is the formula for Need?

Q14

What condition is checked in Banker's safety algorithm?

Q15

Write TAT, WT and RT formulas.

Q16

Which CPU algorithm suffers convoy effect?

Q17

What is SRTF?

Q18

What happens to RR as quantum becomes very large?

Q19

Difference between SCAN and LOOK?

Q20

Internal vs external fragmentation?

Q21

Page size 8 KB: offset bits?

Q22

32-bit VA, 4-KB pages: page number bits?

Q23

What remains unchanged when converting a virtual address to a physical address in paging?

Q24

Difference between TLB miss and page fault?

Q25

Which page-replacement algorithm can show Belady's anomaly?

Q26

What property prevents LRU from showing Belady's anomaly?

Q27

What happens to CPU utilization during severe thrashing?

Q28

What does inode contain?

Q29

If block=4KB and pointer=4 bytes, how many addresses fit in one indirect block?

Q30

Difference between hard link and symbolic link?

ANSWERS

A1

Program is passive code; process is program in execution.

A2

$\boxed{\text{Ready}}$

A3

PID, state, PC, registers, scheduling, memory and I/O information, etc.

A4

$2^3 = \boxed{8}$

A5

Code, data, heap, files/resources.

Private: PC, registers, stack.

A6

Outcome depends on timing/interleaving of concurrent access to shared data.

A7

TEXT

Mutual Exclusion
Progress
Bounded Waiting

A8

$\boxed{}$

A9

$\boxed{\text{mutex}=1, \text{empty}=10, \text{full}=0}$

A10

A continuously arriving stream of readers may indefinitely delay writer.

A11

TEXT

Mutual Exclusion
Hold and Wait
No Preemption
Circular Wait

A12

When each resource type has one instance.

A13

$\boxed{\text{Need}=\text{Max}-\text{Allocation}}$

A14

$\boxed{\text{Need}_i \leq \text{Work}}$

A15

$\boxed{\text{TAT}=\text{CT}-\text{AT}}$

$\boxed{\text{WT}=\text{TAT}-\text{BT}}$

$\boxed{\text{RT}=\text{FirstStart}-\text{AT}}$

A16

$\boxed{\text{FCFS}}$

A17

Preemptive version of SJF using shortest remaining time.

A18

$\boxed{RR \rightarrow FCFS}$

A19

SCAN reaches physical disk end; LOOK reverses at extreme pending request.

A20

Internal = wasted space inside allocation. External = free space fragmented into holes.

A21

$8KB = 2^{13}$

$\boxed{13}$

A22

$32 - 12 = \boxed{20}$

A23

$\boxed{\text{Offset}}$

A24

TLB miss = translation absent from TLB. Page fault = page absent from RAM.

A25

$\boxed{\text{FIFO}}$

A26

$\boxed{\text{Stack property}}$

A27

$\boxed{\text{It falls}}$

A28

Metadata + pointers to data blocks.

A29

$4096/4 = \boxed{1024}$

A30

Hard link = another directory name for same inode. Symlink = separate pathname-based link.

PART X — FINAL 10 NUMERICAL CHECKS

1. Scheduling

Process:

TEXT

AT=3

BT=6

First start=5
CT=12

$$\lfloor RT=5-3=\boxed{2} \rfloor$$

$$\lfloor TAT=12-3=\boxed{9} \rfloor$$

$$\lfloor WT=9-6=\boxed{3} \rfloor$$

2. Banker

Max:

$$\lfloor (7,5,3) \rfloor$$

Allocation:

$$\lfloor (2,1,1) \rfloor$$

Need:

$$\lfloor \boxed{(5,4,2)} \rfloor$$

3. Paging

Page size:

$$\lfloor 4KB \rfloor$$

Logical address:

$$\lfloor 9000 \rfloor$$

Page:

$$\lfloor \lfloor 9000/4096 \rfloor = \boxed{2} \rfloor$$

Offset:

$$\lfloor 9000-8192=\boxed{808} \rfloor$$

4. Address bits

VA = 36-bit.

Page size:

$$\lfloor 8KB=2^{\{13\}} \rfloor$$

VPN bits:

$$\lfloor 36-13=\boxed{23} \rfloor$$

5. Page table

36-bit VA, 8KB pages, 8-byte PTE.

Pages:

$$\lfloor 2^{23} \rfloor$$

Table:

$$\lfloor 2^{23} \times 2^3 = 2^{26} \rfloor$$

$$\lfloor \boxed{64\text{MB}} \rfloor$$

6. TLB

TLB = 10ns, Memory = 100ns, hit = 90%.

$$\lfloor \text{EAT} = .9(110) + .1(210) \rfloor$$

$$\lfloor = \boxed{120\text{ns}} \rfloor$$

7. Deadlock resources

5 processes each require at most 4 identical resources.

Minimum guarantee:

$$\lfloor 5(4-1) + 1 \rfloor$$

$$\lfloor = \boxed{16} \rfloor$$

8. Bitmap

Disk:

$$\lfloor 8\text{GB} = 2^{33} \rfloor$$

Block:

$$\lfloor 4\text{KB} = 2^{12} \rfloor$$

Blocks:

$$\lfloor 2^{21} \rfloor$$

Bitmap:

$$\lfloor 2^{21} \text{ bits} = 2^{18} \text{ bytes} = \boxed{256\text{KB}} \rfloor$$

9. inode

4KB block, 4B pointer.

$$\lfloor N = 4096/4 = \boxed{1024} \rfloor$$

Double indirect:

$$\lfloor 1024^2 = \boxed{1,048,576 \text{ blocks}} \rfloor$$

10. File allocation

File = 18KB, block = 4KB.

Blocks:

$\lceil \frac{18}{4} \rceil = 5$

Waste:

$20 - 18 = 2 \text{ KB}$

FINAL 5-MINUTE OS REVISION SHEET

If you have only 5 minutes before a test, revise this:

TEXT

PROCESS

Ready = waits CPU

Blocked = waits event

fork !' process creation

exec !' replace program

THREAD

CDHF shared

PRS private

CRITICAL SECTION

MPB

SEMAPHORE

1 = mutex

0 = ordering

N = resources

PRODUCER CONSUMER

mutex=1

empty=N

full=0

DEADLOCK

MHNC

Need = Max" Allocation

Need "d Work

Work += Allocation

CPU

TAT = CT" AT

WT = TAT" BT

RT = FirstStart" AT

FCFS!' convoy

SJF!' shortest

SRTF!' remaining

Priority!' aging

RR large q!' FCFS

DISK

SCAN!' wall

LOOK!' last request

C!' one direction

MEMORY

Internal=inside

External=holes

Paging!' pages / frames

Page=LA/PageSize

Offset=LA%PageSize

```
TLB
miss "` page fault

SEGMENT
check offset<limit
PA=base+offset

VM
Demand paging
FIFO!' arrival
LRU!' past
OPT!' future
Belady!' FIFO

THRASHING
Frames!" Faults!' CPU!"

FILES
Contiguous / Linked / Indexed
inode = metadata + pointers
N=block/pointer
Single=N
Double=N2
Triple=N3
Bitmap=1 bit/block
Hard=same inode
Symlink=path
```

' OPERATING SYSTEMS — GATE SYLLABUS COMPLETED

You have now covered:

System calls, processes, threads, inter-process communication, concurrency and synchronization, deadlocks, CPU and I/O scheduling, memory management, virtual memory, and file systems.

The complete conceptual teaching phase is finished.

The most productive next stage is **PYQ-oriented practice**, because OS questions are now mainly about converting these concepts into fast exam decisions and numericals.

A good revision order is:

```
TEXT
Semaphores
!' Dead lock
!' CPU Scheduling
!' Paging / TLB
!' Page Replacement
!' inode
!' Processes / fork
!' remaining conceptual topics
```

That sequence covers a large portion of the calculation-heavy and trap-heavy OS material.

USER

NEXT

ASSISTANT

CHAPTER 20 — GATE OS PYQ-ORIENTED PROBLEM SOLVING

Now that the syllabus is complete, the next phase is:

Concept !' GATE Question Pattern !' Fast Solving Method

This chapter focuses on the **most repeated styles of OS questions** and teaches you how to identify them quickly.

1. THE 10 MOST IMPORTANT OS QUESTION PATTERNS

For GATE, heavily prioritize:

1. `fork()` process-count questions
2. Semaphore ordering/output questions
3. Critical-section/race-condition questions
4. Deadlock/RAG questions
5. Banker's Algorithm
6. CPU Scheduling
7. Paging/address translation
8. TLB effective access time
9. Page replacement
10. inode/file-system numericals

Think:

TEXT

Processes

!"

Synchronization

!"

Deadlock

!"

Scheduling

!"

Memory

!"

Files

PART A — `fork()` QUESTIONS

2. BASIC PATTERN


```
c
fork();
fork();
fork();
```

If all `fork()` calls succeed and every process reaches every call:

$2^3 = 8$

total processes.

3. BUT GATE USUALLY MAKES IT TRICKIER

Example:

```
c
if (fork() == 0)
    fork();
```

Do **not** use:

$2^2 = 4$

blindly.

Trace which process actually executes the second `fork()`.

4. SOLVE IT

Initially:

```
TEXT
P0
```

First:

```
c
fork()
```

creates:

```
TEXT
Parent P0
Child P1
```

Condition:

```
c
fork() == 0
```

is true only in:

child P1

So only P1 executes the inner `fork()`.

P1 creates P2.

Total:

$\boxed{3}$

MEMORY RULE

Count processes reaching each fork, not forks appearing in code.

5. TRICKY EXAMPLE

```
c
fork();

if (fork())
    fork();
```

Let's trace.

After first `fork()`:

$\boxed{2}$

processes.

Both execute second `fork()`.

Each produces parent + child:

$\boxed{4}$

processes now.

But:

```
c
if(fork())
```

is true only for the two parents created by that second fork.

Therefore:

$\boxed{2}$

processes execute final `fork()`.

Each creates one child.

Total:

$\boxed{4+2=6}$

FAST METHOD

For each fork:

TEXT

How many processes reach it?

If $\forall(k)$ reach it:

it creates $\forall(k)$ new processes.

Then:

$\forall[\text{NewTotal} = \text{OldTotal} + k]$

PART B — SEMAPHORE ORDERING QUESTIONS $\emptyset = \acute{Y} \% \emptyset = \acute{Y} \% \emptyset = \acute{Y} \%$

6. CORE PATTERN

Suppose:

TEXT

$S = 0$

P1:

C

A;
signal(S);

P2:

C

wait(S);
B;

Guaranteed:

$\forall[\boxed{A \rightarrow B}]$

7. MULTIPLE DEPENDENCIES

Suppose:

TEXT

$S1 = 0$
 $S2 = 0$

P1:

```
C
A;
signal(S1);
```

P2:

```
C
wait(S1);
B;
signal(S2);
```

P3:

```
C
wait(S2);
C;
```

Dependency:

```
TEXT
A  !'  B  !'  C
```

Therefore only ordering:

$\boxed{ABC}$

8. PARALLEL DEPENDENCY

P1:

```
C
A;
signal(S1);
```

P2:

```
C
wait(S1);
B;
```

P3:

```
C
wait(S1);
C;
```

Be careful.

A single `signal(S1)` generally releases only one waiter under standard semaphore semantics.

So one signal is **not enough** to guarantee both B and C proceed.

This is a classic trap.

9. IF BOTH MUST PROCEED

P1 would need something such as:

```
C
A;
signal(S1);
signal(S1);
```

with counting-semaphore semantics.

Then both blocked waiters can eventually continue.

MEMORY

One signal != one permit/wakeup.

PART C — SEMAPHORE VALUE QUESTIONS

10. EXAMPLE

Suppose semaphore:

$\boxed{S=3}$

Five processes execute:

```
C
wait(S);
```

and none performs `signal()` yet.

Under the classic blocking definition:

```
TEXT
wait:
S--
if S < 0 !' block
```

Values:

```
TEXT
3 !' 2 !' 1 !' 0 !' - 1 !' - 2
```

So:

$\boxed{2}$

processes are waiting.

And:

$\boxed{S=-2}$

11. IMPORTANT CAVEAT

If question defines semaphore differently, follow its pseudocode.

Never force the negative-value interpretation onto every model.

PART D — RACE CONDITION QUESTIONS

12. CLASSIC `x++`

Initial:

$x=5$

Two threads each execute:

```
C
x++;
```

Non-atomically:

```
TEXT
Read
Increment local
Write
```

Possible:

Serial execution

$5 \rightarrow 6 \rightarrow 7$

Final:

$\boxed{7}$

Lost update

Both read 5.

Both compute 6.

Both write 6.

Final:

$\boxed{6}$

So typical possible finals:

$\boxed{6, 7}$

13. EXAM METHOD

Expand every compound operation.

Never reason from:

```
C
x++;
```

as though it were one indivisible instruction unless question explicitly says atomic.

PART E — DEADLOCK / RAG QUESTIONS

14. SINGLE-INSTANCE RESOURCES

Suppose:

```
TEXT
P1 holds R1 and wants R2
P2 holds R2 and wants R1
```

Graph:

```
TEXT
P 1  !'   R 2  !'   P 2  !'   R 1  !'   P 1
```

Cycle exists.

Each resource has one instance.

Therefore:

$\boxed{\text{Deadlock}}$

15. MULTIPLE INSTANCES

If a resource type has multiple instances:

Cycle does not automatically prove deadlock.

You must inspect whether an additional instance can allow some process to finish.

MEMORY

```
TEXT
One instance:
C y c l e ! 0   D e a d l o c k

Multiple instances:
C y c l e ! 0   P o s s i b l e   d e a d l o c k
```

PART F — BANKER'S ALGORITHM

16. FIRST CALCULATION ALWAYS

$\boxed{\text{Need} = \text{Max} - \text{Allocation}}$

Suppose:

Process	Allocation	Max
P1	(1,0,1)	(3,2,2)
P2	(1,1,0)	(1,2,1)
P3	(1,0,1)	(2,1,2)

Available:

$(1,1,1)$

17. FIND NEED

P1:

$(3,2,2) - (1,0,1) = \boxed{(2,2,1)}$

P2:

$(1,2,1) - (1,1,0) = \boxed{(0,1,1)}$

P3:

$(2,1,2) - (1,0,1) = \boxed{(1,1,1)}$

18. SAFETY CHECK

Initial:

$\text{Work} = (1,1,1)$

P2 fits:

$(0,1,1) \leq (1,1,1)$

P2 finishes.

Return Allocation:

$\text{Work} = (1,1,1) + (1,1,0)$

$= (2,2,1)$

Now P1 fits:

$(2,2,1)$

Finish P1:

$\text{Work} = (2,2,1) + (1,0,1)$

$\forall (3,2,2) \forall$

P3 fits.

Safe sequence:

$\forall \boxed{P2 \rightarrow P1 \rightarrow P3} \forall$

Hence state is:

$\forall \boxed{\text{Safe}} \forall$

BANKER FAST METHOD

Create only:

TEXT

Need

Work

Safe sequence

Avoid unnecessary tables if question is small.

PART G — CPU SCHEDULING Ø=Ý%Ø=Ý%Ø=Ý%

19. MASTER NUMERICAL

Processes:

Process	AT	BT
P1	0	5
P2	1	3
P3	2	1

Let's compare algorithms.

20. FCFS

Arrival order:

TEXT

P 1 ! ' P 2 ! ' P 3

Gantt:

TEXT

0 5 8 9
| P1 | P2 | P3 |

Completion:

TEXT

P1=5

P2=8

P3=9

TAT:

$\lfloor P1=5-0=5 \rfloor$

$\lfloor P2=8-1=7 \rfloor$

$\lfloor P3=9-2=7 \rfloor$

WT:

$\lfloor P1=5-5=0 \rfloor$

$\lfloor P2=7-3=4 \rfloor$

$\lfloor P3=7-1=6 \rfloor$

Average waiting:

$\lfloor \frac{0+4+6}{3} = \boxed{3.33} \rfloor$

21. NON-PREEMPTIVE SJF

At time 0, only P1 exists.

So P1 must execute.

This is important:

Do not wait for future short processes.

After P1 finishes at 5:

Ready:

TEXT

P2 BT=3

P3 BT=1

Choose P3.

Gantt:

TEXT

0	5	6	9
P1	P3	P2	

GATE TRAP

SJF chooses:

shortest among currently ready processes.

Not shortest among all processes that will ever arrive.

22. SRTF

Same:

TEXT

P1 AT0 BT5
P2 AT1 BT3
P3 AT2 BT1

Time 0–1:

P1 executes.

Remaining P1:

$\{ 4 \}$

At time 1:

P2 arrives with 3.

Since:

$\{ 3 < 4 \}$

P2 preempts P1.

Time 1–2:

P2 executes.

Remaining:

$\{ 2 \}$

At time 2:

P3 arrives BT=1.

Since:

$\{ 1 < 2 \}$

P3 runs.

TEXT

0-1 P1
1-2 P2
2-3 P3

Then compare:

TEXT

P1 remaining=4
P2 remaining=2

Run P2:

```
TEXT
3-5 P2
```

Then P1:

```
TEXT
5-9 P1
```

Gantt:

```
TEXT
0 1 2 3 5 9
|P1|P2|P3| P2 | P1 |
```

23. SRTF CHECKPOINT RULE

Only reconsider scheduling at important events:

- process arrival
- process completion

At each arrival:

1. subtract work already done
2. compare remaining times

PART H — ROUND ROBIN Ø=Ý%Ø=Ý%Ø=Ý%

24. RR METHOD

Never mentally jump around.

Maintain an actual queue.

Example:

```
TEXT
P1=5
P2=4
P3=2
q=2
All AT=0
```

Initial queue:

```
TEXT
P1 P2 P3
```

P1:

TEXT

0-2
remaining 3

Queue:

TEXT

P2 P3 P1

P2:

TEXT

2-4
remaining 2

Queue:

TEXT

P3 P1 P2

Continue.

This avoids most RR errors.

RR MEMORY

Execute !' if unfinished, append to rear.

PART I — DISK SCHEDULING Ø=Ý%Ø=Ý%

25. FAST SETUP

Given:

TEXT

Head = 50
Requests = 10, 20, 35, 60, 70, 90
Direction = higher
Disk = 0-99

Immediately write:

TEXT

10 20 35 | 50 | 60 70 90

26. LOOK

Moving right:

TEXT

5 0 !' 6 0 !' 7 0 !' 9 0 !' 3 5 !' 2 0 !' 1 0

Movement:

$\lfloor 90-50 + 90-10 \rfloor$

$\lfloor =40+80 \rfloor$

$\lfloor \boxed{120} \rfloor$

27. SCAN

Must touch right boundary 99:

$\lfloor 50 \rightarrow 99 \rightarrow 10 \rfloor$

Movement:

$\lfloor 99-50+99-10 \rfloor$

$\lfloor 49+89 \rfloor$

$\lfloor \boxed{138} \rfloor$

Difference:

LOOK stops at 90; SCAN goes to 99.

PART J — PAGING QUESTIONS Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

28. GOLDEN FOUR STEPS

Given:

TEXT

VA bits
Page size
Physical memory
PTE size

Find in this order:

1. Offset bits

$\lfloor \log_2(\text{PageSize}) \rfloor$

2. VPN bits

$\lfloor \text{VA-Offset} \rfloor$

3. Frame bits

$\lfloor \text{PA-Offset} \rfloor$

4. Page table size

$$[2^{\text{VPNbits}} \times \text{PTEsize}]$$

29. EXAMPLE

Virtual address:

$$[32 \text{ bits}]$$

Page:

$$[4\text{KB} = 2^{12}]$$

Physical memory:

$$[1\text{GB} = 2^{30}]$$

PTE:

$$[4\text{B}]$$

Then:

Offset:

$$[\boxed{12}]$$

VPN bits:

$$[32 - 12 = \boxed{20}]$$

Frame bits:

$$[30 - 12 = \boxed{18}]$$

Number PTEs:

$$[2^{20}]$$

Page-table size:

$$[2^{20} \times 4 = \boxed{4\text{MB}}]$$

30. CLASSIC GATE TRAP

Which determines number of PTEs?

'L Physical frames.

' Virtual pages.

Which determines frame-number bits?

' Physical frames.

PART K — TLB QUESTIONS Ø=Ý%Ø=Ý%Ø=Ý%

31. FAST METHOD

Don't memorize final formula.

Create path table.

Suppose:

TEXT

TLB = 10ns

Memory = 100ns

Hit ratio = 0.95

Path	Probability	Time
Hit	0.95	110
Miss	0.05	210

Then:

$EAT = .95(110) + .05(210)$

$104.5 + 10.5$

$\boxed{115\text{ns}}$

MEMORY

Path × Probability

PART L — PAGE REPLACEMENT

32. DON'T MIX THE RULES

FIFO

Track:

TEXT

oldest loaded !' newest loaded

Hit:

no order change.

LRU

Track:

TEXT

Least recently used !' most recently used

Hit:

move page to newest.

OPT

Look at:

future.

33. MINI QUESTION

Frames = 3.

Reference:

TEXT

1 2 3 1 4

Before 4:

Frames contain:

TEXT

1, 2, 3

FIFO victim

1 entered first.

$\boxed{1}$

LRU victim

Last uses:

TEXT

```
1 !'   j u s t   u s e d
3 !'   b e f o r e   t h a t
2 !'   l o n g e s t   a g o
```

Victim:

$\boxed{2}$

This one example should permanently separate FIFO from LRU in your mind.

PART M — BELADY'S ANOMALY

34. MUST KNOW

TEXT

```
F I F O ! '   p o s s i b l e
L R U ! '   i m p o s s i b l e
O P T ! '   i m p o s s i b l e
```

Why LRU/OPT?

Stack property.

PART N — THRASHING QUESTIONS

35. SYMPTOM PATTERN

If question says:

TEXT

```
P a g e   f a u l t   r a t e ! '
P a g i n g   d i s k   a c t i v i t y ! '
C P U   u t i l i z a t i o n ! "
```

Answer:

$\boxed{\text{Thrashing}}$

36. WHAT SHOULD OS DO?

Usually:

- give more frames
- reduce multiprogramming

Do **not** automatically add more processes.

PART O — INODE NUMERICALS $\emptyset = \acute{Y}\% \emptyset = \acute{Y}\% \emptyset = \acute{Y}\%$

37. GOLDEN FORMULA

$\boxed{N = \frac{\text{BlockSize}}{\text{PointerSize}}}$

Then:

TEXT

```
Direct = D
Single = N
Double = N2
Triple = N3
```

38. EXAMPLE

Block:

$\backslash[2KB=2048 \backslash]$

Pointer:

$\backslash[4B \backslash]$

Then:

$\backslash[N=\frac{2048}{4}=\boxed{512} \backslash]$

If inode has:

TEXT

```
10 direct
1 single
1 double
```

Max data blocks:

$\backslash[10+512+512^2 \backslash]$

$\backslash[=10+512+262144 \backslash]$

$\backslash[=\boxed{262666} \backslash]$

Max user file size:

$\backslash[262666\times 2KB \backslash]$

39. DISK ACCESS SHORTCUT

If inode already in RAM:

TEXT

```
D i r e c t !' 1
S i n g l e !' 2
D o u b l e !' 3
T r i p l e !' 4
```

PART P — BITMAP / FAT QUESTIONS

40. BITMAP

Disk:

$\backslash[2^{40}B \backslash]$

Block:

$\backslash[2^{12}B \backslash]$

Number blocks:

$\backslash[2^{28} \backslash]$

Bitmap:

$\lfloor 2^{28} \rfloor$ bits

Bytes:

$\lfloor 2^{25} \rfloor$

$\lfloor \boxed{32\text{MB}} \rfloor$

41. FAT

If FAT entry:

$\lfloor 4B = 2^2 \rfloor$

then FAT:

$\lfloor 2^{28} \times 2^2 = 2^{30} B \rfloor$

$\lfloor \boxed{1\text{GB}} \rfloor$

PART Q — 20 GATE-STYLE RAPID QUESTIONS

Try answering before reading the answers.

Q1

Three unconditional `fork()` calls create how many total processes?

Answer

$\lfloor \boxed{8} \rfloor$

Q2

Threads of the same process share stack.

Answer

$\lfloor \boxed{\text{False}} \rfloor$

Stack is private.

Q3

Which critical-section property prevents indefinite postponement after requesting entry?

Answer

$\lfloor \boxed{\text{Bounded Waiting}} \rfloor$

Q4

Semaphore for 8 identical resources should initially be?

$\boxed{8}$

Q5

Producer-consumer buffer size 20:

TEXT

```
mutex = ?  
empty = ?  
full = ?
```

Answer

$\boxed{1, 20, 0}$

Q6

Which Coffman condition is broken by imposing increasing resource-number acquisition?

Answer

$\boxed{\text{Circular Wait}}$

Q7

A state with no safe sequence is?

Answer

$\boxed{\text{Unsafe}}$

Not necessarily already deadlocked.

Q8

Arrival=4, Completion=13.

Turnaround?

$\boxed{13-4=9}$

Q9

TAT=9, Burst=6.

Waiting?

$\boxed{9-6=3}$

Q10

First start=7, arrival=4.

Response?

$\boxed{7-4=3}$

Q11

Which algorithm is preemptive SJF?

$\boxed{\text{SRTF}}$

Q12

Very large RR quantum approximates?

$\boxed{\text{FCFS}}$

Q13

Which disk algorithm stops at last pending request rather than physical boundary?

$\boxed{\text{LOOK}}$

Q14

Page size = 16 KB.

Offset bits?

$\boxed{16\text{KB}=2^{14}}$

$\boxed{14}$

Q15

40-bit VA, 4KB pages.

VPN bits?

$\boxed{40-12=28}$

Q16

A TLB miss necessarily causes disk access.

$\boxed{\text{False}}$

Q17

Which algorithm may show Belady anomaly?

$\boxed{\text{FIFO}}$

Q18

What happens to page faults if LRU gets an extra frame?

They can:

$\boxed{\text{decrease or remain same, but not increase}}$

Q19

4KB block and 8-byte pointer.

Pointers/indirect block?

$4096/8 = \boxed{512}$

Q20

A hard link points to?

$\boxed{\text{Same underlying inode/file object}}$

PART R — GATE EXAM DECISION TREE

When you see a question, identify its family first.

TEXT

Question

!"

Does it contain fork?

!' Trace process tree

Semaphore?

!' Draw ordering / dependency

Resource matrices?

!' Banker / Deadlock

AT/BT/quantum?

!' Scheduling Gantt

Cylinder queue?

!' Disk scheduling

VA/Page/PTE?

!' Paging formulas

TLB/hit ratio?

!' Probability $\times$ path cost

Reference string + frames?

!' FIFO / LRU / OPT table

Block/pointer/inode?
! ' D , N , N ² , N ³

This classification itself saves exam time.

42. THE BIGGEST NUMERICAL MISTAKES

Scheduling

Using:

$$\backslash [WT=CT-BT \backslash]$$

when arrival is non-zero.

Correct:

$$\backslash [WT=CT-AT-BT \backslash]$$

Paging

Using physical frames to determine page-table entries.

Wrong.

TLB

Treating TLB miss as page fault.

Wrong.

FIFO

Updating queue on a hit.

Wrong.

Banker

Adding Need to Work.

Wrong.

Add:

$$\backslash [\text{Allocation} \backslash]$$

inode

Adding indirect blocks to maximum **user file size**.

Wrong unless question asks total disk space.

43. 10 NUMERICALS YOU SHOULD BE ABLE TO SOLVE FAST

Before considering OS exam-ready, make sure you can solve:

1. conditional `fork()`
2. semaphore execution order
3. race-condition possible values
4. Banker safe sequence
5. SRTF
6. Round Robin with arrivals
7. SCAN/C-SCAN/LOOK/C-LOOK
8. page-table size + address bits
9. LRU/OPT/FIFO page-fault count
10. inode maximum-file-size

If any one of these feels slow, revise that topic before doing large PYQ sets.

'p WRITE THIS FINAL PYQ SHORTCUT PAGE

TEXT

FORK

Count processes reaching each fork.

SEMAPHORE

1 protects

0 orders

N counts

BANKER

Need = Max " Allocation

Need "d Work

Work += Allocation

CPU

Gantt !' CT !' TAT !' WT

RT = first start " AT

DISK

Sort around head.

Direction matters.

SCAN = wall

LOOK = last request

PAGING

Page exponent = offset bits

V A" offset = V P N

P A" offset = P F N

Pages $\times$ PTE = PT size

TLB

Probability $\times$ path cost

REPLACEMENT

FIFO = arrival

LRU = past
OPT = future

THRASHING

F r a m e s ! " F a u l t s ! ' C P U ! "

INODE

N = Block/Pointer

Direct=D

Single=N

Double= N^2

Triple= N^3

30-SECOND REVISION

Conditional fork? !' Count who reaches each fork.

Semaphore ordering? !' Draw dependencies.

Banker? !' Need Fits !' Finish !' Free.

Scheduling? !' Gantt first.

Disk? !' Sort + direction.

Paging? !' Offset first.

TLB? !' Weighted path cost.

FIFO? !' Hits don't reorder.

LRU? !' Hits do reorder recency.

OPT? !' Future.

inode? !' D, N, N^2 , N^3 .

NEXT — CHAPTER 21 Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

The next useful step is a **full mixed GATE-style Operating Systems test**, built from the exact syllabus you completed.

It will include:

- 1-mark conceptual questions
- 2-mark numericals
- MCQ
- MSQ
- NAT
- `fork()`
- semaphore
- Banker's
- CPU scheduling
- disk scheduling
- paging/TLB

- page replacement
- inode

I'll give the questions first and then teach the solution to each one in GATE style.

USER

NEXT

ASSISTANT

CHAPTER 21 — FULL MIXED GATE-STYLE OPERATING SYSTEMS TEST

Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

Now we test whether you can **apply** the complete syllabus instead of only remembering definitions.

This test mixes:

- MCQ
- MSQ
- NAT
- conceptual traps
- numericals

Difficulty

Ø=Easy !' Ø=Medium !' Ø=GATE-tricky

PART A — QUESTIONS

Try each question before reading its solution.

Q1 — PROCESS STATES Ø=[MCQ]

A process is waiting for completion of a disk I/O operation. Which state is it normally in?

A. Ready B. Running C. Blocked/Waiting D. Terminated

Q2 — fork() Ø=[NAT]

Consider:

```
c
fork();

if (fork() == 0)
    fork();
```

Assume every fork() succeeds.

How many total processes exist after completion?

Q3 — THREADS 0=[11SQ]

Which of the following are normally **private to each thread** of a multithreaded process?

A. Stack B. Heap C. Registers D. Program Counter E. Code

Q4 — RACE CONDITION 0=[11SQ]

Initially:

$\{ x=10 \}$

Two threads each execute once:

```
C
x = x + 1;
```

Assume read-modify-write is non-atomic.

Which final values are possible?

A. 10 B. 11 C. 12 D. 13

Q5 — CRITICAL SECTION 0=[11SQ]

A correct classical critical-section solution should satisfy:

A. Mutual Exclusion B. Progress C. Bounded Waiting D. Circular Wait

Q6 — SEMAPHORE ORDERING 0=[11CQ]

Initially:

$\{ S=0 \}$

Process P1:

```
C
printf("A");
signal(S);
printf("B");
```

Process P2:

```
C
wait(S);
printf("C");
```

Which statement must be true?

A. B always precedes C B. C always precedes B C. A always precedes C D. Output must be ABC

Q7 — PRODUCER CONSUMER $\emptyset = [1 \text{ AT}]$

A bounded buffer contains 25 slots.

What should the initial value of semaphore `empty` be?

Q8 — DEADLOCK $\emptyset = [1 \text{ SQ}]$

Which are Coffman necessary conditions for deadlock?

A. Mutual exclusion B. Hold and wait C. Preemption must be allowed D. No preemption E. Circular wait

Q9 — RESOURCE ALLOCATION GRAPH $\emptyset = [1 \text{ MCQ}]$

A resource-allocation graph contains a cycle. Every resource type has exactly one instance.

Which conclusion is correct?

A. State is always safe B. Deadlock definitely exists C. Deadlock may or may not exist D. Starvation definitely exists

Q10 — BANKER'S ALGORITHM $\emptyset = [1 \text{ AT}]$

For a process:

$[\text{Max} = (7, 5, 3)]$

$[\text{Allocation} = (2, 2, 1)]$

Find the sum of all components of its `Need` vector.

Q11 — BANKER'S SAFETY $\emptyset = [1 \text{ MCQ}]$

Suppose:

$[\text{Work} = (2, 1, 1)]$

Processes have:

$[\text{Need}(P_1) = (1, 1, 0)]$

$[\text{Need}(P_2) = (3, 0, 0)]$

$[\text{Need}(P_3) = (2, 2, 1)]$

Which process can definitely be selected **right now** by the safety algorithm?

A. Only P1 B. Only P2 C. P1 and P3 D. All three

Q12 — CPU SCHEDULING $\emptyset = [1 \text{ AT}]$

A process has:

$[\text{AT} = 3, \text{BT} = 7, \text{CT} = 15]$

Find its waiting time.

Q13 — SJF O=[MCQ]

Processes:

Process	AT	BT
P1	0	8
P2	1	2
P3	2	1

Using **non-preemptive SJF**, which process executes first?

A. P1 B. P2 C. P3 D. Cannot determine

Q14 — SRTF O=[MCQ]

Processes:

Process	AT	BT
P1	0	5
P2	1	2

Using SRTF, what is the completion time of P2?

Q15 — ROUND ROBIN O=[MCQ]

All processes arrive at time 0.

TEXT

P1 = 5
P2 = 3
P3 = 1

Time quantum:

[q=2]

Find the completion time of P3.

Q16 — DISK SCHEDULING O=[MCQ]

Disk head starts at cylinder:

[50]

Requests arrive in FCFS order:

Find total head movement.

Q17 — SCAN VS LOOK $\emptyset = [MCQ]$

The head is moving toward higher-numbered cylinders.

The highest pending request is 180, while the physical disk ends at 199.

Which algorithm normally reverses at 180?

A. SCAN B. LOOK C. C-SCAN D. FCFS

Q18 — PAGING $\emptyset = [MCQ]$

A system has a 32-bit virtual address and 4-KB pages.

How many bits are used for the virtual page number?

Q19 — PAGE TABLE $\emptyset = [MCQ]$

A system has:

- 32-bit virtual addresses
- 4-KB pages
- 4-byte PTE

Find the size of a single-level page table in MB.

Q20 — ADDRESS TRANSLATION $\emptyset = [MCQ]$

Page size:

[1024\ bytes]

Logical address:

[2500]

The relevant page maps to frame 6.

Find the physical address.

Q21 — TLB $\emptyset = \%$ [MCQ]

Given:

[TLB\ lookup=10ns]

[Memory=100ns]

TLB hit ratio:

[0.8]

On hit:

[10+100]

On miss:

[10+100+100]

Find effective memory access time in ns.

Q22 — SEGMENTATION ~~Q=1~~ [MAT]

For Segment 2:

[Base=5000]

[Limit=600]

Logical address:

[(2,450)]

Find the physical address.

Q23 — SEGMENTATION TRAP ~~Q=1~~ [MCQ]

For the same segment:

[Base=5000, Limit=600]

Logical address:

[(2,600)]

What happens?

A. Physical address = 5600 B. Physical address = 5000 C. Invalid address D. Page fault

Q24 — PAGE REPLACEMENT ~~Q=1~~ [MAT]

Three initially empty frames use FIFO.

Reference string:

TEXT

1 2 3 1 4

How many page faults occur?

Q25 — FIFO VS LRU ~~Q=1~~ [MCQ]

After processing:

TEXT

1, 2, 3, 1

with three frames, page 4 is referenced.

Which page does LRU replace?

A. 1 B. 2 C. 3 D. 4

Q26 — BELADY'S ANOMALY $\emptyset = [11SQ]$

Which algorithms among these have the stack property and therefore do not exhibit Belady's anomaly?

A. FIFO B. LRU C. OPT D. FCFS

Q27 — THRASHING $\emptyset = [11SQ]$

Which observations are consistent with severe thrashing?

A. High page-fault rate B. Heavy paging I/O C. Low CPU utilization D. Very few page faults

Q28 — WORKING SET $\emptyset = [11AT]$

Recent reference sequence:

TEXT

1 2 3 2 4

Working-set window:

$\backslash \Delta = 4 \backslash$

How many pages are in the working set at the final point?

Q29 — INODE $\emptyset = [11AT]$

Block size:

$\backslash 4KB \backslash$

Pointer size:

$\backslash 4B \backslash$

How many block addresses fit in one indirect block?

Q30 — INODE DOUBLE INDIRECT $\emptyset = [11AT]$

Using the answer from Q29, how many data blocks can one double-indirect pointer address?

Q31 — FILE SYSTEM BITMAP ~~Q=~~[WAT]

Disk size:

[1GB]

Block size:

[4KB]

A free-space bitmap uses one bit per block.

What is the bitmap size in **KB**?

Q32 — HARD LINKS ~~Q=~~[MCQ]

A file has hard-link count 2.

One of its names is deleted.

Ignoring open-file references, what is the new hard-link count?

A. 0 B. 1 C. 2 D. File must immediately disappear

PART B — SOLUTIONS

Q1

Disk I/O completion is an event.

The process cannot continue until that event occurs.

[\boxed{\text{C — Blocked/Waiting}}]

Trap

Ready means:

waiting for CPU.

Blocked means:

waiting for an event/resource/I/O.

Q2 — fork()

Initially:

[1]

First:

```
c
fork();
```

All one process reaches it:

$\rightarrow 2$

Now both execute:

```
c
fork()
```

Each creates one child:

$\rightarrow 4$

But only the children of this second fork satisfy:

```
c
fork() == 0
```

There are:

2

such children.

Each executes one more fork:

$4+2=6$

6

Shortcut

Count how many processes reach each `fork()`.

Q3 — THREADS

Private to each thread:

- Stack
- Registers
- Program Counter

Shared:

- Heap
- Code

Therefore:

$\{A, C, D\}$

Memory:

CDHF shared, PRS private.

Q4 — RACE CONDITION

Serial:

TEXT

1 0 !' 1 1 !' 1 2

Final:

$\lfloor 12 \rfloor$

Lost update:

TEXT

T1 reads 10
T2 reads 10
T1 writes 11
T2 writes 11

Final:

$\lfloor 11 \rfloor$

Therefore:

$\lfloor \boxed{B,C} \rfloor$

Q5

Classical requirements:

$\lfloor \boxed{\text{Mutual\ Exclusion, \ Progress, \ Bounded\ Waiting}} \rfloor$

Therefore:

$\lfloor \boxed{A,B,C} \rfloor$

Mnemonic:

MPB

Q6 — SEMAPHORE ORDERING

P2 cannot execute **C** until:

C

wait(S)

succeeds.

Initially:

$\lfloor S=0 \rfloor$

P1 executes:

```
C
A;
signal(S);
```

Therefore:

$\{ A \rightarrow C \}$

But after the signal:

TEXT
B and C may occur in either order.

Possible:

TEXT
ABC
ACB

Hence:

$\{ \boxed{C} \}$

Important

A semaphore gives a **partial ordering**, not necessarily one complete output.

Q7

Initially all 25 positions are empty:

$\{ \boxed{25} \}$

Producer-consumer:

$\{ \text{mutex}=1, \text{empty}=N, \text{full}=0 \}$

Q8

Coffman conditions:

- Mutual Exclusion
- Hold and Wait
- No Preemption
- Circular Wait

Therefore:

$\{ \boxed{A,B,D,E} \}$

Mnemonic:

MHNC

Q9

Single instance of every resource type:

$$\lceil \text{Cycle} \rceil \text{iff Deadlock} \rceil$$

Therefore:

$$\lceil \boxed{B} \rceil$$

Q10 — BANKER

$$\lceil \text{Need} = \text{Max} - \text{Allocation} \rceil$$

$$\lceil = (7, 5, 3) - (2, 2, 1) \rceil$$

$$\lceil = (5, 3, 2) \rceil$$

Sum:

$$\lceil 5 + 3 + 2 \rceil$$

$$\lceil \boxed{10} \rceil$$

Q11 — BANKER SAFETY

Compare component-wise.

P1

$$\lceil (1, 1, 0) \leq (2, 1, 1) \rceil$$

Yes.

P2

$$\lceil (3, 0, 0) \leq (2, 1, 1) \rceil$$

No because:

$$\lceil 3 > 2 \rceil$$

P3

$$\lceil (2, 2, 1) \leq (2, 1, 1) \rceil$$

No because:

$$\lceil 2 > 1 \rceil$$

for second resource.

Only P1 fits.

$$\lceil \boxed{A} \rceil$$

Q12 — CPU SCHEDULING

$$[TAT=CT-AT]$$

$$[=15-3=12]$$

Then:

$$[WT=TAT-BT]$$

$$[=12-7]$$

$$[\boxed{5}]$$

Shortcut:

$$[WT=CT-AT-BT]$$

Q13 — NON-PREEMPTIVE SJF

At time:

$$[t=0]$$

only P1 has arrived.

Even though P2 and P3 are shorter, they do not yet exist in the ready queue.

Therefore:

$$[\boxed{P1}]$$

Answer:

$$[\boxed{A}]$$

Huge GATE trap

Never schedule a future arrival.

Q14 — SRTF

At:

$$[0]$$

P1 starts.

Run:

TEXT

0-1 P1

Remaining P1:

$$[4]$$

At time 1, P2 arrives:

$$[BT=2]$$

Since:

$\lfloor 2 < 4 \rfloor$

P2 preempts P1.

P2 runs:

TEXT

1-3

Therefore:

$\lfloor \boxed{CT(P2)=3} \rfloor$

Q15 — ROUND ROBIN

Initial queue:

TEXT

P1 P2 P3

Quantum:

$\lfloor 2 \rfloor$

P1

TEXT

0-2

remaining:

$\lfloor 3 \rfloor$

Queue:

TEXT

P2 P3 P1

P2

TEXT

2-4

remaining:

$\lfloor 1 \rfloor$

Queue:

TEXT

P3 P1 P2

P3

Burst:

$\lfloor 1 \rfloor$

runs:

TEXT

4-5

Therefore:

$\lfloor \boxed{CT(P3)=5} \rfloor$

Q16 — FCFS DISK

Path:

TEXT

5 0 ! ' 8 0 ! ' 2 0 ! ' 9 0

Movement:

$\lfloor |80-50|=30 \rfloor$

$\lfloor |20-80|=60 \rfloor$

$\lfloor |90-20|=70 \rfloor$

Total:

$\lfloor 30+60+70 \rfloor$

$\lfloor \boxed{160} \rfloor$

Q17

LOOK reverses at:

last pending request.

Therefore it stops at:

$\lfloor 180 \rfloor$

rather than 199.

$\lfloor \boxed{B} \rfloor$

Memory:

SCAN sees wall; LOOK sees request.

Q18 — PAGING

Page size:

$$4KB = 2^{12}$$

Therefore offset:

$$12 \text{ bits}$$

Virtual address:

$$32$$

VPN:

$$32 - 12$$

$$\boxed{20}$$

Q19 — PAGE TABLE SIZE

VPN bits:

$$20$$

Number of pages:

$$2^{20}$$

Each PTE:

$$4B = 2^2 B$$

Page table:

$$2^{20} \times 2^2$$

$$= 2^{22} B$$

$$= \boxed{4MB}$$

Q20 — ADDRESS TRANSLATION

Page size:

$$1024$$

Logical address:

$$2500$$

Page:

$$\left\lfloor \frac{2500}{1024} \right\rfloor = 2$$

Offset:

$$2500 - 2048$$

$$= 452$$

Page 2 maps to frame:

$$6$$

Physical address:

$$6(1024) + 452$$

$\lfloor 6144+452 \rfloor$

$\lfloor \boxed{6596} \rfloor$

Q21 — TLB

Hit:

$\lfloor 110\text{ns} \rfloor$

Miss:

$\lfloor 210\text{ns} \rfloor$

Hit probability:

$\lfloor 0.8 \rfloor$

Miss:

$\lfloor 0.2 \rfloor$

Therefore:

$\lfloor \text{EAT}=0.8(110)+0.2(210) \rfloor$

$\lfloor =88+42 \rfloor$

$\lfloor \boxed{130\text{ns}} \rfloor$

Memory:

Probability × path cost

Q22 — SEGMENTATION

Check validity:

$\lfloor 450 < 600 \rfloor$

Yes.

Physical:

$\lfloor 5000+450 \rfloor$

$\lfloor \boxed{5450} \rfloor$

Q23 — SEGMENTATION TRAP

Valid offsets:

$\lfloor 0 \dots 599 \rfloor$

Offset:

$\lfloor 600 \rfloor$

fails:

$\lfloor 600 < 600 \rfloor$

Therefore:

$\lfloor \boxed{\text{Invalid}} \rfloor$

Answer:

$\lfloor \boxed{C} \rfloor$

Classic off-by-one trap.

Q24 — FIFO PAGE REPLACEMENT

Reference:

TEXT

1 2 3 1 4

3 frames.

1

Fault:

TEXT

1 - -

2

Fault:

TEXT

1 2 -

3

Fault:

TEXT

1 2 3

1

Hit.

FIFO order does **not** change.

4

Fault.

Oldest:

$\lfloor 1 \rfloor$

is replaced.

Total:

$\boxed{4}$

Q25 — LRU

Before 4 arrives:

TEXT
1 2 3

Past reference order:

TEXT
1, 2, 3, 1

Last uses:

TEXT
1 !' most recent
3 !' previous
2 !' least recent

Therefore LRU removes:

$\boxed{2}$

Answer:

$\boxed{B}$

This demonstrates:

TEXT
FIFO would remove 1
LRU removes 2

Q26 — BELADY

Stack algorithms:

$\boxed{\{LRU, \text{OPT}\}}$

Therefore:

$\boxed{B, C}$

FIFO may exhibit Belady's anomaly.

Q27 — THRASHING

Severe thrashing:

TEXT

P a g e f a u l t s !'
P a g i n g I / O !'
C P U u s e f u l u t i l i z a t i o n !"

Therefore:

$\boxed{A,B,C}$

Q28 — WORKING SET

Reference:

TEXT

1 2 3 2 4

Window:

$\Delta=4$

Take last four references:

TEXT

2 3 2 4

Distinct:

$\{2,3,4\}$

Working-set size:

$\boxed{3}$

Q29 — INODE

Block:

$4\text{KB}=4096\text{B}$

Pointer:

4B

Pointers:

$4096/4$

$\boxed{1024}$

Q30 — DOUBLE INDIRECT

If:

$N=1024$

then double indirect:

$$\lfloor N^2 \rfloor$$

$$\lfloor 1024^2 \rfloor$$

$$\lfloor \boxed{1,048,576} \rfloor$$

data blocks.

Q31 — BITMAP

Disk:

$$\lfloor 1\text{GB}=2^{\{30\}} \rfloor$$

Block:

$$\lfloor 4\text{KB}=2^{\{12\}} \rfloor$$

Number blocks:

$$\lfloor 2^{\{18\}} \rfloor$$

One bit per block:

$$\lfloor 2^{\{18\}} \backslash \text{bits} \rfloor$$

Convert to bytes:

$$\lfloor 2^{\{15\}}\text{B} \rfloor$$

$$\lfloor =32\text{KB} \rfloor$$

Therefore:

$$\lfloor \boxed{32} \rfloor$$

KB.

Q32 — HARD LINK

Initial:

$$\lfloor \text{LinkCount}=2 \rfloor$$

Delete one hard-link directory entry:

$$\lfloor 2-1=1 \rfloor$$

Therefore:

$$\lfloor \boxed{B} \rfloor$$

The underlying file still exists through the remaining hard link.

PART C — SCORE YOURSELF

Total:

$$\lfloor 32 \backslash \text{questions} \rfloor$$

A rough readiness interpretation:

Score	Meaning
28–32	Ø=Ý% Excellent conceptual control
24–27	' Strong; start serious PYQs
19–23	&p Good base, but numerical topics need revision
13–18	Revise high-priority chapters before PYQs
<13	Rebuild synchronization, scheduling, memory first

More important than the raw score:

Which topic caused your mistakes?

PART D — 8 QUESTIONS YOU MUST NEVER MISS

If you want maximum GATE return, these should become automatic.

1. Banker

$\boxed{\text{Need} = \text{Max-Allocation}}$

2. CPU

$\boxed{\text{TAT} = \text{CT} - \text{AT}}$

$\boxed{\text{WT} = \text{TAT} - \text{BT}}$

3. Paging

$\boxed{\text{OffsetBits} = \log_2(\text{PageSize})}$

4. Page Table

$\boxed{\#\text{VirtualPages} \times \text{PTEsize}}$

5. TLB

$\boxed{\sum \text{Probability} \times \text{PathCost}}$

6. FIFO / LRU / OPT

TEXT

FIFO !' arrival
LRU !' past
OPT !' future

7. inode

$\boxed{N = \text{BlockSize} / \text{PointerSize}}$

TEXT

Single=N
Double=N²
Triple=N³

8. Semaphore

1 protects, 0 orders, N counts.

PART E — ERROR-DIAGNOSIS MAP

If you make mistakes in:

fork()

Don't memorize formulas.

Draw the process tree.

Semaphore

Don't guess output.

Draw dependency arrows.

Banker

Don't mentally simulate.

Calculate Need and Work explicitly.

CPU scheduling

Don't calculate WT directly.

Gantt !' CT !' TAT!' WT.

Round Robin

Don't keep queue in your head.

Write the queue.

Disk scheduling

Don't immediately add numbers.

Sort requests around the head first.

Paging

Don't start from number of pages.

Find offset bits first.

TLB

Don't memorize one formula.

List paths + costs.

Page replacement

Don't mix algorithms.

TEXT

FIFO = entry order
LRU = usage order
OPT = future

inode

Don't multiply randomly.

First find $\lfloor (N-B/P) \rfloor$.

PART F — FINAL OS FORMULA MEMORY CHAIN

TEXT

DEADLOCK

$Need = Max \text{ " Allocation }$

CPU

$TAT = CT \text{ " AT}$
 $WT = TAT \text{ " BT}$
 $RT = FirstStart \text{ " AT}$

DISK

$\mathcal{E} \mid Next \text{ " Current} \mid$

PAGING

$Page = LA / PageSize$
 $Offset = LA \% PageSize$
 $PA = Frame \times PageSize + Offset$

PAGE TABLE

$VirtualPages \times PTEsize$

TLB

$Probability \times Path \text{ Cost}$

SEGMENTATION

$Offset < Limit$
 $PA = Base + Offset$

INODE

$N = Block / Pointer$
 D, N, N^2, N^3

BITMAP

$1 \text{ bit} / \text{disk block}$

CHAPTER 21 COMPLETE '

At this stage, the **full GATE OS teaching + revision + mixed application phase is complete.**

The strongest next stage is not another theory chapter. It is:

CHAPTER 22 — GATE OS HIGH-DIFFICULTY NUMERICAL DRILL Ø=Ý%Ø=Ý%Ø=Ý%Ø=

Next I'll focus only on the hardest calculation patterns:

1. tricky conditional `fork()`
2. semaphore output/order
3. semaphore final-value questions
4. race-condition possible values
5. Banker safe sequence
6. SRTF with multiple arrivals
7. Round Robin with arrivals
8. SCAN/C-SCAN/LOOK numerical
9. page-table + multilevel paging
10. TLB + page-fault combined EAT
11. FIFO/LRU/OPT long reference string
12. inode indirect-block indexing

These are the questions where GATE most often converts a known concept into a time-consuming trap.

USER

NEXT

ASSISTANT

CHAPTER 22 — GATE OS HIGH-DIFFICULTY NUMERICAL DRILL Ø=Ý%Ø=Ý%Ø=Ý%Ø=

Now we focus only on **tricky GATE-style problems.**

Goal:

You should learn not only the answer, but the fastest safe method.

1. TRICKY `fork()` QUESTION

Consider:

```
c
fork();

if (fork() == 0)
    fork();
```

```
fork();
```

Assume all `fork()` calls succeed.

Find total number of processes.

Step 1 — First `fork()`

Initially:

$\backslash 1 \backslash$

After:

```
c
fork();
```

$\backslash 1 \rightarrow 2 \backslash$

So:

```
TEXT
P0
P1
```

Step 2 — Conditional `fork()`

Both 2 processes execute:

```
c
fork();
```

Each creates one child.

So:

$\backslash 2 \rightarrow 4 \backslash$

But only the children get:

```
c
fork() == 0
```

Therefore only:

$\backslash 2 \backslash$

processes execute the inner:

```
c
fork();
```

Each creates one child:

$\lfloor 4+2=6 \rfloor$

Step 3 — Final `fork()`

Now all:

$\lfloor 6 \rfloor$

processes execute:

```
c
fork();
```

Therefore:

$\lfloor 6 \rightarrow 12 \rfloor$

Answer

$\lfloor \boxed{12} \rfloor$

GATE shortcut

At every `fork()`, count how many processes actually reach that line.

2. `fork()` WITH `&&`

Consider:

```
c
if (fork() && fork())
    fork();
```

How many total processes?

This is tricky because C uses:

short-circuit evaluation

First `fork()`

Initial P0 executes first fork.

Creates:

```
TEXT
Parent P0 !' fork returns positive !' TRUE
Child P1 !' fork returns 0 !' FALSE
```

Condition:

```
c
fork() && fork()
```

For P1:

First operand is false.

Because of `&&` short circuit:

P1 does NOT execute second fork.

For P0:

First operand true.

So P0 executes second fork.

Second fork

P0 creates P2.

For P0:

second fork returns positive.

So:

```
TEXT
TRUE && TRUE
```

condition true.

P0 executes body:

```
c
fork();
```

P2 receives 0 from second fork.

So:

```
TEXT
TRUE && FALSE
```

condition false.

P2 does not execute body.

Final body fork

Only P0 executes it.

Creates one more process.

Current processes before body:

TEXT

P0, P1, P2

After:

$\lfloor 3+1=\boxed{4} \rfloor$

Answer

$\lfloor \boxed{4} \rfloor$

3. `fork()` WITH `||`

Consider:

```
c
if (fork() || fork())
    fork();
```

Again short-circuit evaluation matters.

First fork

P0 parent:

TEXT

TRUE

Because `||` already has true first operand:

parent does NOT execute second fork.

Child P1:

TEXT

FALSE

So P1 must evaluate second fork.

Second fork executed only by P1

P1 creates P2.

For P1:

second fork positive != TRUE.

For P2:

second fork 0 != FALSE.

So condition:

P0

TRUE !' enters body.

P1

FALSE || TRUE !' enters body.

P2

FALSE || FALSE !' does not enter.

Thus 2 processes execute body `fork()` .

Before body:

$\lfloor 3 \rfloor$

Two additional children:

$\lfloor 3+2=\boxed{5} \rfloor$

Answer

$\lfloor \boxed{5} \rfloor$

MASTER SHORT-CIRCUIT RULE

A && B

If A is false:

B is skipped.

A || B

If A is true:

B is skipped.

Essential for `fork()` questions.

PART B — SEMAPHORE OUTPUT ORDER

4. ORDERING QUESTION

Initially:

$\lfloor S_1=0, \text{ } S_2=0 \rfloor$

P1:


```
c
printf("A");
signal(S1);
wait(S2);
printf("D");
```

P2:

```
c
wait(S1);
printf("B");
printf("C");
signal(S2);
```

Find the guaranteed output.

Dependency 1

P2 cannot print B until:

```
c
signal(S1)
```

which occurs after A.

Therefore:

$\text{A} \rightarrow \text{B}$

Within P2:

$\text{B} \rightarrow \text{C}$

Then P2 signals S2.

P1 cannot print D until that signal.

Therefore:

$\text{C} \rightarrow \text{D}$

Complete chain:

$\text{A} \rightarrow \text{B} \rightarrow \text{C} \rightarrow \text{D}$

Answer

$\boxed{\text{ABCD}}$

5. PARTIAL ORDER QUESTION

Initially:

$S=0$

P1:

```
C
printf("A");
signal(S);
printf("B");
```

P2:

```
C
wait(S);
printf("C");
```

What outputs are possible?

Guaranteed:

$\{A \rightarrow C\}$

Within P1:

$\{A \rightarrow B\}$

But no ordering exists between B and C.

So possible:

```
TEXT
ABC
ACB
```

Answer

$\{ \boxed{ABC, ACB} \}$

Not:

```
TEXT
CAB
BAC
```

because A must occur before both.

6. COUNTING SEMAPHORE VALUE

Semaphore initially:

$\{S=2\}$

Four processes execute:

```
C
wait(S);
```

No `signal()` occurs.

Assume:

TEXT

```
wait:
S--
  if S < 0 !' b l o c k
```

Values:

TEXT

```
2 !' 1
1 !' 0
0 !' - 1
- 1 !' - 2
```

So:

$\boxed{S=-2}$

and:

$\boxed{2\text{ processes blocked}}$

7. SIGNAL AFTER BLOCKING

Continue previous example:

$\boxed{S=-2}$

Now one process executes:

```
c
signal(S);
```

Then:

$\boxed{S=-1}$

Because:

$\boxed{S \leq 0}$

one blocked process is awakened.

Blocked processes remaining:

$\boxed{1}$

Important:

Awakened process becomes Ready, not necessarily immediately Running.

PART C — RACE CONDITION TRICK

8. INCREMENT + DECREMENT

Initial:

$\{ x=5 \}$

P1:

```
C
x = x + 1;
```

P2:

```
C
x = x - 1;
```

Assume each becomes:

```
TEXT
Read
Modify local copy
Write
```

What final values are possible?

Serial P1 then P2

$\{ 5 \rightarrow 6 \rightarrow 5 \}$

Final:

$\{ 5 \}$

Serial P2 then P1

$\{ 5 \rightarrow 4 \rightarrow 5 \}$

Final:

$\{ 5 \}$

Both read 5

P1 computes:

$\{ 6 \}$

P2 computes:

$\{ 4 \}$

If P1 writes then P2:

Final:

$\{ 4 \}$

If P2 writes then P1:

Final:

[6]

Therefore possible:

[\boxed{4, \ 5, \ 6}]

9. GATE METHOD FOR RACES

Never guess.

Do:

1. Expand compound operation.
2. Give each process its own local register.
3. Preserve order inside each process.
4. Interleave legally.
5. Track final shared write.

PART D — BANKER'S ALGORITHM Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

10. FULL SAFE-SEQUENCE QUESTION

Given:

Allocation

Process	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Max

Process	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

Available:

$\vee (3,3,2) \vee$

Find whether state is safe.

11. FIND NEED

$\vee \text{Need} = \text{Max} - \text{Allocation} \vee$

So:

Process	Need
P0	(7,4,3)
P1	(1,2,2)
P2	(6,0,0)
P3	(0,1,1)
P4	(4,3,1)

Start:

$\vee \text{Work} = (3,3,2) \vee$

P1 fits

$\vee (1,2,2) \leq (3,3,2) \vee$

Finish P1.

Add Allocation P1:

$\vee (2,0,0) \vee$

So:

$\vee \text{Work} = (5,3,2) \vee$

P3 fits

$\vee (0,1,1) \leq (5,3,2) \vee$

Add Allocation:

$\vee (2,1,1) \vee$

$\vee \text{Work} = (7,4,3) \vee$

P4 fits

$\vee (4,3,1) \leq (7,4,3) \vee$

Add:

$\vee (0,0,2) \vee$

$\text{Work} = (7, 4, 5)$

P0 fits

$(7, 4, 3) \leq (7, 4, 5)$

Add:

$(0, 1, 0)$

$\text{Work} = (7, 5, 5)$

P2 fits

$(6, 0, 0) \leq (7, 5, 5)$

All complete.

Safe sequence:

$P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$

Hence:

Safe State

12. BANKER'S BIGGEST TRAP

When process finishes:

Do not add **Need**.

Add:

Allocation

because those are the resources actually released.

PART E — SRTF WITH MULTIPLE ARRIVALS

13. QUESTION

Process	AT	BT
P1	0	8
P2	1	4
P3	2	2
P4	3	1

Use SRTF.

Find completion times.

14. TIME 0-1

Only P1.

Run P1:

```
TEXT
0-1 P1
```

P1 remaining:

\[7 \]

15. TIME 1

P2 arrives:

\[BT=4 \]

Compare:

```
TEXT
P1 rem = 7
P2 rem = 4
```

Run P2.

```
TEXT
1-2 P2
```

P2 remaining:

\[3 \]

16. TIME 2

P3 arrives:

\[BT=2 \]

Compare:

```
TEXT
P1=7
P2=3
P3=2
```

Run P3.

```
TEXT
2-3 P3
```


P3 remaining:

$\lfloor 1 \rfloor$

17. TIME 3

P4 arrives:

$\lfloor BT=1 \rfloor$

Now:

TEXT

P3 remaining = 1
P4 burst = 1

Tie-breaking depends on stated convention.

A common choice is to let currently running P3 continue.

Assume that.

P3:

TEXT

3-4 P3

So:

$\lfloor CT(P3)=4 \rfloor$

18. NEXT

Remaining:

TEXT

P1=7
P2=3
P4=1

Run P4:

TEXT

4-5 P4

$\lfloor CT(P4)=5 \rfloor$

Then P2:

TEXT

5-8 P2

$\lfloor CT(P2)=8 \rfloor$

Then P1:

TEXT

8-15 P1

$\lfloor \text{CT}(P1)=15 \rfloor$

GANTT

TEXT

0 1 2 3 4 5 8 15
|P1|P2|P3|P3|P4| P2 | P1 |

Completion times:

$\lfloor \boxed{P1=15, P2=8, P3=4, P4=5} \rfloor$

19. SRTF EXAM RULE

At every arrival:

Compare new burst with remaining times, not original bursts.

PART F — ROUND ROBIN WITH ARRIVALS $\emptyset=\dot{Y}\%\emptyset=\dot{Y}\%\emptyset=\dot{Y}\%$

20. QUESTION

Quantum:

$\lfloor q=2 \rfloor$

Process	AT	BT
P1	0	5
P2	1	3
P3	2	2

Assume newly arriving processes join the ready queue as they arrive, and at a quantum boundary all arrivals up to that time are already enqueued before the expired process is re-enqueued.

21. TIME 0

Queue:

TEXT

P1

Run:

TEXT
0-2 P1

P1 remaining:

\[3 \]

During this interval:

TEXT
t = 1 !' P 2 a r r i v e s
t = 2 !' P 3 a r r i v e s

At time 2 queue becomes:

TEXT
P2, P3, P1

22. P2

Run:

TEXT
2-4 P2

remaining:

\[1 \]

Queue:

TEXT
P3, P1, P2

23. P3

BT=2.

Run:

TEXT
4-6 P3

finishes.

\[CT(P3)=6 \]

Queue:

TEXT

P1, P2

24. P1

TEXT

6-8 P1

remaining:

$\lfloor 1 \rfloor$

Queue:

TEXT

P2, P1

25. P2

TEXT

8-9 P2

finishes.

$\lfloor \text{CT}(P2)=9 \rfloor$

26. P1

TEXT

9-10 P1

finishes.

$\lfloor \text{CT}(P1)=10 \rfloor$

Thus:

$\lfloor \boxed{\text{CT}(P1)=10, \text{CT}(P2)=9, \text{CT}(P3)=6} \rfloor$

RR MASTER RULE

Keep a literal queue.

Never solve Round Robin only mentally.

PART G — SCAN FAMILY NUMERICAL $\emptyset=\acute{Y}\%\emptyset=\acute{Y}\%\emptyset=\acute{Y}\%$

27. QUESTION

Disk cylinders:

$\backslash \text{0\texttt{to }199} \backslash$

Head:

$\backslash \text{50} \backslash$

Requests:

TEXT

10, 20, 35, 60, 70, 90, 150

Initial direction:

toward higher cylinders.

Find total movement for SCAN, LOOK, C-SCAN and C-LOOK.

28. SORT AROUND HEAD

TEXT

10 20 35 | 50 | 60 70 90 150

29. SCAN

Path:

TEXT

5 0 !' 6 0 !' 7 0 !' 9 0 !' 1 5 0 !' 1 9 9 !' 3 5 !' 2 0 !' 1 0

Shortcut:

$\backslash (199-50)+(199-10) \backslash$

$\backslash 149+189 \backslash$

$\backslash \boxed{338} \backslash$

30. LOOK

Path:

TEXT

5 0 !' 6 0 !' 7 0 !' 9 0 !' 1 5 0 !' 3 5 !' 2 0 !' 1 0

Movement:

$\backslash (150-50)+(150-10) \backslash$

$\lfloor 100+140 \rfloor$

$\lfloor \boxed{240} \rfloor$

31. C-SCAN

Path:

TEXT

5 0 !' 6 0 !' 7 0 !' 9 0 !' 1 5 0 !' 1 9 9
!' 0
!' 1 0 !' 2 0 !' 3 5

Movement:

$\lfloor (199-50)+199+35 \rfloor$

$\lfloor 149+199+35 \rfloor$

$\lfloor \boxed{383} \rfloor$

32. C-LOOK

Path:

TEXT

5 0 !' 6 0 !' 7 0 !' 9 0 !' 1 5 0
!' 1 0
!' 2 0 !' 3 5

Movement:

$\lfloor (150-50)+(150-10)+(35-10) \rfloor$

$\lfloor 100+140+25 \rfloor$

$\lfloor \boxed{265} \rfloor$

MASTER TABLE

Algorithm	Movement
SCAN	338
LOOK	240
C-SCAN	383
C-LOOK	265

Memory:

SCAN touches wall; LOOK touches request.

PART H — MULTILEVEL PAGING Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

33. QUESTION

System has:

- 48-bit virtual address
- page size = 4 KB
- PTE size = 8 bytes
- every page-table level occupies one page

How many levels are required?

34. OFFSET BITS

$$\lfloor 4\text{KB}=2^{\{12\}} \rfloor$$

Therefore:

$$\lfloor \text{Offset}=12 \rfloor$$

VPN bits:

$$\lfloor 48-12=36 \rfloor$$

35. ENTRIES PER PAGE TABLE PAGE

Page-table page:

$$\lfloor 4096\text{B} \rfloor$$

PTE:

$$\lfloor 8\text{B} \rfloor$$

Entries:

$$\lfloor 4096/8=512 \rfloor$$

$$\lfloor 512=2^9 \rfloor$$

Each full level indexes:

$$\lfloor 9 \rfloor \text{ bits}$$

36. LEVELS

VPN bits:

$$\lfloor 36 \rfloor$$

Per level:

$$\lfloor 9 \rfloor$$

Therefore:

$$\lfloor 36/9=4 \rfloor$$

Answer

$$\lfloor \boxed{4} \rfloor$$

37. ACCESS COUNT

With 4 page-table levels and no TLB:

$$\lfloor 4 \text{ page-table accesses} + 1 \text{ data access} \rfloor$$

$$\lfloor \boxed{5} \text{ memory accesses} \rfloor$$

PART I — TLB + PAGE FAULT COMBINED EAT $\emptyset = \text{Y}\% \emptyset = \text{Y}\% \emptyset = \text{Y}\% \emptyset = \text{Y}\%$

38. QUESTION

Suppose:

- TLB lookup = 10 ns
- memory access = 100 ns
- TLB hit ratio = 0.9
- page-fault probability = (10^{-6})
- page-fault service time = 8 ms

Assume page-fault probability applies to references overall and fault service time includes all fault handling.

Find approximate EAT.

39. FIRST — NORMAL ACCESS COST

If no page fault:

TLB hit:

$$\lfloor 110\text{ns} \rfloor$$

TLB miss:

$$\lfloor 210\text{ns} \rfloor$$

Normal EAT:

$$\lfloor 0.9(110)+0.1(210) \rfloor$$

$$\lfloor 99+21 \rfloor$$

$$\lfloor \boxed{120\text{ns}} \rfloor$$

40. CONVERT FAULT TIME

$$\lfloor 8\text{ms} \rfloor$$

$$\lfloor 8 \times 10^6 \text{ ns} \rfloor$$

41. COMBINE

$$\lfloor \text{EAT} = (1-p)(120) + p(8 \times 10^6) \rfloor$$

with:

$$\lfloor p = 10^{-6} \rfloor$$

Approximately:

$$\lfloor 120 + 8 \rfloor$$

$$\lfloor \boxed{128 \text{ ns}} \rfloor$$

More precisely the tiny subtraction from $\lfloor 120(1-p) \rfloor$ is negligible.

IMPORTANT LESSON

Even one page fault in a million references increases EAT noticeably.

PART J — FIFO / LRU / OPT LONG TRACE Ø=Ý%Ø=Ý%Ø=Ý%

42. QUESTION

Reference string:

TEXT

1 2 3 4 1 2 5 1 2 3 4 5

Frames:

$$\lfloor 3 \rfloor$$

Find page faults for FIFO, LRU and OPT.

43. FIFO

Trace using queue.

TEXT

```

1  F  !'  [ 1 ]
2  F  !'  [ 1 2 ]
3  F  !'  [ 1 2 3 ]
4  F  !'  [ 2 3 4 ]
1  F  !'  [ 3 4 1 ]
2  F  !'  [ 4 1 2 ]
5  F  !'  [ 1 2 5 ]
1  H
2  H
3  F  !'  [ 2 5 3 ]
4  F  !'  [ 5 3 4 ]
5  H

```

Faults:

$\boxed{9}$

44. LRU

Maintain least recent !' most recent.

TEXT

```
1  F  !'  [ 1 ]
2  F  !'  [ 1  2 ]
3  F  !'  [ 1  2  3 ]
4  F  !'  [ 2  3  4 ]
1  F  !'  [ 3  4  1 ]
2  F  !'  [ 4  1  2 ]
5  F  !'  [ 1  2  5 ]
1  H  !'  [ 2  5  1 ]
2  H  !'  [ 5  1  2 ]
3  F  !'  [ 1  2  3 ]
4  F  !'  [ 2  3  4 ]
5  F  !'  [ 3  4  5 ]
```

Faults:

$\boxed{10}$

This is a useful reminder:

LRU does not always beat FIFO on every individual string.

45. OPT

Let's trace key replacements.

TEXT

```
1 F
2 F
3 F
```

Frames:

TEXT

```
1 2 3
```

Reference 4.

Future:

TEXT

```
1 2 5 1 2 3 4 5
```

Among 1,2,3:

3 is used farthest.

Replace 3.

Faults = 4.

Reference 1 hit.

Reference 2 hit.

Reference 5:

Frames:

TEXT

1 2 4

Future:

TEXT

1 2 3 4 5

4 is used farthest.

Replace 4.

Faults = 5.

Then 1 hit, 2 hit.

Reference 3:

Frames:

TEXT

1 2 5

Future:

TEXT

4 5

1 and 2 never used again.

Replace either.

Faults = 6.

Reference 4 faults.

Faults = 7.

Reference 5 hits.

So:

$\boxed{\text{OPT}=7}$

COMPARISON

$\boxed{\text{FIFO}=9, \text{LRU}=10, \text{OPT}=7}$

Critical:

OPT must be minimum.

PART K — BELADY CHECK Ø=Ý%

46. SAME REFERENCE STRING WITH FIFO

Classic string:

TEXT

1 2 3 4 1 2 5 1 2 3 4 5

FIFO faults:

3 frames

\[\boxed{9} \]

4 frames

\[\boxed{10} \]

Thus:

\[Frames\uparrow \]

but:

\[Faults\uparrow \]

This is:

\[\boxed{\text{Belady's Anomaly}} \]

PART L — INODE INDEXING Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

47. QUESTION

An inode has:

- block size = 4 KB
- pointer size = 4 bytes
- 12 direct pointers
- 1 single indirect
- 1 double indirect

For logical file block:

\[5000 \]

find which structure is used and exact double-indirect indices.

48. POINTERS PER BLOCK

$$\lfloor N=4096/4=1024 \rfloor$$

Direct capacity:

$$\lfloor 12 \rfloor$$

Single:

$$\lfloor 1024 \rfloor$$

So double-indirect starts at logical block:

$$\lfloor 12+1024 = \boxed{1036} \rfloor$$

49. RELATIVE BLOCK

$$\lfloor r=5000-1036 \rfloor$$

$$\lfloor =\boxed{3964} \rfloor$$

50. DOUBLE-INDIRECT INDICES

First:

$$\lfloor i_1 = \left\lfloor \frac{3964}{1024} \right\rfloor \rfloor$$

$$\lfloor =\boxed{3} \rfloor$$

Second:

$$\lfloor i_2 = 3964 \bmod 1024 \rfloor$$

$$\lfloor 3964 - 3(1024) \rfloor$$

$$\lfloor 3964 - 3072 \rfloor$$

$$\lfloor =\boxed{892} \rfloor$$

So path:

TEXT

Double-indirect block

!"

Entry 3

!"

Second-level block

!"

Entry 892

!"

Data block

51. DISK ACCESSSES

If inode is already in RAM and nothing else cached:

TEXT

L1 pointer block
L2 pointer block
Data

Therefore:

$\boxed{\text{disk accesses}}$

PART M — HARD MIXED QUESTIONS

52. QUESTION — MINIMUM PTE BITS

Physical memory:

$8\text{GB} = 2^{33}$

Page size:

$8\text{KB} = 2^{13}$

Number frames:

2^{20}

Minimum frame number bits:

$\boxed{20}$

If PTE additionally requires:

TEXT

Valid = 1 bit
Dirty = 1 bit
Reference = 1 bit

minimum:

$20 + 3 = \boxed{23 \text{ bits}}$

before any alignment/rounding assumptions.

53. QUESTION — PAGE TABLE SIZE

Virtual address:

36bits

Page:

$8\text{KB} = 2^{13}$

PTE:

$8\text{B} = 2^3$

VPN bits:

$36 - 13 = 23$

Entries:

$$\lfloor 2^{23} \rfloor$$

Page table size:

$$\lfloor 2^{23} \times 2^3 = 2^{26} \text{B} \rfloor$$

$$\lfloor \boxed{64\text{MB}} \rfloor$$

54. QUESTION — BITMAP

Disk:

$$\lfloor 16\text{GB} = 2^{34} \rfloor$$

Block:

$$\lfloor 4\text{KB} = 2^{12} \rfloor$$

Blocks:

$$\lfloor 2^{22} \rfloor$$

Bitmap:

$$\lfloor 2^{22} \text{ bits} \rfloor$$

Bytes:

$$\lfloor 2^{19} \rfloor$$

$$\lfloor \boxed{512\text{KB}} \rfloor$$

PART N — 15-QUESTION HARD DRILL

Try these before reading answers.

Q1

```
c
fork();
if (fork())
    fork();
```

Total processes?

Q2

```
c
if (fork() && fork())
    fork();
```

Total?

Q3

Semaphore $(S=1)$. Three processes perform `wait(S)` with no signals. Under negative blocking semantics, what is final S ?

Q4

Initial $(x=10)$. Two threads each execute non-atomic `x++`. Minimum final x ?

Q5

Initial:

$[\text{Work}=(2,2)]$

Which Need can immediately finish?

A. (2,2) B. (3,1) C. (1,3)

Q6

P1: AT=0 BT=7 P2: AT=2 BT=2

Under SRTF, what is CT(P2)?

Q7

Disk range 0–199, head 100, moving right, highest request 180. Which algorithm reaches 199 before reversing?

Q8

48-bit VA, 4-KB page. VPN bits?

Q9

Page size 4KB, PTE 8B. Entries per page-table page?

Q10

3-level paging without TLB: number of memory accesses for one data reference?

Q11

FIFO can show Belady anomaly: True/False?

Q12

LRU hit changes recency order: True/False?

Q13

Block=2KB, pointer=4B. What is $\lfloor N \rfloor$?

Q14

If $\lfloor N=512 \rfloor$, triple indirect addresses how many data blocks?

Q15

Disk has $\lfloor 2^{24} \rfloor$ blocks. Bitmap size in bytes?

ANSWERS

A1

First fork:

$\lfloor 1 \rfloor$

Both execute second fork:

$\lfloor 2 \rfloor$

Only parents of second fork satisfy `if(fork())`.

There are 2 such processes.

They execute final fork:

$\lfloor 4+2=6 \rfloor$

A2

As solved earlier:

$\lfloor 4 \rfloor$

A3

$\lfloor 1 \rfloor$

$\lfloor -2 \rfloor$

Two blocked.

A4

Both may read 10 and both write 11.

$\boxed{11}$

A5

Component-wise:

$(2,2) \leq (2,2)$

So:

$\boxed{A}$

A6

P1 runs 0–2.

Remaining:

5

P2 arrives with 2, preempts, runs 2–4.

$\boxed{4}$

A7

$\boxed{\text{SCAN}}$

LOOK reverses at 180.

A8

$48-12 = \boxed{36}$

A9

$4096/8 = \boxed{512}$

A10

$3+1 = \boxed{4}$

A11

$\boxed{\text{True}}$

A12

$\boxed{\text{True}}$

A13

$2048/4 = \boxed{512}$

A14

$512^3 = 134,217,728$

$\boxed{134,217,728}$

A15

2^{24} bits

Bytes:

2^{21}

$= \boxed{2\text{MB}}$

HIGH-DIFFICULTY MASTER CHECKLIST

Before moving on, these should become automatic:

fork()

Count who reaches every fork; respect $\&\&$ / $\|\|$ short circuit.

Semaphore

Convert waits/signals into ordering constraints.

Race

Expand to Read-Modify-Write.

Banker

Need = Max " Allocation; Need "dWork; Work += Allocation.

SRTF

Re-evaluate on arrival using remaining time.

RR

Maintain a literal ready queue.

SCAN family

Sort around head; check direction; wall vs request.

Paging

Offset first.

TLB/Page fault

Weighted path cost.

Replacement

FIFO arrival, LRU past, OPT future.

inode

$\lfloor (N=B/P) \rfloor$, then $\lfloor (D, N, N^2, N^3) \rfloor$.

CHAPTER 22 COMPLETE '

You have now gone beyond theory and covered the **main high-difficulty numerical patterns** that GATE can build from this Operating Systems syllabus.

NEXT — CHAPTER 23: OS FINAL ONE-PAGE REVISION + 100 RAPID-FIRE QUESTIONS Ø=Ý%

Next I'll switch to **speed-training mode**:

- ultra-short OS revision sheet
- 100 rapid-fire GATE facts/traps
- answer immediately after each question
- formulas repeated only where they matter
- designed for last-minute recall and PYQ readiness.

USER

NEXT

CHAPTER 23 — OS FINAL ONE-PAGE REVISION + 100 RAPID-FIRE GATE QUESTIONS

This is speed-training mode.

Goal:

See the question !' recall the concept in a few seconds !' avoid common GATE traps.

PART A — ONE-PAGE OPERATING SYSTEMS REVISION

Processes

TEXT

Program = passive
Process = program in execution

Ready = waiting for CPU
Blocked = waiting for I/O/event

Running !' Ready = preemption
Running !' Blocked = I/O/event wait
Blocked !' Ready = event completion

PCB stores:

TEXT

PID
State
PC
Registers
Scheduling
Memory
I/O information

`fork()`:

$\boxed{\text{Unconditional } n \text{ forks}} \rightarrow 2^n \text{ total processes}$

But for conditional forks:

Trace who actually reaches each `fork()`.

TEXT

`fork` !' creates process
`exec` !' replaces current program
`wait` !' parent waits
`exit` !' terminates

Zombie:

child dead, status not collected.

Orphan:

parent dead, child alive.

Threads

Shared:

CDHF

TEXT

Code
Data
Heap
Files/resources

Private:

PRS

TEXT

Program Counter
Registers
Stack

IPC

TEXT

Shared Memory
Message Passing
Pipes
Message Queues
Sockets
Signals

Memory:

Share or Send

Critical Section

Race condition:

Result depends on concurrent execution/interleaving.

Non-atomic:

TEXT
Read
Modify
Write

Critical-section requirements:

MPB

TEXT
Mutual Exclusion
Progress
Bounded Waiting

Semaphores

TEXT
wait = P = down
signal = V = up

Master memory:

$\boxed{1 \text{ protects}, \quad 0 \text{ orders}, \quad N \text{ counts}}$

Producer-consumer buffer size (N) :

$\boxed{\text{mutex}=1, \text{ empty}=N, \text{ full}=0}$

Readers-writers:

First reader locks, last reader unlocks.

Deadlock

Coffman:

MHNC

TEXT
Mutual Exclusion
Hold and Wait
No Preemption
Circular Wait

Single-instance RAG:

$\boxed{\text{Cycle} \text{ iff } \text{Deadlock}}$

Banker:

$\boxed{\text{Need} = \text{Max-Allocation}}$

$\boxed{\text{Need}_i \neq \text{Work}}$

After process finishes:

$\boxed{\text{Work} = \text{Work} + \text{Allocation}_i}$

Unsafe:

$\neq$

necessarily deadlocked.

CPU Scheduling

$\boxed{\text{TAT} = \text{CT} - \text{AT}}$

$\boxed{\text{WT} = \text{TAT} - \text{BT}}$

$\boxed{\text{RT} = \text{FirstStart} - \text{AT}}$

TEXT

```
FCFS !'  convoy  effect
SJF  !'  shortest burst
SRTF !'  shortest remaining
Priority !' starvation; Aging fixes
RR  !'  quantum
```

Large RR quantum:

$\boxed{\text{RR} \rightarrow \text{FCFS}}$

Disk Scheduling

$\boxed{\text{Movement} = \sum |\text{Next} - \text{Current}|}$

TEXT

```
FCFS !'  arrival order
SSTF !'  nearest
SCAN !'  physical wall + reverse
C-SCAN !' wall + circular jump
LOOK !'  last request + reverse
C-LOOK !' extreme request + circular jump
```

Memory Management

TEXT

```
Logical !' CPU generated
Physical !' actual RAM
MMU !'  translation
```

Base/limit:

$\boxed{\text{LA} < \text{Limit}}$

then:

$$\boxed{PA = \text{Base} + LA}$$

Fragmentation:

TEXT

Internal = waste inside

External = scattered free holes

Paging

$$\boxed{\text{PageSize} = \text{FrameSize}}$$

Logical:

TEXT

Page Number | Offset

Physical:

TEXT

Frame Number | Offset

Offset stays unchanged.

$$\boxed{\text{Page} = \lfloor LA / S \rfloor}$$

$$\boxed{\text{Offset} = LA \bmod S}$$

$$\boxed{PA = \text{Frame} \times S + \text{Offset}}$$

If page size:

$$2^n$$

then:

$$\boxed{\text{OffsetBits} = n}$$

Page-table size:

$$\boxed{\# \text{VirtualPages} \times \text{PTEsize}}$$

TLB

Fast cache of Page !' Frame translations.

$$\boxed{\text{TLB} \setminus \text{miss} \neq \text{Page} \setminus \text{fault}}$$

Universal EAT:

$$\boxed{\sum \text{Probability} \times \text{PathCost}}$$

Segmentation

Logical address:

$\lfloor (s,d) \rfloor$

Check:

$\lfloor d < \text{Limit}[s] \rfloor$

Then:

$\lfloor \boxed{\text{PA} = \text{Base}[s] + d} \rfloor$

TEXT

Paging !' fixed size !' internal fragmentation
Segmentation !' variable size !' external fragmentation

Virtual Memory

Demand paging:

bring page only when referenced.

Page fault:

valid page absent from RAM.

TEXT

Temporal locality = recent !' reuse
Spatial locality = nearby !' soon

Page Replacement

TEXT

FIFO !' arrival
LRU !' past usage
OPT !' future usage

Belady:

TEXT

FIFO !' possible
LRU !' no
OPT !' no

Thrashing

TEXT

```
F r a m e s / p r o c e s s !"  
P a g e   f a u l t s !'  
P a g i n g   I / O !'  
C P U   u t i l i z a t i o n !"
```

Cure:

- give more frames
- reduce multiprogramming

File Systems

Allocation:

TEXT

```
C o n t i g u o u s !'   f a s t   b u t   r i g i d  
L i n k e d !'   e a s y   g r o w t h ,   p o o r   r a n d o m   a c c e s s  
I n d e x e d !'   p o i n t e r / i n d e x   s t r u c t u r e  
F A T !'   l i n k e d   a l l o c a t i o n   i n   t a b l e
```

inode:

TEXT

metadata + block pointers

Let:

$N = \frac{\text{BlockSize}}{\text{PointerSize}}$

Then:

TEXT

```
D i r e c t   =   D  
S i n g l e   =   N  
D o u b l e   =   N2  
T r i p l e   =   N3
```

Bitmap:

1 bit per disk block.

Hard link:

same inode.

Symbolic link:

pathname-based link.

PART B — 100 RAPID-FIRE GATE QUESTIONS

Try answering each in **2–5 seconds** before reading the answer.

1. What is a process?

Answer: Program in execution.

2. Which process state waits for CPU?

Answer: Ready.

3. Which state waits for I/O completion?

Answer: Blocked/Waiting.

4. Running !' Ready is called?

Answer: Preemption.

5. Waiting !' Ready occurs when?

Answer: Waiting event/I/O completes.

6. What does PCB stand for?

Answer: Process Control Block.

7. Does PCB contain CPU register values?

Answer: Yes.

8. Does a program itself necessarily have a PCB?

Answer: No. A process does.

9. Three unconditional successful `fork()` calls give total processes?

$\boxed{8}$

10. Three unconditional forks create how many new children?

$\boxed{7}$

11. Does `exec()` normally create another process?

Answer: No.

12. `fork()` returns what in the child?

`0`

13. A terminated child whose parent hasn't collected its status is?

Answer: Zombie.

14. A child whose parent terminates first is?

Answer: Orphan.

15. Mode switch and context switch are identical.

Answer: False.

16. Threads of the same process share code?

Answer: Yes.

17. Threads share heap?

Answer: Yes.

18. Threads share stack?

Answer: No.

19. Each thread has its own program counter?

Answer: Yes.

20. Many-to-One thread model supports true multicore parallelism through multiple kernel threads?

Answer: No.

21. Concurrency always means simultaneous execution.

Answer: False.

Concurrency can be interleaving.

22. Parallelism requires simultaneous execution.

Answer: Yes.

23. Two main IPC models?

Answer: Shared memory and message passing.

24. Which IPC model usually needs explicit synchronization around shared data?

Answer: Shared memory.

25. A signal is mainly used for bulk data transfer.

Answer: False.

Mostly notification/event signaling.

CONCURRENCY & SYNCHRONIZATION

26. What is a race condition?

Answer: Outcome depends on timing/interleaving of concurrent accesses to shared data.

27. Is `x++` necessarily atomic?

Answer: No.

28. Expand `x++` conceptually.

Answer: Read !' Modify !' Write.

29. Three classical critical-section requirements?

Answer: Mutual Exclusion, Progress, Bounded Waiting.

30. Which requirement ensures at most one contender enters the critical section?

Answer: Mutual Exclusion.

31. Which requirement is associated with fairness/no indefinite postponement after requesting entry?

Answer: Bounded Waiting.

32. Busy waiting consumes CPU while waiting.

Answer: True.

33. A spinlock normally involves busy waiting.

Answer: True.

34. Does basic Test-and-Set automatically guarantee bounded waiting?

Answer: No.

35. Peterson's classical algorithm is primarily for how many processes?

$\boxed{2}$

36. Semaphore operation names?

Answer: `wait()` and `signal()`.

Also P/V, down/up.

37. Semaphore for mutual exclusion usually starts at?

$\boxed{1}$

38. Semaphore for enforcing A-before-B usually starts at?

$\boxed{0}$

39. Semaphore for 7 identical resources starts at?

$\boxed{7}$

40. Give the master semaphore mnemonic.

Answer: 1 protects, 0 orders, N counts.

CLASSICAL SYNCHRONIZATION

41. Buffer size N : initial empty ?

$\boxed{N}$

42. Initial full ?

$\boxed{0}$

43. Initial producer-consumer mutex?

$\boxed{1}$

44. Stable producer-consumer relation?

$\boxed{\text{empty} + \text{full} = N}$

45. Producer should normally wait(empty) before wait(mutex) .

Answer: True.

46. Why?

Answer: Avoid holding mutex while waiting for an empty slot, which can deadlock the classical solution.

47. In classic readers-writers, first reader does what?

Answer: Locks the writer semaphore/resource.

48. Last reader does what?

Answer: Releases it.

49. Reader-preference can starve whom?

Answer: Writers.

50. Dining philosophers' naive one-fork-each situation demonstrates which deadlock condition?

Answer: Circular Wait.

DEADLOCK

51. State the Coffman conditions.

Answer: Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.

52. If even one Coffman condition is impossible, deadlock can occur?

Answer: No.

53. RAG edge P !' R means?

Answer: Process requests resource.

54. RAG edge R !' P means?

Answer: Resource assigned to process.

55. Single instance per resource: graph cycle means?

Answer: Deadlock.

56. Multiple instances: cycle always guarantees deadlock?

Answer: No.

57. Banker is deadlock prevention or avoidance?

Answer: Avoidance.

58. Formula for Need?

$$\boxed{\text{Need} = \text{Max} - \text{Allocation}}$$

59. Safety test condition?

$$\boxed{\text{Need}_{i \in \text{Work}}}$$

component-wise.

60. After process hypothetically finishes, what is added to Work?

Answer: Its Allocation.

61. Unsafe state automatically means deadlock currently exists.

Answer: False.

62. Every deadlocked state is unsafe.

Answer: True.

63. Minimum identical resources for n processes, max demand k each?

$$\boxed{n(k-1)+1}$$

under the standard single-resource-type assumptions.

CPU SCHEDULING

64. Turnaround-time formula?

$$\boxed{TAT=CT+AT}$$

65. Waiting-time formula?

$$\boxed{WT=TAT-BT}$$

66. Response-time formula?

$$\boxed{RT=FirstStart+AT}$$

67. Does response time include all later waiting periods?

Answer: No.

Only until first CPU service.

68. FCFS is preemptive?

Answer: No.

69. FCFS famous problem?

Answer: Convoy effect.

70. SJF chooses the shortest job among which processes?

Answer: Processes currently ready/arrived.

71. SJF may cause starvation?

Answer: Yes.

72. Preemptive SJF is called?

Answer: SRTF.

73. SRTF compares original burst or remaining burst?

Answer: Remaining burst.

74. Priority scheduling may cause?

Answer: Starvation of low-priority processes.

75. Remedy?

Answer: Aging.

76. Round Robin is preemptive?

Answer: Yes.

77. RR with very large quantum approaches?

Answer: FCFS.

78. Too-small RR quantum causes?

Answer: High context-switch overhead.

DISK SCHEDULING

79. Disk-head movement between 30 and 80?

$\boxed{50}$

80. SSTF chooses?

Answer: Nearest pending request.

81. SSTF may starve?

Answer: Far-away requests.

82. SCAN is also called?

Answer: Elevator algorithm.

83. SCAN reverses at physical boundary or last request?

Answer: Physical boundary under standard SCAN.

84. LOOK reverses where?

Answer: At the extreme pending request.

85. C-SCAN services requests in one or two directions?

Answer: One direction.

86. **c** in C-SCAN/C-LOOK indicates?

Answer: Circular behavior.

MEMORY / PAGING

87. Logical address is generated by?

Answer: CPU.

88. Who translates logical !' physical?

Answer: MMU.

89. Base = 5000, LA = 300. Physical address?

$\boxed{5300}$

assuming address is within limit.

90. Internal fragmentation means?

Answer: Wasted space inside allocated block.

91. External fragmentation means?

Answer: Free memory scattered into non-contiguous holes.

92. Paging eliminates classic external fragmentation?

Answer: Yes.

93. Paging can have internal fragmentation?

Answer: Yes.

94. Page size equals?

Answer: Frame size.

95. 4-KB page has how many offset bits?

$\lceil \log_2(4\text{KB}) \rceil = 12$

$\boxed{12}$

96. 32-bit VA with 4-KB pages: VPN bits?

$32 - 12 = \boxed{20}$

97. Does paging change offset during translation?

Answer: No.

98. Normal single-level page-table entries correspond to virtual pages or physical frames?

Answer: Virtual pages.

99. TLB miss automatically means page fault?

Answer: No.

100. FIFO/LRU/OPT victim rule in one line?

Answer:

TEXT

```
FIFO !' oldest arrival
LRU !' least recently used in past
OPT !' farthest use in future
```

BONUS — 25 ULTRA-HIGH-YIELD FACTS YOU SHOULD NOT MISS

These are worth revising even though you've completed the 100.

1.

Belady's anomaly:

$\boxed{\text{FIFO}}$

2.

Stack algorithms:

$\boxed{\text{LRU}, \text{OPT}}$

3.

Demand paging:

Bring page when referenced.

4.

Page fault:

Valid page absent from RAM.

5.

Dirty page:

Modified; may require disk write before eviction.

6.

Temporal locality:

Recently used !' likely reused.

7.

Spatial locality:

Nearby location !' likely used.

8.

Thrashing:

TEXT

```
F a u l t s !'  
P a g i n g !'  
C P U   u t i l i z a t i o n !"
```

9.

Thrashing cure:

More frames or fewer active processes.

10.

Working set:

Distinct recently referenced pages.

11.

Segmentation units:

Variable-sized logical segments.

12.

Segment translation:

$\backslash [\text{Offset} < \text{Limit} \backslash]$

then:

$\backslash [\text{PA} = \text{Base} + \text{Offset} \backslash]$

13.

Paging vs segmentation:

TEXT

```
P a g i n g !'   f i x e d  
S e g m e n t a t i o n !'   v a r i a b l e
```

14.

Contiguous file allocation:

Fast random access, difficult growth.

15.

Linked file allocation:

Easy growth, poor random access.

16.

Indexed allocation:

Uses pointer/index block.

17.

FAT:

Linked allocation information stored centrally.

18.

inode:

Metadata + block pointers.

19.

Pointer count per indirect block:

$$\boxed{N = \frac{\text{BlockSize}}{\text{PointerSize}}}$$

20.

Single indirect:

$$N$$

21.

Double indirect:

$$N^2$$

22.

Triple indirect:

$$N^3$$

23.

Bitmap:

One bit per disk block.

24.

Hard link:

Same inode.

25.

Symbolic link:

PART C — THE 20 MOST IMPORTANT THINGS TO MEMORIZE WORD-FOR-WORD

If you remember only twenty things, make them these:

1. Ready waits for CPU; blocked waits for an event.
2. Fork creates; exec replaces.
3. CDHF shared, PRS private.
4. Concurrency is not the same as parallelism.
5. Race conditions come from non-atomic shared-data accesses.
6. Critical-section requirements = MPB.
7. Semaphore: 1 protects, 0 orders, N counts.
8. Producer-consumer = mutex 1, empty N, full 0.
9. Deadlock conditions = MHNC.
10. Banker: Need = Max " Allocation.
11. Gantt !' CT !' TAT!' WT.
12. SRTF uses remaining time.
13. SCAN sees wall; LOOK sees request.
14. Internal = inside; external = holes outside.
15. Page !' Frame, offset stays.
16. TLB miss is not a page fault.
17. FIFO = arrival, LRU = past, OPT = future.
18. FIFO can show Belady's anomaly.
19. Thrashing: Frames !" Faults !" ,CPU !" .
20. inode = D, N, N², N³.

PART D — FINAL FORMULA WALL

Process creation

$$\boxed{2^n}$$

for (n) unconditional forks reached by all processes.

Banker

$$\boxed{\text{Need} = \text{Max} - \text{Allocation}}$$

CPU

$$\boxed{\text{TAT} = \text{CT} - \text{AT}}$$

$$\boxed{WT = TAT - BT}$$

$$\boxed{RT = FirstStart - AT}$$

Disk

$$\boxed{\sum |Next - Current|}$$

Rotation

$$\boxed{T_{\text{rotation}} = \frac{60}{RPM}}$$

$$\boxed{AvgLatency = \frac{T_{\text{rotation}}}{2}}$$

Paging

$$\boxed{Page = \left\lfloor \frac{LA}{PageSize} \right\rfloor}$$

$$\boxed{Offset = LA \bmod PageSize}$$

$$\boxed{PA = Frame \times PageSize + Offset}$$

Page-table size

$$\boxed{PTSize = VirtualPages \times PTESize}$$

Effective time

$$\boxed{EAT = \sum Probability \times Cost}$$

Segmentation

$$\boxed{Offset < Limit}$$

$$\boxed{PA = Base + Offset}$$

inode

$$\boxed{N = \frac{BlockSize}{PointerSize}}$$

$$\boxed{MaxBlocks = D + N + N^2 + N^3}$$

when one single/double/triple pointer exists.

Bitmap

$$\boxed{BitmapBits = \frac{DiskSize}{BlockSize}}$$

CHAPTER 23 COMPLETE '

At this point, continuing to add more theory would give diminishing returns.

The best next stage is:

CHAPTER 24 — GATE OS PYQ SIMULATION SET 1

It should now work like an actual exam session:

- 15 carefully selected GATE-level questions
- no answers shown initially
- MCQ + MSQ + NAT
- mixed topics
- realistic traps
- you solve first
- then the solutions can be checked one by one
- each mistake gets mapped back to the exact chapter/concept you need to revise.

USER

NEXT

ASSISTANT

CHAPTER 24 — GATE OS PYQ SIMULATION SET 1 Ø=Ý%Ø=Ý%Ø=Ý%

Now we shift into **actual exam mode**.

This time:

Do not scroll looking for solutions — there are no solutions in this section.

Try to solve all 15 first.

Use this answer format:

TEXT

1-B
2-6
3-AC
4-...

For NAT questions, write only the numerical answer.

SECTION A — 1-MARK STYLE

Q1 — PROCESS STATES[MCQ]

A process P_i is currently blocked waiting for disk I/O. The disk operation completes.

What is the immediate state transition of P_i ?

A. Blocked $\rightarrow$ Running B. Blocked $\rightarrow$ Ready C. Blocked $\rightarrow$ Terminated D. Blocked $\rightarrow$ New

Q2 — THREADS [MSQ]

Consider two threads belonging to the same process.

Which of the following are normally shared between them?

A. Heap B. Stack C. Code segment D. CPU registers E. Global variables

Q3 — SYSTEM CALLS [MCQ]

Which statement is correct?

A. Every function call causes a context switch. B. Every system call creates a new process. C. A system call provides a controlled mechanism to request an OS kernel service. D. A system call always causes the running process to be replaced by another process.

Q4 — CRITICAL SECTION [MSQ]

Which of the following are requirements of the classical critical-section problem?

A. Mutual Exclusion B. Progress C. Hold and Wait D. Bounded Waiting

Q5 — DEADLOCK [MCQ]

A system guarantees that every process must request resources strictly in increasing order of resource number.

Which Coffman condition is prevented?

A. Mutual Exclusion B. Hold and Wait C. No Preemption D. Circular Wait

SECTION B — NUMERICAL / APPLICATION

Q6 — `fork()` 0=[MAT]

Assume every `fork()` succeeds.

```
c
fork();

if (fork() == 0)
    fork();

fork();
```

How many total processes exist after execution?

Q7 — SEMAPHORE ORDERING $\theta = [1/6]$

Initially:

$\forall [S=0]$

P1:

```
c
print("A");
signal(S);
print("B");
```

P2:

```
c
wait(S);
print("C");
```

Which outputs are possible?

A. ABC B. ACB C. CAB D. BAC

Q8 — RACE CONDITION $\theta = [1/6]$

Initially:

$\forall [x=10]$

Thread T1 executes once:

```
c
x = x + 1;
```

Thread T2 executes once:

```
c
x = x - 1;
```

Assume each statement is implemented as non-atomic Read–Modify–Write.

Which final values are possible?

A. 9 B. 10 C. 11 D. 12

Q9 — BANKER'S ALGORITHM $\theta = [1/6]$

For a process:

$\forall [Max=(8,5,4)]$

$\forall [Allocation=(3,2,1)]$

Find the **sum of the components of the Need vector**.

Q10 — CPU SCHEDULING

Processes:

Process	Arrival Time	Burst Time
P1	0	7
P2	2	4
P3	4	1

Use **SRTF** scheduling.

Assume the currently running process continues in case of an exact remaining-time tie.

Find:

$\boxed{\text{Completion Time of P3}}$

SECTION C — MEMORY MANAGEMENT

Q11 — PAGING

A system has:

- 36-bit virtual addresses
- page size = 8 KB

How many bits are used for the **virtual page number**?

Q12 — PAGE TABLE

A system has:

- 32-bit virtual address
- 4-KB page size
- 8-byte page-table entry

Find the size of a single-level page table in **MB**.

Q13 — TLB

Given:

$\boxed{\text{TLB lookup}=20\text{ns}}$

$\boxed{\text{Main memory}=100\text{ns}}$

TLB hit ratio:

$\boxed{0.9}$

Assume:

Hit

$\lfloor 20+100 \rfloor$

Miss

$\lfloor 20+100+100 \rfloor$

Find effective memory access time in ns.

SECTION D — VIRTUAL MEMORY + FILE SYSTEM

Q14 — PAGE REPLACEMENT

Three initially empty frames use **LRU**.

Reference string:

TEXT

1 2 3 1 4 2 5 1 2

Find the total number of page faults.

Q15 — INODE

A file system uses:

- block size = 4 KB
- block pointer size = 4 bytes
- inode with:
 - 12 direct pointers
 - 1 single-indirect pointer
 - 1 double-indirect pointer

What is the **first logical file-block number handled by the double-indirect pointer**, assuming logical blocks are numbered starting from 0?

STOP HERE — SOLVE FIRST '

Use this compact answer sheet:

TEXT

Q1 =
Q2 =
Q3 =
Q4 =
Q5 =
Q6 =
Q7 =
Q8 =
Q9 =
Q10 =
Q11 =

Q12 =
Q13 =
Q14 =
Q15 =

Suggested exam rule

Try to finish without notes.

For calculations, use only these reminders:

$\backslash$ Need=Max-Allocation $\backslash$

$\backslash$ TAT=CT-AT $\backslash$

$\backslash$ WT=TAT-BT $\backslash$

$\backslash$ OffsetBits= $\log_2$ (PageSize) $\backslash$

$\backslash$ PageTableSize= $\backslash$ #VirtualPages $\times$ PTEsize $\backslash$

$\backslash$ EAT= $\sum$ Probability $\times$ PathCost $\backslash$

$\backslash$ $N=\frac{\text{BlockSize}}{\text{PointerSize}}$ $\backslash$

For Q15 specifically, remember:

TEXT

```
Direct blocks first
!' then Single indirect
!' then Double indirect
```

When you send your 15 answers, I'll score them and explain **only the questions you got wrong or were unsure about**, then we can continue to **PYQ Simulation Set 2**.